Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

TEIL 1

Inhaltsverzeichnis

1 ÜBERBLICK ÜBER R UND INSTALLATION 3

1.1 Zielgruppe und Voraussetzungen 3

1.2 Allgemeiner Überblick 3

1.3 Benutzte Softwarepakete und deren Installation 4

1.4 Übungen 5

2 ERSTE SCHRITTE 6

2.1 Numerische Ausdrücke und Zuweisungen 7

2.2 Variablen 7

2.3 Ein Vektor als einfacher Datensatz 9
2.3.1 Recycling 11

2.4 Mehr zu RStudio 13

2.5 Beispiele und Demos 16

2.6 Skripte 22

2.7 Übungen 23

3 ERSTER ÜBERBLICK: KONTROLLSTRUKTUREN UND FUNKTIONEN 24

3.1 Namen und Objekte: Ein erster Vergleich mit C und C++ 24

3.2 Funktionen 28
3.2.1 Benannte und optionale Argumente 32
3.2.2 Call-by-value und Copy-on-modify 33
3.2.3 Keine Zeiger und kein Call-by-reference 33
3.2.4 Operator versus Funktionsaufruf 34

3.3 Kontrollstrukturen 35
3.3.1 Verzweigung: `if-else` 35
3.3.2 Verzweigung: `ifelse` 36
3.3.3 Schleife: `while` und `repeat` 37
3.3.4 Schleife: `for` 37
3.3.5 Mehrfachverzweigung: `switch` 38

3.4 Übungen 38

1 Überblick über R und Installation

1.1 Zielgruppe und Voraussetzungen

Dieses Skript richtet sich an die Studierenden des berufsbegleitenden Masterstudiums der FH-Südwestfalen am Standort Meschede. Neben Grundkenntnissen in der Informatik werden solide Kenntnisse einer imperativen Programmiersprache sowie erste grundlegende Kenntnisse in einer objektorientierten Programmiersprache vorausgesetzt.

Vorausgesetzt werden:

- Grundkenntnisse in der Informatik
- solide Kenntnisse einer imperativen Programmiersprache (wie C, C#, Java oder Python)
- erste grundlegende Kenntnisse in einer objektorientierten Programmiersprache (wie C++ oder Java)

Insbesondere werden grundlegende und in allen imperativen Programmiersprachen zu findende Konzepte wie Variablen und Funktionen vorausgesetzt ebenso wie Kenntnisse über Kontrollstrukturen wie Verzweigungen und Schleifen.

Im Folgenden werden wir oft Vergleiche zu den Programmiersprachen C und C++ ziehen; daher sind Kenntnisse in C und C++ hilfreich, aber nicht zwingend notwendig. Außerdem gehen wir davon aus, dass die Studierenden in der Lage sind, englische Fachtexte zu lesen.

1.2 Allgemeiner Überblick

Da es viel gute Quellen und ausführliche Informationen im Internet über R gibt, wollen wir an dieser Stelle auf einen allgemeinen Überblick über die Programmiersprache R verzichten. Stattdessen sollen Sie folgende Teile des Wikipedia-Artikels „R (Programmiersprache)“[1] lesen:

- von Anfang bis zum Zitat von John M. Chambers
  „Um Berechnungen in R zu verstehen sind zwei Sätze hilfreich:
  Alles, was existiert, ist ein Objekt.
  Alles, was passiert, ist ein Funktionsaufruf“
  im Beginn des Abschnitts „Programmierparadigmen“
- der Abschnitt „Pakete“
- der Abschnitt „Unterstützung durch Stiftungen“
- der Abschnitt „Rezeption“

Danach sollen Sie folgende Fragen (siehe unten) beantworten. Nur ein Teil der Antworten ergibt sich allerdings direkt aus dem Wikipedia-Artikel; für den Rest sind grundlegende Informatikkenntnisse gefragt, die Sie hoffentlich im Bachelorstudium gelernt haben. Die Diskussion der Antworten wird Bestandteil der ersten Präsenzveranstaltung sein.

[1] https://de.wikipedia.org/wiki/R_(Programmiersprache). Im Downloadbereich finden Sie eine entsprechende PDF-Datei des Wikipedia-Eintrags, die am 26.7.2019 heruntergeladen wurde.

1.3 Benutzte Softwarepakete und deren Installation

Neben dem Softwarepaket R werden wir auch *RStudio* intensiv benutzen. *RStudio* (genauer *RStudio Desktop*) ist eine integrierte Entwicklungsumgebung, die wie R als Open Source Lizenz zur Verfügung steht. Insgesamt gehen wir also davon aus, dass Sie für diese Veranstaltung folgende Softwarepakete auf Ihrem persönlich verfügbaren Rechner installieren:

- R: Version R 3.6.0 (oder höher, am besten die aktuellste Version):
  - Installationsquelle: https://cran.rstudio.com/
  - Installieren Sie eine dem Betriebssystem entsprechende vorkompilierte Binär-Distribution („precompiled binary distribution“). Unter Windows brauchen Sie lediglich die „base distribution“
  - Wählen Sie die 64-bit-Version, wenn Sie ein 64-bit Betriebssystem benutzen
- RStudio Desktop: Version 1.2.1335 (oder höher, am besten aktuellste Version)
  - Installationsquelle: https://www.rstudio.com/products/rstudio/download/ ("Free"-Version auszuwählen)
- plus verschiedenen Packages, auf die wir weiter unten eingehen werden

Im Verlauf dieses Moduls werden zusätzlich unterschiedliche Packages benötigt. R benutzt sogenannte *Pakete* (packages), um thematisch zusammengehörende Software zu bündeln. Pakete sind vergleichbar mit Bibliotheken in anderen Programmiersprachen. Einige Pakete werden je nach Konfiguration automatisch geladen wie zum Beispiel das base-Paket. Pakete werden in bestimmten Verzeichnissen gespeichert. Der R-Befehl `.libPaths()` listet die entsprechenden Verzeichnisse auf. Abhängig vom benutzten Rechner könnte eine Ausgabe wie folgt aussehen (eingegeben in RStudio im Konsolen-Bereich, siehe weiter unten):

```
> .libPaths()
[1] "C:/Users/willms/Documents/R/win-library/3.6"
[2] "C:/Program Files/R/R-3.6.0/library"
```

Aus Zeit- und Speichergründen werden aber nicht alle verfügbaren Pakete beim Start von R geladen. Später werden wir sehen, wie bei Bedarf neue Pakete installiert und geladen werden können.

1.4 Übungen

Aufgabe 1

Beantworten Sie folgende Fragen.

- Wird R interpretiert oder kompiliert?
- Was ist der Unterschied zwischen einem Interpreter und einem Compiler? Erläutern Sie Vor- und Nachteile.[2]
- Nennen Sie populäre Programmiersprachen, die kompiliert bzw. interpretiert werden.
- Was ist ein JIT-Compiler?
- Gibt es für R einen JIT-Compiler?[3]
- Ist R Open Source?
- Welche Lizenzkosten entstehen?
- Was ist RStudio?
- Was ist eine integrierte Entwicklungsumgebung (IDE)?
- Nennen Sie Beispiele populärer Entwicklungsumgebungen, z. B. für C und C++.
- Was ist Scheme?
- Was ist S?
- Ist R für alle populären Desktop-Betriebssysteme verfügbar?
- Was sind die Vorteile von R?
- Was ist in R ein Paket? Paket ist hier die deutsche Übersetzung für welchen englischen Begriff?
- Was ist aus programmiertechnischer Sicht der Unterschied zwischen einem Paket und einer Bibliothek?

[2] Vgl. z. B. Willms, Jürgen: *Studienbuch: Grundlagen der Programmierung (Informatik 2)*. 1. Aufl.. WGS, 2009., Abschnitte 1.2.1 - 3

[3] Diese Frage wollen wir erst einmal offen lassen

2 Erste Schritte

Es gibt mehrere Möglichkeiten, R zu starten. Eine interaktive R-Sitzung kann zum Beispiel direkt in einem Terminalfenster (unter Windows oft auch Eingabefenster genannt) durch die Eingabe des Kommandos „R“ gestartet werden. Statt eine interaktive R-Sitzung zu beginnen, können auch sogenannte R-Skripte in einem Terminalfenster ausgeführt werden.

Im Verlauf dieser Veranstaltung werden wir fast ausschließlich R indirekt über *RStudio* (genauer *RStudio Desktop*) starten, was viele Vorteile bietet. *RStudio* (genauer *RStudio Desktop*) ist eine integrierte Entwicklungsumgebung (IDE), die das Arbeiten mit R stark vereinfacht, und wie R als Open Source Lizenz zur Verfügung steht.

Ein typisches Fenster in *RStudio* umfasst mehrere Bereiche. In Abbildung 1 sehen Sie z. B. drei Bereiche: „Konsole“, „Umgebung“ und „Hilfe“. Im dort abgebildeten Konsolen-Bereich sehen wir zum Beispiel, welche R-Version und Plattform benutzt wird.

Jeder Bereich wiederum kann durch Aktivierung unterschiedlicher Reiter verändert werden. Das gesamte Layout der Bereiche kann individuell angepasst werden (siehe Menüeintrag „View“ / „Panes“). Es passt sich auch dynamisch an: so wird zum Beispiel bei einer Datenvisualisierung statt des „Hilfe“-Bereichs der „Plot“-Bereich aktiviert. Wird ein Skript geladen, erscheint zusätzlich ein Bereich „Editor“. Wir werden später sehen, was R-Skripte (kurz: Skripte) sind und welche Bedeutung sie haben. Wir starten jedoch zuerst mit ganz einfachen Befehlen in der Konsole.

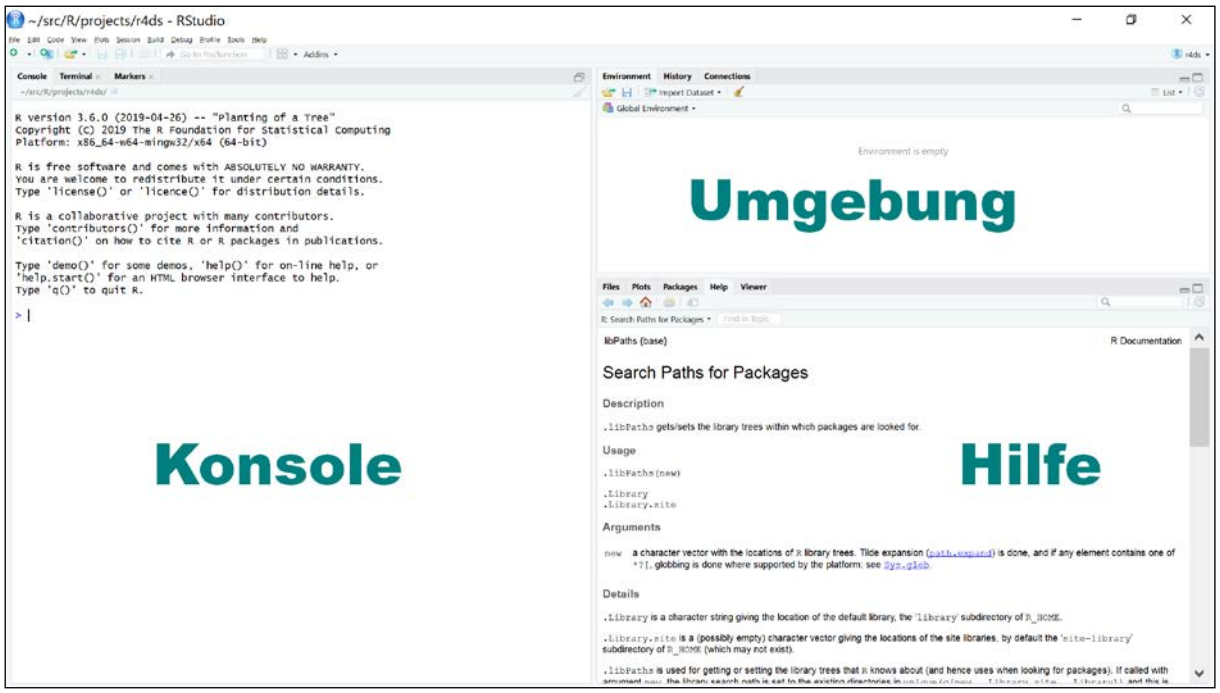


Abbildung 1: RStudio Bereiche

Im Bereich „Hilfe“ in Abbildung 1 sehen Sie den Manual-Eintrag zum Befehl `.libPaths()`. Sie erhalten diesen Eintrag, indem Sie `?.libPaths()` im Konsolenbereich hinter dem Prompt-Zeichen `'>'` eingeben. Wir werden später auf das Hilfesystem etwas ausführlicher eingehen.

2.1 Numerische Ausdrücke und Zuweisungen

Befehle (engl.: commands) können direkt im Bereich „Konsole“ hinter dem Prompt-Zeichen `'>'` eingegeben werden.

```
> 6*7
[1] 42
```

Sie erhalten sofort das Ergebnis des eingegebenen Ausdrucks. Ist der eingegebene Ausdruck syntaktisch noch nicht abgeschlossen, dann wechselt in der Konsole das Prompt-Zeichen `'>'` zu `'+'`; mit einem Semikolon kann ein Abschluss erzwungen werden.

```
> 2^10   # Potenzrechnung: 2 hoch 10
[1] 1024
```

Wie wir in diesem Beispiel zusätzlich sehen können, werden wie in vielen anderen Programmiersprachen auch alle Zeichen in einer Zeile, die nach einem '#' folgen, als Kommentar interpretiert.

```
> 10/3   # ganzzahlige Division?
[1] 3.333333
```

Auch sehen wir in diesem Beispiel, dass R wie C, C++ oder Java nicht ein Komma, sondern einen Punkt als *Dezimaltrennzeichen* verwendet.

Frage

- Welchen Wert hat in den Programmiersprachen C und C++ der Ausdruck `10/3` ?

WICHTIG

- Alle Zeichen in einer Zeile, die nach einem '#' folgen, werden als Kommentar interpretiert
- Punkt als Dezimaltrennzeichen

2.2 Variablen

Wir können das Ergebnis einer Berechnung natürlich auch einer Variablen zuweisen. Wie wir später sehen werden, handelt es sich dabei um eine **globale** Variable!

```
> res = 10/3       # '=' sollte vermieden werden
```

Allerdings bleibt dann das Ergebnis der Berechnung verborgen. Dies sehen wir erst, wenn wir `print(res)` oder kürzer `res` eingeben

```
> res
[1] 3.333333
```

Es ist allerdings in R üblich, anstelle von `'='` den Operator `'<-'`, der aus zwei Buchstaben besteht, für eine Zuweisung zu benutzen. Das hat zum Teil historische Gründe; aber es gibt auch spezielle Situationen, wo `'='` als Zuweisungsoperator nicht wie erwartet funktioniert. Also besser:

```
> res <- 10/3       # Besser
> res
[1] 3.333333
```

Weil Zuweisungen oft benutzt werden, gibt es für `'<-'` in *RStudio* eine besondere Tastenkombination: `Alt-'-'`.

Im Gegensatz zu C oder C++ müssen in R Variablen nicht vorher vereinbart werden und sind nicht fest für den Rest ihrer Geltungsdauer an einem bestimmten Datentyp gebunden, wie das nächste Beispiel zeigt:

```
> name <- 10
> name
[1] 10
> name <- "zehn"
> name
[1] "zehn"
```

Zuerst verweist in dem obigen Beispiel die Variable `name` auf den numerischen Wert 10. Nach einer erneuten Zuweisung weist dann die Variable `name` auf die Zeichenkette "zehn", also auf ein Objekt eines ganz anderen Datentyps. Dies ist in C oder C++ so nicht möglich! Wie wir sehen werden, ist R vektorbasiert und unsere vorläufige Analyse, dass in dem obigen Beispiel `name` auf den numerischen Wert 10 bzw. auf eine Zeichenkette verweist, ist ungenau. Besser wäre die Formulierung, dass im ersten Fall die Variable `name`[4] auf einen numerischen Vektor zeigt, der aus nur einem Element besteht. Für den zweiten Fall gilt Ähnliches.

R unterscheidet grundsätzlich zwischen Groß- und Kleinschreibung. So bezeichnen zum Beispiel `res` und `Res` zwei unterschiedliche Variablen:

```
> Res = 1.4   # Groß- und Kleinschreibung beachten!
> Res
[1] 1.4
> res
[1] 3.333333
```

Es gibt Einschränkungen für Variablennamen, auf die wir aber nicht detailliert eingehen werden. Abgesehen von Schlüsselworten wie `TRUE` oder `if` (siehe auch `?Reserved` für eine Liste aller Schlüsselworte) sind grundsätzlich alphanumerische Zeichen aus dem ursprünglichen ASCII-Zeichensatz erlaubt, aber keine Umlaute oder Sonderzeichen mit Ausnahme von `'_'` und `'.'`[5]. Das erste Zeichen sollte ein Buchstabe sein. Der Begriff `syntatic names` bezieht sich auf erlaubte Namen (im Gegensatz zu `non-syntatic names`).

[4] in Verbindung mit der entsprechenden Umgebung (Environment); auf dieses Konzept werden wir später genauer eingehen

[5] In C und C++ hingegen ist das Zeichen `'.'` Bestandteil der sogenannten Punktnotation (genutzt in Strukturen und Klassen) und darf nicht in einem Variablennamen vorkommen.

Mittlerweile gibt es eine ganze Reihe von unterschiedlichen Style-Guides, wie:

- https://jef.works/R-style-guide/
- https://google.github.io/styleguide/Rguide.xml
- http://adv-r.had.co.nz/Style.html

Wir wollen im Folgenden der Empfehlung folgen, Variablen- und Funktionsnamen klein zu schreiben und Großbuchstaben lediglich für die Trennung separater Worte zu nutzen (der sogenannte „*camelCase*“, im Gegensatz zu dem von manchen Autoren bevorzugten „*snake_case*“). Insbesondere wollen wir auch nicht das Sonderzeichen `'.'` in Variablen- und Funktionsnamen nutzen, da es in manchen Bereichen wie Klassen eine unterschiedliche Bedeutung hat.

WICHTIG

- Für eine Zuweisung `'<-'` benutzen
- Die Tastenkombination `Alt-'-'` steht in RStudio für `'<-'`
- In R werden Anweisungen nicht durch ein Semikolon abgeschlossen
- Variablen müssen nicht vorher vereinbart werden
- Variablen sind nicht fest an einen bestimmten Datentyp gebunden
- R unterscheidet zwischen Groß- und Kleinschreibung

2.3 Ein Vektor als einfacher Datensatz

Vektoren spielen in R eine zentrale Rolle und wir werden in einem späteren Kapitel detailliert auf Vektoren eingehen. An dieser Stelle wollen wir zur Einführung lediglich ein erstes Beispiel mit numerischen Vektoren vorstellen. Ein Vektor ist wie ein Feld (array) eine Datenstruktur, die eine Folge von Elementen desselben Typs enthält. Erzeugt werden soll im nächstem Beispiel ein einfacher Datensatz `p`, der die ersten zehn Primzahlen enthält:

```
> p <- c(2,3,5,7,11,13,17,19,23,29) # Erzeugt einen Vektor der Läng
e 10
> p
 [1]  2  3  5  7 11 13 17 19 23 29
```

`p` verweist nun auf einen Vektor, der die ersten zehn Primzahlen enthält. Die Größen `2, 3, 5, 7, 11, 13, 17, 19, 23, 29` werden mithilfe der Funktion `c()` zu einem numerischen Vektor zusammengefasst und der Variablen `p` zugewiesen; `c` steht hier für *concatenate* (deutsch: aneinanderhängen) und gibt immer einen Vektor zurück. Wie in C haben wir mithilfe des Indexoperators `[]` wahlfreien Zugriff auf die einzelnen Elemente des Vektors. Aber im Gegensatz zu C und vielen anderen Programmiersprachen beginnt die Indexierung in R nicht bei 0 sondern bei 1!

```
> p[1]
[1] 2
> p[2]
[1] 3
> p[10]
[1] 29
```

Was passiert, wenn wir über die Indexgrenzen hinweggehen? In C und C++ ist dies ein Fehler!

```
> p[11]
[1] NA
> p[12]
[1] NA
> p[13]
[1] NA
```

In R hingegen ist dies nicht fehlerhaft! R benutzt das Symbol `NA` (*not available*), um anzuzeigen, dass ein Wert nicht vorhanden ist (im Englischen oft als „missing data“ bezeichnet). Unser Beispieldatensatz besteht nach wie vor aus 10 Elementen. Was passiert aber, wenn wir dem 13. Element einen Wert zuweisen?

```
> p[13] <- 41
> p
 [1]  2  3  5  7 11 13 17 19 23 29 NA NA 41
```

Offenbar hat sich jetzt die Größe des Vektors `p` dynamisch angepasst: `p` besteht nun aus dreizehn Elementen. Sowohl das elfte als auch das zwölfte Element hat jedoch keinen regulären numerischen Wert. Auf Vektoren werden wir später noch sehr ausführlich eingehen.

WICHTIG

- Mit `c()` können Vektoren erzeugt werden
- Indexierung beginnt in R nicht bei 0 sondern bei 1!
- `NA` (not available) wird benutzt, um anzuzeigen, dass ein Wert nicht vorhanden ist.

In R kann man ähnlich wie z. B. in MATLAB Operatoren direkt auf Vektoren anwenden:

```
> p
 [1]  2  3  5  7 11 13 17 19 23 29 NA NA 41
> p^2
 [1]    4    9   25   49  121  169  289  361  529  841   NA   NA 16
81
> 2*p +1
 [1]  5  7 11 15 23 27 35 39 47 59 NA NA 83
> p%%4      % Modulo-Operator
 [1]  2  3  1  3  3  1  1  3  3  1 NA NA  1
```

Hier bezeichnet „`%%`“ den Modulo-Operator; er gibt den Rest bei einer ganzzahligen Division zurück. Ebenso können Funktionen auf Vektoren angewandt werden. So gibt es zum Beispiel die Funktionen `mean()` und `sd()`, die das arithmetische Mittel und die Standardabweichung zurückgeben.

```
> mean(p)
[1] NA
> sd(p)
[1] NA
```

Hier bereiten die fehlenden Werte Probleme und die beiden Funktionen können die statistischen Größen nicht berechnen. Allerdings kann man bei vielen Funktionen durch den Parameter `na.rm =TRUE` angeben, dass fehlende Werte nicht berücksichtigt werden sollen:

```
> mean(p, na.rm=TRUE)     # rm steht für remove
[1] 15.45455
> sd(p, na.rm=TRUE)
[1] 12.04461
```

Es ist nicht überraschend, dass man Funktionen wie `mean()` oder `sd()` auf Vektoren anwenden kann. Aber wie sieht dies mit Funktionen wie `sqrt()` aus? Die Funktion `sqrt()` gibt die Wurzel einer Zahl zurück und wird in vielen Programmiersprachen üblicherweise auf skalare Werte angewandt.

```
> sqrt(x)
 [1] 1.000000 1.414214 1.732051 2.000000 2.236068 2.449490 2.645751 2.828427 3.000000
[10] 3.162278 3.316625 3.464102 3.605551 3.741657 3.872983 4.000000
```

Auch hier haben wir ein ähnliches Verhalten wie in MATLAB; die Funktion `sqrt()` wird auf jedes Element des Vektors angewandt. Als Ergebnis wird ein Vektor gleicher Länge zurückgegeben („*vector in, vector out*“). Solche Funktionen nennt man auch *vektorisierbar*. In Abschnitt 3.3.2 werden wir sehen, dass zum Beispiel Funktionen, die eine `if-else` Verzweigung enthalten, nicht vektorisierbar sind.

WICHTIG

- `%%` Modulo-Operator
- Operatoren und Funktion können direkt auf Vektoren angewandt werden
- Durch `na.rm =TRUE` werden bei vielen Funktionen `NA`-Werte ignoriert
- `mean()`, `sd()`, `sqrt()`
- vektorisierbare Funktionen

2.3.1 Recycling

Manchmal verlangt eine Operation, dass alle beteiligten Vektoren dieselbe Länge haben. In diesem Fall ergänzt R die kürzeren Vektoren, indem es seine Werte periodisch wiederholt. Dieses Verhalten nennt man auch *Recycling*.

```
> x <- rep(1,6) # entspricht c(1,1,1,1,1,1)
> x
[1] 1 1 1 1 1 1
> y <- c(1,2)
> x+y
[1] 2 3 2 3 2 3
```

Der Funktionsaufruf `rep(x,n)` (für *replicate*) erzeugt ein Objekt des gleichen Typs wie `x`, das den Wert von `x` genau `n`-mal wiederholt. Berechnet wurde in diesem Fall also

```
> c(1,1,1,1,1,1) + c(1,2,1,2,1,2)
[1] 2 3 2 3 2 3
```

Allerdings gibt es eine Warnung, wenn die Länge des größeren Vektors kein Vielfaches des kleineren Vektors ist:

```
> x <- rep(1,6)
> y <- 1:4   # wie c(1L, 2L, 3L, 4L)
> str(y)
 int [1:4] 1 2 3 4
> x+y
[1] 2 3 4 5 2 3
Warning message:
In x + y : longer object length is not a multiple of shorter object
length
```

Allgemein wird durch einen Ausdruck wie `n:m` (wobei `n` und `m` ganze Zahlen sind) ein Vektor erzeugt, dessen erstes Element `n` ist, dessen letztes Element `m` ist und aufeinanderfolgende Elemente sich entweder alle um 1 oder alle um -1 unterscheiden.

Wie man in der Manual-Hilfe sieht, ist `rep_len(x, length.out)` eine Variante von `rep()`. `rep_len(x, length.out)` gibt eine periodisch fortgesetzte Kopie von `x` mit der Länge `length.out` zurück:

```
> rep_len(1:3, 5)
[1] 1 2 3 1 2
> rep_len(1:3, 7)
[1] 1 2 3 1 2 3 1
```

Frage

- Welchen Wert hat der Vektor `7.1:3.3` ?
- Welchen Wert hat der Vektor `2:5 + 10` ?

Wenn man weiß, ob der Additions- oder der Bereichsoperator eine höhere Priorität hat, dann kann man die letzte Frage leicht beantworten. `?Syntax` liefert dabei folgende Antwort; die Operatoren sind gemäß ihrer Priorität (mit höchster Priorität zuerst) aufgelistet:

`:: :::`	access variables in a namespace
`$ @`	component / slot extraction
`[ [[`	indexing
`^`	exponentiation (right to left)
`- +`	unary minus and plus
`:`	sequence operator
`%any%`	special operators (including %% and %/%)
`* /`	multiply, divide
`+ -`	(binary) add, subtract
`< > <= >= == !=`	ordering and comparison
`!`	negation
`& &&`	and
`| ||`	or
`~`	as in formulae
`-> ->>`	rightwards assignment
`<- <<-`	assignment (right to left)
`=`	assignment (right to left)
`?`	help (unary and binary)

Tabelle 2-1: Auszug aus dem Manual-Eintrag `?Syntax`

WICHTIG

- `rep()`
- Recycling
- `y <- 1:4` (oder allgemeiner `n:m`)
- Priorität der Operatoren

2.4 Mehr zu RStudio

Im Bereich „Umgebung“ sehen wir nun die von uns erzeugten globalen Variablen und deren aktuelle Werte. Hier werden die Namen aller erzeugten Objekte aufgelistet; dies bezieht sich sowohl auf Daten als auch auf Funktionen (und andere Objekte). Zusätzlich erhält man neben den Namen eine kurze und kompakte Zusammenfassung der entsprechenden Werte.

Mithilfe der Cursortasten ↑ und ↓ kann im Konsolenbereich von RStudio innerhalb der bereits eingegebenen Befehle navigiert werden. Mit den → und ← kann der Cursor innerhalb der aktuellen Befehlszeile bewegt werden und gegebenenfalls Korrekturen vorgenommen werden. Alternativ kann der Reiter „History“ (in dem Bereich rechts oben) aktiviert werden, so dass alle eingegebenen Befehle sichtbar werden.

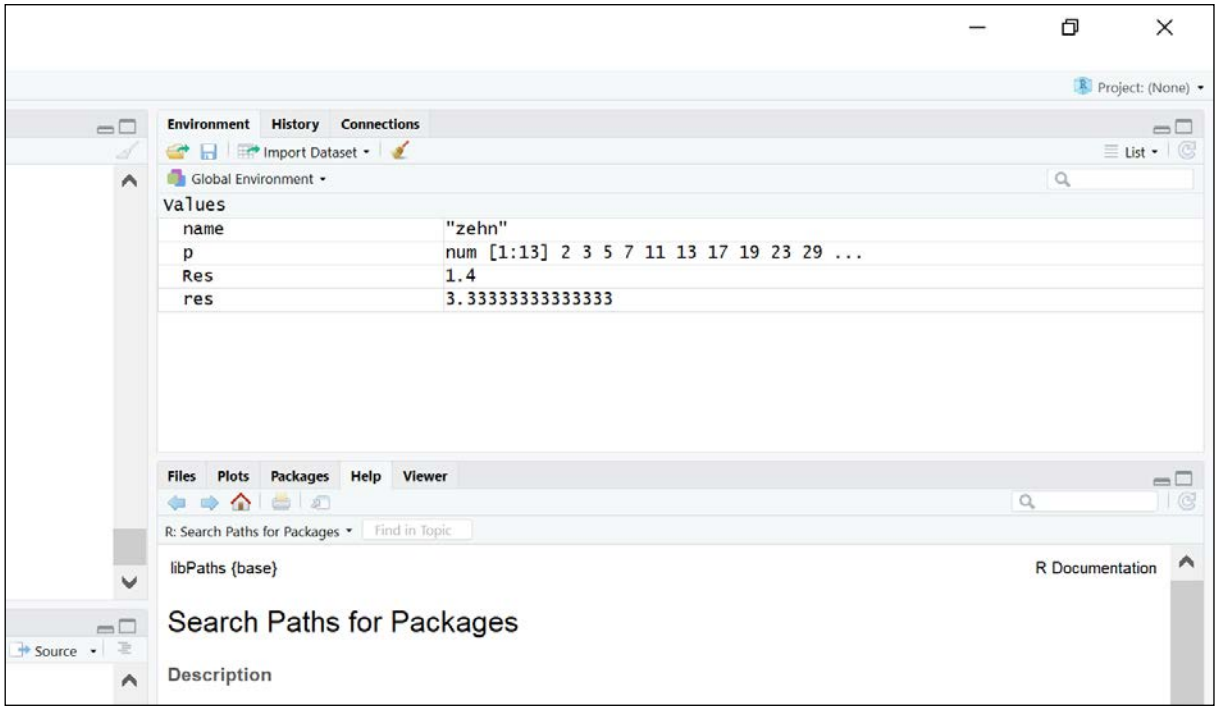


Abbildung 2: Die erzeugten globalen Variablen RStudio im Bereich „Umgebung“

RStudio bietet viele hilfreiche Unterstützungen an, von denen wir allerdings nur einige vorstellen werden. Weiterführende Dokumentationen finden Sie u. a. unter dem Menüeintrag „Help“ / „RStudio Docs“.

RStudio bietet ein umfangreiches Hilfesystem an. Wir wollen dies an einem Beispiel demonstrieren.

```
> x <- seq(0, 4*pi, by=0.1)
> x
  [1]  0.0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9  1.0  1.1  1.2  1.3  1.4
 [16]  1.5  1.6  1.7  1.8  1.9  2.0  2.1  2.2  2.3  2.4  2.5  2.6  2.7  2.8  2.9
 [31]  3.0  3.1  3.2  3.3  3.4  3.5  3.6  3.7  3.8  3.9  4.0  4.1  4.2  4.3  4.4
 [46]  4.5  4.6  4.7  4.8  4.9  5.0  5.1  5.2  5.3  5.4  5.5  5.6  5.7  5.8  5.9
 [61]  6.0  6.1  6.2  6.3  6.4  6.5  6.6  6.7  6.8  6.9  7.0  7.1  7.2  7.3  7.4
 [76]  7.5  7.6  7.7  7.8  7.9  8.0  8.1  8.2  8.3  8.4  8.5  8.6  8.7  8.8  8.9
 [91]  9.0  9.1  9.2  9.3  9.4  9.5  9.6  9.7  9.8  9.9 10.0 10.1 10.2 10.3 10.4
[106] 10.5 10.6 10.7 10.8 10.9 11.0 11.1 11.2 11.3 11.4 11.5 11.6 11.7 11.8 11.9
[121] 12.0 12.1 12.2 12.3 12.4 12.5
```

Aus diesem Beispiel kann man bereits erahnen, was die Funktion `seq()` bewirkt. Durch `help(seq)` oder kürzer durch `?seq` erhält man in RStudio den entsprechenden Manual-Eintrag. Weiterhin bezeichnet hier `pi` eine in R eingebaute Konstante, die der Kreiszahl entspricht (Verhältnis des Umfangs eines Kreises zu seinem Durchmesser).

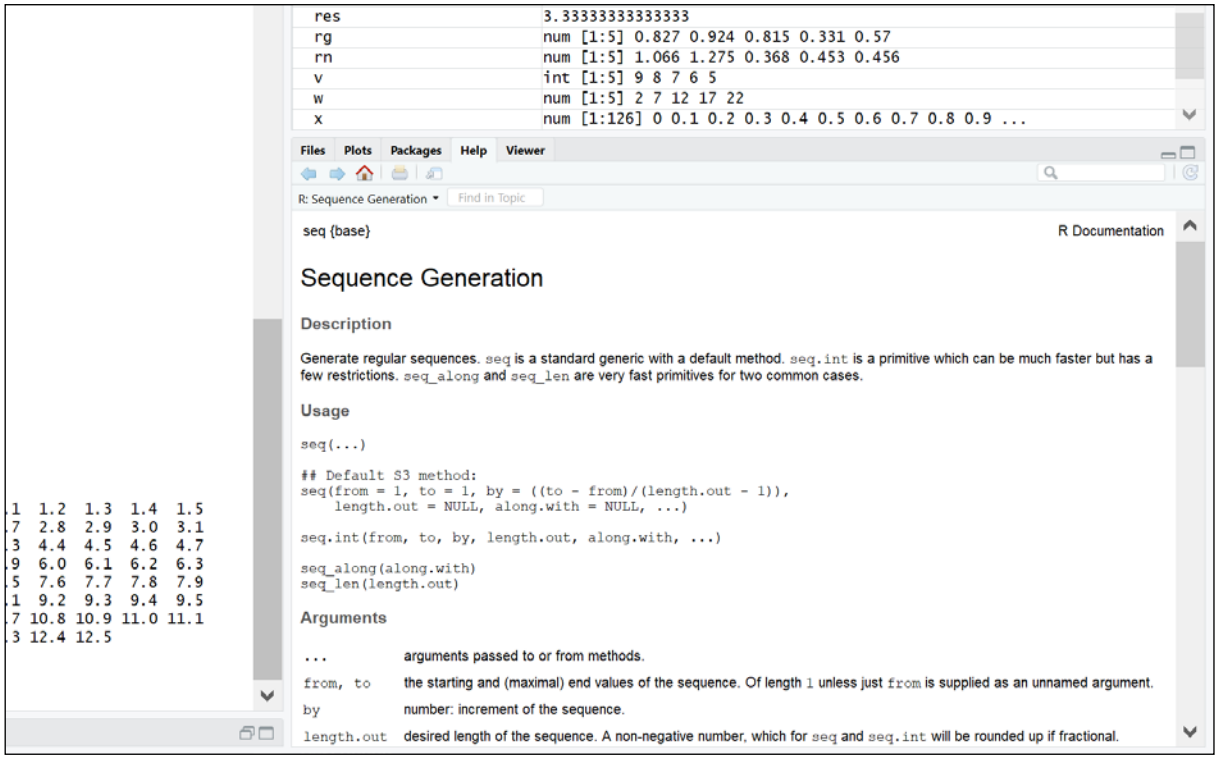


Abbildung 3: Manual-Eintrag angezeigt in RStudio

Die Eingabe von `x` im Konsolenbereich (hinter dem Prompt-Zeichen `'>'`) bewirkt, dass die Funktion `print(x)` aufgerufen wird. Enthält `x` viele Daten, kann dies schnell sehr unübersichtlich werden. Daher gibt es die Funktion `str()`, die eine alternative kompakte Darstellung bereitstellt (`str` steht hier für *structure*):

```
> str(x)
 num [1:126] 0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 ...
```

Hieraus entnehmen wir, dass `x` ein numerischer Vektor mit insgesamt 126 Elementen ist.

Schließlich wollen wir noch den „Plot“-Bereich in RStudio kurz vorstellen. Dazu geben wir folgenden Befehl ein:

```
> plot(x, sin(x), type="l", col="blue")
```

Durch `type="l"` legen wir fest, dass die einzelnen Punkte durch Linien verbunden werden sollen, und durch `col="blue"`, dass die Zeichenfarbe blau sein soll.

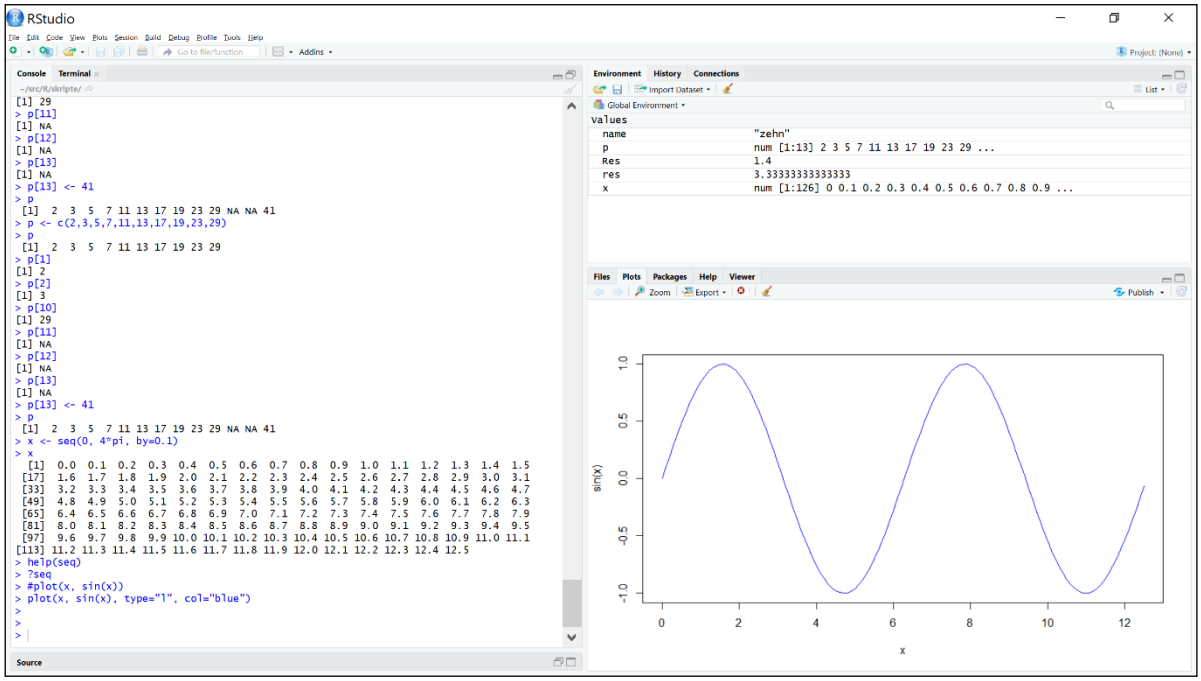


Abbildung 4: Plot in RStudio

WICHTIG

- Cursortasten im Konsolenbereich von RStudio
- Durch `?xyz` erhält man den Manual-Eintrag von `xyz`
- `str()` für eine kompakte textuelle Darstellung

2.5 Beispiele und Demos

R bietet standardmäßig unterschiedliche Präsentationen an, siehe `?demo`. Um z. B. eine kurze Demo über die Grafikmöglichkeiten in R zu erhalten, geben sie `demo(graphic)` ein.

Daneben gibt es hauptsächlich zu Übungszwecken unterschiedliche Datensätze, siehe `?datasets` bzw. `library(help = "datasets")`. Wir werden kurz auf den Datensatz `Nile` eingehen, der die jährliche Durchflussmenge (in [10^8 m^3]) am Nil in Aswan (früher Assuan) zwischen 1871 und 1970 beschreibt.

```
> Nile
Time Series:
Start = 1871
End = 1970
Frequency = 1
  [1] 1120 1160  963 1210 1160 1160  813 1230 1370 1140  995  935 1110  994 1020  960
 [17] 1180  799  958 1140 1100 1210 1150 1250 1260 1220 1030 1100  774  840  874  694
 [33]  940  833  701  916  692 1020 1050  969  831  726  456  824  702 1120 1100  832
 [49]  764  821  768  845  864  862  698  845  744  796 1040  759  781  865  845  944
 [65]  984  897  822 1010  771  676  649  846  812  742  801 1040  860  874  848  890
 [81]  744  749  838 1050  918  986  797  923  975  815 1020  906  901 1170  912  746
 [97]  919  718  714  740
```

Oder in kompakter Darstellung:

```
str(Nile)
 Time-Series [1:100] from 1871 to 1970: 1120 1160 963 1210 1160 1160 813 1230 1370 1140 ..
```

Hierbei werden wir jedoch nicht auf alle Einzelheiten eingehen, sondern wir wollen beispielhaft einige Möglichkeiten von R aufzeigen. Mittelwert und Standardabweichung sind schnell berechnet:

```
> mean(Nile)
[1] 919.35
> sd(Nile)
[1] 169.2275
```

Mit `summary()` erhalten wir diese beiden Werte zusammen mit vier anderen wichtigen statistischen Kennzahlen (Minimum, untere und obere Quartil und Median), auf die wir an dieser Stelle nicht weiter eingehen wollen (siehe auch `?summary`):

```
> summary(Nile)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  456.0   798.5   893.5   919.4  1032.5  1370.0
```

Ein Plot der Daten ist ebenfalls schnell erzeugt:

```
> plot(Nile)
```

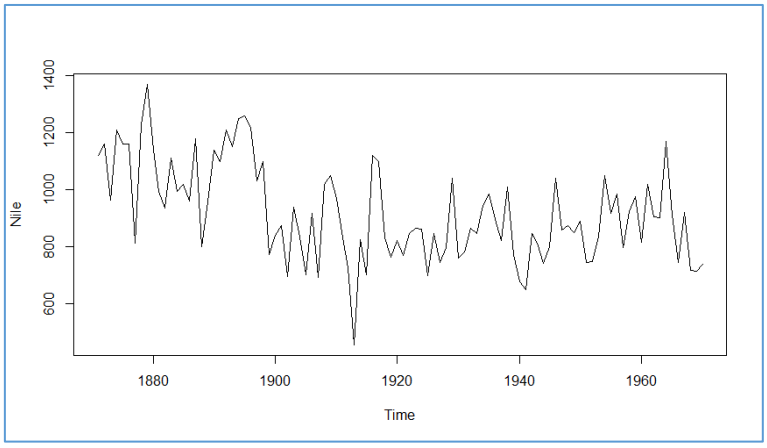


Abbildung 5

Ebenso ist ein Histogramm:

```
> hist(Nile)
```

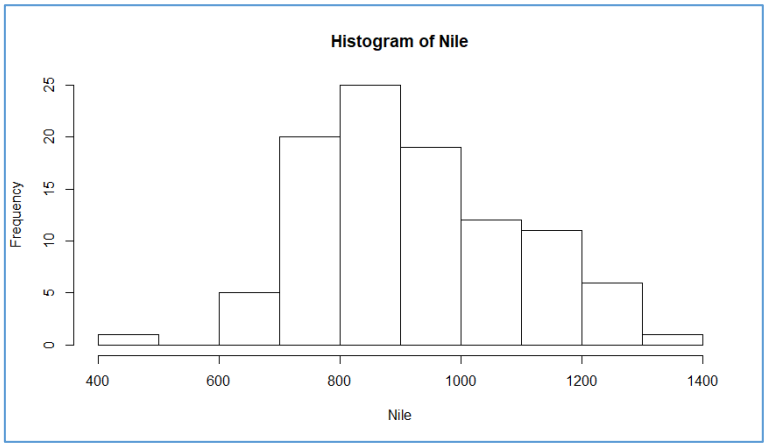


Abbildung 6

und ein Boxplot schnell erstellt:

```
> boxplot(Nile)
```

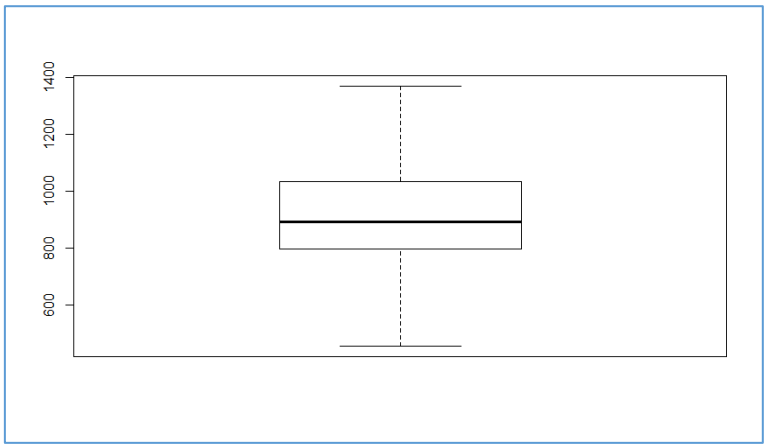


Abbildung 7

In R sind bereits viele Funktionen zu häufig verwendeten Wahrscheinlichkeitsverteilungen implementiert. Exemplarisch wollen wir an dieser Stelle die Funktionen `rnorm()` und `runif()` betrachten (siehe `?rnom` und `?runif`). Beide Funktionen geben *Pseudozufallszahlen* gemäß der Normalverteilung bzw. der Gleichverteilung zurück. Werden keine weiteren Argumente spezifiziert, dann wird bei der Normalverteilung Mittelwert 0 und Standardabweichung 1 angenommen; entsprechend gibt `rnorm(n)` einen Vektor mit `n` normalverteilten Pseudozufallszahlen zurück:

```
> rnorm(10)
 [1] -0.002234582 -1.370477832 -0.634437683  0.818618540 -1.533895474 -0.009951146
-0.058751592 -1.689363070  0.468880835  1.024772154
```

Bei der Gleichverteilung wird standardmäßig vom Intervall [0,1] ausgegangen; entsprechend gibt `runiform(n)` einen Vektor mit `n` in [0,1] gleichverteilten Pseudozufallszahlen zurück:

```
> runif(10)
 [1] 0.55942101 0.76979148 0.76647418 0.92896448 0.98320026 0.27809910 0.69253050
0.26390423 0.59879014 0.05161719
```

Wenn wir im Folgenden von erzeugten Zufallszahlen (oder zufälligen Werten) sprechen, dann meinen wir stets *Pseudozufallszahlen*.

```
> y <- rnorm(1000)
> summary(y)
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
-3.98153 -0.61783  0.06902  0.01713  0.68989  3.73673
> x <- rnorm(1000000)
> summary(x)
     Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
-4.999760 -0.674315 -0.000336 -0.000143  0.674947  4.727007
```

Um zu prüfen, ob die erzeugten Zufallszahlen einer Normalverteilung genügen, können wir als ersten Test ein Histogramm erzeugen:

```
> hist(y)
```

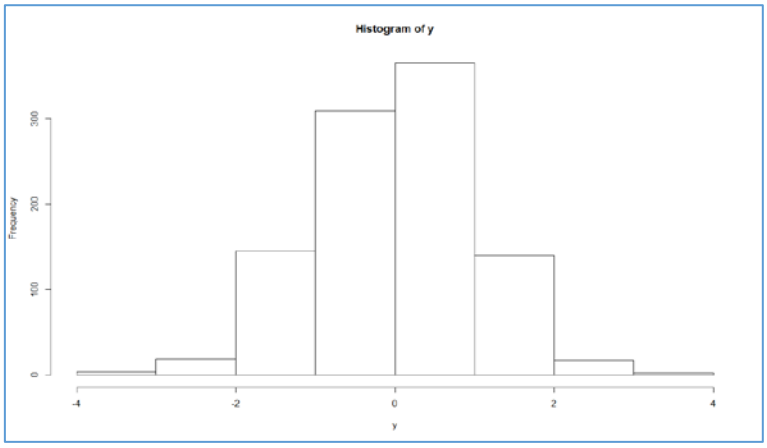


Abbildung 8

Die Anzahl der Klassen im Histogramm ist selbst für den kleinen Datensatz `y` eigentlich zu klein. Es ist oft nicht ganz so einfach, die „richtige“ Anzahl der Klassen in einem Histogramm zu bestimmen. Für den größeren Datensatz `x` wollen wir 100 unterschiedliche Klassen im Histogramm benutzen. Mit `breaks` kann die Anzahl der Klassen im Histogramm beeinflusst werden (siehe `?hist`).

```
> hist(x, breaks = 100)
```

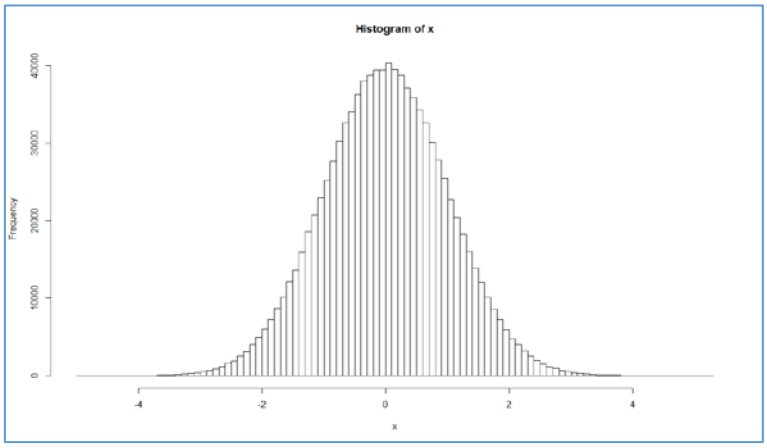


Abbildung 9

Wenn man dieses Bild mit der „hochskalierten“ theoretischen Wahrscheinlichkeitsdichte der Normalverteilung in der nächsten Abbildung vergleicht, dann sieht man, dass die so erzeugten Zufallszahlen diesen ersten groben Test bestanden haben. Für die Erzeugung der nächsten Abbildung benutzen wir dabei in der Plot-Funktion die Funktion `dnorm()`, die die Werte der Wahrscheinlichkeitsdichte (density) der Normalverteilung zurückgibt:

```
> z <- seq(-4, 4, by=0.1)
> plot(z, 100000*dnorm(z), type="l", col="blue")
```

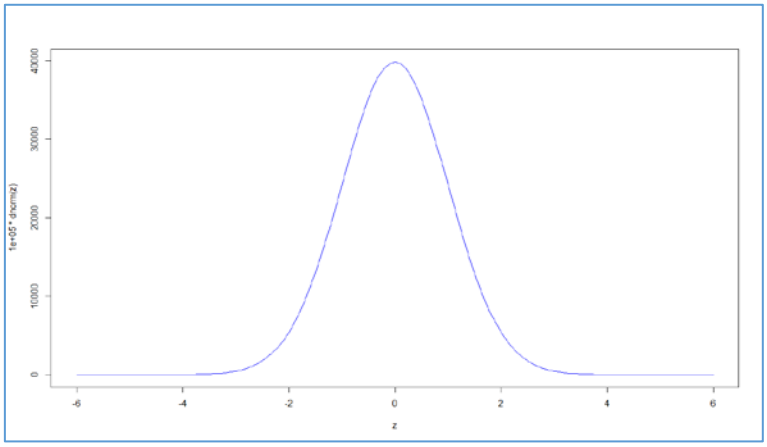


Abbildung 10: Wahrscheinlichkeitsdichte der Normalverteilung $\mathcal{N}(0,1)$ multipliziert mit 10^5

Das Gleiche wollen wir mit zufällig erzeugten gleichverteilten Werten wiederholen:

```
> y <- runif(1000)
> summary(y)
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
0.001823 0.241139 0.508957 0.497238 0.754191 0.998383
> x <- runif(1000000)
> summary(x)
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
0.000001 0.249538 0.499219 0.499566 0.749428 1.000000
```

Auch wollen wir wieder die Histogramme des kleinen und des großen Datensatzes erzeugen:

```
> hist(y)
```

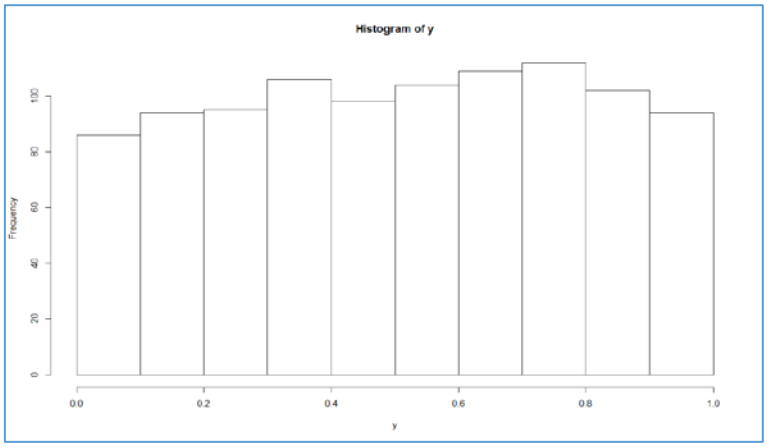


Abbildung 11

```
> hist(x, breaks = 100)
```

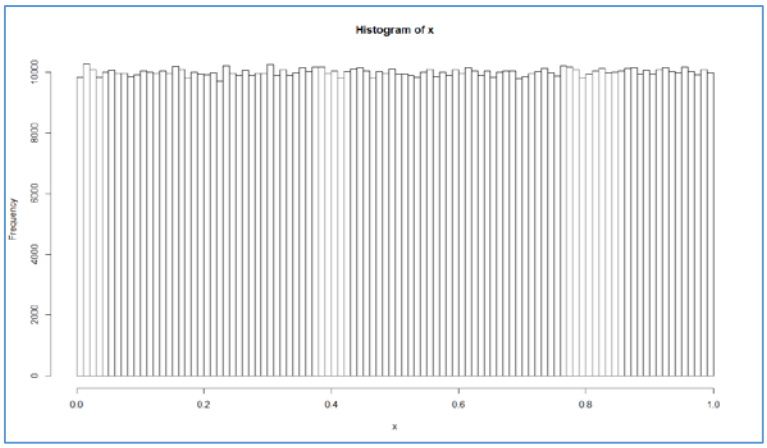


Abbildung 12

Auch hier sieht man, dass die erzeugten Werte im Intervall [0,1] ziemlich gleichverteilt sind. Mit R kann man relativ leicht, komplexere statistische Tests durchführen, um die Güte der erzeugten zufälligen Werte im Vergleich zu den vorgegebenen Verteilungen zu bestimmen. Dazu sind aber umfangreichere statistische Kenntnisse erforderlich, die den Rahmen an dieser Stelle sprengen würden.

Um „zufällige“ Zahlen zu erzeugen, benutzt R intern einen Zufallszahlengenerator (*RNG*, *random number generator*), der beim Start mithilfe der aktuellen Systemzeit immer unterschiedlich (und scheinbar zufällig) initialisiert wird. Um jedoch reproduzierbare „zufällige“ Werte zu erhalten, muss der Zufallszahlengenerator mit einer festen Zahl initialisiert werden; genau dies leistet `set.seed()`:

```
> rnorm(3)
[1] 0.4874291 0.7383247 0.5757814
> rnorm(3)
[1] -0.3053884  1.5117812  0.3898432
> set.seed(1)
> rnorm(3)
[1] -0.6264538  0.1836433 -0.8356286
> rnorm(3)
[1]  1.5952808  0.3295078 -0.8204684
> set.seed(1)
> rnorm(3)
[1] -0.6264538  0.1836433 -0.8356286
> rnorm(3)
[1]  1.5952808  0.3295078 -0.8204684
```

2.6 Skripte

Befehle direkt in die Konsole einzugeben kann manchmal sehr hilfreich sein, aber spätestens, wenn man Ergebnisse reproduzieren oder die Herleitung der Ergebnisse dokumentieren will, ist es sinnvoller, mit Skripten zu arbeiten. R-Skripte sind einfache Textdateien mit der Endung „.R“, die R-Befehle und Kommentare enthalten. Im Folgenden werden wir auch nicht mehr von Befehlen sprechen, sondern den mehr oder weniger äquivalenten Begriff „*Anweisung*“ (engl.: statement) benutzen, der im Skript-und Programmierumfeld gebräuchlicher ist.

Abgesehen von kleinen Berechnungen zu Testzwecken und dem Aufrufen von Manual-Seiten sollten Sie im Folgenden fast immer Skripte benutzen! Im Downloadbereich finden Sie Skripte, die alle R-Beispiele dieses Skriptes enthalten sollten.

Als integrierte Entwicklungsumgebung verfügt RStudio über einen Texteditor, mit dem Sie einfach Codebereiche an die Konsole schicken können. Wird ein Skript geladen, erscheint in RStudio ein zusätzlicher Bereich „Editor“, der den Texteditor enthält. RStudio wird beim Beenden automatisch alle offene Skripte speichern und beim erneuten Starten von RStudio werden die Skripte automatisch wieder geladen. Manuell können Sie über den Menüeintrag „File“ / „Open file …“ oder über „New File“ / „R Skript …“ ein Skript laden.

Als Nächstes öffnen Sie das Skript „02_jw_psd_x.R“, das Sie im Downloadbereich finden. Im Texteditor können Sie nun Anweisungen und Kommentare verändern und einfügen.

Mit der Tastenkombination `Strg-Enter` schicken Sie stets den kompletten Befehl, der durch die aktuelle Position des Cursors bestimmt ist, an die Konsole, wo er unmittelbar ausgeführt wird. In der Konsole erscheint sofort das Ergebnis. Mit `Strg-Shift-Enter` wird hingegen das gesamte Skript in der Konsole ausgeführt. Alternativ können Sie auch die entsprechenden Menü-Icons benutzen:

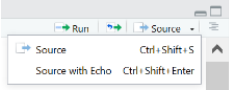


Abbildung 13: Befehlsausführung im RStudio-Texteditor

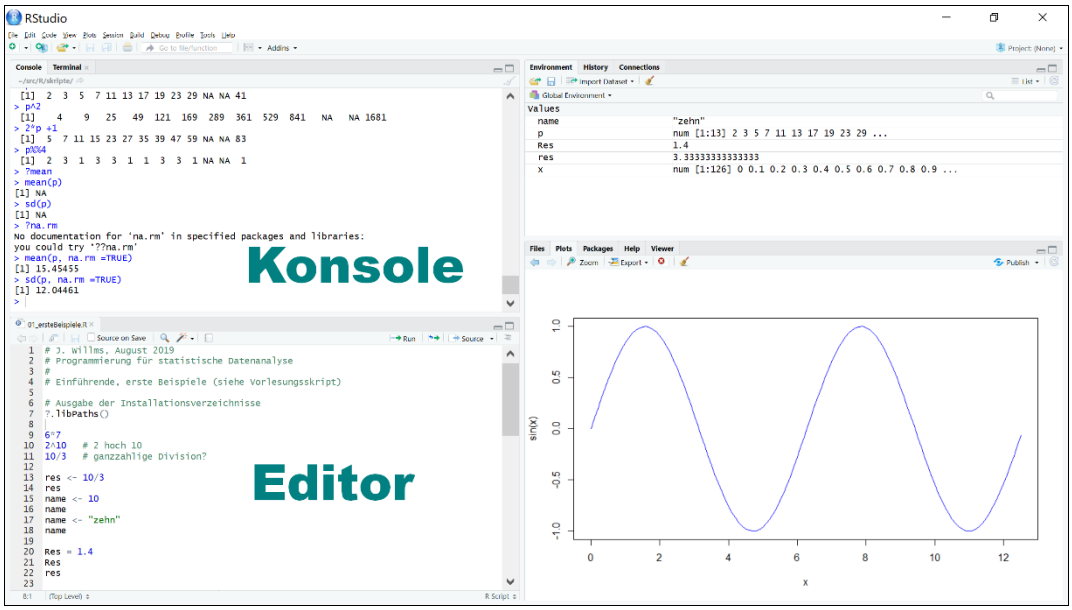


Abbildung 14: Zusätzlicher Editor-Bereich in R-Studio

WICHTIG

- Anweisung versus Befehl
- `Strg-Enter`: die aktuelle Anweisung zur Ausführung an die Konsole senden
- `Strg-Shift-Enter`: das gesamte Skript wird in der Konsole ausgeführt
- `summary()`
- `plot()`, `boxplot()` und `hist()`
- `rnorm()`, `runif()`, `dnorm()`
- `set.seed()`

2.7 Übungen

Die Übungen für dieses Kapitel finden Sie im Downloadbereich in der Datei „jwRSS19_ueb2_x.pdf“.

3 Erster Überblick: Kontrollstrukturen und Funktionen

Im Vergleich zu C++, C# und Java unterscheidet sich R stark von der Art und Weise wie komplexere Datenstrukturen aufgebaut sind. Dies werden wir erst im nächsten Kapitel genauer beleuchten. Große und erst einmal nicht so offensichtliche Unterschiede gibt es aber auch bei Variablen, während es im Bereich Kontrollstrukturen und Funktionen weniger Überraschungen gibt, wie wir in diesem Kapitel sehen werden.

3.1 Namen und Objekte: Ein erster Vergleich mit C und C++

Im Folgenden setzen wir Grundkenntnisse in C und C++ voraus und wollen erste grundlegende Unterschiede zwischen R und C bzw. C++ beleuchten. Wie wir sehen werden, gibt es viele Ähnlichkeiten. R wird ähnlich wie C durch Blöcke strukturiert, die durch geschweifte Klammern gekennzeichnet werden. Auch Kontrollstrukturen wie Verzweigungen mithilfe von `if-else` oder Schleifen (`while` oder `for`) haben eine ähnliche Syntax und Semantik.

Erste augenfällige Unterschiede:

- Im Gegensatz zu C wird R interpretiert! Welche Vor- und Nachteile ergeben sich dadurch?
- In R wird üblicherweise der Operator `<-` für eine Zuweisung benutzt (obwohl auch der in C benutzte Zuweisungsoperator `=` benutzt werden kann)
- In R werden Anweisungen nicht durch ein Semikolon abgeschlossen.

Variablen sind ein grundlegendes Konzept beider Sprachen; aber es gibt große und weitreichende Unterschiede. Wir wollen dies an einem Beispiel veranschaulichen. Um R wirklich zu verstehen, ist es hilfreich, diese Unterschiede zu kennen.

Hier und im Folgenden werden wir den Begriff „*Objekt*“ benutzen. Damit meinen wir nicht ein Objekt im objektorientierten Sinn (also eine Instanz einer Klasse), sondern wollen damit einen Speicherbereich in der Ausführungsumgebung bezeichnen, der neben eventuell zusätzlichen Metadaten Bitfolgen enthält, die als Werte eines bestimmten Datentyps interpretiert werden können.

In C:

```
int n = 5;
```

Es wird ein Objekt im Speicher angelegt, das groß genug ist, einen `int`-Wert zu repräsentieren, aber nicht größer. Auf einem üblichen PC ist das so erzeugte Objekte genau 4 Byte groß und der entsprechende Speicherbereich enthält die Dualdarstellung von 5 (also 29 Nullen gefolgt von 101). Der Variablenname `n` ist nun der Bezeichner für dieses Objekt (oder Speicherbereich); man kann auch sagen: das so erzeugte Objekt hat den Namen `n`.

Solange `n` gültig (und sichtbar) ist, ist `n` immer ein Bezeichner für einen `int`-Wert, dessen Binärdarstellung stets im selben Speicherbereich hinterlegt ist. Also bezieht sich `n` immer auf denselben Speicherbereich.

Der zugehörige Wert von `n` kann sich ändern, z. B. durch eine Zuweisung oder eine Anweisung wie `n++`. Nicht ändern kann sich aber der Datentyp (in unserem Fall `int`). Bereits beim Kompilieren wird `n` an den Datentyp `int` gebunden. Man spricht in diesem Zusammenhang von *frühen Binden (early binding)* im Gegensatz zum *späten Binden (late binding)*, die wir in R vorfinden.

In R:

```
n<-5
```

Auch hier wird ein Objekt in einem Speicherbereich erzeugt. Im Gegensatz zu C kann man aber in R nicht direkt auf diesen Speicherbereich zugreifen. Auch ist der benutzte Speicherbereich viel größer: auf dem hier benutzten System sind dies 56 Byte für eine 64-Bit-Version von R; eine 32-Bit-Version von R braucht ein paar Byte weniger (wieso?), vielleicht 32 Byte:

```
> n <- 5
> str(n)
 num 5
> object.size(n)
56 bytes
```

Die Funktion `object.size()` gibt die Speichergröße des übergebenen Objekts zurück; dies ist allerdings nur ein geschätzter Wert und kann ungenau sein, insbesondere wenn Teile der Daten mit anderen Objekten geteilt werden.

Bemerkung

- Die Punktnotation `object.size()` ist für C++-Programmierer in diesem Zusammenhang recht ungewohnt. `size()` ist hier keine Methode des Objekts (oder der Klasse bzw. Struktur) `object` , sondern bezeichnet eine Funktion und keine Methode. Viele C++-Programmierer würden die Funktion stattdessen `objectSize()` oder `object_size()` nennen.

Das nächste Quelltextbeispiel zeigt, dass sowohl durch `n <- c(5,7)` als auch durch `n <- 5` ein Vektorobjekt (vom Typ `double`) erzeugt wird:

```
> n <- c(5,7)
> str(n)
 num [1:2] 5 7
> typeof(n)
[1] "double"
> is.vector(n)
[1] TRUE
> length(n)
[1] 2
> n <- 5
> str(n)
 num 5
> typeof(n)
[1] "double"
> is.vector(n)
[1] TRUE
> length(n)
[1] 1
```

Die Funktion `length()` gibt die Länge, also die Anzahl der Elemente im Vektor, zurück, während die Funktion `typeof()` den Typ zurückgibt. Die Funktion `is.vector()` gibt den booleschen Wert `TRUE` zurück, falls es sich um einen „echten“ Vektor handelt (dazu später mehr, oder siehe `?is.vector`); in allen anderen Fällen wird `FALSE` zurückgegeben.[6]

R kennt also keine skalaren Werte. Durch die Zuweisung `n<-5` wird `n` nicht an den skalaren Wert 5 gebunden, sondern an einen Vektor der Länge 1! Wenn wir im Folgenden trotzdem von skalaren Werten sprechen, dann meinen wir implizit Vektoren der Länge 1.

Fragen

- Welche Funktion gibt es in R, deren Funktionsnamen mit „is.“ beginnen und was bewirken sie?
- Wird durch `n <- TRUE` auch ein Vektorobjekt erzeugt?

Im Gegensatz zu Programmiersprachen wie C, C++, C# oder Java haben Variablen in R keinen festen Datentyp. In R verweisen (oder zeigen) Variablen auf Objekte; man sagt auch, dass eine Variable an ein Objekt gebunden ist.

Während in C und C++ dieser Verweis (ähnlich wie eine Referenz) dauerhaft ist, kann sich in R diese Bindung durch eine Zuweisung ändern:

```
> x <- 5
> typeof(x)
[1] "double"
> str(x)
 num 5
> x[1]
[1] 5
> x[2]
[1] NA

> x<-1:1000
> typeof(x)
[1] "integer"
> str(x)
 int [1:1000] 1 2 3 4 5 6 7 8 9 10 ...

> x<-c("eins", "zwei", "drei")
> typeof(x)
[1] "character"
> str(x)
 chr [1:3] "eins" "zwei" "drei"
```

`1:1000` ist ein Vektor vom Typ `integer`, der die ganzen Zahlen von 1 bis 1000 umfasst; dies ist identisch zu `seq(1,1000,1)` (was wiederum identisch zu `seq(1,1000)` ist, siehe `?seq`).

[6] `TRUE` und `FALSE` können in R auch durch `T` und `F` abgekürzt werden

Durch `x<-c("eins", "zwei", "drei")` wird `x` an einen Vektor vom Typ `character` gebunden, der die Länge 3 hat.

Variablen werden in R erst während der Laufzeit (und nicht schon beim Kompilieren) an ein Objekt gebunden (*late binding*). Somit hat eine Variable keinen festen Datentyp, sondern eine Variable ist an ein Objekt gebunden, dem ein fester Datentyp zugeordnet ist. Wir sagen in diesem Zusammenhang auch, dass die Variable auf das Objekt *verweist*.

Im Gegensatz zu C enthält in R jedes Objekt zusätzlich sogenannte *Metadaten*, die u. a. den Typ des Objekts beschreiben (siehe z. B. [Wickham_AdvR], Abschnitt 18.1: Object size, Seite 379 oder [R_ints], Abschnitt 1.1).

Zwei oder mehrere Variablen können auf dasselbe Objekt verweisen (d.h. an dasselbe Objekt gebunden sein):

```
> a <- c(1, 5, 3, 2)
> b <- a
```

Durch die Zuweisung `b <- a` wird kein neues Objekt erzeugt (`a` wird also nicht in `b` kopiert)!

Werden allerdings `a` oder `b` verändert, dann wird umkopiert und die Variablen `a` und `b` zeigen dann auf unterschiedliche Objekte[7]

```
> a <- c(1, 5, 3, 2)
> b <- a
> b[2]<-0
> a
[1] 1 5 3 2
> b
[1] 1 0 3 2
```

Erst durch die Änderung `b[2]<-0`

- erzeugt R eine Kopie des Objekts, das an den Bezeichner `a` gebunden ist
- ändert sich die Kopie entsprechend
- wird der Name `b` an das neue Objekt gebunden.

Dieses Verhalten wird auch als *copy-on-modify* bezeichnet.

Ein Objekt kann mithilfe der Funktion `rm()` auch wieder gelöscht werden:

```
> rm(b)
> b
Error: object 'b' not found
```

Die Anweisung `rm(list=ls())` löscht ohne Warnung alle Objekte! (Mit `ls()` erhalten Sie ein Vektor aller erzeugten Objekte)

Im Gegensatz zu C und C++ gibt es in R einen *Garbage Collector*, kurz auch mit GC bezeichnet Er ist für die Speicherverwaltung von R zuständig. So fordert er zum Beispiel zusätzlichen Speicher vom Betriebssystem an, falls der interne vom GC verwaltete Memory-Pool nicht mehr ausreicht. Wenn Speicherbereiche nicht mehr benutzt werden, z. B. weil sie Objekte enthalten,

[7] Dies kann z. B. mithilfe der Funktion `tracemem()` verfolgt werden; siehe `?tracemem`

die an keinen Namen mehr gebunden sind, dann gibt sie der GC bei Bedarf wieder frei. Dies alles passiert automatisch im Hintergrund ohne dass der GC explizit aufgerufen werden muss (siehe auch `?gc`). Man kann den GC explizit mit `gc()` aufrufen (siehe `?gc`), aber dies ist eigentlich nie wirklich erforderlich; als Seiteneffekt erhält man allerdings Informationen über den Status des GCs: u. a. über die Größe des aktuell benutzten Speichers (siehe `used`) und wie viel Speicher benutzt werden kann, bevor ein neuer GC-Aufruf gestartet wird (siehe `gc trigger`).

```
> gc()
            used  (Mb) gc trigger   (Mb)  max used  (Mb)
Ncells    438523  23.5     938047   50.1    621888  33.3
Vcells 100729739 768.6  147642767 1126.5 100729820 768.6
```

WICHTIG

- Objekt
- R kennt keine skalaren Werte (sondern es werden Vektoren der Länge 1 für skalare Werte genutzt)
- In R verweisen Variablen auf Objekte; man sagt auch, dass eine Variable an ein Objekt gebunden ist
- Eine Variable hat keinen festen Datentyp, sondern ist an ein Objekt gebunden, dem ein fester Datentyp zugeordnet ist
- Jedes Objekt hat zusätzlich sogenannte *Metadaten*, die u. a. den Typ des Objekts beschreiben
- `object.size()`
- `length()`
- `typeof()`
- `is.vector()`
- R nutzt *copy-on-modify*
- Garbage Collection
- `1:1000` ist ein Vektor vom Typ `integer`, der die ganzen Zahlen von 1 bis 1000 umfasst
- *late binding*: Variablen werden erst während der Laufzeit an ein Objekt gebunden

3.2 Funktionen

Neben Objekten sind Funktionen ein zentraler Bestandteil von R. Mit Funktionen können Anweisungen zu einer Einheit zusammengefasst werden. Dadurch kann Quelltext strukturiert werden und die durch die Funktion beschriebene Funktionalität mehrfach in einem Programm verwandt werden. Funktion sind in R sogenannte „*first-class-objects*“; d.h. es sind in der Tat Objekte, die fast immer wie andere „Datenobjekte“ (Vektoren, Tabellen, Listen, …) benutzt werden können.

Im Folgenden gehen wir davon aus, dass Sie mit dem Konzept einer Funktion vertraut sind, so wie es zum Beispiel in C, C++ oder Java verwandt wird.

Ähnlich wie in anderen Programmiersprachen wird in R eine Funktion durch eine Anweisung der folgenden Form definiert:

```
function(arg_1, arg_2, ...) expression
```

Hierbei bezeichnet `arg_1, arg_2, ...` eine Liste der *formalen Funktionsparameter* und `expression` den *Funktionskörper*, der in der Regel aus einem durch geschweifte Klammern begrenzten Block von Anweisungen besteht. Insgesamt wird hierdurch ein Objekt erzeugt (eine sogenannte *anonyme Funktion*), das aber an keinem Namen gebunden ist. Ein Name wird aber gebraucht, wenn man die Funktion aufrufen will! Daher wird in der Regel eine Funktion in einer Zuweisung definiert, die dann folgende Form hat:

```
name <-function(arg_1, arg_2, ...) expression
```

Solange die Variable `name` an das Funktionsobjekt gebunden ist, kann die Funktion dann durch `name (arg_1, arg_2, ...)` aufgerufen werden.

Ein einfaches Beispiel mit drei formalen Funktionsparametern `x`, `y` und `z` soll dies illustrieren:

```
f1 <- function (x,y,z){
  tmp <- 2*y+3*z
  return(x+tmp)
}
```

Solange `f1` nicht durch eine Zuweisung an ein anderes Objekt gebunden wird, kann nun unsere Beispielfunktion aufgerufen werden (und `f1` kann als Funktionsname betrachtet werden, der allerdings eventuell nur temporär ist):

```
> f1(1,1,1)
[1] 6
> f1(1,2,3)
[1] 14
```

Wenn wir wollen, können wir auch einen weiteren Namen an unser Funktionsobjekt binden:

```
> neuerName <- f1
> neuerName(1,2,3)
[1] 14
```

Unsere einfache Beispielfunktion `f1` besitzt neben den formalen Parametern `x`, `y`, `z` eine *lokale Variable* `tmp`. Lokale Variablen können nur im entsprechenden Funktionskörper angesprochen werden:

```
> a <- 1
> f1(a,a,a)
[1] 7
> a
[1] 1
> tmp
Error: object 'tmp' not found
```

In unserem Beispiel ist die lokale Variable `tmp` eigentlich unnötig; man hätte auch kürzer schreiben können:

```
f2 <- function (x,y,z){
  return(x+2*y+3*z)
}
```

Wie in C beendet eine `return`-Anweisung den Funktionsaufruf. Im Gegensatz zu C ist eine `return`-Anweisung aber nicht notwendig: Fehlt die `return`-Anweisung, dann wird der letzte in der Funktion berechnete Wert zurückgegeben:

```
f3 <- function (x,y,z){
   x+2*y+3*z
}
```

Theoretisch hätte man hier sogar auch auf die geschweiften Klammern verzichten können. Insgesamt liefern also alle drei Funktionsaufrufe denselben Wert zurück:

```
> f1(4,2,1)
[1] 11
> f2(4,2,1)
[1] 11
> f3(4,2,1)
[1] 11
```

Wie wir gesehen haben, gibt es in R keine skalaren Werte; stattdessen werden Vektoren der Länge 1 benutzt. Was passiert, wenn wir unserer Beispielfunktion Vektoren übergeben, deren Länge größer als eins ist?

```
> a <- 1:10
> f1(a,a,a)
 [1]  6 12 18 24 30 36 42 48 54 60
```

Wie wir schon bei der Funktion `sqrt()` in Abschnitt 2.3 gesehen haben, wird nun die Funktion elementweise angewandt und als Ergebnis wird ein Vektor gleicher Länge zurückgegeben.

Bestehen die Funktionsargumente aus Vektoren unterschiedlicher Länge, dann tritt automatisch Recyling ein (siehe Abschnitt 2.3.1):

```
> a <- 1:10
> b <- 1:2
> f1(a,a,b)
 [1]  6 12 12 18 18 24 24 30 30 36
> c <- 0
> f1(a,b,c)
 [1]  3  6  5  8  7 10  9 12 11 14
```

Zur Kontrolle sollen für jeden der beiden Funktionsaufrufe noch einmal das jeweils letzte Ergebnis explizit berechnet werden:

```
> f1(10,10,2)
[1] 36
> f1(10,2,0)
[1] 14
```

Informationen über einzelne Funktionsobjekte erhält man, indem man die Funktionsnamen ohne Klammern und Argumente eingibt:

```
> f1
function (x,y,z){
  tmp <-  2*y+3*z
  return(x+tmp)
}
<bytecode: 0x0000000003b1a848>
> sqrt
function (x)  .Primitive("sqrt")
> atan2
function (y, x)
.Internal(atan2(y, x))
<bytecode: 0x0000000010cffa20>
<environment: namespace:base>
```

WICHTIG

- `name <-function(arg_1, arg_2, ...) expression`
- Formale Funktionsparameter
- Funktionskörper
- Lokale Variablen
- Fehlt die `return`-Anweisung, dann wird der letzte in der Funktion berechnete Wert zurückgegeben
- Elementweises Anwenden der Funktion bei Vektorargumenten (und eventuelles Recycling)
- Unterschied `sqrt` und `sqrt(2)`

3.2.1 Benannte und optionale Argumente

```
> f4 <- function (x,y=1,z=0){
+   x+2*y+3*z
+ }
> a <- 1:10
> b <- 1:2
> f4(a)
 [1]  3  4  5  6  7  8  9 10 11 12
> f4(a,b)
 [1]  3  6  5  8  7 10  9 12 11 14
> f4(a,a,b)
 [1]  6 12 12 18 18 24 24 30 30 36
> f4()
Error in f4() : argument "x" is missing, with no default
```

Ähnlich wie in C# kann ein Funktionsaufruf sogenannte *benannte Argumente* beinhalten. Benannte Argumente ermöglichen es, durch Nennung des Parameternamens ein Argument für einen bestimmten Parameter anzugeben. Dies hat den Vorteil, dass die Position des Parameters dann keine Rolle spielt.

```
> f4(z=b, x=a, y=a)
 [1]  6 12 12 18 18 24 24 30 30 36
> f4(a,z=a)   # äquivalent zu: f4(a,1,a)
 [1]  6 10 14 18 22 26 30 34 38 42
> f4(a,1,a)
 [1]  6 10 14 18 22 26 30 34 38 42
```

Benannte Argumente werden sehr häufig benutzt. Ein einfaches und hoffentlich selbsterklärendes Beispiel sind folgende Funktionen:

```
rnorm(n, mean = 0, sd = 1)
runif(n, min = 0, max = 1)
```

WICHTIG

- Benannte und optionale Argumente (Default-Werte)
- `rnorm(n, mean = 0, sd = 1)`
- `runif(n, min = 0, max = 1)`

3.2.2 Call-by-value und Copy-on-modify

Wie in C ist das Standardverhalten bei Funktionsaufrufen *Call-by-value;* also ändert ein Funktionsaufruf die ursprünglich übergebenen Argumente nicht:

```
> f <- function(x) {
    x[1] <- 42
    x <- 2*x
  }
> v <- 1:10
> w <- f(v)
> v
 [1]  1  2  3  4  5  6  7  8  9 10
> w
 [1] 84  4  6  8 10 12 14 16 18 20
```

Die Funktion `f` arbeitet mit einer Kopie des übergebenen Objekts `x`. Lediglich diese Kopie wird durch die Anweisung `x[1] <- 42` verändert; das ursprüngliche Objekt bleibt hingegen unverändert.

Allerdings wird zumindest in neueren R-Versionen Call-by-value in Verbindung mit der copy-on-modify Semantik benutzt: Nur wenn die aufgerufene Funktion ein übergebenes Argument verändern will, wird zuvor eine Kopie dieses Arguments erstellt und die Kopie entsprechend verändert. Im folgenden Beispiel sollte also beim Funktionsaufruf keine Kopie erstellt werden, dies ist natürlich wichtig bei großen Datenmengen:

```
> g <- function(x) {
    return(length(x))
  }
> v <- 1:1e9 # E-Notation: 1e9 entspricht einer Milliarde
> w <- g(v)
> w
[1] 1000000000
```

3.2.3 Keine Zeiger und kein Call-by-reference

In R sind Variablennamen an Objekte gebunden; Variablennamen „verweisen“ also auf Objekte, ähnlich wie Referenzen oder Zeiger. In R gibt es jedoch keine expliziten Zeiger oder Referenzen. Ist es in R möglich, eine Funktion zu schreiben, die die übergebenen Argumente direkt verändert? In C kann man dies mithilfe von Zeigern lösen; Referenzen sind in C++ eine zusätzliche Möglichkeit, dies mit wenig Aufwand und sehr effektiv zu lösen. In R ist dies hingegen, mit einfachen Mitteln nicht möglich. Um dies zu verdeutlichen, versuchen wir eine Funktion zu schreiben, die die ersten beiden Elemente eines Vektors vertauschen soll. Ein erster nicht erfolgreicher Versuch könnte so aussehen:

```
> vertausche1 <- function(x){
    tmp = x[1]
    x[1] = x[2]
    x[2] = tmp
  }
> z <- 1:4
> vertausche1(z)
> x
[1] 1 2 3 4
```

Da R bei Funktionsaufrufen Call-by-value benutzt, kann dies nicht funktionieren: es wird eine Kopie von `z` erstellt und die beiden ersten Elemente dieser Kopie vertauscht!

Was würde man stattdessen in R machen? Die Antwort ist einfach, aber leider beinhaltet sie unnötiges Kopieren; insbesondere wenn `x` extrem viele Daten enthält, sollte eigentlich unnötiges Kopieren vermieden werden:

```
> vertausche2 <- function(x){
    tmp = x[1]
    x[1] = x[2]
    x[2] = tmp
    return(x)
  }
> z <- vertausche2(z)
> z
[1] 2 1 3 4
```

In der Funktion `vertausche2()` wird eine Kopie von `z` erstellt, die beiden ersten Elemente vertauscht und die Variable `z` an die von der Funktion modifizierte Kopie gebunden.

3.2.4 Operator versus Funktionsaufruf

Abgesehen von der Syntax gibt es keinen Unterschied zwischen einem Operatorausdruck und einem entsprechenden Funktionsaufruf; das Operatorsymbol dient dann als Funktionsname.[8] So kann unser erstes Beispiel `6*7` auch folgendermaßen geschrieben werden:

```
> '*' (6,7)
[1] 42
```

Da * kein erlaubter Namen ist (`non-syntatic`, siehe Abschnitt 2.2), muss es allerdings in Anführungszeichen gesetzt werden.

[8] Siehe [R-lang], S. 11

WICHTIG

- Call-by-value in Verbindung mit Copy-on-modify
- Keine Zeiger und kein Call-by-reference
- E-Notation wie z.B. `1e9`

3.3 Kontrollstrukturen

Einzelne Anweisungen werden in R üblicherweise durch einen Zeilenumbruch getrennt; sie können aber auch durch ein Semikolon abgegrenzt werden. Wie in C auch, können Anweisungen zu einem *Block* (auch *Blockanweisung* genannt) mithilfe geschweifter Klammern gruppiert werden. Syntaktisch ist ein Block äquivalent zu einer einzelnen Anweisung.

Da die elementaren Kontrollstrukturen in R sehr ähnlich zu den Kontrollstrukturen in Sprachen wie C, C++, C#, Java, JavaScript, … sind, werden wir sie an dieser Stelle nicht detailliert behandeln (siehe auch `?Control` oder `?while`). Einen großen Unterschied gibt es allerdings: in R werden Kontrollstrukturen intern durch Funktionen realisiert!

Schleifen haben in R den Ruf, recht langsam zu sein und viele Autoren empfehlen deshalb, sie wenn möglich zu vermeiden. Wie wir gesehen haben (siehe Abschnitt 2.3), können viele Funktionen und Operatoren direkt auf Vektoren angewandt werden, so dass in diesem Fall keine expliziten Schleifen notwendig sind:

```
> x <- 1:12
> sin(x^3-x^2/(2*pi))
 [1]  0.7452069  0.8820497  0.4212873 -0.9815359  0.9975643  0.2146034  0.8128370
 [8] -0.7451375 -0.1737885 -0.6932617 -0.9919059  0.7195282
```

Des Weiteren gibt es eine Familie von Funktionen, bestehend aus `apply()`, `lapply()`, `sapply()` … , die oft im Vergleich zu expliziten Schleifen viele Vorteile bieten; darauf werden wir später eingehen.

3.3.1 Verzweigung: `if-else`

Wie in C lautet die formale Syntax:

```
if ( statement1 )
statement2
else
statement3
```

Wie in C ist der `else`-Teil optional. Falls `statement2` oder `statement3` aus mehreren Anweisungen besteht, müssen geschweifte Klammern benutzt werden! Auch ist ein Zeilenumbruch zwischen ‚`}`‘ und `else` nicht erlaubt.

`if-else` gibt den zuletzt berechneten Wert zurück. Dies wird oft in Zuweisungen ausgenutzt. Statt:

```
if (x%%2 == 0){
  z <- x
} else {
  z <-x+1
}
```

kann daher auch kürzer geschrieben werden:

```
z <- if (x%%2 == 0) x else x+1
```

Wie in C bezeichnet == den Vergleichsoperator! Der Operator = entspricht hingegen einer Zuweisung (siehe Abschnitt 2.2). Und zur Erinnerung: %% bezeichnet den Modulo-Operator (siehe Abschnitt 2.3)

Genau wie in C sind auch „else-if“-Ketten erlaubt:

```
if ( statement1 ) {
statement2
} else if ( statement3 ) {
statement4
} else if ( statement5 ) {
statement6
} else
statement8
```

3.3.2 Verzweigung: `ifelse`

Die Auswertung der `if`-Bedingung in der Kontrollstruktur `if-else` berücksichtigt lediglich das erste Element eines Vektors. `if-else` ist daher nur bedingt für Vektoren geeignet. Insbesondere sind Funktionen, die im Funktionsrumpf ein `if-else` benutzten, nicht vektorisierbar!

```
> x <- 1:10
> if (x%%2 == 0){
+   z <- x
+ } else {
+   z <-x+1
+ }
Warning message:
In if (x%%2 == 0) { :
  the condition has length > 1 and only the first element will be u
sed
> z
 [1]  2  3  4  5  6  7  8  9 10 11
```

Ebenso:

```
> x <- 1:10
> z <- if (x%%2 == 0) x else x+1
Warning message:
In if (x%%2 == 0) x else x + 1 :
  the condition has length > 1 and only the first element will be u
sed
> z
 [1]  2  3  4  5  6  7  8  9 10 11
```

Für diesen Zweck gibt es die Funktion `ifelse()`:

```
ifelse(test, wahr, falsch)
```

Zurückgegeben wird ein Objekt der gleichen Größe und Form wie `test`, der abhängig vom Wahrheitswert von `test` elementweise gefüllt wird von den entsprechenden Elementen aus `wahr` bzw. `falsch`.

```
> x <- 1:10
> z <- ifelse(x%%2 == 0, x, x+1)
> z
 [1]  2  2  4  4  6  6  8  8 10 10
```

Entsprechend ergibt sich mithilfe von Recycling (siehe Abschnitt 2.3.1):

```
> x <- 2:13
> z <- ifelse(x%%2 == 0, 5, 9)
> z
 [1] 5 9 5 9 5 9 5 9 5 9 5 9
```

3.3.3 Schleife: `while` und `repeat`

Auch das Grundgerüst einer `while`-Schleife ist wie in C:

```
while ( statement1 ) statement2
```

Ebenso wie in C wird `break` verwendet, um eine Schleife vorzeitig zu beenden. Statt `continue` wie in C wird in R allerdings `next` verwendet, um innerhalb der Schleife sofort (und ohne die folgenden Anweisungen im Schleifenrumpf auszuführen) zur nächsten Schleifenbewertung zu springen.

Weiterhin entspricht `repeat` im Wesentlichen `while(TRUE)`.

3.3.4 Schleife: `for`

Die Syntax für die `for`-Schleife (ähnlich wie in Python) lautet:

```
for ( name in vector )
statement1
```

Hierzu ein kleines Beispiel:

```
> x <- 11:15
> for(i in x ) print(i^2)
[1] 121
[1] 144
[1] 169
[1] 196
[1] 225
```

`break` und `next` werden ähnlich wie in einer `while`-Schleife (siehe oben) behandelt.

3.3.5 Mehrfachverzweigung: `switch`

In R gibt es auch eine `switch`-Anweisung für eine Mehrfachverzweigung, deren Syntax sich aber nicht an C anlehnt. Die Syntax ist unterschiedlich zu C und zeigt aber auch deutlich, dass es sich intern um einen Funktionsaufruf handelt:

```
switch (statement, list)
```

Zuerst wird `statement` ausgewertet und der erhaltene Wert als Index der Liste `list` interpretiert. Ist er kein gültiger Indexwert, dann wird NULL zurückgegeben.

```
> switch(1, "eins", "zwei", "drei", "vier")
[1] "eins"
> switch(3, "eins", "zwei", "drei", "vier")
[1] "drei"
```

`NULL` ist ein spezielles Objekt in R, das nur einmal in R existiert. Es wird in der Regel benutzt, um anzuzeigen, dass es kein entsprechendes Objekt gibt (im Gegensatz zu `NA`, das anzeigt, dass kein Wert verfügbar ist).

WICHTIG

- Block
- `if-else`
- `ifelse`
- `while, repeat`
- `break, next`
- `for`
- `switch`
- `NULL`

3.4 Übungen

Die Übungen für dieses Kapitel finden Sie im Downloadbereich in der Datei „jwRSS19_ueb3_x.pdf“.

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

TEIL 2

Inhaltsverzeichnis

4 HOMOGENE DATENSTRUKTUREN 3

4.1 Wichtige Eigenschaften von Objekten 3
4.1.1 typeof() 3
4.1.2 length() 4
4.1.3 Attribute 4

4.2 (Atomic) Vector 5
4.2.1 Filter und Vektor-Indizierung 7
4.2.2 Elemente einfügen 8
4.2.3 Test auf Gleichheit 8
4.2.4 Elementnamen 9

4.3 Matrix und Array 11
4.3.1 Matrizen erzeugen 11
4.3.2 Indizierung 14
4.3.3 Zeilen- und Spaltennamen 16
4.3.4 Einfügen und Löschen von Zeilen und Spalten 17
4.3.5 Matrixoperationen 19
4.3.6 Lineare Gleichungssysteme 21
4.3.7 Eigenwerte und Eigenvektoren 22
4.3.8 apply() 25
4.3.9 Array 27

4.4 Übungen 31

4 Homogene Datenstrukturen

Bevor wir auf die wesentlichen Strukturen für Daten eingehen werden, wollen wir kurz grundsätzlich auf Objekte eingehen.

4.1 Wichtige Eigenschaften von Objekten

Wie bereits gesehen gilt laut John M. Chambers: *„alles, was [in R] existiert, ist ein Objekt“*. Dies trifft sowohl für Datencontainer zu wie z. B. Vektoren als auch für Funktionen oder Namen.

4.1.1 typeof()

Jedes Objekt hat in R einen zugrundeliegenden Datentyp (kurz Typ), den man mithilfe der Funktion
`typeof()` abrufen kann:

```
> a <- 42
> typeof(a)
[1] "double"

> x <- 1:10
> typeof(x)
[1] "integer"

> z <- c(1,2,3)
> typeof(z)
[1] "double"

> f3 <- function (x,y,z){
+   x+2*y+3*z
+ }
> typeof(f3)
[1] "closure"

> typeof(sqrt)
[1] "builtin"
```

Aus der Ausgabe entnehmen wir, dass `x` ein `integer`-Vektor ist und `a` und `z` jeweils `double`-Vektoren sind. Wir haben gesehen, dass R keine skalaren Datentypen hat: ein skalarer Wert wird stets dargestellt durch einen Vektor der Länge eins. Es gibt auch keinen Datentyp Vektor, sondern es gibt verschiedene Vektor-Datentypen wie `integer`-Vektor oder `double`-Vektor.

Weiterhin zeigt die Ausgabe, dass die Funktion `f3()` den Datentyp `closure` besitzt, während die Wurzelfunktion `sqrt()` vom Datentyp `builtin` ist. Zum Teil werden wir auf die einzelnen Datentypen im Verlauf dieses Skriptes näher eingehen. Neben dem Typ spielen Attribute bei Objekten eine wichtige Rolle.

Fragen

- Welchen Datentyp hat `NULL` ?
- Welchen Datentyp hat `NA` ?
- Welchen Datentyp hat `Nile` ?

4.1.2 length()

Neben dem Typ hat jedes Objekt in R eine Länge, die die Funktion `length()` zurückgibt:

```
> x <- 1:10
> length(x)
[1] 10
 2
```

Für manche Objekttypen wie z. B. Funktionsobjekte beinhaltet die Längeninformation allerdings keine zusätzlichen Informationen. Die Eigenschafen Typ und Länge werden oft auch „intrinsische Attribute“ genannt.[1]

4.1.3 Attribute

Grundsätzlich kann jedem Objekt eine Menge von Attributen zugeordnet sein und zwar in Form einer „`key-value`“ Beziehung. `key` ist dabei stets eine Zeichenkette, wohingegen `value` ein beliebiger Datentyp sein kann.

Mit `attributes()` erhält man eine Liste aller zugeordneten Attribute.

```
> x <- 1:10
> attributes(x)
NULL
```

Die Ausgabe zeigt, dass der Vektor `x` keine zugeordneten Attribute besitzt. Wir werden bald sehen, dass komplexere Datenstrukturen wie Matrix oder Data Frame durchaus unterschiedliche Attribute besitzen; dies sieht man auch an dem Datensatz `Nile`:

```
> attributes(Nile)
$tsp
[1] 1871 1970    1

$class
[1] "ts"
```

Das Objekt Nile hat also zwei Attribute: „tsp“, das Anfangs- und Endzeit und Frequenz beschreibt, und „class“, das den Wert „ts“ hat (und für *time series* steht).

Wie wir später ebenfalls sehen werden, können mithilfe der Funktion `attr()` Attribute gesetzt und verändert werden. Einige Attribute nehmen dabei jedoch eine Sonderrolle[2] ein, nämlich „class“, „comment“, „dim“, „dimnames“, „names“, „row.names“ und „tsp“. Für diese Attribute

[1] Siehe: R Core Team, *R Internals*, Version 3.6.0 (2019-04-26), S. 13
[2] Siehe: R Core Team, *R Internals*, Version 3.6.0 (2019-04-26), S. 7

gibt es auch bestimmte Einschränkungen und in der Regel werden spezielle Funktionen bereitgestellt (wie z. B. `names()` oder `class()`), um das entsprechende Attribut zu verändern oder zu setzen.

Das Attribut „class“ spielt eine zentrale Rolle bei der objektorientierten Programmierung; es ist ein character-Vektor, der die Namen der Oberklassen enthält. Genaugenommen unterstützt R unterschiedliche objektorientierte Systeme, das gebräuchlichste ist S3.

Auch wenn das class-Attribut nicht explizit gesetzt ist, dann gibt der Funktionsaufruf `class()` einen Wert zurück wie „matrix“, „array“, „function“ or „numeric“ oder das Resultat von `typeof()`.

```
> class(1:10)
[1] "integer"
> class(c(1,2,3))
[1] "numeric"
> class(sqrt)
[1] "function"
> class(NULL)
[1] "NULL"
> class(Nile)
[1] "ts"
```

WICHTIG

- `typeof()`
- `length()`
- `attributes()`
- `class()`

4.2 (Atomic) Vector

Vektoren haben wir bereits kennengelernt. Streng genommen gibt es zwei Arten von Vektoren: *atomare Vektoren* und *Listen*. Im Folgenden meinen wir jedoch immer atomare Vektoren, wenn wir von Vektoren reden!

Ein Vektor ist wie ein Feld (array) eine Datenstruktur, die eine Folge von Elementen desselben Typs enthält, die in aufeinander folgenden Speicherbereichen abgelegt sind. Wie in C haben wir mithilfe des Indexoperators `[]` wahlfreien Zugriff auf die einzelnen Elemente des Vektors. Aber im Gegensatz zu C und vielen anderen Programmiersprachen beginnt die Indexierung in R nicht bei 0 sondern bei 1!

Es gibt sechs unterschiedliche Vektor-Datentypen: `logical`, `integer`, `double`, `complex`, `character` und `raw`, die den jeweils zugrundeliegenden Datentyp (d. h. den Typ eines Vektorelements) charakterisieren.

Auf `raw` werden wir im Folgenden nicht weiter eingehen. Im nächsten Quelltextbeispiel sehen wir Vektoren der Länge 2, die jeweils einen unterschiedlicher Vektor-Datentypen haben:

```
> x1 <- c(T, F)           # logical   (entspricht booleschen Wert)
> x2 <- 4:5               # integer
> x3 <- c(4, 5)           # double
> x4 <- c(4+1i, 5+2i)     # complex
> x5 <- c("vier", "5")    # character  (entspricht einer Zeichenket
te)
```

Dabei wurden die Abkürzungen `T` und `F` für `TRUE` und `FALSE` benutzt. `character` bezieht sich nicht auf ein einzelnes Zeichnen, sondern stets auf eine Zeichenkette (die leer sein kann oder aus ein oder mehreren Zeichen bestehen kann). Zeichenketten können in R mit einfachen oder doppelten Anführungszeichen geschrieben werden. Statt `c("vier", "5")` hätten wir also auch `c('vier','5')` schreiben können. Wir werden in der Regel die doppelten Anführungszeichen benutzten, es sei denn, wir wollen eine Zeichenkette verwenden, die selbst doppelte Anführungszeichen enthält wie `'Eine Zeichenkette die "doppelte" Anführungszeichen enthält'`.

Aus zum Teil historischen Gründen sind die Bezeichnungen nicht immer konsistent. Aus Kompatibilitätsgründen mit S gibt es neben `typeof()` auch die ähnlichen Funktionen `mode()` und `storage.mode()`, die aber zum Teil unterschiedliche Bezeichnungen zurückgeben, wie aus der folgenden Tabelle hervorgeht:

typeof	mode	storage.mode
logical	logical	logical
integer	numeric	integer
double	numeric	double
complex	complex	complex
character	character	character
raw	raw	raw

Tabelle 1-1

`numeric` kann also sowohl für `integer` als auch `double` für stehen. Die Inkonsistenz der Bezeichnung wird auch bei `str()` deutlich:

```
> str(x2)
 int [1:2] 4 5
> str(x3)
 num [1:2] 4 5
```

Ein Vektor ist *homogen*, d. h. alle Elemente haben den gleichen zugrundeliegenden Datentyp. Was passiert aber, wenn wir versuchen, einen `integer`-Wert wie 3L und einen `double`-Wert wie 5.1 zu einem neuen Vektor zusammenzufassen?

```
> typeof(x)
[1] "double"
> typeof(x[1])
[1] "double"
```

Offensichtlich wird der „kleinere“ Datentyp in den „größeren“ Datentyp umgewandelt (*coercion*) und wir erhalten einen `double`-Vektor. Ebenso:

```
> z <- c(3.1, 1L,"drei", 2+4i)
> typeof(z)
[1] "character"
> str(z)
 chr [1:4] "3.1" "1" "drei" "2+4i"
```

4.2.1 Filter und Vektor-Indizierung

Wir haben bereits gesehen, dass durch `x[n]` das `n`-te Element im Vektor `x` angesprochen wird, falls `n` eine natürliche Zahl ist. Dies ist in R verallgemeinert worden: `n` kann auch ein Vektor sein, wie man aus der folgenden Tabelle entnehmen kann:

`n`	Ausgangswerte:	`4`
`u`	Ausgangswerte:	`6 1 3`
`x`	Ausgangswerte:	`3 6 9 12 15 18`
`x[n]`	das `n`-te Element	`12`
`x[-n]`	alle, bis auf `n`-te Element	`3 6 9 15 18`
`x[1:n]`	die ersten `n` Elemente	`3 6 9 12`
`x[-(1:n)]`	alle bis auf die ersten `n` Elemente	`15 18`
`x[u]`	die Elemente `x[u[1]]`, `x[u[2]]`, `x[u[3]]`	`18 3 9`
`x[x > 7]`	alle Elemente größer als 7	`9 12 15 18`
`x[x >= 3 & x <= 7]`	die Elemente zwischen 3 und 7	`3 6`
`x[x %in% u]`	alle Elemente, die in `u` enthalten sind	`3 6`

Tabelle 1-2: Beispiele Vektor-Indizierung

Beispielhaft wollen wir die drittletzte Zeile kurz erläutern. Was bedeutet `x > 7`? Durch Recycling wird aus dem skalaren Wert 7 der Vektor (7, 7, 7, 7, 7, 7), der nun mit `x` elementweise verglichen wird. Das Ergebnis dieses Vergleichs ist der Vektor (`FALSE, FALSE, TRUE, TRUE, TRUE, TRUE`). Entsprechend besteht `x[x > 7]` daher aus dem 3., 4., 5. und 6. Element von `x`.

Ganz ähnlich ergibt sich das Beispiel in der letzten Zeile; hierzu sei angemerkt, dass `x%in%u` den Vektor `(T,T,F,F,F,F)`[3] zurückgibt.

Auf eine weitere Möglichkeit der Vektor-Indizierung mithilfe des Namens (also der Form `x["einNamen"]`) werden wir in 4.2.4 eingehen.

Die Funktion `subset()` bietet eine ähnliche Filterfunktion wie `x[x > 7]`, unterscheidet sich jedoch dadurch, dass sie automatisch NA-Werte ausschließt. In unserem Fall liefert sie ein identisches Ergebnis:

```
> subset(x, x>7)
[1]  9 12 15 18
```

Die Funktion `which()` kann ebenfalls für Filteraufgaben benutzt werden. Im Gegensatz zu `x[x > 7]` gibt `which(x > 7)` nicht die entsprechenden Elemente von `x` sondern die korrespondierenden Indexwerte zurück:

[3] wie in R üblich steht `T` für `TRUE` und `F` für `FALSE`

```
> which(x > 7)
[1] 3 4 5 6
```

Ein Sonderfall sollte vielleicht noch an dieser Stelle erwähnt werden: `x[0]` liefert einen leeren Vektor zurück.

Nützlich sind auch die Funktionen `any()` und `all()`. Sie geben zurück, ob mindestens ein Element oder alle Elemente die übergebene Bedingung erfüllen (eine detaillierte Erklärung benutzt wieder Recycling):

```
> any(x%%4==0)
[1] TRUE
> all(x > 4)
[1] FALSE
```

4.2.2 Elemente einfügen

Mithilfe von Wiederzuweisung und Verkettung kann man Elemente in Vektoren einfügen:

```
> x <- 3L*(1:8)
> x
[1]  3  6  9 12 15 18 21 24
> x <- c(x,5L) # 5 am Ende hinzufügen
> x
[1]  3  6  9 12 15 18 21 24  5
> x <- c(x[1:3],20L,x[4:length(x)]) # 20 als 4. Element einfügen
> x
 [1]  3  6  9 20 12 15 18 21 24  5
```

4.2.3 Test auf Gleichheit

Die Funktion `identical()` testet, ob zwei Objekte exakt gleich sind. Betrachten Sie hierzu folgendes Beispiel:

```
> u <- 1:3
> v <- c(1,2,3)
> all(u==v)
[1] TRUE
> identical(u,v)
[1] FALSE
```

Die Vektoren u und v haben zwar gleiche Werte aber unterschiedliche Datentypen; daher liefert `identical(u,v)` den Wert `FALSE` zurück, im Gegensatz zu `u==v`, das den Vektor (`TRUE, TRUE, TRUE`) zurückgibt.

4.2.4 Elementnamen

Den Elementen eines Vektors können optional Namen zugeordnet werden:

```
> x <- c(a=1, b=2, 3, delta=4)
> x
    a     b       delta
    1     2     3     4
> x[1]
a
1
> typeof(x[1])
[1] "double"
> is.vector(x)
[1] TRUE
```

Das so erzeugte Objekt hat nun ein Attribut „names“, das eine Liste von Zeichenketten enthält:

```
> attributes(x)
$names
[1] "a"     "b"     ""      "delta"
```

Mithilfe der Funktion `names()` können die Namen der Elemente sowohl abgerufen als auch gesetzt werden:

```
> names(x)
[1] "a"     "b"     ""      "delta"
> names(x) <- c("alpha", "beta","gamma", "delta")
> names(x)
[1] "alpha" "beta"  "gamma" "delta"
> str(x)
 Named num [1:4] 1 2 3 4
 - attr(*, "names")= chr [1:4] "alpha" "beta" "gamma" "delta"
```

Nun können wir auch über den Namen auf einzelne Elemente zugreifen:

```
> x["beta"]
beta
   2
```

Dies wollen wir als Nächstes in einem kleinen Beispiel illustrieren. `letters` und `LETTERS` sind (ebenso wie `pi`) in R eingebaute Konstanten. Sie sind Vektoren, die jeweils die 26 Buchstaben des Alphabets (ohne Umlaute) in Klein- bzw. Großschreibung beinhalten:

```
> letters
 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p
" "q" "r" "s" "t"
[21] "u" "v" "w" "x" "y" "z"
> LETTERS
 [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P
" "Q" "R" "S" "T"
[21] "U" "V" "W" "X" "Y" "Z"
```

Somit können wir nun schreiben:

```
> x <- 1:20
> names(x) <- LETTERS[1:length(x)]
> x
 A  B  C  D  E  F  G  H  I  J  K  L  M  N  O  P  Q  R  S  T
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
> str(x)
 Named int [1:20] 1 2 3 4 5 6 7 8 9 10 ...
 - attr(*, "names")= chr [1:20] "A" "B" "C" "D" ...
> x["H"]
H
8
```

WICHTIG

- `logical`, `integer`, `double`, `complex`, `character`
- `numeric`
- Ein Vektor ist homogen
- *Coercion*
- Vektor-Indizierung
- `x > 7`
- `x%in%u`
- leerer Vektor: `x[0]`
- `which()`
- `subset()`
- `any()` und `all()`
- `identical()` vs. `==`
- `names()`
- `letters` und `LETTERS`

4.3 Matrix und Array

Matrizen sind aus der Mathematik bekannt; in Sprachen wie C entsprechen sie zweidimensionalen Feldern. In R sind Matrizen einfach Vektoren, die mit einem zusätzlichen Attribut „dim“ (und einem optionalen Attribut „dimnames“) versehen sind; das Attribut „dim“ ist ein numerischer Vektor der Länge zwei, der die Matrixdimensionen festlegt: die Anzahl der Zeilen und die Anzahl der Spalten. Wie wir sehen werden gilt:

Matrix = Vektor + Attribut: dim (Vektor der Länge 2) + Attribut class = "matrix“ + Optionales Attribut: „dimnames“

4.3.1 Matrizen erzeugen

In R können Matrizen u. a. erzeugt werden durch:

```
matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE, dimnames = NULL)
```

`data`	zugrundeliegender Datenvektor
`nrow`	Anzahl der Zeilen
`ncol`	Anzahl der Spalten
`byrow`	Falls TRUE, wird der Datenvektor zeilenweise interpretiert, sonst spaltenweise
`dimnames`	Setzt das Attribut “dimnames“, mit dem Namen für die Matrixdimensionen gesetzt werden können: siehe `?matrix`

Tabelle 1-3: Argumente der Funktion matrix()

Da in `matrix()` alle Argumente optional sind, ergibt sich im einfachsten Fall:

```
> matrix()
     [,1]
[1,]   NA
```

Im nächsten Beispiel sehen wir, dass die Daten spaltenweise eingelesen werden:

```
> A <- matrix(1:6, nrow=2)
> A
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> str(A)
 int [1:2, 1:3] 1 2 3 4 5 6
```

Durch `byrow=TRUE` kann dies geändert werden:

```
> B <- matrix(1:6, nrow=2, byrow = TRUE)
> B
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
> str(B)
 int [1:2, 1:3] 1 4 2 5 3 6
```

Intern wird jedoch eine Matrix stets spaltenweise im Speicher abgelegt. Also sind für eine 2×3-Matrix wie A (d.h. zwei Zeilen und drei Spalten) die Elemente: `A[1,1], A[2,1], A[1,2], A[2,2], A[1,3], A[2,3]` in aufeinanderfolgenden Speicherbereichen abgelegt.

Mit `cbind()` (column bind) erhält man das gleiche Ergebnis:

```
> C <- cbind(1:2,3:4,5:6)
> C
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Alternativ kann auch `rbind()` (row bind)

```
> D <- rbind(c(1,3,5), c(2,4,6))
> D
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Einen kleinen Unterschied gibt es allerdings: da wir nicht `rbind(c(1L,3L,5L), c(2L,4L,6L))` geschrieben haben, ist der zugrundeliegende Datentyp nun `double` und nicht wie vorher `integer`.

Auf die einzelnen Elemente kann wie gewohnt zugegriffen werden:

```
> D[2,3] <- 24
> D
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4   24
```

Frage

- Durch `A <- matrix(1:6, nrow=2); A[1,1] = 3` wird auf jeden Fall die Matrix A umkopiert, wieso?

Die Matrix A ist eine 2×3-Matrix, wie man leicht mithilfe der Funktion `dim()` oder den Funktionen `nrow()` bzw `ncol()` herausfindet:

```
> A <- matrix(1:6, nrow=2)
> dim(A)
[1] 2 3
> nrow(A)
[1] 2
> ncol(A)
[1] 3
```

Die nächste Ausgabe zeigt auch, dass die Funktionen `nrow()` und `ncol()` einfache „Helferfunktionen“ sind, die jeweils das erste bzw. zweite Element von `dim()` zurückgeben.

```
> nrow
function (x)
dim(x)[1L]
<bytecode: 0x000000000f859ea8>
<environment: namespace:base>
> ncol
function (x)
dim(x)[2L]
<bytecode: 0x000000000f113b98>
<environment: namespace:base>
```

Wir wollen die Matrix A genauer untersuchen:

```
> str(A)
 int [1:2, 1:3] 1 2 3 4 5 6
> typeof(A)
[1] "integer"
```

Welche Attribute hat A und zu welcher Klasse und Typ gehört es?

```
> attributes(A)
$dim
[1] 2 3
> class(A)
[1] "matrix"
> typeof(A)
[1] "integer"
```

Die Funktion `attributes()` zeigt lediglich ein Attribut an, nämlich „dim“: ein Vektor der Länge 2, der der Dimension der Matrix (und der Rückgabe der Funktion `dim()`) entspricht). Wie bereits erwähnt, nimmt das „class“-Attribut eine Sonderrolle ein. Erst mithilfe der Funktion `class()` wird das „class“-Attribut sichtbar.

Um zu zeigen, dass folgende Gleichung gilt:

Matrix = Vektor + Attribut: dim (Vektor der Länge 2) + Attribut class = "matrix“ + Optionales Attribut: „dimnames“

wollen wir einen Vektor in eine Matrix umwandeln:

```
> E <- 1:6
> attr(E,"dim") <- c(2,3) # oder: dim(E) <- c(2,3)
> str(E)
 int [1:2, 1:3] 1 2 3 4 5 6
> attributes(E)
$dim
[1] 2 3
> class(E)
[1] "matrix"
> typeof(E)
[1] "integer"
```

Statt `attr(E,"dim") <- c(2,3)` hätten wir auch kürzer `dim(E) <- c(2,3)` schreiben können. Wie wir sehen, erhalten wir eine Matrix und das „class“-Attribut wurde automatisch gesetzt.

4.3.2 Indizierung

In diesem Abschnitt wollen wir die 5×6-Matrix `F` zugrunde legen:

```
> F <- matrix(1:30, nrow=5)
> F
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    6   11   16   21   26
[2,]    2    7   12   17   22   27
[3,]    3    8   13   18   23   28
[4,]    4    9   14   19   24   29
[5,]    5   10   15   20   25   30
```

Das folgende Beispiel zeigt, wie auf Spalten und Zeilen einer Matrix zugegriffen werden kann:

```
> c2 <- F[,2] # 2. Spalte
> c2
[1]  6  7  8  9 10
> z3 <- F[3,] # 3. Zeile
> z3
[1]  3  8 13 18 23 28
```

Hierbei sollte beachtet werden, dass `c2` und `z3` keine Objekte der Klasse Matrix sind, sondern einfache Vektoren sind:

```
> class(c2)
[1] "integer"
> class(z3)
[1] "integer"
```

Da in R Matrizen erweiterte Vektoren sind, können Filter und Index-Operationen für Vektoren (siehe Abschnitt 1.2.1) direkt auf Matrizen übertragen werden:

```
> F[1:2,5:6]     # Matrix
     [,1] [,2]
[1,]   21   26
[2,]   22   27
```

Das Ergebnis ist eine 2×2-Untermatrix von F. Das Ergebnis des Beispiels ist hingegen ein einfacher Vektor:

```
> F[1:2,6]       # Vektor
[1] 26 27
```

Mithilfe des zusätzlichen Arguments drop=FALSE kann jedoch dieses Umwandeln in einen Vektor vermieden werden:

```
> F[1:2,6, drop=FALSE] # Matrix
     [,1]
[1,]   26
[2,]   27
```

Wie in Abschnitt 1.2.1 besprochen, können negative Indexwerte benutzt werden, um Bereiche auszuschließen. Die folgende Anweisung gibt F ohne die erste Zeile zurück:

```
> F[-1,]
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    2    7   12   17   22   27
[2,]    3    8   13   18   23   28
[3,]    4    9   14   19   24   29
[4,]    5   10   15   20   25   30
```

Möglich ist es auch, Teilbereichen von F neue Werte zuzuweisen:

```
> F[1:3,1:2] <- 100:105
> F
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]  100  103   11   16   21   26
[2,]  101  104   12   17   22   27
[3,]  102  105   13   18   23   28
[4,]    4    9   14   19   24   29
[5,]    5   10   15   20   25   30
```

Das nächste Beispiel zeigt, wie man einen Vektor erhält, der alle Werte von F umfasst, die kleiner als 13 sind:

```
> F[F< 13]
[1]  4  5  9 10 11 12
```

Schwieriger ist das folgende Beispiel:

```
> F[F[,3]< 13,]
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]  100  103   11   16   21   26
[2,]  101  104   12   17   22   27
```

Um das obige Beispiel zu analysieren betrachten wir zuerst

```
> F[,3]
[1] 11 12 13 14 15
```

`F[,3]` ist die dritte Spalte von `F` und daher ergibt die folgende Anweisung einen booleschen Vektor, bei dem nur die ersten beiden Elemente den Wert `TRUE` haben:

```
> F[,3] < 13
[1]  TRUE  TRUE FALSE FALSE FALSE
```

Entsprechend werden daher in `F[F[,3]< 13,]` nur die ersten beiden Zeilen von `F` ausgegeben.

4.3.3 Zeilen- und Spaltennamen

Wir haben in Abschnitt 4.2.4 gesehen, dass Elemente eines Vektors optional Namen erhalten können. Ähnlich können Namen für die Zeilen und Spalten einer Matrix vergeben werden. Hierzu betrachten wir folgendes Beispiel:

```
> A <- matrix(1:6, nrow=2)
> A
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> attributes(A)
$dim
[1] 2 3
```

Mithilfe der Funktionen `colnames()` und `rownames()` können nun Namen für die Zeilen bzw. Spalten vergeben werden:

```
> colnames(A)
NULL
> colnames(A) <- c("a", "b", "c")
> rownames(A) <- c("X", "Y")
> A
  a b c
X 1 3 5
Y 2 4 6
```

Im Folgenden sehen wir, dass A hierdurch das zusätzliche Attribut „dimnames“ erhalten hat, bestehend aus einer Liste mit zwei Elementen. Beide Listenelemente sind Vektoren, die die Zeilen- bzw. Spaltennamen enthalten. Auf Listen werden wir im nächsten Kapitel genauer eingehen.

```
> str(A)
 int [1:2, 1:3] 1 2 3 4 5 6
 - attr(*, "dimnames")=List of 2
  ..$ : chr [1:2] "X" "Y"
  ..$ : chr [1:3] "a" "b" "c"
> attributes(A)
$dim
[1] 2 3

$dimnames
$dimnames[[1]]
[1] "X" "Y"

$dimnames[[2]]
[1] "a" "b" "c"
```

4.3.4 Einfügen und Löschen von Zeilen und Spalten

Durch Selbstzuweisung und mithilfe von cbind() und rbind() können Zeilen und Spalten zu einer gegebenen Matrix hinzugefügt werden. Als Beispiel wollen wir die Matrix A betrachten:

```
> A <- matrix(1:12, nrow=3)
> A
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

Die nächsten Beispiele zeigen exemplarisch, wie eine zusätzliche Spalte hinzugefügt werden kann:

```
> A <-cbind(A,c(97,98,99))
> A
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    4    7   10   97
[2,]    2    5    8   11   98
[3,]    3    6    9   12   99
```

```
> A <-cbind(A[,1:2],c(44,55,66),A[,3:5])
> A
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    4   44    7   10   97
[2,]    2    5   55    8   11   98
[3,]    3    6   66    9   12   99
```

Ähnliches gilt für Zeilen:

```
> A <-rbind(A[1,], -1:-6, A[2:3,])
> A
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    4   44    7   10   97
[2,]   -1   -2   -3   -4   -5   -6
[3,]    2    5   55    8   11   98
[4,]    3    6   66    9   12   99
```

Das Löschen von Zeilen und Spalten funktioniert ähnlich:

```
> A <-A[c(1,2,4),]
> A
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    4   44    7   10   97
[2,]   -1   -2   -3   -4   -5   -6
[3,]    3    6   66    9   12   99
```

```
> A <-A[c(1,3),c(2,4,6)]
> A
     [,1] [,2] [,3]
[1,]    4    7   97
[2,]    6    9   99
```

4.3.5 Matrixoperationen

Es gibt eine Reihe von Matrixoperationen. Wir werden uns im Wesentlichen auf die in der folgenden Tabelle aufgelisteten Funktionen beschränken.

`colSums(A)`	Spaltensummen von A
`rowSums(A)`	Zeilensummen von A
`colMeans(A)`	Mittelwerte der Spalten von A
`rowMeans(A)`	Mittelwerte der Zeilen von A
`A %*% B`	Matrizenmultiplikation von A und B
`t(A)`	Transponierte von A
`det(A)`	Determinante von A
`solve(A)`	Inverse zu A
`solve(A,b)`	Lösung des linearen Gleichungssystems Ax=b
`eigen(A)`	Eigenwerte und Eigenvektoren von A

Tabelle 4-4: Matrixoperationen

Für die folgenden Rechnungen legen wir die folgende Matrix A zugrunde:

```
> A <- matrix(1:9,3)
> A[1,1]=0
> A
     [,1] [,2] [,3]
[1,]    0    4    7
[2,]    2    5    8
[3,]    3    6    9
```

Dann gilt:

```
> colSums(A)
[1]  5 15 24
> rowSums(A)
[1] 11 15 18
> colMeans(A)
[1] 1.666667 5.000000 8.000000
> rowMeans(A)
[1] 3.666667 5.000000 6.000000
```

Die transponierte Matrix von A und die Determinante erhält man durch:

```
> t(A)
     [,1] [,2] [,3]
[1,]    0    2    3
[2,]    4    5    6
[3,]    7    8    9
> det(A)
[1] 3
```

Die Matrixaddition und die Multiplikation einer Matrix mit einem skalaren Wert werden wie erwartet berechnet:

```
> A+A
     [,1] [,2] [,3]
[1,]    0    8   14
[2,]    4   10   16
[3,]    6   12   18
> 1.1*A
     [,1] [,2] [,3]
[1,]  0.0  4.4  7.7
[2,]  2.2  5.5  8.8
[3,]  3.3  6.6  9.9
```

Beachten Sie aber den Unterschied zwischen den Operatoren `%*%` und `*`:

```
> A*A
     [,1] [,2] [,3]
[1,]    0   16   49
[2,]    4   25   64
[3,]    9   36   81
> A %*% A
     [,1] [,2] [,3]
[1,]   29   62   95
[2,]   34   81  126
[3,]   39   96  150
```

Der Operator `%*%` steht hier für die *Matrixmultiplikation*, während der Operator `*` die *element-weise Multiplikation* bezeichnet.

Bemerkung

In MATLAB hat man sich für die umgekehrte Notation entschieden: `A*B` bezeichnet in MATLAB die Matrixmultiplikation, während `A.*B` für die elementweise Multiplikation der Matrizen `A` und `B` steht.

4.3.6 Lineare Gleichungssysteme

Als Nächstes wollen wir `Ax` berechnen, wobei der Vektor `x` den Wert (1,2,3) haben soll:

```
> x <- c(1,2,3)
> b <- A %*% x
> b
```

```
     [,1]
[1,]   29
[2,]   36
[3,]   42
```

Wie wir sehen, ist `b` eine 3×1-Matrix. Im Ausdruck `A%*%x` ist `A` eine Matrix und `x` ein Vektor. Bevor die Matrixmultiplikation ausgeführt wird, wird `x` zu einer Matrix umgewandelt, die aus nur einer Spalte besteht. Dies können wir mithilfe der Funktion `as.matrix()`, die versucht, ein übergebenes Objekt in eine Matrix umzuwandeln, auch explizit verdeutlichen:

```
> as.matrix(x)
     [,1]
[1,]    1
[2,]    2
[3,]    3
```

Wir haben `b` so konstruiert, dass `x` die eindeutige Lösung des linearen Gleichungssystems `Ax=b` ist. Dies können wir in R auch umgekehrt verifizieren: die Lösung des durch `A` und `b` gegebenen linearen Gleichungssystems `Ax=b` muss gleich `x` sein:

```
> solve(A,b)
     [,1]
[1,]    1
[2,]    2
[3,]    3
```

Um die inverse Matrix von A zu bestimmen, nutzen wir ebenfalls `solve()`:

```
> B <- solve(A)
> A %*% B
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

Zur Kontrolle haben wir `A·B` berechnet, was die Einheitsmatrix ergeben muss. Dasselbe muss eigentlich auch für `B·A` gelten:

```
> B %*% A
              [,1]         [,2]          [,3]
[1,]  1.000000e+00 7.105427e-15  1.065814e-14
[2,] -1.776357e-15 1.000000e+00 -7.105427e-15
[3,]  0.000000e+00 0.000000e+00  1.000000e+00
```

Aufgrund von Rundungsfehlern sehen wir hier allerdings keine exakte Übereinstimmung. Wenn wir jedoch das Ergebnis mit `round()` runden (zum Beispiel auf 12 Dezimalstellen), erhalten wir das erwartete Ergebnis:

```
> E <- B %*% A
> round(E,12)
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

4.3.7 Eigenwerte und Eigenvektoren

Als Letztes wollen wir kurz die Eigenwerte und Eigenvektoren von A bestimmen:

```
> eig <- eigen(A)
> eig
eigen() decomposition
$values
[1] 16.01040388 -1.91242451 -0.09797938

$vectors
           [,1]        [,2]       [,3]
[1,] -0.4437999 -0.95012609  0.2357165
[2,] -0.5781140 -0.07533974 -0.8451953
[3,] -0.6847086  0.30262905  0.4796694
```

Als Ergebnis erhalten wir eine Liste. Auf Listen werden wir im nächsten Kapitel ausführlicher eingehen. Die Liste besteht aus den beiden Elementen `eig$values` und `eig$vectors`. Das Listenelement `eig$values` ist ein Vektor, der alle Eigenwerte enthält und `eig$vectors` ist eine Matrix, deren Spalten aus den entsprechenden Eigenvektoren der Länge 1 bestehen.

Die erste Spalte von `eig$vectors` ist also ein Eigenvektor zum Eigenwert `eig$values[1]`. Dies lässt sich leicht (bis auf Rundungsfehler) verifizieren:

```
> e <-eig$values
> E <- eig$vectors
> v <-  as.matrix(E[,1]) # Eigenvektor zum Eigenwert e[1]
> str(v)
 num [1:3, 1] -0.444 -0.578 -0.685
> b <- A %*% v
> c <- e[1]*v
> str(b)
 num [1:3, 1] -7.11 -9.26 -10.96
> str(b - c)
 num [1:3, 1] 1.15e-14 8.88e-15 7.11e-15
> all.equal(b,c)
[1] TRUE
```

Theoretisch müssten alle Elemente von `b-c` gleich null sein; durch Rundungsfehler ergibt sich ein leicht anderes Ergebnis. Die Funktion `all.equal(x, y)` testet, ob die beiden übergebenen Objekte „fast gleich“ sind (siehe `?all.equal`)

```
> all.equal(b,c)
[1] TRUE
Diese Rechnung lässt sich für alle drei Spalten leicht durchführen:
> for(i in 1:3) {
+   v <-  as.matrix(E[,i])
+   b <- A %*% v
+   c <- e[i]*v
+   str(b)
+   str(b - c)
+   print(all.equal(b,c))
+ }
 num [1:3, 1] -7.11 -9.26 -10.96
 num [1:3, 1] 1.15e-14 8.88e-15 7.11e-15
[1] TRUE
 num [1:3, 1] 1.817 0.144 -0.579
 num [1:3, 1] -1.33e-15 -9.71e-16 -1.78e-15
[1] TRUE
 num [1:3, 1] -0.0231 0.0828 -0.047
 num [1:3, 1] 1.64e-15 7.91e-16 1.46e-16
[1] TRUE
```

Um eine kompaktere Ausgabe zu erhalten, haben wir diesmal die Matrix `b-c` mithilfe der Funktion `str()` ausgegeben.

Grundsätzlich sollten aus Laufzeitgründen in R Schleifen vermieden werden, wenn sie durch Matrix- oder Vektoroperationen ersetzt werden können. Statt dreimal `A%*%v` in der Schleife zu berechnen, kann man auch einmal `A%*%E` berechnen.

```
> A %*% E
           [,1]       [,2]        [,3]
[1,]  -7.105416  1.8170444 -0.02309535
[2,]  -9.255838  0.1440816  0.08281170
[3,] -10.962461 -0.5787552 -0.04699771
```

Das Ergebnis muss dann mit der Matrix übereinstimmen, die man erhält, wenn die erste Spalte von `E` mit `e[1]`, die zweite Spalte von E mit `e[2]` und schließlich die dritte Spalte von E mit `e[3]` skalar multipliziert wird. Um dies in R zu erreichen, ist die Funktion `rep()` hilfreich, die im Kapitel 2 bereits kurz besprochen wurde (siehe auch `?rep`):

```
> rep(1:3, 3)
[1] 1 2 3 1 2 3 1 2 3
> rep(1:3, times=3)
[1] 1 2 3 1 2 3 1 2 3
> rep(1:3,each=3)
[1] 1 1 1 2 2 2 3 3 3
> rep(e,each=3)
[1] 16.01040388 16.01040388 16.01040388 -1.91242451 -1.91242451 -1.
91242451 -0.09797938 -0.09797938 -0.09797938
```

Also muss das Ergebnis (bis auf Rundungsfehler) von `A%*%E` mit `rep(e,each=3)*E` übereinstimmen:

```
> rep(e,each=3)*E
           [,1]       [,2]        [,3]
[1,]  -7.105416  1.8170444 -0.02309535
[2,]  -9.255838  0.1440816  0.08281170
[3,] -10.962461 -0.5787552 -0.04699771
```

4.3.8 apply()

In R gibt es eine Reihe von „apply“-Funktionen (wie `apply()`, `lapply()`, `sapply()` oder `tapply()`), die eine wichtige Rolle spielen; sie bilden einen zentralen Baustein in der funktionalen Programmierung. In vielen funktionalen Sprachen wie Haskell werden sie als „map“ bezeichnet, ebenso in Python; in C++ hingegen wird der Name „std::transform()“ benutzt.

Wir beschränken uns hier erst einmal auf die Funktion `apply()`. Mit `apply()` können vektorisierbare Funktionen auf jede Spalte oder Zeile einer Matrix angewendet werden, ohne dass explizit Schleifen benutzt werden. Dies hat neben einer häufig besseren Laufzeit auch den Vorteil, übersichtlicheren und klaren Quelltext zu erzeugen. Die Funktion `apply()` hat die folgende allgemeine Form:

```
apply(A, margin, f, ...)
```

`A`	Matrix (oder Array)
`margin`	1: Zeilen 2: Spalten
`f`	Die anzuwendende Funktion
`...`	Weitere Argumente, die an die anzuwendende Funktion weitergegeben werden

Tabelle 1-5: Argumente von apply()

In den nächsten Beispielen wollen die folgende 3×5 Matrix A zugrunde legen:

```
> A <- matrix(1:15, nrow=3)
> A
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    4    7   10   13
[2,]    2    5    8   11   14
[3,]    3    6    9   12   15
```

Um die einzelnen Zeilensummen von A zu berechnen, kann die Funktion `rowSums()` benutzt werden. Alternativ können wir die Funktion `sum()` mithilfe von `apply()` auf die Zeilen von A anwenden:

```
> rowSums(A)
[1] 35 40 45
> apply(A,1,sum)
[1] 35 40 45
```

Entsprechend erhalten wir:

```
> colSums(A)
[1]  6 15 24 33 42
> apply(A,2,sum)
[1]  6 15 24 33 42
```

Ähnlich gilt für die Funktion `mean()`

```
> rowMeans(A)
[1] 7 8 9
> apply(A,1,mean)
[1] 7 8 9
```

```
> colMeans(A)
[1]  2  5  8 11 14
> apply(A,2,mean)
[1]  2  5  8 11 14
```

Statt den Funktionen `sum()` oder `mean()` kann jede andere vektorisierbare Funktion genutzt werden:

```
> quadraticMean <- function(x){
+   return(mean(x*x))
+ }
> quadraticMean(A)
[1] 82.66667
> apply(A,1,quadraticMean)
[1] 67 82 99
> apply(A,2,quadraticMean)
[1]   4.666667  25.666667  64.666667 121.666667 196.66666
```

Im Folgenden gehen wir immer davon aus, dass `A` eine Matrix ist. Die anzuwendende Funktion `f` wird dann auf die einzelnen Zeilen bzw. Spalten von `A` angewandt.

Gibt `f` wie in den oberen Beispielen einen skalaren Wert zurück, dann gibt `apply()` einen Vektor zurück, dessen Länge der Anzahl der Zeilen bzw. Spalten entspricht.

Gibt hingegen `f` einen Vektor der Länge k zurück, dann gibt `apply()` eine Matrix `B` mit k Zeilen zurück. Dabei ist die erste Spalte von `B` das Ergebnis von `f` angewandt auf die erste Zeile bzw. Spalte. Hierzu wollen wir folgendes Beispiel betrachten:

```
> bereich <- function(x){
+   return(c(min(x), max(x)))
+ }
> bereich(A)
[1]  1 15
> apply(A,1,bereich)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]   13   14   15
> apply(A,2,bereich)
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    4    7   10   13
[2,]    3    6    9   12   15
```

Statt unserer Funktion `bereich()`, hätten wir auch die Funktion `range()` benutzen können, die wie `bereich()` einen Vektor mit dem minimalen und maximalen Wert des übergebenen Arguments zurückgibt.

4.3.9 Array

Matrizen sind konzeptionell zweidimensionale Datenstrukturen; intern sind es aber (eindimensionale) Vektoren mit einem zusätzlichen Attribut „dim“. Dies kann in natürlicherweise auf höhere Dimensionen verallgemeinert werden:

Array = Vektor + Attribut: „dim“ (Vektor der Länge größer 0) + Attribut class = "array“ + Optionales Attribut: „dimnames“

In R können Matrizen erzeugt werden durch:

```
array(data = NA, dim = length(data), dimnames = NULL)
```

Hier ist ein erstes Beispiel:

```
> a=1:30
> str(a)
 int [1:30] 1 2 3 4 5 6 7 8 9 10 ...
> A=matrix(a, nrow=5)
> str(A)
 int [1:5, 1:6] 1 2 3 4 5 6 7 8 9 10 ...
> B=array(a,dim=c(5,3,2))
> str(B)
 int [1:5, 1:3, 1:2] 1 2 3 4 5 6 7 8 9 10 ...
```

Weiterhin gilt dann:

```
> B
, , 1

     [,1] [,2] [,3]
[1,]    1    6   11
[2,]    2    7   12
[3,]    3    8   13
[4,]    4    9   14
[5,]    5   10   15

, , 2

     [,1] [,2] [,3]
[1,]   16   21   26
[2,]   17   22   27
[3,]   18   23   28
[4,]   19   24   29
[5,]   20   25   30
```

```
> B[,,1]
     [,1] [,2] [,3]
[1,]    1    6   11
[2,]    2    7   12
[3,]    3    8   13
[4,]    4    9   14
[5,]    5   10   15
> B[,2,1]
[1]  6  7  8  9 10
> B[5,3,2]
[1] 30
> length(B)
[1] 30
> dim(B)
[1] 5 3 2
```

Ein Array `A` heißt *N*-dimensional, wenn `length(dim(A))` gleich *N* ist. Im obigen Beispiel ist `B` also ein dreidimensionales Array.

Arrays können auch eindimensional sein:

```
> A <- array(1:12, dim=1)
> str(A)
 int [1(1d)] 1
> class(A)
[1] "array"
> is.vector(A)
[1] FALSE
```

Eindimensionale Arrays unterscheiden sich allerdings von Vektoren, da sie im Gegensatz zu Vektoren das Attribut „dim“ besitzen. Hingegen gibt es keinen Unterschied zwischen Matrizen und zweidimensionalen Arrays:

```
> B <- array(1:12, dim=c(2,6))
> str(B)
 int [1:2, 1:6] 1 2 3 4 5 6 7 8 9 10 ...
> class(B)
[1] "matrix"
> is.matrix(B)
[1] TRUE
> C <- matrix(1:12, nrow = 2)
> str(C)
 int [1:2, 1:6] 1 2 3 4 5 6 7 8 9 10 ...
> identical(B, C)
[1] TRUE
```

Dies zeigt auch, dass das class-Attribut bei zweidimensionalen Arrays überraschenderweise den Wert `"matrix"` hat (und nicht wie vermutet `"array"`).

In TensorFlow, einer populären Deep-Learning-Plattform, sind Tensoren grundlegende Datenstrukturen. Tensoren werden gemäß der Anzahl *N* der *Achsen* unterschieden. Ein *N*-D-Tensor *A* besitzt genau *N* Achsen und entspricht in R einem *N*-dimensionalen Array `A`.

2-D-Tensoren entsprechen also Matrizen, 1-D-Tensoren entsprechen eindimensionalen Arrays (oder manchmal auch Vektoren) und 0-D-Tensoren sind skalare Werte. Wie wir mehrfach gesehen haben, gibt es in R keinen Datentyp für skalare Werte, stattdessen wird in R ein Vektor der Länge 1 benutzt, was konzeptionell einem skalaren Wert entspricht.

Mit *Shape* von A wird in TensorFlow ein Vektor bezeichnet, der angibt, wie viele Dimensionen der Tensor A bezüglich der verschiedenen Achsen besitzt; Shape von A entspricht somit `dim(A)`.

Für das nächste Beispiel benötigen wir die Datei „ziffern.rds“, die im Downloadbereich hinterlegt ist und die in das aktuelle Arbeitsverzeichnis kopiert werden muss. Die Datei „ziffern.rds“ besteht aus einem mit `saveRDS(ziffern, "ziffern.rds")` serialisierten Objekt. Mithilfe der Funktion `readRDS()` kann das Objekt wiederhergestellt werden (und bei Bedarf unter einem anderen Namen gespeichert werden).

```
> ziffern <- readRDS("ziffern.rds")
> str(ziffern)
 int [1:100, 1:28, 1:28] 0 0 0 0 0 0 0 0 0 0 ...
> object.size(ziffern)
313824 bytes
```

Wie wir sehen, handelt es sich hier bei dem eingelesenen Objekt `ziffern` um ein dreidimensionales Array, das ca. 300 KB umfasst. `ziffern` ist ein Datensatz, der aus hundert Grauwertbildern besteht, die die Ziffern von 0 bis 9 in ganz unterschiedlichen handgeschriebenen Varianten zeigen. Jedes Grauwertbild besteht aus 28×28 Pixeln und jeder Pixelwert liegt zwischen 0 und 255. Somit beschreibt der erste Dimensionswert von `ziffern` den Bildindex, während die beiden anderen Dimensionswerte die Pixelposition im entsprechenden Bild beschreiben. Die Bilder stammen ursprünglich aus der MNIST-Datensammlung[4], die insgesamt 70 000 handgeschriebene Varianten von Ziffern umfasst. Die MNIST-Datensammlung wird sehr häufig als Referenzbeispiel für Machine-Learning-Algorithmen benutzt.

[4] Siehe z. B. https://en.wikipedia.org/wiki/MNIST_database

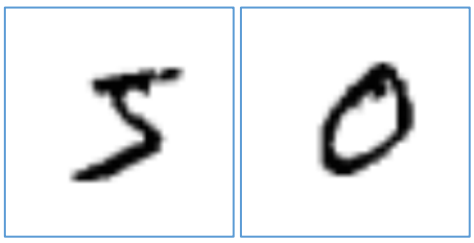
Als Nächstes wollen wir die Daten des ersten Bilds ausgeben:

```
> ziffern[1,,]
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13] [,14] [,15] [,16] [,17] [,18] [,19] [,20] [,21] [,22] [,23] [,24] [,25] [,26] [,27] [,28]
 [1,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
 [2,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
 [3,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
 [4,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
 [5,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
 [6,]    0    0    0    0    0    0    0    0    0     0     0     0     3    18    18    18   126   136   175    26   166   255   247   127     0     0     0     0
 [7,]    0    0    0    0    0    0    0    0   30    36    94   154   170   253   253   253   253   253   225   172   253   242   195    64     0     0     0     0
 [8,]    0    0    0    0    0    0    0   49  238   253   253   253   253   253   253   253   253   251    93    82    82    56    39     0     0     0     0     0
 [9,]    0    0    0    0    0    0    0   18  219   253   253   253   253   253   198   182   247   241     0     0     0     0     0     0     0     0     0     0
[10,]    0    0    0    0    0    0    0    0   80   156   107   253   253   205    11     0    43   154     0     0     0     0     0     0     0     0     0     0
[11,]    0    0    0    0    0    0    0    0    0    14     1   154   253    90     0     0     0     0     0     0     0     0     0     0     0     0     0     0
[12,]    0    0    0    0    0    0    0    0    0     0     0   139   253   190     2     0     0     0     0     0     0     0     0     0     0     0     0     0
[13,]    0    0    0    0    0    0    0    0    0     0     0    11   190   253    70     0     0     0     0     0     0     0     0     0     0     0     0     0
[14,]    0    0    0    0    0    0    0    0    0     0     0     0    35   241   225   160   108     1     0     0     0     0     0     0     0     0     0     0
[15,]    0    0    0    0    0    0    0    0    0     0     0     0     0    81   240   253   253   119    25     0     0     0     0     0     0     0     0     0
[16,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0    45   186   253   253   150    27     0     0     0     0     0     0     0     0
[17,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0    16    93   252   253   187     0     0     0     0     0     0     0     0
[18,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0   249   253   249    64     0     0     0     0     0     0     0
[19,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0    46   130   183   253   253   207     2     0     0     0     0     0     0     0
[20,]    0    0    0    0    0    0    0    0    0     0     0     0    39   148   229   253   253   253   250   182     0     0     0     0     0     0     0     0
[21,]    0    0    0    0    0    0    0    0    0     0    24   114   221   253   253   253   253   201    78     0     0     0     0     0     0     0     0     0
[22,]    0    0    0    0    0    0    0    0   23    66   213   253   253   253   253   198    81     2     0     0     0     0     0     0     0     0     0     0
[23,]    0    0    0    0    0    0   18  171  219   253   253   253   253   195    80     9     0     0     0     0     0     0     0     0     0     0     0     0
[24,]    0    0    0    0   55  172  226  253  253   253   253   244   133    11     0     0     0     0     0     0     0     0     0     0     0     0     0     0
[25,]    0    0    0    0  136  253  253  253  212   135   132    16     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
[26,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
[27,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
[28,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0     0
```

Es ist in R nicht schwer, einzelne Bilder anzuzeigen. Mithilfe der Funktion `as.raster()` können Bilddaten in ein entsprechendes Raster-Objekt umgewandelt werden, das mit der Funktion `plot()` angezeigt werden kann:

```
> plot(as.raster(ziffern[1,,], max = 255))
> plot(as.raster(ziffern[2,,], max = 255))
```

Das erste Bild zeigt die Ziffer 5, während das zweite die Ziffer 0 darstellt. Wie erhält man die folgenden zwei Bilder?



Die Anwort ist einfach. Dank Recycling kann man schreiben:

```
> plot(as.raster(255-ziffern[1,,], max = 255))
> plot(as.raster(255-ziffern[2,,], max = 255))
```

WICHTIG

- Matrix = Vektor + Attribut „dim“
- `matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE, dimnames = NULL)`
- `cbind(), rbind()`
- `dim(), nrow(), ncol()`
- `F[2,], F[,3], F[2:3, 3:5], …`
- `Drop = FALSE`
- `colnames(), rownames()`
- `colSums(), rowSums(), colMeans(), rowMeans()`
- `2*A, A+B, A*B, A %*% B`
- `t()`
- `det()`
- `as.matrix()`
- `solve(A), solve(A,b)`
- `eigen()`
- `round()`
- `apply()`
- `array(data = NA, dim = length(data), dimnames = NULL)`
- *N*-dimensionales Array, Tensoren, Achsen
- `saveRDS(), readRDS(),` Serialisierung
- `plot(), as.raster()`

4.4 Übungen

Die Übungen für dieses Kapitel finden Sie im Downloadbereich in der Datei „jwRscript_ueb_Kapitel4x.pdf“.

Jürgen Willms

Programmierung für statistische Datenanalyse

TEIL 3

Inhaltsverzeichnis

5 WEITERE DATENSTRUKTUREN 3

5.1 List 3
5.1.1 Erzeugung von Listen und Indexierung 3
5.1.2 Listen ändern 7
5.1.3 Vergleich: Liste versus Vektor 8
5.1.4 lapply() und sapply() 10

5.2 Factor 11

5.3 Data Frame 16
5.3.1 Data Frame erzeugen 16
5.3.2 Der Indexoperator 19
5.3.3 apply() 20
5.3.4 Filtern, Einfügen und Löschen 20

Literaturverzeichnis 21

5 Weitere Datenstrukturen

Als Nächstes betrachten wir weitere grundlegende Datenstrukturen in R; im Vordergrund hierbei stehen *Listen* und *Data Frames*. Die bisher besprochenen Datenstrukturen *(atomare) Vektoren*, *Matrizen* und *Arrays* sind homogene Datenstrukturen: alle Elemente haben den gleichen zugrundeliegenden Datentyp, der nicht weiter unterteilt werden kann. Er ist entweder `logical`, `integer`, `double`, `complex`, `character` oder `raw`.

Listen und Data Frames sind hingegen heterogen: in ihnen können Objekte unterschiedlicher Datentypen gespeichert werden. Das erste Listenelement kann z. B. ein Vektor der Länge 10 sein, während das zweite Listenelement eine 28×28-Matrix sein kann.

5.1 List

Wie wir gesehen haben, können in einem einzelnen (atomaren) Vektor nicht Objekte gespeichert werden, die unterschiedliche Datentypen besitzen; hierzu benutzt man Listen. Eine Liste ist ein linear angeordneter Datencontainer, dessen Elemente auf andere Objekte verweisen; eine Liste kann somit also konzeptionell als ein Vektor von Referenzen aufgefasst werden. Intern ist in R der Datentyp Liste als Vektor implementiert und es gibt strenggenommen in R die Unterscheidung zwischen atomaren und nicht-atomaren Vektoren. Da Listen auch Listen als Elemente enthalten können, die ihrerseits wiederum Listen als Elemente enthalten können, … werden Listen auch manchmal *rekursive* Vektoren genannt. Wir werden aber bei unserer Konvention bleiben und im Folgenden immer atomare Vektoren meinen, wenn wir von Vektoren reden.

5.1.1 Erzeugung von Listen und Indexierung

Als erstes einfaches Beispiel soll eine Liste in R erzeugt werden, die Prüfungsdaten eines Studenten enthält:

```
> x <- list(nachname="Meier", matNr = 123456, noten=c(1.3,2.7))
> x
$nachname
[1] "Meier"

$matNr
[1] 123456

$noten
[1] 1.3 2.7
```

Es empfiehlt sich, `str()` zu benutzen, um eine kompakte und übersichtliche Ausgabe der Liste zu erhalten:

```
> str(x)
List of 3
 $ nachname: chr "Meier"
 $ matNr   : num 123456
 $ noten   : num [1:3] 1.3 2.7
```

x ist also eine Liste mit drei Elementen; jedes Element hat dabei einen Namen.[1] Das dritte Listenelement zeigt auf einen numerischen Vektor der Länge 2. Weiterhin erhalten wir:

```
> length(x)
[1] 3
> typeof(x)
[1] "list"
> class(x)
[1] "list"
> is.list(x)
[1] TRUE
> attributes(x)
$names
[1] "nachname" "matNr"    "noten"
```

In vielen Programmiersprachen kann mithilfe der Punktnotation auf ein Listenelement zugegriffen werden. Dies ist in R nicht möglich! In R wird stattdessen das Dollarzeichen benutzt, um über den Namen auf ein Listenelement zuzugreifen:

```
> x.noten
Error: object 'x.noten' not found
> x$noten
[1] 1.3 2.7 1.0
> x$matNr
[1] 123456
```

Auch mithilfe des Indexoperators können Listenelemente angesprochen werden, allerdings unterscheidet sich der *„einfache“* Indexoperator grundlegend vom *„doppelten“* Indexoperator:

- `x[3]` und `x[[3]]`
- `x["noten"]` und `x[["noten"]]`

```
> str(x["noten"])
List of 1
 $ noten: num [1:2] 1.3 2.7
> str(x[3])
List of 1
 $ noten: num [1:2] 1.3 2.7
> str(x[["noten"]])
 num [1:2] 1.3 2.7
> str(x[[3]])
 num [1:2] 1.3 2.7
> str(x$noten)
```

[1] daher oft auch *named* list (oder *benannte* Liste) genannt. Die Namen werden auch als „Tags“ bezeichnet.

```
num [1:2] 1.3 2.7
```

Identische Ergebnisse erhält man jeweils durch:

- `x$noten`, `x[["noten"]]` und `x[[3]]`.
- `x["noten"]` und `x[3]`

Überraschenderweise geben `x["noten"]` und `x[3]` eine Liste zurück. Der einfache Indexoperator angewandt auf eine Liste gibt immer eine Teilliste zurück! Will man also das Element einer Liste erhalten, so muss der doppelte Indexoperator benutzt werden. Aus Konsistenzgründen empfehlen daher manche Autoren sogar, auch bei Vektoren den doppelten Indexoperator zu benutzen, da es bei Vektoren keinen Unterschied zwischen beiden Schreibweisen gibt:

```
> v <- 1:4
> v[[1]]
[1] 1
> identical(v[1], v[[1]])
[1] TRUE
> xv <- as.list(1,2,3,4)
> str(xv)
List of 1
 $ : num 1
> str(xv[1])
List of 1
 $ : num 1
> str(xv[[1]])
 num 1
> identical(xv[1], xv[[1]])
[1] FALSE
```

Indizierung kann wie bei Vektoren angewandt werden:

```
> x[1:2]
$nachname
[1] "Meier"

$matNr
[1] 123456
```

Hingegen:

```
> x[[1:2]]
Error in x[[1:2]] : subscript out of bounds
```

Unser erstes Listenbeispiel hätte auch ohne Namen erzeugt werden können:

```
> x1 <- list("Meier", 123456, c(1.3,2.7, 1.0))
> x1
[[1]]
[1] "Meier"

[[2]]
[1] 123456

[[3]]
[1] 1.3 2.7 1.0

> str(x1)
List of 3
 $ : chr "Meier"
 $ : num 123456
 $ : num [1:3] 1.3 2.7 1
```

Aber es ist in der Regel besser und übersichtlicher, Namen zu verwenden.

Beachten Sie auch den Unterschied zwischen den nächsten drei einzelnen Anweisungen:

```
> str(list(1:4))
List of 1
 $ : int [1:4] 1 2 3 4
> str(list(1,2,3,4))
List of 4
 $ : num 1
 $ : num 2
 $ : num 3
 $ : num 4
> str(as.list(1:4))
List of 4
 $ : int 1
 $ : int 2
 $ : int 3
 $ : int 4
```

Listen können auch Listen als Element haben (und zwar grundsätzlich in „beliebiger“ Verschachtelungstiefe). Dies demonstrieren die nächsten drei Beispiele:

```
> yx <- list(pvl=c(T,T,F), x)
> str(yx)
List of 2
 $ pvl: logi [1:3] TRUE TRUE FALSE
 $    :List of 3
  ..$ nachname: chr "Meier"
  ..$ matNr   : num 123456
  ..$ noten   : num [1:2] 1.3 2.7
```

```
> yx2 <-list(yx,yx[2])
> str(yx2)
List of 2
 $ :List of 2
  ..$ pvl: logi [1:3] TRUE TRUE FALSE
  ..$    :List of 3
  .. ..$ nachname: chr "Meier"
  .. ..$ matNr   : num 123456
  .. ..$ noten   : num [1:2] 1.3 2.7
 $ :List of 1
  ..$ :List of 3
  .. ..$ nachname: chr "Meier"
  .. ..$ matNr   : num 123456
  .. ..$ noten   : num [1:2] 1.3 2.7
```

```
> yx3 <-list(list(list(T)))
> str(yx3)
List of 1
 $ :List of 1
  ..$ :List of 1
  .. ..$ : logi TRUE
```

Um aus einer verschachtelten Liste wieder einen „flachen“ Vektor zu erhalten, der alle atomaren Komponenten der Liste enthält, kann die Funktion `unlist()` benutzt werden:

```
> unlist(yx2)
    pvl1     pvl2     pvl3 nachname    matNr   noten1   noten2
  "TRUE"   "TRUE"  "FALSE"  "Meier" "123456"    "1.3"    "2.7"
nachname    matNr   noten1   noten2
 "Meier" "123456"    "1.3"    "2.7"
```

5.1.2 Listen ändern

Wir haben bereits gesehen, wie auf einzelne Listenelemente zugegriffen werden kann. Listen können in natürlicherweise durch Zuweisungen geändert werden. Elemente können gelöscht werden, indem sie den Wert `NULL` erhalten,

Wir betrachten wieder unser einführendes Beispiel:

```
> str(x)
List of 3
 $ nachname: chr "Meier"
 $ matNr   : num 123456
 $ noten   : num [1:2] 1.3 2.7
```

Ein Vorname soll eingefügt, die Matrikelnummer geändert und die Noten gelöscht werden:

```
> x$vorname <- "Udo"
> x[[2]] <- 222333     # besser ?:  x[["matNr"]] <- 222333
> x$noten <- NULL
> str(x)
List of 3
 $ nachname: chr "Meier"
 $ matNr   : num 222333
 $ vorname : chr "Udo"
```

Durch Kombination von Listen und Vektoren können Listen auch mit der bereits bekannten Funktion `c()` erstellt werden:

```
> z <- list(11,12,13)
> z1 <- c(z, c(14,15))
> str(z1)
List of 5
 $ : num 11
 $ : num 12
 $ : num 13
 $ : num 14
 $ : num 15
> z2 <- c(c(14,15), z)
> str(z2)
List of 5
 $ : num 14
 $ : num 15
 $ : num 11
 $ : num 12
 $ : num 13
```

Vergleichen Sie dies mit den folgenden beiden Anweisungen:

```
> z3 <- list(z, c(11,12,13))
> str(z3)
List of 2
 $ :List of 3
  ..$ : num 11
  ..$ : num 12
  ..$ : num 13
 $ : num [1:3] 11 12 13
```

```
> z <- list(11,12,13)
> str(z)
List of 3
 $ : num 11
 $ : num 12
 $ : num 13
> z[4:5] <- c(14,15)
> str(z)
List of 5
 $ : num 11
 $ : num 12
 $ : num 13
 $ : num 14
 $ : num 15
```

Frage

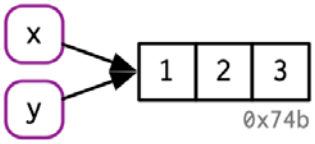
Sind jetzt z und z1 identisch?

5.1.3 Vergleich: Liste versus Vektor

Wir wollen noch einmal kurz Listen und Vektoren gegenüberstellen und dabei insbesondere den Effekt von Copy-on-modify betrachten. Die im Folgenden benutzten Diagramme und die zugehörigen Beispiele sind aus [Wickham 2019a][2], Kapitel 2 entnommen.

```
> x <- c(1,2,3)
> y <- x
```

Wie wir bereits in Kapitel 3 erläutert haben, sind nun beide Namen (x und y) an dasselbe Objekt gebunden:

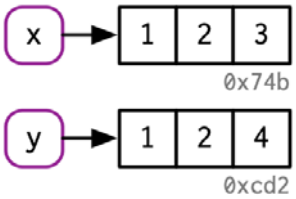


Quelle: [Wickham 2019a]

Wird y geändert,

```
> y[3] <- 4
```

dann „zeigen“ x und y auf verschiedene Objekte:



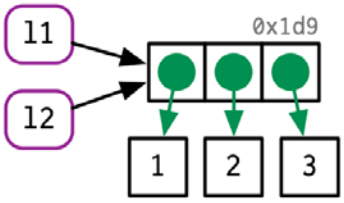
Quelle: [Wickham 2019a]

[2] Unbedingt empfehlenswert, wenn man R genauer verstehen möchte. [Wickham 2019a] ist die elektronische Version von [Wickham 2019b]

Wie sieht die vergleichbare Situation bei Listen aus?

```
> l1 <- list(1,2,3)
> l2 <- l1
```

Da eine Liste ein linear angeordneter Datencontainer ist, dessen Elemente auf andere Objekte verweisen und der als ein Vektor von Referenzen aufgefasst werden kann, ergibt sich im Vergleich zum Vektor eine andere interne Implementierung. Dies wird im folgenden Diagramm deutlich:

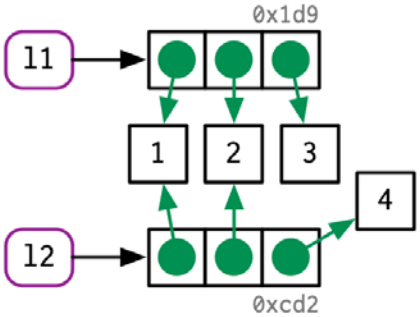


Quelle: [Wickham 2019a]

Ändern wir l2,

```
l2[[3]] <- 4
```

dann ergibt sich folgendes Bild:



Quelle: [Wickham 2019a]

5.1.4 lapply() und sapply()

Die Funktion `lapply()` (für *list apply*) bezieht sich auf Listen und entspricht der Funktion `apply()`. Sie wird auf Listen angewandt (oder auf Vektoren die in Listen umgewandelt werden), während `apply()` auf Matrizen angewandt wird. Zur Illustration wollen wir folgendes Beispiel betrachten:

```
> l <- list(9:0, 1:100, 4)
> str(l)
List of 3
 $ : int [1:10] 9 8 7 6 5 4 3 2 1 0
 $ : int [1:100] 1 2 3 4 5 6 7 8 9 10 ...
 $ : num 4
```

Mithilfe von `lapply()` können wir die `mean()`-Funktion auf alle Listenelemente anwenden:

```
> lapply(l, mean)
[[1]]
[1] 4.5

[[2]]
[1] 50.5

[[3]]
[1] 4
```

Das Ergebnis ist wiederum eine Liste.

Die benutzerfreundlichere Version `sapply()` (für *simplified list apply*) kann ebenfalls auf Listen angewandt werden; sie gibt in der Regel eine homogene Datenstruktur wie Vektor oder Matrix zurück. In unserem Beispiel ist dies ein Vektor:

```
> sapply(l,mean)
[1]  4.5 50.5  4.0
```

`sapply()` kann aber auch eine Matrix zurückgeben:

```
> sapply(l,range)
     [,1] [,2] [,3]
[1,]    0    1    4
[2,]    9  100    4
```

Vergleichen Sie hierzu:

```
> lapply(l,range)
[[1]]
[1] 0 9

[[2]]
[1]   1 100

[[3]]
[1] 4 4
```

WICHTIG

- `list()`
- `[], [[ ]], $`
- `is.list() as.list()`
- `unlist()`
- `lapply(), sapply()`

5.2 Factor

Mithilfe von Faktoren ist es in R möglich, nominalskalierte oder ordinalskalierte Daten[3] zu bearbeiten.

Frage

- Was sind nominalskalierte Daten?
- Was sind ordinalskalierte Daten?
- Was sind kardinalskalierte Daten?

Als Beispiel wollen wir einen Datensatz `data` betrachten, der aus 30 nominalskalierten Werten besteht, die den belegten Studiengang Maschinenbau (MB), Elektrotechnik (ET), Wirtschaftsingenieur (WIng) und Wirtschaftsinformatik (WInfo) von 30 Studierenden angibt:

```
> studiengang <- c("MB", "ET", "WIng", "WInfo")
> data <-  sample(studiengang, 30, replace = TRUE)
> data
 [1] "MB"    "ET"    "WIng"  "WIng"  "MB"    "WInfo" "WInfo" "WInfo" "MB"    "WInfo" "WInfo"
[12] "WInfo" "WInfo" "MB"    "MB"    "ET"    "ET"    "ET"    "ET"    "WInfo" "WInfo" "MB"
[23] "ET"    "WIng"  "ET"    "WIng"  "MB"    "MB"    "ET"    "MB"
> str(data)
 chr [1:30] "MB" "ET" "WIng" "WIng" "MB" "WInfo" "WInfo" "WInfo" "MB" "WInfo" "WInfo" ...
```

Der Datensatz `data` besitzt vier unterschiedliche Kategorien: "MB", "ET", "WIng", "WInfo", die wir im Folgenden, wie in R üblich, als *Level* bezeichnen wollen. Somit können wir den Datensatz auch als einen `integer`-Vektor der Länge 30 interpretieren, der nur die Werte 1, 2, 3 und 4 enthält. Genau dies erreichen wir mit einer Umwandlung in einen Faktor:

```
> f1 <- factor(data)
> f1
 [1] MB    ET    WIng  WIng  MB    WInfo WInfo WInfo MB    WInfo WInfo WInfo WInfo MB
[15] MB    ET    ET    ET    ET    WInfo WInfo MB    ET    WIng  ET    WIng  MB    MB
[29] ET    MB
Levels: ET MB WInfo WIng
> str(f1)
 Factor w/ 4 levels "ET","MB","WInfo",..: 2 1 4 4 2 3 3 3 2 3 ...
> levels(f1)
[1] "ET"    "MB"    "WInfo" "WIng"
> attributes(f1)
$levels
[1] "ET"    "MB"    "WInfo" "WIng"

$class
[1] "factor"
```

Mit der Funktion `levels()` können wir die entsprechenden Level erhalten, die bei der Umwandlung automatisch erzeugt wurden. Wenn wir wollen, dass "MB" für 1, "ET" für 2, "WIng" für 3 und "WInfo" für 4 steht, dann müssen wir die Level explizit in der entsprechenden Reihenfolge angeben:

```
> f2 <- factor(data, c("MB", "ET", "WIng", "WInfo"))
> str(f2)
 Factor w/ 4 levels "MB","ET","WIng",..: 1 2 3 3 1 4 4 4 1 4 ...
```

[3] Oft wird in diesem Zusammenhang auch von kategorialen Variablen gesprochen

Wenn wir uns stattdessen auf nur drei Level beschränken, dann erhalten wir entsprechende `NA`-Werte:

```
> f3 <- factor(data, c("MB", "ET", "WIng"))
> str(f3)
 Factor w/ 3 levels "MB","ET","WIng": 1 2 3 3 1 NA NA NA 1 NA ...
```

Um aus einem Faktor wieder einen Vektor zu erzeugen, gibt es verschiedene Möglichkeiten, die allerdings zu unterschiedlichen Ergebnissen führen:

```
> str(as.vector(f1))
 chr [1:30] "MB" "ET" "WIng" "WIng" "MB" "WInfo" "WInfo" "WInfo" "MB" "WInfo" "WInfo
" ...
> str(as.numeric(f1))
 num [1:30] 2 1 4 4 2 3 3 3 2 3 ...
> str(levels(f)[f])
 chr [1:30] "MB" "ET" "WIng" "WIng" "MB" "WInfo" "WInfo" "WInfo" "MB" "WInfo" "WInfo
" ...
```

Mithilfe der Funktion `table()`[4] kann ein Faktor auch in Tabellenform angezeigt werden:

```
> table(f1)
f1
   ET    MB WInfo  WIng
    7    10     6     7
```

Ein ähnliches Ergebnis erhält man auch mit der generischen Funktion `summary()`[5]:

```
> summary(f1)
   ET    MB WInfo  WIng
    7    10     6     7
```

Mit der Funktion `barplot()`, mit der leicht Balkendiagramme erzeugt werden können, können die Daten aus `f1` visualisiert werden. `barplot(table(f1))` liefert folgendes Balkendiagramm:

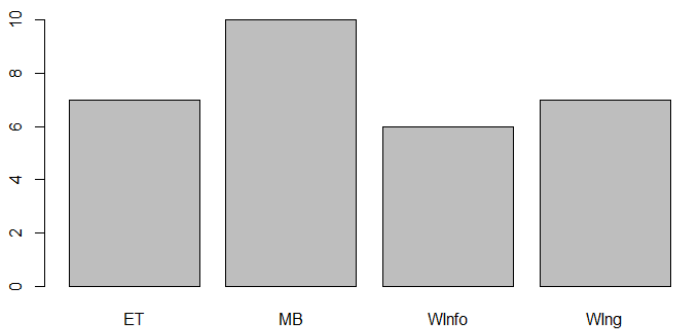


Abbildung 1: `barplot(table(f1))`

[4] Siehe Übungen zu Kapitel 4

[5] Die erzeugten Ausgaben sind fast identisch, aber die Rückgabewerte sind unterschiedlich: Während `table()` ein Objekt der Klasse `table` zurückgibt, gibt in diesem Fall `summary()` einen benannten Vektor zurück

Die Daten in `f1` sind nominalskaliert, jedoch nicht ordinalskaliert. Daher ist die folgende Fehlermeldung nicht überraschend:

```
> min(f1)
Error in Summary.factor(c(2L, 4L, 2L, 4L, 1L, 4L, 4L, 2L, 1L, 4L, 3L,  :
  'min' not meaningful for factors
```

Allerdings ist R hier nicht konsequent, da `f1` sortiert werden kann[6]:

```
> sort(f1)
 [1] ET    ET    ET    ET    ET    ET    ET    MB    MB    MB    MB    MB    MB    MB
[15] MB    MB    MB    WInfo WInfo WInfo WInfo WInfo WInfo WIng  WIng  WIng  WIng  WIng
[29] WIng  WIng
Levels: ET MB WInfo WIng
```

Ein Faktor kann aber auch mit ordinalskalierten Daten umgehen. Angenommen wir haben in unseren Studiengängen eine feste Reihenfolge vorgegeben, wie

"ET" < "WIng" < "MB" < "WInfo".

Mit der option `ordered = TRUE` können wir dies mit einbringen:

```
f4 <- factor(data, levels= c( "ET", "WIng","MB", "WInfo"), ordered = TRUE)
```

und erhalten dann:

```
> min(f4)
[1] ET
Levels: ET < WIng < MB < WInfo
> range(f4)
[1] ET    WInfo
Levels: ET < WIng < MB < WInfo
> sort(f4)
 [1] ET    ET    ET    ET    ET    ET    ET    WIng  WIng  WIng  WIng  WIng  WIng  WIng
[15] MB    MB    MB    MB    MB    MB    MB    MB    MB    MB    WInfo WInfo WInfo WInfo
[29] WInfo WInfo
Levels: ET < WIng < MB < WInfo
```

Das entsprechende Balkendiagramm berücksichtigt dann auch die vorgegebene Reihenfolge:

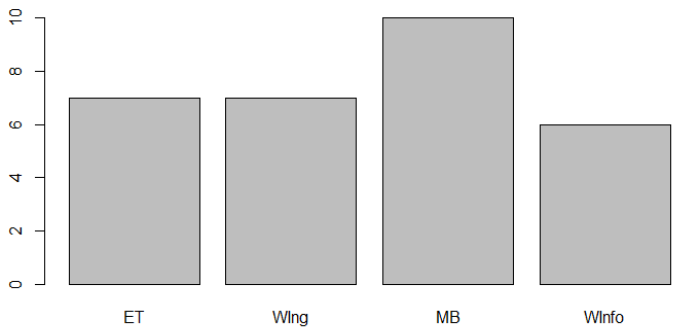


Abbildung 2: `barplot(table(f4))`

[6] entspricht hier einer alphabetischen Sortierung

Um kardinalskalierte Daten in Bereiche aufzuteilen, kann die Funktion `cut()` benutzt werden, die aus einem numerischen Vektor einen Faktor erzeugt. Wir wollen dies kurz an einem Beispiel illustrieren. Der Vektor `punkte` soll hierbei die Anzahl der erreichten Punkte in einer Klausur mit 200 Teilnehmern simulieren:

```
> punkte <-  rnorm(200, mean = 100, sd = 10)
> summary(punkte)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  66.67   93.50  100.99  100.08  106.92  128.14
```

Im einfachsten Fall können wir den Wertebereich in 8 gleich große Bereiche unterteilen:

```
> c1 <- cut(punkte, breaks = 8)
> str(c1)
 Factor w/ 8 levels "(66.6,74.4]",..: 4 5 5 5 6 4 5 2 3 6 ...
> table(c1)
c1
(66.6,74.4]   (74.4,82]   (82,89.7] (89.7,97.4]  (97.4,105]   (105,113]   (113,120]
          4           7          21          44          62          39          18
  (120,128]
          5
> barplot(table(c1))
```

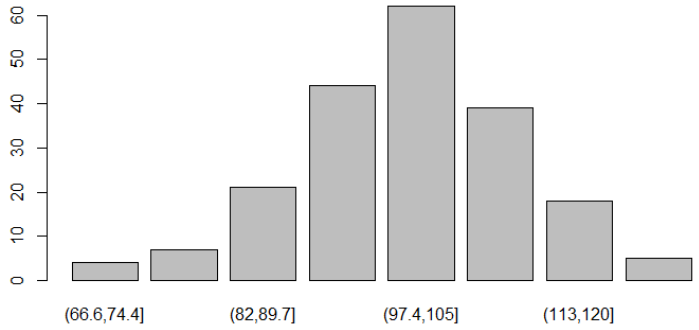


Abbildung 3 barplot(table(c1))

Für die einzelnen Level wurde die mathematische Notation für halboffene Intervalle (a,b] benutzt. Es ist aber auch möglich, die einzelnen Bereiche explizit anzugeben:

```
> barplot(table(c2))
> c2 <- cut(punkte, breaks = seq(65, 135, by = 10))
> str(c2)
 Factor w/ 7 levels "(65,75]","(75,85]",..: 4 4 4 4 5 3 4 2 3 5 ...
> table(c2)
c2
  (65,75]   (75,85]   (85,95]  (95,105] (105,115] (115,125] (125,135]
        4        10        47        77        49        11         2
> barplot(table(c2))
```

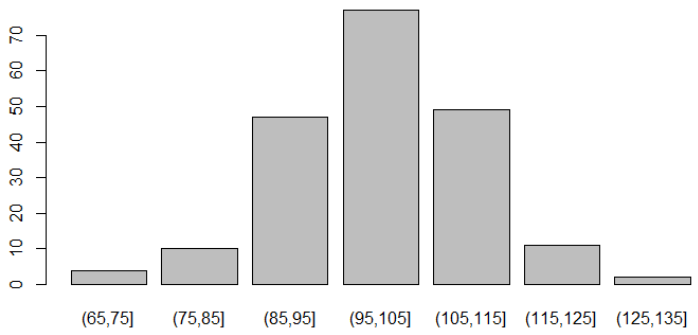


Abbildung 4 barplot(table(c2))

WICHTIG

- `factor(), levels(), ordered = TRUE`
- `barplot(), table()`
- `cut()`

5.3 Data Frame

Viele Datensätze liegen in Form von Tabellen vor, da Tabellen oft als grundlegende Datenstruktur eingesetzt werden. Dies ist zum Beispiel der Fall bei relationalen Datenbanken: aus konzeptioneller oder externer Sicht betrachtet, bestehen Datensätze in relationalen Datenbanken aus Tabellen[7]. Auch in der Statistik werden Tabellen betrachtet: üblich ist es, dass dann die Spalten Variablen und die Zeilen Beobachtungen (Observationen) entsprechen.

Wie wir sehen werden, besitzt R mit Data Frame eine mächtige Datenstruktur, die aus konzeptioneller Sicht einer Tabelle entspricht. Da eine Tabelle (und somit ein Data Frame) aus Zeilen und Spalten besteht, also zweidimensional ist, ist sie ähnlich zu einer Matrix. Aber es gibt einen bedeutenden Unterschied zwischen einer Matrix und einer Tabelle: in einer Tabelle können den einzelnen Spalten unterschiedliche Datentypen zugrunde liegen; eine Matrix hingegen ist homogen. Intern ist ein Data Frame im einfachsten Fall eine Liste mit der einschränkenden Bedingung, dass jedes Listenelement ein Vektor gleicher Länge ist. Insgesamt finden wir in R also folgende grundlegende Datenstrukturen vor:

Dimension	homogen	heterogon
1	vector[8], factor	list
2	matrix	data frame
n	array	

Tabelle 5: Vergleich von Datenstrukturen in R

Wir unterscheiden hier zwischen Dimension[9] und Länge! Zum Beispiel betrachten wir einen (atomaren) Vektor als eine eindimensionale Datenstruktur, der allerdings eine beliebig große Länge haben kann.

5.3.1 Data Frame erzeugen

Als nächstes Beispiel wollen wir folgenden Datensatz betrachten:

n	Messung 1	Messung 2	Messung 3	Messung 4
40000000	3,656	3,781	3,719	3,766
50000000	4,672	4,735	4,719	4,688
60000000	5,719	5,672	5,672	5,625
70000000	6,720	6,782	6,688	6,610
80000000	7,641	7,673	7,704	7,750
90000000	8,750	8,798	9,048	8,735

Tabelle 5: Beispieldatensatz Zeitmessung

Dieser Datensatz enthält für sechs unterschiedliche *n*-Werte jeweils vier Zeitmessungen. Gemessen wurde dabei, wie lange eine bestimmte Implementierung des Quicksort-Algorithmus auf einem gegebenen Rechner braucht, um *n* ganze Zahlen zu sortieren. Die zu sortierenden Daten wurden vor jeder einzelnen Zeitmessung „zufällig“[10] erzeugt.

[7] streng genommen eigentlich aus Relationen
[8] wie üblich meinen wir dabei einen atomaren Vektor
[9] Wie bereits am Ende von Kapitel 4 erläutert, wird statt Dimension manchmal auch der Begriff „Anzahl der Achsen“ verwendet
[10] mithilfe eines Pseudo-Zufallsgenerators

Die entsprechenden Werte wurden in einer CSV-Datei „zeiten1.csv“ hinterlegt:

```
n        ; Messung 1; Messung 2 ; Messung 3;  Messung 4
40000000;     3,656;      3,781;     3,719;     3,766
50000000;     4,672;      4,735;     4,719;     4,688
60000000;     5,719;      5,672;     5,672;     5,625
70000000;     6,720;      6,782;     6,688;     6,610
80000000;     7,641;      7,673;     7,704;     7,750
90000000;     8,750;      8,798;     9,048;     8,735
```

Wir können den Datensatz dieser Datei leicht in R einlesen, vorausgesetzt die Datei „zeiten1.csv“ befindet sich im Arbeitsverzeichnis:

```
> df1 <- read.csv2("zeiten1.csv")
> df1
         n Messung.1 Messung.2 Messung.3 Messung.4
1 40000000     3.656     3.781     3.719     3.766
2 50000000     4.672     4.735     4.719     4.688
3 60000000     5.719     5.672     5.672     5.625
4 70000000     6.720     6.782     6.688     6.610
5 80000000     7.641     7.673     7.704     7.750
6 90000000     8.750     8.798     9.048     8.735
```

Zum Einlesen wurde die Funktion `read.csv2()` benutzt, eine Variante der Funktion `read.table()`. Die Funktion `read.table()` ist die Standardfunktion in R zum Einlesen von Tabellendaten. Abgesehen von bereits für CSV-Dateien[11] geeignet gesetzten Defaultwerten, sind die beiden Varianten `read.csv()` und `read.csv2()` identisch zu `read.table()`. Die Funktion `read.csv2()` eignet sich besonders für Dateien, in denen das Komma als Dezimaltrennzeichen und das Semikolon als Datentrennzeichen benutzt wird; genau dies ist bei unserer Datendatei „zeiten1.csv“ der Fall.

`df1` ist nun ein Data Frame bestehend aus 5 Variablen (der Anzahl „n“ plus vier Messvariablen) und 6 Observationen:

```
> str(df1)
'data.frame': 6 obs. of  5 variables:
 $ n        : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ Messung.1: num  3.66 4.67 5.72 6.72 7.64 ...
 $ Messung.2: num  3.78 4.74 5.67 6.78 7.67 ...
 $ Messung.3: num  3.72 4.72 5.67 6.69 7.7 ...
 $ Messung.4: num  3.77 4.69 5.62 6.61 7.75 ...
```

Die Namen der Spaltenvariablen sind mit leichter Veränderung aus den Daten der Datei übernommen worden:

```
> names(df1)
[1] "n"         "Messung.1" "Messung.2" "Messung.3" "Messung.4"
```

[11] comma separated values

Ein Data Frame kann natürlich auch explizit erzeugt werden. Hierzu benutzen wir die Funktion `data.frames()`:

```
> n <- (4:9)*1e7L
> mess1 <- c(3.656, 4.672, 5.719, 6.720, 7.641, 8.750)
> mess2 <- c(3.781, 4.735, 5.672, 6.782, 7.673, 8.798)
> mess3 <- c(3.719, 4.719, 5.672, 6.688, 7.704, 9.048)
> mess4 <- c(3.766, 4.688, 5.625, 6.610, 7.750, 8.735)
> df2 <- data.frame(n, mess1, mess2, mess3, mess4)
```

Interessant ist, dass die Namen der Argumente aus dem Funktionsaufruf `data.frame(n, mess1, mess2, mess3, mess4)` direkt übernommen werden, wenn kein Name explizit angegeben wird.

Wie wir sehen, sind die beiden Data Frames `df1` und `df2` nahezu identisch:

```
> str(df2)
'data.frame':  6 obs. of  5 variables:
 $ n    : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ mess1: num  3.66 4.67 5.72 6.72 7.64 ...
 $ mess2: num  3.78 4.74 5.67 6.78 7.67 ...
 $ mess3: num  3.72 4.72 5.67 6.69 7.7 ...
 $ mess4: num  3.77 4.69 5.62 6.61 7.75 ...
```

Sie unterscheiden sich lediglich in den Namen:

```
> names(df2)
[1] "n"     "mess1" "mess2" "mess3" "mess4"
```

Allerdings muss bei der Funktion `data.frames()` eine Besonderheit beachtet werden: in der Standardeinstellung werden `character`-Vektoren stets in Faktoren umgewandelt! Wir wollen hierzu folgendes Beispiel betrachten:

```
> bezeichnung <- paste("M", 1:6, sep="")
> str(bezeichnung)
 chr [1:6] "M1" "M2" "M3" "M4" "M5" "M6"
> df_a <- data.frame(n, bezeichnung, mess1)
> str(df_a)
'data.frame':  6 obs. of  3 variables:
 $ n          : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ bezeichnung: Factor w/ 6 levels "M1","M2","M3",..: 1 2 3 4 5 6
 $ mess1      : num  3.66 4.67 5.72 6.72 7.64 ...
```

In diesem Beispiel wird ein Data Frame erzeugt, bestehend aus den Variablen `n`, `bezeichnung` und `mess1`. Allerdings wurde der `character`-Vektor `bezeichnung` in einen Faktor umgewandelt! Dies kann durch das benannte Argument `stringsAsFactors` verhindert werden:

```
> df_b <- data.frame(n, bezeichnung, mess1, stringsAsFactors = FALSE)
> str(df_b)
'data.frame':  6 obs. of  3 variables:
 $ n          : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ bezeichnung: chr  "M1" "M2" "M3" "M4" ...
 $ mess1      : num  3.66 4.67 5.72 6.72 7.64 ...
```

5.3.2 Der Indexoperator

Da ein Data Frame eine Liste der Spaltenvariablen ist, können die einzelnen Daten wie bei einer Liste angesprochen werden. Am übersichtlichsten kann über die Namen zugegriffen werden:

```
> str(df1$n)
 int [1:6] 40000000 50000000 60000000 70000000 80000000 90000000
> str(df1$Messung.1)
 num [1:6] 3.66 4.67 5.72 6.72 7.64 ...
> df1$n[2]
[1] 50000000
```

Möglich ist es aber auch, den Indexoperator zu benutzen. Wie bei Listen gibt es auch hier einen gravierenden Unterschied zwischen dem einfachen und dem doppelten Indexoperator. Der einfache Indexoperator angewandt auf ein Data Frame gibt ein Data Frame zurück! Will man einzelne Elemente als Vektor erhalten, so muss ähnlich wie bei Listen der doppelte Indexoperator benutzt werden:

```
str(df1[1])
'data.frame':  6 obs. of  1 variable:
 $ n: int  40000000 50000000 60000000 70000000 80000000 90000000
> str(df1[[1]])
 int [1:6] 40000000 50000000 60000000 70000000 80000000 90000000
> str(df1["n"])
'data.frame':  6 obs. of  1 variable:
 $ n: int  40000000 50000000 60000000 70000000 80000000 90000000
> str(df1[["n"]])
 int [1:6] 40000000 50000000 60000000 70000000 80000000 90000000
```

Auf einzelne Tabellenwerte kann wie gewohnt zugegriffen werden:

```
> str(df1[2,1])
 int 50000000
> str(df1[1,2])
 num 3.66
```

Auf den ersten Blick überraschend ist, dass Zugriffe auf Zeilen und Spalten unterschiedliche Datentypen zurückgeben. Zeilen werden als einzelne Observationen aufgefasst, während Spalten sich auf den Datenvektor einer Variablen beziehen:

```
> str(df1[1,])  // 1. Zeile
'data.frame':  1 obs. of  5 variables:
 $ n        : int 40000000
 $ Messung.1: num 3.66
 $ Messung.2: num 3.78
 $ Messung.3: num 3.72
 $ Messung.4: num 3.77
> str(df1[,1])  // 1. Spalte
 int [1:6] 40000000 50000000 60000000 70000000 80000000 90000000
```

In unserem Beispiel ist `df1[-1]` der Teil des Data Frames, der die Messwerte erhält:

```
> str(df1[-1])
'data.frame':  6 obs. of  4 variables:
 $ Messung.1: num  3.66 4.67 5.72 6.72 7.64 ...
 $ Messung.2: num  3.78 4.74 5.67 6.78 7.67 ...
 $ Messung.3: num  3.72 4.72 5.67 6.69 7.7 ...
 $ Messung.4: num  3.77 4.69 5.62 6.61 7.75 ...
```

In unserem Fall hätten wir auch `df1[2:5]` statt `df1[-1]` schreiben können:

```
> identical(df1[-1], df1[2:5])
[1] TRUE
```

5.3.3 apply()

Mithilfe von `apply()` können wir die Mittelwerte der gemessenen Zeiten ermitteln. Allgemein kann `apply()` angewandt werden, wenn die beteiligten Spalten denselben Datentyp haben. Die Mittelwerte für die 6 Observationen erhalten wir durch[12]:

```
> apply(df1[-1],1, mean)
[1] 3.73050 4.70350 5.67200 6.70000 7.69200 8.83275
```

Für n = 40 000 000 ist also 3.73050 der Mittelwert der 4 Messwerte.

Die Mittelwerte für jede der vier Messvariablen erhalten wir hingegen durch:

```
> apply(df1[-1],2, mean)
Messung.1 Messung.2 Messung.3 Messung.4
 6.193000  6.240167  6.258333  6.195667
```

Da ein Date Frame eine Liste der Spaltenvariablen ist, kann für die letzte Abfrage auch `lapply()` oder besser noch `sapply()` benutzt werden:

```
> sapply(df1[-1], mean)
Messung.1 Messung.2 Messung.3 Messung.4
 6.193000  6.240167  6.258333  6.195667
```

5.3.4 Filtern, Einfügen und Löschen

Wie bei Matrizen und Listen kann man in Data Frames filtern:

```
> df2$mess1 > 6
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE
> df2$mess1 > df2$mess3
[1] FALSE FALSE  TRUE  TRUE FALSE FALSE

> df2[df2$mess1 > df2$mess3,]
         n mess1 mess2 mess3 mess4
3 60000000 5.719 5.672 5.672 5.625
4 70000000 6.720 6.782 6.688 6.610
```

Der letzte Funktionsaufruf kann sogar noch kürzer geschrieben werden:

```
> df2[mess1 > mess3,]
```

Ähnlich wie bei Matrizen hätte man alternativ auch die Funktion `subset()` benutzen können.

[12] Zur Erinnerung: bei `apply()` steht 1 für Zeilen und 2 für Spalten

Als Nächstes wollen wir eine Kopie von `df2` erstellen und eine neue Variable `diff1` zum Data Frame hinzufügen. Die neue Spaltenvariable soll gleich der Differenz der ersten beiden Messwerte sein:

```
> df3 <- df2
> df3$diff1 <- df2$mess2 -df2$mess1
> str(df3)
'data.frame':  6 obs. of  6 variables:
 $ n    : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ mess1: num  3.66 4.67 5.72 6.72 7.64 ...
 $ mess2: num  3.78 4.74 5.67 6.78 7.67 ...
 $ mess3: num  3.72 4.72 5.67 6.69 7.7 ...
 $ mess4: num  3.77 4.69 5.62 6.61 7.75 ...
 $ diff1: num  0.125 0.063 -0.047 0.062 0.032 ...
```

Wie bei Matrizen können auch `cbind()` und `rbind()` benutzt werden, um neuen Daten an jeder beliebigen Stelle einzufügen:

```
> Nummer <- 1:6
> df3 <- cbind(df3[1], Nummer, df3[-1])
> str(df3)
'data.frame':  6 obs. of  7 variables:
 $ n     : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ Nummer: int  1 2 3 4 5 6
 $ mess1 : num  3.66 4.67 5.72 6.72 7.64 ...
 $ mess2 : num  3.78 4.74 5.67 6.78 7.67 ...
 $ mess3 : num  3.72 4.72 5.67 6.69 7.7 ...
 $ mess4 : num  3.77 4.69 5.62 6.61 7.75 ...
 $ diff1 : num  0.125 0.063 -0.047 0.062 0.032 ...
```

Auffällig ist, dass der Name „Nummer“ der hinzugefügten Spalte automatisch im Data Frame übernommen wird! Durch `df3$diff1 <-NULL` wird die Spaltenvariable `diff1` wieder gelöscht:

```
> df3$diff1 <-NULL
> str(df3)
'data.frame':  6 obs. of  6 variables:
 $ n     : int  40000000 50000000 60000000 70000000 80000000 90000000
 $ Nummer: int  1 2 3 4 5 6
 $ mess1 : num  3.66 4.67 5.72 6.72 7.64 ...
 $ mess2 : num  3.78 4.74 5.67 6.78 7.67 ...
 $ mess3 : num  3.72 4.72 5.67 6.69 7.7 ...
 $ mess4 : num  3.77 4.69 5.62 6.61 7.75 ...
```

WICHTIG

- `data.frame()`
- `read.table(), read.csv(), read.csv2()`

Literaturverzeichnis

[Wickham_2019a] https://adv-r.hadley.nz/names-values.html, Hadley Wickham, *Advanced R*, (Zugriff 24.10.2019) (elektronische Version von [Wickham 2019b])

[Wickham_2019b] Wickham, Hadley: Advanced R, Second Edition. 2. Aufl.. Boca Raton, Fla: CRC Press, 2019.

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

TEIL 4

Inhaltsverzeichnis

6 DATENVISUALISIERUNG MIT GGPLOT2 2

6.1 Erste Beispiele 2
6.1.1 Beispiel: Ästhetische Eigenschaften (aesthetic) festlegen 5
6.1.2 Beispiel: Ästhetische Eigenschaften (aesthetic) für `geom_point()` 6

6.2 Statistische Transformation 7

7 DATA TRANSFORMATION 8

7.1 Der Datensatz `flights` 8

7.2 `dplyr`-Grundlagen 10
7.2.1 `filter()`: Zeilen auswählen 11
7.2.2 `arrange()`: nach Spalten sortieren 11
7.2.3 `select()`: Spalten auswählen 11
7.2.4 `mutate()`: neue Spalten erzeugen 12
7.2.5 `summarize()`: Spalten verdichten 12
7.2.6 Der Pipe-Operator `%>%` 13
7.2.7 Gruppierungen mit `group_by()` 14

8 EXPLORATORY DATA ANALYSIS (EDA) 18

8.1 Der Datensatz `diamonds` 18

6 Datenvisualisierung mit ggplot2

Parallel zu diesem Kapitel sollen Sie das Kapitel "3 Data visualisation"[1] aus *"R for Data Sciences"* durcharbeiten. *"R for Data Sciences"* gibt es in mindestens drei Versionen:

- https://r4ds.had.co.nz ist die Webseite zu dem Buch; im Folgenden werden wir diese Quelle mit [R4DS] bezeichnen.
- Wickham, Hadley and Grolemund, Garrett: *R for Data Science : Import, Tidy, Transform, Visualize, and Model Data*. Sebastopol: O'Reilly Media, Inc., 2016.
- Mittlerweile gibt es auch eine deutsche Übersetzung:
  Wickham, Hadley und Grolemund, Garrett : *R für Data Science : Daten importieren, bereinigen, umformen, modellieren und visualisieren.* Sebastopol: O'Reilly, 2017

Es gibt mehrere Möglichkeiten, Grafiken in R zu erstellen. Bis jetzt haben wir die im Base-R enthaltenen Funktionen wie `plot()`, `barplot()`, `boxplot()` und `hist()` benutzt, um einfache Grafiken zu erstellen. Das `ggplot2`-Package[2] bietet darüber hinaus viele interessante Möglichkeiten, mit relativ wenig Aufwand aussagekräftige Grafiken zu erstellen. Nicht umsonst ist es mittlerweile weit verbreitet. Allerdings unterstützt `ggplot2` keine dreidimensionalen Grafiken. Wie wir sehen werden, ist die `ggplot2` zugrunde liegende Idee, die einzelnen Komponenten einer Grafik durch eine spezielle Grammatik zu beschreiben.[3]

6.1 Erste Beispiele

Das Data Frame `mpg` ist Teil des `ggplot2`-Packages (siehe `?mpg`). Es besteht aus 234 Datensätzen mit jeweils 11 Variablen. Es enthält u. a. Informationen über den Energieverbrauch von unterschiedlichen Automodellen aus den Jahren 1999 und 2008, die von der US Environment Protection Agency gesammelt wurden.[4]

[1] https://r4ds.had.co.nz/data-visualisation.html
[2] https://cran.r-project.org/web/packages/ggplot2/index.html
[3] https://vita.had.co.nz/papers/layered-grammar.html
[4] siehe http://fueleconomy.gov

```
> library(ggplot2)
> str(mpg)
Classes ‘tbl_df’, ‘tbl’ and 'data.frame': 234 obs. of  11 variables:
 $ manufacturer: chr  "audi" "audi" "audi" "audi" ...
 $ model       : chr  "a4" "a4" "a4" "a4" ...
 $ displ       : num  1.8 1.8 2 2 2.8 2.8 3.1 1.8 1.8 2 ...
 $ year        : int  1999 1999 2008 2008 1999 1999 2008 1999 1999 2008 ...
 $ cyl         : int  4 4 4 4 6 6 6 4 4 4 ...
 $ trans       : chr  "auto(l5)" "manual(m5)" "manual(m6)" "auto(av)" ...
 $ drv         : chr  "f" "f" "f" "f" ...
 $ cty         : int  18 21 20 21 16 18 18 18 16 20 ...
 $ hwy         : int  29 29 31 30 26 26 27 26 25 28 ...
 $ fl          : chr  "p" "p" "p" "p" ...
 $ class       : chr  "compact" "compact" "compact" "compact" ...
```

manufacturer	Manufacturer/brand name	Hersteller
model	Car model name	Modell-Name
displ	Engine displacement, in litres	Hubraum in Litern
year	Year of manufacture	Baujahr
cyl	Number of cylinders	Anzahl der Zylinder
trans	Type of transmission	Schaltung (Automatik / Anzahl der Gänge)
drv	f = front-wheel drive, r = rear wheel drive, 4 = 4wd	Antrieb
cty	City miles per gallon	Verbrauch: Stadt
hwy	Highway miles per gallon	Verbrauch: Autobahn
fl	Fuel type. e = ethanol, d = diesel, r = regular, p =premium, c = natural gas	Antriebsart
class	The 'type' of car	Fahrzeugtyp

Tabelle 6-1: Variablen des Datensatzes mpg

Umrechnung: miles per gallon (mpg) nach Liter pro 100 km

10 mpg	entsprechen 23,5	l /100 km	15 mpg	entsprechen 15,7	l /100 km
20 mpg	entsprechen 11,8	l /100 km	25 mpg	entsprechen 9,4	l /100 km
30 mpg	entsprechen 7,8	l /100 km	35 mpg	entsprechen 6,7	l /100 km

Die Umrechnung von Miles per Gallon (mpg) nach Liter pro 100 km kann z. B. durch eine Funktion geschehen:

```
# Umrechnung von Miles per Gallon (mpg) nach Liter pro 100 km
literPer100km <- function(mpg) {
  return(round(235.195 / mpg, digits = 1))
}
```

Wir erhalten dann die obigen Werte:

```
> fuel.mpg <- seq(10,40,by=5)
> fuel.mpg
[1] 10 15 20 25 30 35 40
> literPer100km(fuel.mpg)
[1] 23.5 15.7 11.8  9.4  7.8  6.7  5.9
```

Wie wir sehen, hat der Datensatz `mpg` einigermaßen vernünftige „sprechende“ Bezeichnungen für die verschiedenen Variablen (Spaltennamen)[5]; ebenso haben die nicht nominal skalierten Daten aussagekräftige Werte. Dies wird uns bei der Erstellung der unterschiedlichen Plots helfen.

Der erste Plot in Kapitel 3 in [R4DS] wurde erzeugt durch:

```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy))
```

Es geht mithilfe der `qplot`-Funktion sogar noch kürzer:

```
qplot(displ, hwy, data = mpg)
```

Allerdings bezeichnet selbst Hadley Wickham, der `ggplot2` entwickelt hat, die Funktion `qplot` als „Krücke“, die zwar in vielen Situationen vernünftige Defaultwerte benutzt, empfiehlt aber, das konzeptionell übersichtlichere `ggplot` für komplizierte Grafiken zu benutzen.

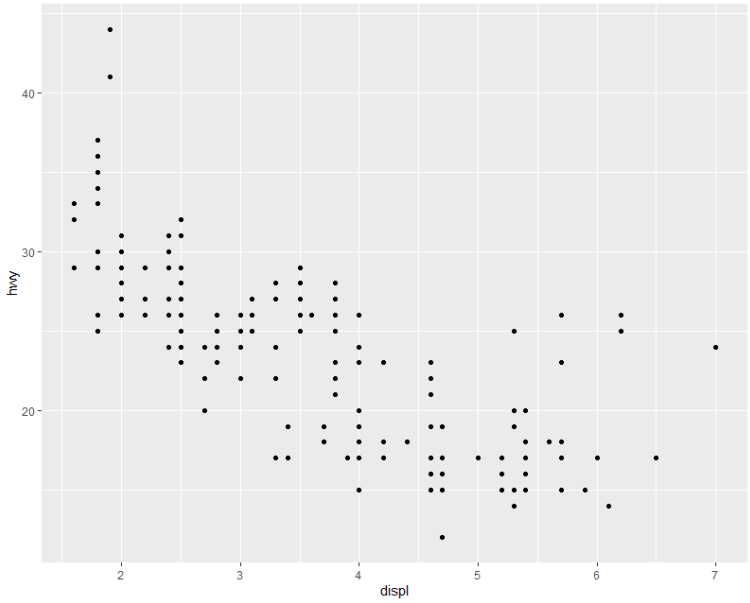


Abbildung 1: Der erste Plot: Hubraum versus Verbrauch

Wir gehen vom Folgenden aus:

- Die zugehörigen Daten liegen immer in Form eines Data Frame vor
- Eine Grafik besteht aus mehreren Ebenen
- Ein „***Geom***“ ist ein geometrisches Objekt, das die Erscheinungsform festlegt. Beispiele:
  - Balkendiagramm: `geom_bar()`
    (gut geeignet für die Visualisierung der Werte einer diskreten Variable)
  - Histogramm: `geom_histogram()`
    (gut geeignet für die Visualisierung der Werte einer kontinuierlichen Variable)
  - Boxplot: `geom_boxplot()`
    (gut geeignet für die Visualisierung des Zusammenhangs einer diskreten und einer kontinuierlichen Variable)
  - x-y-Plot (scatterplot): `geom_point()`
    (gut geeignet für die Visualisierung des Zusammenhangs zweier kontinuierlicher Variable)

[5] für Europäer manchmal vielleicht zu sehr abgekürzt

6.1.1 Beispiel: Ästhetische Eigenschaften (aesthetic) festlegen

Ästhetische Eigenschaften wie `colour` oder `size` können direkt mithilfe eines festen Werts festgelegt werden (und zwar außerhalb der Funktion `aes()`):

```
### Alle Punkte blau
ggplot(data = mpg) +                                # outside of aes()
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

Sie können aber auch an eine Variable gebunden werden und nehmen dann abhängig von diesem Variablenwert unterschiedliche Werte an. Dieses „Binden“ geschieht *innerhalb* der Funktion `aes()`, dessen Rückgabewert mithilfe des Arguments `mapping` an das Geom-Objekt übergeben wird:

```
ggplot(data = mpg) +                                 # inside aes()
  geom_point(mapping = aes(x = displ, y = hwy, color = year))
```

`x` und `y` sind ebenfalls ästhetische Eigenschaften; da sie (in der Regel) an Variablen gebunden werden, stehen sie somit stets innerhalb der Funktion `aes()`.

Im dem folgenden erzeugten Plot sind die Punkte zwar einfarbig, aber nicht blau! „blue“ wird hier als kategoriale Variable interpretiert, die nur einen Wert annimmt und in der erstellten Legende angezeigt wird:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = "blue"))
```

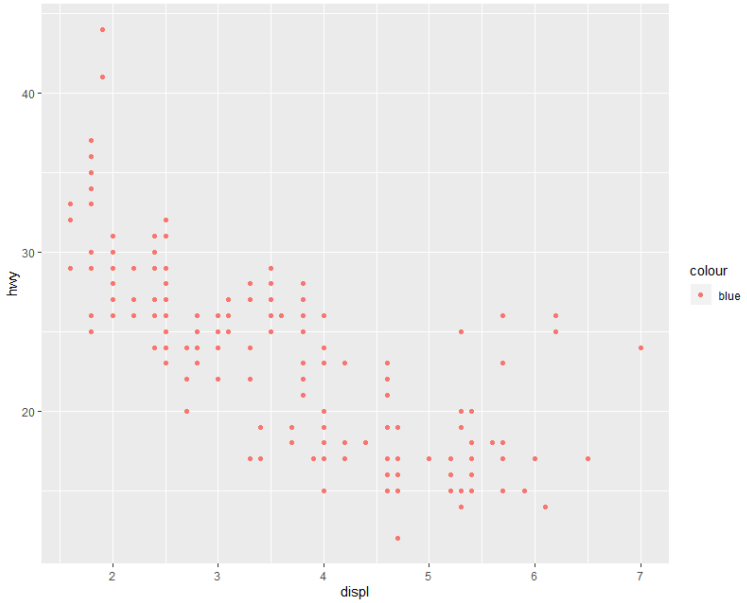


Abbildung 2: Unerwartete Farbe

Entsprechend erhält man durch den folgenden Plot eine Vielzahl von Farbtönen und eine entsprechende Legende:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = 1:234))
```

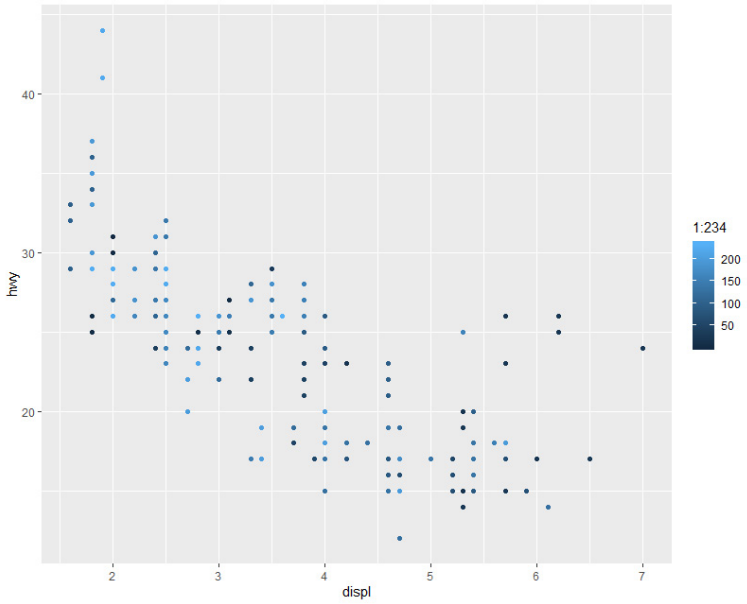


Abbildung 3: Vielzahl von Farben

6.1.2 Beispiel: Ästhetische Eigenschaften (aesthetic) für `geom_point()`

`geom_point()` versteht folgende ästhetische Eigenschaften (`x` und `y` müssen immer gesetzt werden):

`x, y, alpha, colour, fill, group, shape, size, stroke`

Beispielsweise erzeugt:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy),
      size = 3, stroke = 1.5, shape = 21, fill = "blue", color ="orange")
```

den folgenden Plot:

Abbildung 4: `fill` und `color`

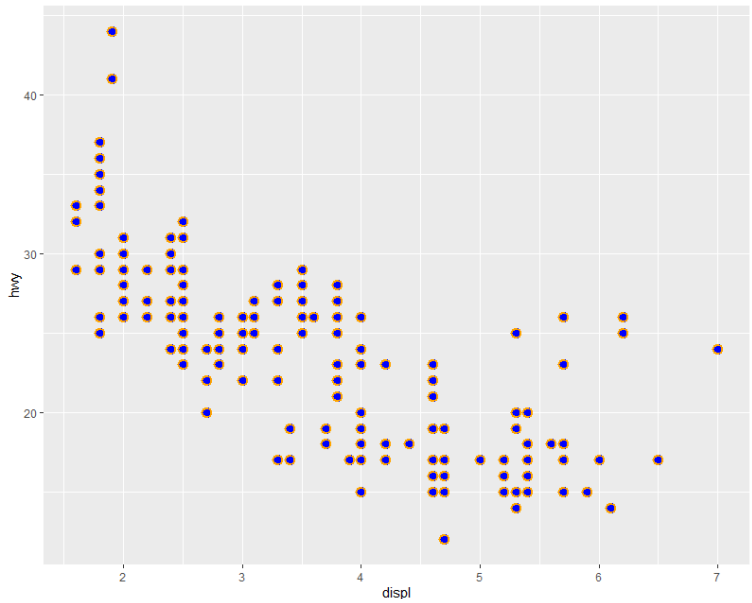


Abbildung 5: `fill` und color

6.2 Statistische Transformation

Bevor Daten visualisiert werden, müssen sie in der Regel transformiert und verdichtet werden. Dazu dienen `stat`-Objekte. Jedes `geom`-Objekt ist einem Standard-`stat`-Objekt zugeordnet und umgekehrt. In der folgenden Tabelle werden die häufigsten `geom-stat-` Beziehungen aufgelistet:

geom	stat
geom_bar()	stat_count()
geom_bin2d()	stat_bin_2d()
geom_boxplot()	stat_boxplot()
geom_contour()	stat_contour()
geom_count()	stat_sum()
geom_density()	stat_density()
geom_density_2d()	stat_density_2d()
geom_hex()	stat_hex()
geom_freqpoly()	stat_bin()
geom_histogram()	stat_bin()
geom_qq_line()	stat_qq_line()
geom_qq()	stat_qq()
geom_quantile()	stat_quantile()
geom_smooth()	stat_smooth()
geom_violin()	stat_violin()
geom_sf()	stat_sf()

Tabelle 6-2: Gebräuchliche `geom-stat-` Beziehungen[6]

[6] Entnommen aus https://jrnold.github.io/r4ds-exercise-solutions/data-visualisation.html#statistical-transformations (Stand 27.11.19)

7 Data Transformation

Parallel zu diesem Kapitel sollen Sie das Kapitel „5 Data transformation“[7] aus *„R for Data Sciences“* [R4DS] durcharbeiten, das sich mit dem `dplyr`[8]-Paket auseinandersetzt, einer weiteren Kernkomponente der `tidyverse`-Welt.

Im Folgenden setzen wir voraus:

```
library(tidyverse)
```

Dadurch werden die folgenden Pakete (packages) geladen:[9]

`ggplot2` `purrr` `tibble` `dplyr` `tidyr` `stringr` `readr` `forcats`

In diesem Kapitel wollen wir mit der großen Datensammlung `flights` arbeiten, die im `nycflights13`-Paket enthalten ist; daher setzen wir zusätzlich voraus:

```
library(nycflights13)
```

7.1 Der Datensatz `flights`

Der Datensatz `flights` beschreibt alle Flüge, die 2013 von einem der drei New Yorker Flughäfen (JFK, LGA or EWR) gestartet sind.[10]

```
> str(flights)
Classes ‘tbl_df’, ‘tbl’ and 'data.frame':   336776 obs. of  19 variables:
 $ year          : int  2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
 $ month         : int  1 1 1 1 1 1 1 1 1 1 ...
 $ day           : int  1 1 1 1 1 1 1 1 1 1 ...
 $ dep_time      : int  517 533 542 544 554 554 555 557 557 558 ...
 $ sched_dep_time: int  515 529 540 545 600 558 600 600 600 600 ...
 $ dep_delay     : num  2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
 $ arr_time      : int  830 850 923 1004 812 740 913 709 838 753 ...
 $ sched_arr_time: int  819 830 850 1022 837 728 854 723 846 745 ...
 $ arr_delay     : num  11 20 33 -18 -25 12 19 -14 -8 8 ...
 $ carrier       : chr  "UA" "UA" "AA" "B6" ...
 $ flight        : int  1545 1714 1141 725 461 1696 507 5708 79 301 ...
 $ tailnum       : chr  "N14228" "N24211" "N619AA" "N804JB" ...
 $ origin        : chr  "EWR" "LGA" "JFK" "JFK" ...
 $ dest          : chr  "IAH" "IAH" "MIA" "BQN" ...
 $ air_time      : num  227 227 160 183 116 150 158 53 140 138 ...
 $ distance      : num  1400 1416 1089 1576 762 ...
 $ hour          : num  5 5 5 5 6 5 6 6 6 6 ...
 $ minute        : num  15 29 40 45 0 58 0 0 0 0 ...
 $ time_hour     : POSIXct, format: "2013-01-01 05:00:00" "2013-01-01 05:00:00" "201
3-01-01 05:00:00" ...
```

[7] https://r4ds.had.co.nz/transform.html
[8] dplyr steht für “**d**ata frame ap**ply**e**r**”
[9] In der Version: tidyverse 1.2.1
[10] Siehe: ?flights

Der Datensatz `flights` ist ein „*Tibble*“. Ein Tibble ist ein leicht angepasster Data Frame, der besser mit `tidyverse` zusammenarbeitet. Im Vergleich zu einem Data Frame ist bei einem Tibble die Ausgabe eines großen Datensatzes viel besser gelöst. Auch ist der Zugriff auf Teilbereiche der Daten mit Tibbles konsistenter und strenger: mehr Warnungen werden ausgegeben, wenn angesprochene Spalten nicht existieren.[11] Im Gegensatz zu einem Data Frame wird bei der Erzeugung eines Tibbles nicht automatisch ein `character`-Vektor in einen Faktor umgewandelt.

Mithilfe von `as_tibble()` kann ein Data Frame leicht in einen Tibble umgewandelt werden. Ein Tibble kann auch direkt mit `tibble()` erzeugt werden. Auf Tibbles werden wir später zurückkommen.

Die Ausgabe des Tibbles `flights` ist in der Tat recht übersichtlich:

```
> flights
# A tibble: 336,776 x 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay carrier
   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl> <chr>
 1  2013     1     1      517            515         2      830            819        11 UA
 2  2013     1     1      533            529         4      850            830        20 UA
 3  2013     1     1      542            540         2      923            850        33 AA
 4  2013     1     1      544            545        -1     1004           1022       -18 B6
 5  2013     1     1      554            600        -6      812            837       -25 DL
 6  2013     1     1      554            558        -4      740            728        12 UA
 7  2013     1     1      555            600        -5      913            854        19 B6
 8  2013     1     1      557            600        -3      709            723       -14 EV
 9  2013     1     1      557            600        -3      838            846        -8 B6
10  2013     1     1      558            600        -2      753            745         8 AA
# ... with 336,766 more rows, and 9 more variables: flight <int>, tailnum <chr>, origin <chr>,
#   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dttm>
```

`flights` beschreibt also 336 776 Flüge mithilfe 19 Variablen.

`year, month, day`	Date of departure.
`dep_time, arr_time`	Actual departure and arrival times (format HHMM or HMM), local tz.
`sched_dep_time, sched_arr_time`	Scheduled departure and arrival times (format HHMM or HMM), local tz.
`dep_delay, arr_delay`	Departure and arrival delays, in minutes. Negative times represent early departures/arrivals.
`carrier`	Two letter carrier abbreviation. See airlines to get name.
`flight`	Flight number.
`tailnum`	Plane tail number. See planes for additional metadata.
`origin, dest`	Origin and destination. See airports for additional metadata.
`air_time`	Amount of time spent in the air, in minutes.
`distance`	Distance between airports, in miles.
`hour, minute`	Time of scheduled departure broken into hour and minutes.
`time_hour`	Scheduled date and hour of the flight as a POSIXct date. Along with origin, can be used to join flights data to weather data.

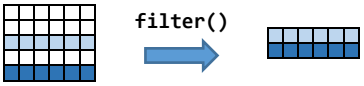
Tabelle 7-1: Variablen des Datensatzes `flights`

[11] „A concise, and fun, way to summarise the main differences is that tibbles are lazy and surly: they do less and complain more. You'll see what that means as you work through this section.“ aus „Advanced R“, Hadley Wickham. Quelle: https://adv-r.hadley.nz/vectors-chap.html#tibble (Zugriff 23.11.2019)

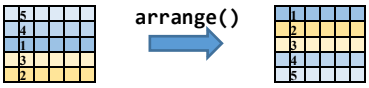
7.2 dplyr-Grundlagen

In `dplyr` gibt es fünf Schlüsselfunktionen[12], die jeweils ein Tibble (bzw. ein Data Frame) als erstes Element erwarten und als Resultat ein Tibble (bzw. ein Data Frame) zurückgeben.

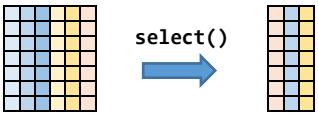
`filter()`	Beobachtungen (Zeilen) gemäß Filterbedingung auswählen



`arrange()`	Nach Spalten sortieren

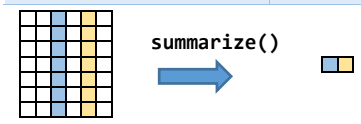


`select()`	Variablen (Spalten) nach Namen auswählen



`mutate()`	Neue Variablen aus bestehenden Variablen erzeugen

`summarize()`	Spalten verdichten

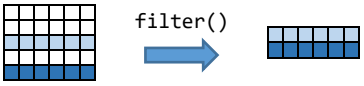


Daneben gibt es noch Variationen dieser fünf Schlüsselfunktionen wie `rename()` oder `transmute()`. Viele dieser Funktionen berücksichtigen Gruppierungen, die mit der Funktion `group_by()` erstellt werden können. Anstatt auf dem gesamten Datensatz zu arbeiten, operieren sie dann auf den erstellten Gruppen. Dies wird insbesondere mit der Funktion `summarize()` häufig ausgenutzt. Wir werden hierauf weiter unten eingehen.

[12] In [R4DS] werden diese Schlüsselfunktionen auch als „*Verben*“ bezeichnet

7.2.1 `filter()`: Zeilen auswählen

`filter()`	Beobachtungen (Zeilen) gemäß Filterbedingung auswählen



Mit `filter()` können gemäß einer Filterbedingung bestimmte Zeilen ausgewählt werden. Mehrere Filterbedingungen werden dabei logisch mit „Und“ (also mit `&`) verknüpft.

`filter()` gibt:

- komplette Datenzeilen zurück
- nur Datenzeilen zurück, bei denen die Filterbedingung wahr ist; NA-Werte werden ausgeschlossen, es sei denn, sie tauchen explizit in der Filterbedingung auf.

Beispiele

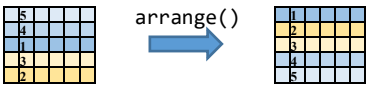
```
filter(flights, month == 1, day == 1)
filter(flights, month == 11 | month == 12)
filter(flights, month %in% c(11, 12))
```

Hinweis[13]:

Der Operator `&&` bezieht sich bei Vektoren nur auf das erste Element; bei Vektoren sollte man statt `&&` den Operator `&` benutzen! Ähnliches gilt für die Operatoren `||` und `|` (siehe z. B. `?'&'`).

7.2.2 `arrange()`: nach Spalten sortieren

`arrange()`	Nach Spalten sortieren



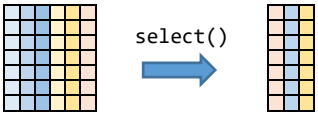
Mit `arrange()` kann nach Spalten sortiert werden. Mithilfe der Funktion `desc()`kann in absteigender Reihenfolge sortiert werden.

Beispiele

```
arrange(flights, year, month, day)
arrange(flights, desc(dep_delay))
```

7.2.3 `select()`: Spalten auswählen

`select()`	Variablen (Spalten) nach Namen auswählen



Mit `select()` wird der Datensatz reduziert, indem nur bestimmte Spalten ausgewählt werden.

`starts_with("abc")`	sucht nach Namen, die mit „abc“ beginnen
`ends_with("xyz")`	sucht nach Namen, die mit „xyz“ beginnen
`contains("ijk")`	sucht nach Namen, die „ijk“ enthalten
`everything()`	Alle Variablen

Tabelle 7-2: Nützliche Hilfsfunktionen

Um Spaltennamen zu ändern, sollte die Funktion `rename()` benutzt werden.

[13] Dies ist der Hinweis aus den Übungen Kapitel 3:

Beispiele

```
select(flights, year, month, day)
select(flights, year:day)
select(flights, -(year:day))
select(flights, year, month, day)
select(flights, year:day)
select(flights, -(year:day))
select(flights, starts_with("dep"))
select(flights, ends_with("time"))
select(flights, contains("dep"))
select(flights, time_hour, air_time, everything())
```

7.2.4 `mutate()`: neue Spalten erzeugen

`mutate()`	Neue Variablen aus bestehenden Variablen erzeugen

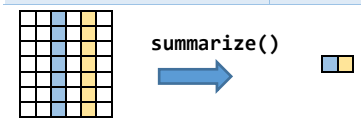
Mit `mutate()` wird der Datensatz durch neue Spalten erweitert, die aus bestehenden Variablen erzeugt werden. Bei Namensgleichheit werden bereits existierende Variablen überschrieben. `transmute()` lässt zusätzlich alle nicht genannten Spalten weg.

Beispiele

```
(flights_sml <- select(flights, year:day, ends_with("delay"), distance, air_time))
mutate(flights_sml, gain = dep_delay - arr_delay, speed = distance / air_time * 60)
mutate(flights_sml, gain = dep_delay - arr_delay, hours = air_time / 60,
       gain_per_hour = gain / hours)
transmute(flights, year, gain = dep_delay - arr_delay,hours = air_time / 60,
        gain_per_hour = gain / hours)
```

7.2.5 `summarize()`: Spalten verdichten

`summarize()`	Spaltenwerte verdichten



Mit `summarize()` kann ein Datensatz auf eine einzelne Zeile reduziert werden, indem ausgewählte Spalten verdichtet werden. `summarise()` und `summarize()` sind Synonyme.

Beispiele

```
summarize(flights, delay = mean(dep_delay, na.rm = TRUE))
summarize(flights, delay = mean(dep_delay, na.rm = TRUE),
               sd = sd(dep_delay, na.rm = TRUE))
```

`summarize()` wird sehr häufig im Zusammenspiel mit `group_by()` verwendet. Wir werden weiter untern hierauf detaillierter eingehen.

7.2.6 Der Pipe-Operator %>%

Der Pipe-Operator `%>%`[14] ist nicht Bestandteil von Base-R; er wird durch das `tidyverse`-Paket automatisch eingeführt und spielt in der `tidyverse`-Welt eine wichtige Rolle.

Allgemein gilt, dass

```
x %>% f(y)
```

äquivalent ist zu

```
f(x, y)
```

Folglich entspricht:

```
x %>% f(y) %>% g(z)
```

dem Ausdruck:

```
g(f(x, y), z)
```

Als Beispiel wollen wir die Ausgabe der Funktion `sd()` auf drei verschiedene Arten nachrechnen. Dazu wollen wir die folgenden drei Funktionen nutzen:

```
potenz <- function(x,n) return(x^n)
zentriere <- function(x) return(x - mean(x))
mittel <- function(x) return( sum(x)/ (length(x)-1)) # Precondion: length(x) > 1
```

Es gilt:

```
> d <- 1:10
> sd(d)
[1] 3.02765
```

In unserer ersten Variante rufen wir die Funktionen geschachtelt auf:

```
> sqrt(mittel(potenz(zentriere(d),2)))
[1] 3.02765
```

In der zweiten Variante gehen wir schrittweise vor:

```
> res <- zentriere(d)
> res <- potenz(res,2)
> res <- mittel(res)
> res <- sqrt(res)
> res
[1] 3.02765
```

[14] In Visual Studio R fügt Strg+Shift+m den %>% ein

In der dritten Variante nutzen wir schließlich den Pipe-Mechanismus[15] (er setzt allerdings voraus, dass Base-R durch ein Paket wie `tidyverse` erweitert worden ist)[16]:

```
d %>%
  zentriere() %>%
  potenz(2)  %>%
  mittel() %>%
  sqrt()
[1] 3.02765
```

In komplexen Situationen kann der Pipe-Mechanismus die Lesbarkeit des Quelltextes deutlich erhöhen. Insbesondere in der `tidyverse`-Welt wird hiervon viel Gebrauch gemacht. Eine Ausnahme bildet `ggplot2`: es benutzt das gleiche Prinzip, nur wird dort leider `+` statt `%>%` benutzt!

7.2.7 Gruppierungen mit `group_by()`

Die Funktion `group_by()` und ihr Gegenstück `ungroup()` ist für sich aus gesehen nicht besonders nützlich. Erst im Zusammenspiel mit Funktionen wie `summarize()`, die die erstellten Gruppierungen berücksichtigen, entfalten sie ihr Potential. Wir wollen dies an einem Beispiel verdeutlichen. Aus Gründen der Übersichtlichkeit wollen wir hierbei nur einen Teil des Datensatzes `flights` benutzen:

```
> fluege <- select(flights, month, day, contains("delay"))
> fluege <- filter(fluege, !is.na(arr_delay), !is.na(dep_delay))
> fluege
# A tibble: 327,346 x 4
   month   day dep_delay arr_delay
   <int> <int>     <dbl>     <dbl>
 1     1     1         2        11
 2     1     1         4        20
 3     1     1         2        33
 4     1     1        -1       -18
 5     1     1        -6       -25
 6     1     1        -4        12
 7     1     1        -5        19
 8     1     1        -3       -14
 9     1     1        -3        -8
10     1     1        -2         8
# ... with 327,336 more rows
```

Durch die beiden Filterbedingungen `!is.na(arr_delay)` und `!is.na(dep_delay)` schließen wir NA-Werte bei den Verspätungen aus.

Mithilfe des Pipe-Operators können wir stattdessen auch alternativ schreiben:

```
fluege <- flights %>%
  select(month, day, contains("delay")) %>%
  filter(!is.na(arr_delay), !is.na(dep_delay))
```

[15] Wie `+` darf `%>%` nicht am Anfang einer Zeile stehen

[16] Wir hätten dies auch durch die Anweisung `require(tidyverse)` verdeutlichen können. Ursprünglich stammt der Pipe-Mechanismus aus dem Paket `magrittr`.

Als Nächstes wollen wir die Funktion `group_by()` auf `fluege` anwenden:

```
> fluege_month <- group_by(fluege, month)
> fluege_month
# A tibble: 327,346 x 4
# Groups:   month [12]
   month   day dep_delay arr_delay
   <int> <int>     <dbl>     <dbl>
 1     1     1         2        11
 2     1     1         4        20
 3     1     1         2        33
 4     1     1        -1       -18
 5     1     1        -6       -25
 6     1     1        -4        12
 7     1     1        -5        19
 8     1     1        -3       -14
 9     1     1        -3        -8
10     1     1        -2         8
# ... with 327,336 more rows
```

Wie man sieht, ist jetzt eine Gruppierung hinzugekommen, bestehend aus 12 Gruppen. Funktionen wie `summarize()` berücksichtigen diese Gruppierung:

```
> summarize(fluege, delay = mean(arr_delay))
# A tibble: 1 x 1
  delay
  <dbl>
1  6.90
> summarize(fluege_month, delay = mean(arr_delay))
# A tibble: 12 x 2
   month  delay
   <int>  <dbl>
 1     1  6.13
 2     2  5.61
 3     3  5.81
 4     4 11.2
 5     5  3.52
 6     6 16.5
 7     7 16.7
 8     8  6.04
 9     9 -4.02
10    10 -0.167
11    11  0.461
12    12 14.9
```

`summarize()` berücksichtigt die Gruppierung in `fluege_month`: die Mittelwerte werden nun pro Monat berechnet. Da hingegen `fluege` keine Gruppierung besitzt, wird im ersten Aufruf `summarize(fluege, delay = mean(arr_delay))` der Mittelwert über alle Daten berechnet.

Mit `ungroup()` kann die Gruppierung wieder rückgängig gemacht werden:

```
> summarize(ungroup(fluege_month), delay = mean(arr_delay))
# A tibble: 1 x 1
  delay
  <dbl>
1  6.90
```

Möglich sind auch Gruppierungen nach mehreren Variablen. Dies zeigt das nächste Beispiel:

```
> fluege_monthDay <- group_by(fluege, month, day)
> summarize(fluege_monthDay, delay = mean(dep_delay))
# A tibble: 365 x 3
# Groups:   month [12]
   month   day delay
   <int> <int> <dbl>
 1     1     1 11.4
 2     1     2 13.7
 3     1     3 10.9
 4     1     4  8.97
 5     1     5  5.73
 6     1     6  7.15
 7     1     7  5.42
 8     1     8  2.56
 9     1     9  2.30
10     1    10  2.84
# ... with 355 more rows
```

Gruppierungen sind in Verbindung mit `summarize()` sehr nützlich, werden oft auch bei `mutate()` and `filter()` eingesetzt. Im nächsten Beispiel wird zu `fluege_month` eine zusätzliche Variable hinzugefügt, die in jeder Zeile die entsprechende monatliche durchschnittliche Verspätung bei der Ankunft angibt (vgl. `summarize(fluege_month, delay = mean(arr_delay))`, siehe oben):

```
> fluege_month  %>%
+   mutate(m_arr_delay = mean(arr_delay))
# A tibble: 327,346 x 5
# Groups:   month [12]
   month   day dep_delay arr_delay m_arr_delay
   <int> <int>     <dbl>     <dbl>       <dbl>
 1     1     1         2        11        6.13
 2     1     1         4        20        6.13
 3     1     1         2        33        6.13
 4     1     1        -1       -18        6.13
 5     1     1        -6       -25        6.13
 6     1     1        -4        12        6.13
 7     1     1        -5        19        6.13
 8     1     1        -3       -14        6.13
 9     1     1        -3        -8        6.13
10     1     1        -2         8        6.13
# ... with 327,336 more rows
```

Entsprechend können wir die Gruppierung in Verbindung mit `filter()` nutzen:

```
> fluege_month  %>%
+   filter(mean(arr_delay) > 10)
# A tibble: 109,952 x 4
# Groups:   month [4]
   month   day dep_delay arr_delay
   <int> <int>     <dbl>     <dbl>
 1    12     1        14         1
 2    12     1        18         6
 3    12     1        -7       -15
 4    12     1         5       -19
 5    12     1        -4        -5
 6    12     1       -10       -22
 7    12     1        -4       -21
 8    12     1         1        -9
 9    12     1       -11       -11
10    12     1       -10       -29
# ... with 109,942 more rows
```

Zum Test können wir die Anzahl der Zeilen in der jeweiligen Gruppe bestimmen und die ausgegebenen Monate mit `summarize(fluege_month, delay = mean(arr_delay))` vergleichen:

```
> fluege_month  %>%
+   filter(mean(arr_delay) > 10) %>%
+   summarize(count = n())
# A tibble: 4 x 2
  month count
  <int> <int>
1     4 27564
2     6 27075
3     7 28293
4    12 27020
```

Die spezielle Funktion `n()` kann nur in den Funktionen `summarize()`, `mutate()` und `filter()` benutzt werden. Sie gibt die Anzahl der Observationen in der jeweiligen Gruppe zurück.

8 Exploratory Data Analysis (EDA)

Parallel zu diesem Kapitel sollen Sie das Kapitel „7 Exploratory Data Analysis“[17] aus *„R for Data Sciences“* [R4DS] durcharbeiten.

Die Inhalte dieses Kapitels werden wir in den Übungen und in einer weiteren Fallstudie aufgreifen und vertiefen.

Im Folgenden setzen wir voraus:

```
library(tidyverse)
```

8.1 Der Datensatz diamonds

Ebenso wie `mpg` ist `diamonds` Teil des `ggplot2`-Packages (siehe `?diamonds`):

```
> diamonds
# A tibble: 53,940 x 10
   carat cut       color clarity depth table price     x     y     z
   <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
 1 0.23  Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
 2 0.21  Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
 3 0.23  Good      E     VS1      56.9    65   327  4.05  4.07  2.31
 4 0.290 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
 5 0.31  Good      J     SI2      63.3    58   335  4.34  4.35  2.75
 6 0.24  Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
 7 0.24  Very Good I     VVS1     62.3    57   336  3.95  3.98  2.47
 8 0.26  Very Good H     SI1      61.9    55   337  4.07  4.11  2.53
 9 0.22  Fair      E     VS2      65.1    61   337  3.87  3.78  2.49
10 0.23  Very Good H     VS1      59.4    61   338  4     4.05  2.39
# ... with 53,930 more rows
```

Der Datensatz `diamonds` ist ein „*Tibble*“, der annähernd 54,000 Diamanten beschreibt. Jede dieser Observation besteht aus 10 Variablen, die unter anderem Informationen über Preis, Karat, Farbe, Reinheit und Schliff enthalten:

`price`	Preis	price in US dollars ($326–$18,823)
`carat`	Karatgewicht	weight of the diamond (0.2–5.01)
`cut`	Schliff	quality of the cut (Fair, Good, Very Good, Premium, Ideal)
`color`	Farbe	diamond colour, from D (best) to J (worst)
`clarity`	Reinheit	a measurement of how clear the diamond is (I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best))
`x`		length in mm (0–10.74)
`y`		width in mm (0–58.9)
`z`		depth in mm (0–31.8)
`depth`		total depth percentage = z / mean(x, y) = 2 * z / (x + y) (43–79)
`table`		width of top of diamond relative to widest point (43–95)

Tabelle 8-1: Variablen des Datensatzes `diamonds`

[17] https://r4ds.had.co.nz/exploratory-data-analysis.html

Programmierung für statistische Datenanalyse
Spezielle Themen und Fallstudien 1

1 Boxplots

Inhaltliche Voraussetzung: Boxplot, unteres Quartil, Median, oberes Quartil

Führen Sie folgenden Quelltext aus:

```
anzahl <- 8
op <- par(mfrow=c(2,anzahl/2), col="blue") # Grafikeinstellung ändern
n <- 100L
x <- 0:n
for (i in 0:(anzahl-1)){
  x[n] <- x[n]+10*i
  x[n-1] <- x[n-1]+5*i
  print(paste("i = ",i, ":"))
  print(summary(x))
  boxplot(x, main = paste("i = ",i),
        xlab = paste("x[n-1] =", x[n-1], ", x[n] =", x[n]))
}
par(op)  # stellt die ursprüngliche Grafikeinstellung wieder her
```

Erklärungen zum Quelltext:

- Durch `par()` können Grafikoptionen gesetzt werden. In unserem Fall werden die nächsten 8 Bilder in einem Plot bestehend aus 2×4 Einzelbildern dargestellt.
- `paste()` konvertiert die übergebenen Argumente zu Zeichenketten (charakter-Vektoren) und fügt sie zusammen.

Ausgegeben wird:

```
[1] "i =  0 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      0      25      50      50      75     100
[1] "i =  1 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   50.15   75.00  109.00
[1] "i =  2 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   50.45   75.00  129.00
[1] "i =  3 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   50.89   75.00  159.00
[1] "i =  4 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   51.49   75.00  199.00
```

```
[1] "i =  5 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   52.23   75.00  249.00
[1] "i =  6 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   53.12   75.00  309.00
[1] "i =  7 :"
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00   25.00   50.00   54.16   75.00  379.00
```

Sie erhalten folgenden Plot:

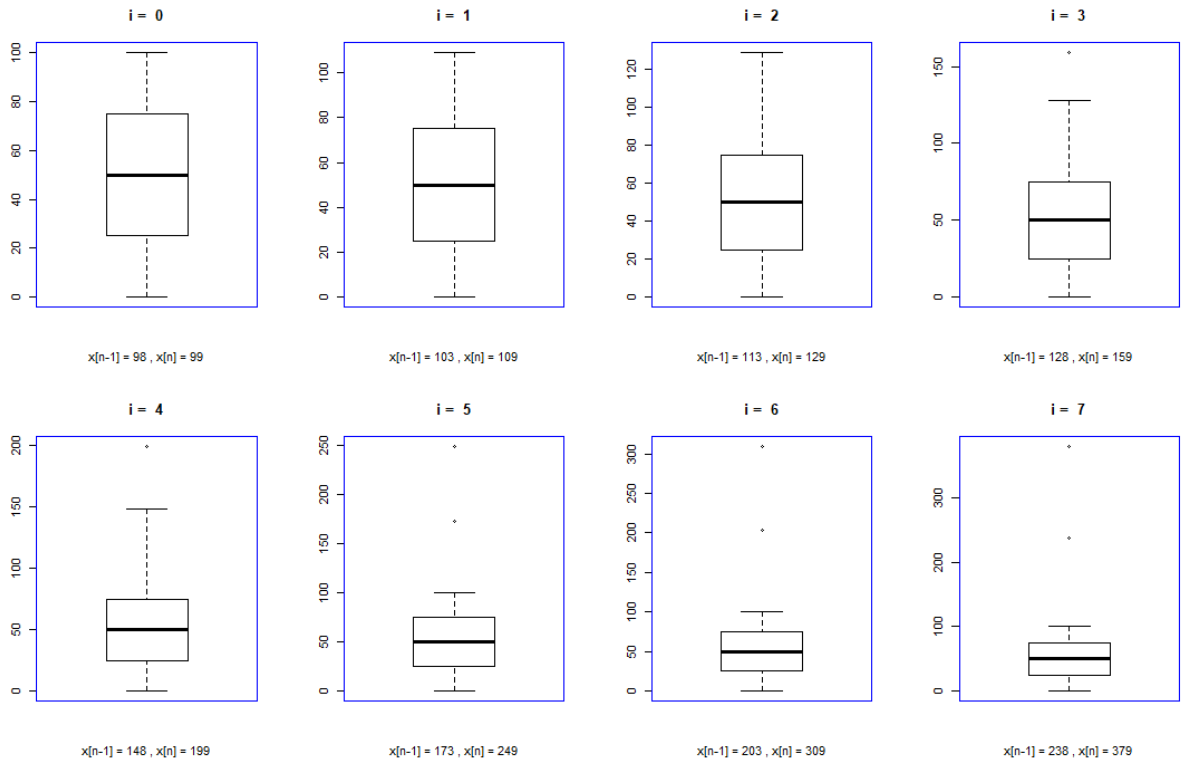


Abbildung 1: Der erzeugte Plot

Beantworten Sie folgende Fragen:

- Einen Ausreißer gibt es für i ≥3; für i ≥ 5 gibt es sogar zwei Ausreißer. Was genau sind Ausreißer? Erklären Sie die unterschiedlichen Bilder.
- Ist die Breite der Box der Quartilabstand? Wie viel Prozent der Daten „liegen“ in der Box?
- Wie lang sind die jeweiligen „Whiskers“?
- Wie müssen Sie den Aufruf von `boxplot()` ändern, so dass es erst bei i ≥ 5 den ersten und für i ≥ 7 den zweiten Ausreißer gibt?

WICHTIG

- `par()`
- `paste()`

2 MNIST-Bilddaten

Wir wollen das Beispiel aus dem Skript vertiefen, das am Ende des Kapitels 4 besprochen wurde. Grundlage für dieses Beispiel war der Datensatz `ziffern`, der aus hundert Grauwertbildern aus der MNIST-Datensammlung[1] besteht, die die Ziffern von 0 bis 9 in ganz unterschiedlichen handgeschriebenen Varianten zeigen. Jedes Grauwertbild besteht dabei aus 28×28 Pixeln und jeder Pixelwert liegt zwischen 0 und 255.

```
> ziffern <- readRDS("ziffern.rds")
> str(ziffern)
 int [1:100, 1:28, 1:28] 0 0 0 0 0 0 0 0 0 0 ...
> nBilder <- dim(ziffern)[1]  # Anzahl der Bilder
> nBilder
[1] 100
```

Abbildung 2 stellt alle Bilder aus dem Datensatz `ziffern` dar.

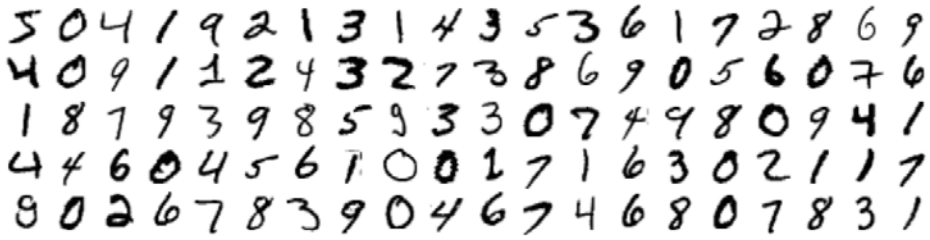


Abbildung 2: Alle Bilder aus dem Datensatz `ziffern`

Zusätzlich benötigen wir nun noch die Datei „label.rds“, die im Downloadbereich hinterlegt ist und die in das aktuelle Arbeitsverzeichnis kopiert werden muss. Die Datei „label.rds“ besteht aus einem mit `saveRDS(label, "label.rds")` serialisierten Objekt. Mithilfe der Funktion `readRDS()` kann das Objekt wiederhergestellt werden.

```
> label<- readRDS("label.rds")
> str(label)
 int [1:100(1d)] 5 0 4 1 9 2 1 3 1 4 ...
> label
  [1] 5 0 4 1 9 2 1 3 1 4 3 5 3 6 1 7 2 8 6 9 4 0 9 1 1 2 4 3 2 7 3 8 6 9 0 5 6 0 7 6 1 8
 [43] 7 9 3 9 8 5 9 3 3 0 7 4 9 8 0 9 4 1 4 4 6 0 4 5 6 1 0 0 1 7 1 6 3 0 2 1 1 7 9 0 2 6
 [85] 7 8 3 9 0 4 6 7 4 6 8 0 7 8 3 1
```

Wie wir sehen, ist `label` ein eindimensionales Array der Länge 100. In `label` ist gespeichert, welche der zehn Ziffern von den jeweiligen Bildern dargestellt werden. Das i-te Bild (also `ziffern[i,,]`) stellt die Ziffer `label[i]` dar. Also stellen die ersten fünf Bilder die Ziffern 5, 0, 4, 1 und 9 währen die letzten fünf Bilder die Ziffern 0, 7, 8, 3 und 1 darstellen. Vergleichen Sie die Werte von label mit Abbildung 2.

[1] Siehe z. B. https://en.wikipedia.org/wiki/MNIST_database

a) Bestimmen Sie mithilfe von `label`, wie viele Bilder es von jeder Ziffer gibt.

b) Erstellen Sie eine (5·28)×(20·28)-Matrix:

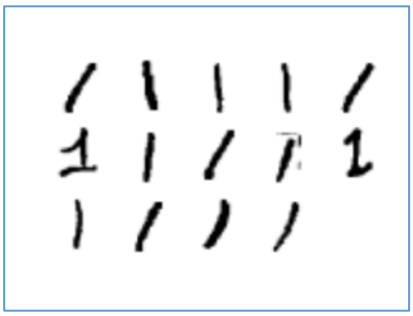
```
bild <- matrix(0, nrow=5*28, ncol = 20*28)
```

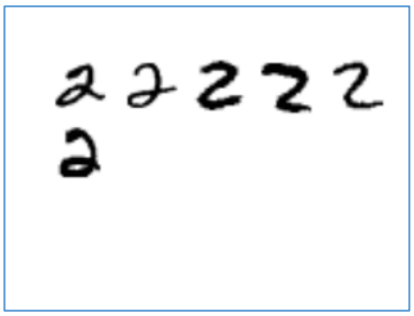
Konvertieren Sie und kopieren Sie die Bilddaten der 100 Grauwertbilder in die Matrix `bild` und zeigen Sie das entsprechende Rasterbild mithilfe der folgenden Anweisung an:

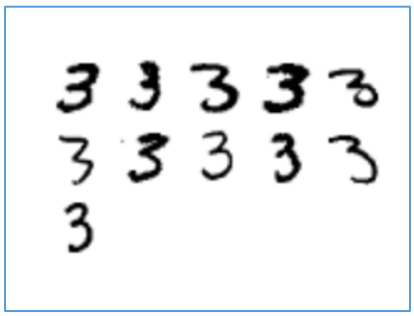
```
plot(as.raster(bild, max = 255))
```

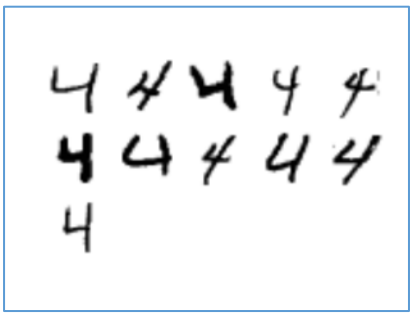
Das Ergebnis soll mit Abbildung 2 übereinstimmen.

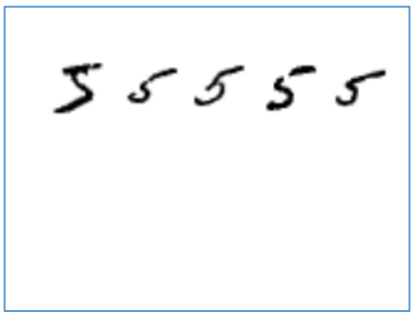
c) Erzeugen Sie für jede Ziffer *z* mithilfe einer (3·28)×(5·28)-Matrix einen entsprechenden Plot, der aus all denjenigen Bildern aus dem Datensatz `ziffern` besteht, die die Ziffer *z* darstellen. Die Ergebnisse sollen mit den nächsten Abbildungen übereinstimmen.

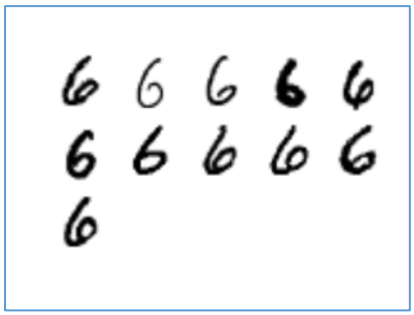


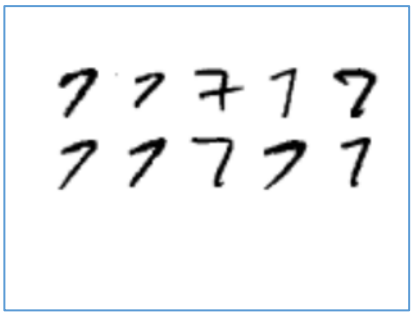


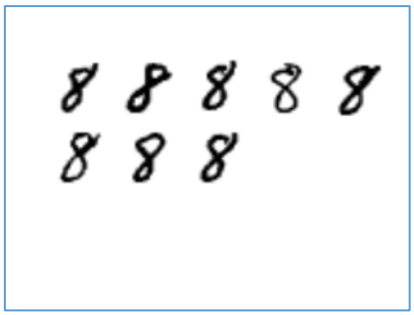


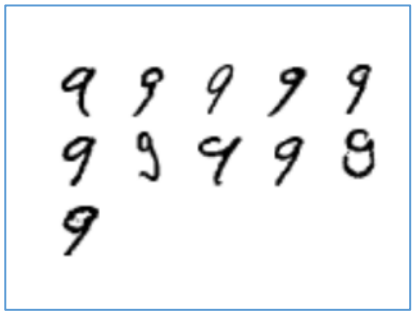












Programmierung für statistische Datenanalyse
Spezielle Themen und Fallstudien 2

1 IMDb Internet Movie Database

Wir werden in dieser Fallstudie Daten aus der Internet Movie Database (IMDb)[1] verwenden, die eine Vielzahl von Filmbewertungen enthält.

Hierfür benötigen wir die Dateien „imdbData.rds“ und „wortIndex.rds“, die im Downloadbereich hinterlegt sind. Die Daten sind aus dem R-Package keras[2] entnommen[3]. Keras ist ein High-Level-Framework für Deep-Learning, das eine einheitliche Schnittstelle für unterschiedliche Deep-Learning-Frameworks wie TensorFlow anbietet.

Die Beispiele aus dieser Fallstudie werden auch im Kapitel 3 des Buchs [Chollet 2018][4] behandelt, wenn auch in einer leicht anderen Form. Dort werden Sie als Trainings- und Testdaten für Neuronale Netze benutzt, die lernen sollen, ob eine Bewertung positiv oder negativ ist.

Um in R die Objekte `imdbData` und `wortIndex` zu erhalten, müssen die Daten deserialisiert werden:

```
imdbData <- readRDS("imdbData.rds")
wortIndex <- readRDS("wortIndex.rds")
```

In unserem Beispiel gehen wir davon aus, dass sich die beiden Dateien im aktuellen Arbeitsverzeichnis befinden.

1.1 imdb_Bewertungen

Das Objekt `imdb`Data ist eine Liste bestehend aus den Elementen `x` und `y`.

```
imdb_Bewertungen <- imdbData$x
imdb_Ergebnisse <- imdbData$y
```

Zuerst wollen wir das Objekt `imdb_Bewertungen` betrachten, das hundert Filmbewertungen enthält. Mithilfe von `str()` erfahren wir, dass es sich bei `imdb_Bewertungen` um eine Liste (ohne Namen) mit hundert Elementen handelt. Jedes Listenelement entspricht einer Filmbewertung und besteht aus einem integer-Vektor:

[1] siehe zum Beispiel https://de.wikipedia.org/wiki/Internet_Movie_Database
[2] https://www.rdocumentation.org/packages/keras
[3] wobei die Anzahl der Filmbewertungen von 50 000 auf 100 reduziert wurde
[4] Chollet, François ; Allaire, J.J.: Deep Learning mit R und Keras : Das Praxis-Handbuch von den Entwicklern von Keras und RStudio. 1. Aufl.. Heidelberg: MITP-Verlags GmbH & Co. KG, 2018.

```
> str(imdb_Bewertungen)
List of 100
 $ : int [1:68] 1 591 202 14 31 6 717 10 10 2 ...
 $ : int [1:260] 1 14 22 3443 6 176 7 5063 88 12 ...
 $ : int [1:603] 1 111 748 4368 1133 2 2 4 87 1551 ...
 $ : int [1:181] 1 13 1228 119 14 552 7 20 190 14 ...
 $ : int [1:108] 1 40 49 85 84 1040 146 6 783 254 ...
 $ : int [1:132] 1 146 427 5718 14 20 218 112 2962 32 ...
 $ : int [1:761] 1 1822 424 8 30 43 6 173 7 6 ...
 $ : int [1:180] 1 4 2 745 2 912 9 2 8 2 ...
  …  # Ausgabe abgeschnitten
```

Abgesehen von den Werten 1, 2 und 3 sind die restlichen Einträge als Indexwerte in eine Wortliste zu interpretieren, die im Prinzip im `wortIndex` enthalten ist. Allerdings sind die gespeicherten Bewertungen nicht vollständig: es wurden nur häufig benutzte Worte berücksichtigt. So lautet zum Beispiel der Anfang der zweiten Bewertung:

„this film requires a lot of patience because it focuses on mood and character development the plot is very simple and many of the scenes take place on the same set in frances ? the sandy dennis character apartment but the film builds to a disturbing climax br br the characters create an atmosphere ? with sexual tension and psychological ? it's very interesting that robert altman directed this considering the style and structure of his other films still the trademark altman audio style is evident here and there i think what really makes this film work is the brilliant performance by sandy dennis it's definitely one of her darker characters but she plays it so perfectly and convincingly that it's scary michael burns does a good job as the mute young …“

Ob eine Filmbewertung positiv oder negativ ist, wird in `imdb_Ergebnisse` festgehalten. Es ist ein Vektor bestehend aus 100 Elementen, die entweder den Wert 0 (negative Bewertung) oder den Wert 1 (positive Bewertung) haben.

a) Was ist in `imdb_Bewertungen` der kleinste und was ist der größte Wert?

1.2 wortIndex

Auch `wortIndex` ist eine Liste. Sie enthält 88584 Elemente. Jedes Element ist ein benannter integer-Vektor der Länge 1, den wir als skalaren Wert betrachten und als ***Wort-Indexwert*** bezeichnen wollen.

```
> head(wortIndex , n= 4)
$fawn
[1] 34701

$tsukino
[1] 52006

$nunnery
[1] 52007

$sonja
[1] 16816
```

```
> head(names(wortIndex), n=20)
 [1] "fawn"       "tsukino"    "nunnery"    "sonja"      "vani"       "woods"      "spiders"
 [8] "hanging"    "woody"      "trawling"   "hold's"     "comically"  "localized"  "disobeying"
[15] "'royale"    "harpo's"    "canet"      "aileen"     "acurately"  "diplomat's"
```

```
> wortIndex[[1]]
[1] 34701
> wortIndex$fawn
[1] 34701
```

Das am häufigsten benutzte Wort („the“) hat den Wort-Indexwert 1, das am zweithäufigsten benutzte Wort („and“) hat den Wort-Indexwert 2, usw.

```
> wortIndex[wortIndex == 1]
$the
[1] 1

> wortIndex[wortIndex == 2]
$and
[1] 2
```

Das Wort „fawn“ wurde nicht so häufig benutzt, es hat den Wort-Indexwert 34701.

b) Was ist in `wortIndex` der kleinste und was ist der größte Wort-Indexwert? Ist jeder Wert dazwischen ein Wort-Indexwert?

c) Erstellen Sie einen character-Vektor `topWorte`, der die hundert häufigsten Worte enthält, geordnet nach der Häufigkeit:

```
> topWorte
 [1] "the"     "and"     "a"       "of"      "to"      "is"      "br"      "in"      "it"      "i"
[11] "this"    "that"    "was"     "as"      "for"     "with"    "movie"   "but"     "film"    "on"
[21] "not"     "you"     "are"     "his"     "have"    "he"      "be"      "one"     "all"     "at"
[31] "by"      "an"      "they"    "who"     "so"      "from"    "like"    "her"     "or"      "just"
[41] "about"   "it's"    "out"     "has"     "if"      "some"    "there"   "what"    "good"    "more"
[51] "when"    "very"    "up"      "no"      "time"    "she"     "even"    "my"      "would"   "which"
[61] "only"    "story"   "really"  "see"     "their"   "had"     "can"     "were"    "me"      "well"
[71] "than"    "we"      "much"    "been"    "bad"     "get"     "will"    "do"      "also"    "into"
[81] "people"  "other"   "first"   "great"   "because" "how"     "him"     "most"    "don't"   "made"
[91] "its"     "then"    "way"     "make"    "them"    "too"     "could"   "any"     "movies"  "after"
```

Wie Sie hoffentlich bei den Lösungen der letzten beiden Teilaufgaben gemerkt haben, ist es sinnvoll, die Liste `wortIndex` in einen Vektor zu überführen:

```
indexVector <- unlist(wortIndex)
```

Wir können jetzt `indexVector` als ein "Dictionary<string, int>"[5] betrachten:

Zu einem Schlüssel (*key*) wie `"fawn"` wird als Wert (*value*) der entsprechende Wort-Indexwert `indexVector["fawn"]` zurückgegeben; in diesem Fall ist dies 34701.

Da es eine 1:1 Beziehung zwischen „key“ und „value“ gibt, können wir dieses "Dictionary" auch „umdrehen“. Dies wird durch den character-Vektor `wortVektor` erreicht und ist die nächste Teilaufgabe.

	Wert	Name	Alternative Schreibweise	Zugrundeliegender Datentyp
indexVektor[1]	34701	"Fawn"	indexVetor["Fawn"]	integer
wortVektor[1]	"Fawn"	"34701"	wortVektor["34701"]	character
worte [34701]	"Fawn"	NULL		character

Tabelle 1: Beispielswerte der erzeugten Datenstrukturen

[5] In C++ entspricht dies der Datenstruktur `map<string, int>`

d) Erzeugen Sie einen benannten character-Vektor `wortVektor`, der wie ein Dictionary aus einem Wort-Indexwert (aufgefasst als Zeichenkette), das entsprechende Wort zurückgibt:

```
> wortVektor[as.character(34701)]
 34701
"fawn"
> unname(wortVektor[as.character(34701)])
[1] "fawn"
```

Für `wortVektor` soll also u. a. gelten:

```
> str(wortVektor)
 Named chr [1:88584] "fawn" "tsukino" "nunnery" "sonja" "vani" "woods" "spiders" "hanging" "woody" ...
 - attr(*, "names")= chr [1:88584] "34701" "52006" "52007" "16816" ...
> head(wortVektor, n=20)
       34701        52006        52007        16816        63951         1408        16115         2345
      "fawn"    "tsukino"    "nunnery"      "sonja"       "vani"      "woods"    "spiders"    "hanging"
        2289        52008        52009        11307        40830        30568        52010        40831
     "woody"   "trawling"     "hold's"  "comically"  "localized" "disobeying"    "'royale"    "harpo's"
       52011        19313        52012        52013
     "canet"     "aileen"  "acurately" "diplomat's"
```

e) Schreiben Sie eine Funktion `getWort(index)`, die aus `wortVektor` das Wort mit dem Wort-Index `index` zurückgibt. Insbesondere soll gelten:

```
> for(i in 1:10) {
+   print(getWort(i))
+ }
[1] "the"
[1] "and"
[1] "a"
[1] "of"
[1] "to"
[1] "is"
[1] "br"
[1] "in"
[1] "it"
[1] "i"
```

f) Erklären Sie folgenden Quelltext und führen Sie ihn aus:

```
for(i in 1:5){
  bewertung <- imdb_Bewertungen[[i]][-1] -3
  w <- getWort(bewertung)
  w[is.na(w)] <- "?"
  text <- paste(w, collapse = " ")
  print(text)
  print("------------------------------")
}
```

g) Sortieren Sie nun `wortVektor` gemäß den Wort-Indexwerten, so dass man einen unbenannten character-Vektor `worte` der Länge 88584 erhält, der alle Worte geordnet nach ihrer Häufigkeit umfasst:[6]

```
> str(worte)
 chr [1:88584] "the" "and" "a" "of" "to" "is" "br" "in" "it" "i" "this" "that" "was" "as" "for" ...
> head(worte, n=100)
  [1] "the"     "and"     "a"       "of"      "to"      "is"      "br"      "in"      "it"      "i"
 [11] "this"    "that"    "was"     "as"      "for"     "with"    "movie"   "but"     "film"    "on"
 [21] "not"     "you"     "are"     "his"     "have"    "he"      "be"      "one"     "all"     "at"
 [31] "by"      "an"      "they"    "who"     "so"      "from"    "like"    "her"     "or"      "just"
 [41] "about"   "it's"    "out"     "has"     "if"      "some"    "there"   "what"    "good"    "more"
 [51] "when"    "very"    "up"      "no"      "time"    "she"     "even"    "my"      "would"   "which"
 [61] "only"    "story"   "really"  "see"     "their"   "had"     "can"     "were"    "me"      "well"
 [71] "than"    "we"      "much"    "been"    "bad"     "get"     "will"    "do"      "also"    "into"
 [81] "people"  "other"   "first"   "great"   "because" "how"     "him"     "most"    "don't"   "made"
```

Insgesamt sind u. a. folgende Objekte erzeugt worden:

	Datentyp	Anzahl der Elemente	Bemerkung
wortIndex	Liste	88584	benannt `indexVector <- unlist(wortIndex)`
indexVektor	integer-Vektor	88584	benannt: „Dictionary“
wortVektor	character-Vektor	88584	benannt: „Dictionary“

Tabelle 2: Erzeugte Datenstrukturen

h) Erklären Sie folgenden Quelltext und führen Sie ihn aus:

```
for(i in 1:5){
  bewertung <- imdb_Bewertungen[[i]][-1] -3
  bewertung[bewertung < 1 | bewertung > length(worte)] <- NA
  w <- worte[bewertung]
  w[is.na(w)] <- "?"
  text <- paste(w, collapse = " ")
  print(text)
  print("------------------------------")
}
```

[6] Dies ist ein Einzeiler!

Damit die einzelnen Bewertungen als Input einem Neuronalen Netz übergeben werden können, müssen sie umcodiert werden. Eine Möglichkeit ist die sogenannte ***One-Hot-Codierung***. In unserem Fall wird hierfür eine Matrix `C` erzeugt, die nur die Werte 0 oder 1 annimmt. Es gilt:

`C[i,j] == 1` genau dann, wenn `j` in der `i`-ten Bewertung[7] (als Wort-Indexwert) vorkommt.

Beispielhaft enthält `C` folgende Werte:

```
> C[1:8, 1:16]
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13] [,14] [,15] [,16]
[1,]    1    1    0    1    1    1    1    1    1     1     0     0     1     1     0     1
[2,]    1    1    0    1    1    1    1    1    1     1     1     1     1     1     1     0
[3,]    1    1    0    1    1    1    1    1    1     1     1     1     0     1     1     1
[4,]    1    1    0    1    1    1    1    1    1     0     1     1     1     1     1     1
[5,]    1    1    0    1    1    1    0    1    1     1     0     1     1     1     1     0
[6,]    1    1    0    1    1    1    0    1    1     0     1     1     1     1     0     0
[7,]    1    1    0    1    1    1    1    1    1     1     1     1     0     1     1     1
[8,]    1    1    0    1    1    1    1    1    1     1     1     0     0     1     1     0
```

Für große Spaltenindexwerte ist `C` hingegen dünn besetzt:

```
C[1:8, 501:516]
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13] [,14] [,15] [,16]
[1,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0
[2,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0
[3,]    0    0    0    0    0    0    0    1    0     0     0     0     0     0     0     0
[4,]    0    0    0    0    0    0    0    0    1     1     0     0     0     0     0     0
[5,]    0    0    0    0    1    0    0    0    0     0     0     0     0     0     1     0
[6,]    0    0    0    0    0    0    0    0    0     0     0     0     0     0     0     0
[7,]    0    0    0    0    0    0    0    0    0     0     0     0     1     0     0     0
[8,]    0    0    0    0    0    0    0    0    0     0     0     0     1     0     0     0
```

i) Die Matrix `C` sollte nicht größer als nötig sein. Wie groß ist `C`?
Erstellen Sie die Matrix `C`.

[7] D.h. in `imdb_Bewertungen[[i]]`

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

Spezielle Themen und Fallstudien 3

Visualisierung mit ggplot2

1 WAHRSCHEINLICHKEITSVERTEILUNGEN IN R 2

2 FUNKTIONSPLOT 3

3 HISTOGRAMME 5

3.1 UNTERSCHIEDLICHE ANZAHL DER GRUPPEN 6
3.2 UNTERSCHIEDLICHE UNTERE GRENZEN 7
3.3 KERNDICHTESCHÄTZER GEOM_DENSITY( ) 8

4 BEISPIEL: ZENTRALER GRENZWERTSATZ 10

5 QUANTILPLOT 13

1 Wahrscheinlichkeitsverteilungen in R

In den folgenden Abschnitten werden wir auf unterschiedliche Wahrscheinlichkeitsverteilungen zurückgreifen. In der nächsten Tabelle sind die in Base-R verfügbaren Verteilungen aufgelistet.

Verteilung	R Bezeichnung	Zusätzliche Argumente
Beta	`beta`	`shape1, shape2, ncp`
Binomial	`binom`	`size, prob`
Cauchy	`cauchy`	`location, scale`
Chi-Quadrat χ^2	`chisq`	`df, ncp`
Exponential	`exp`	`rate`
F	`f`	`df1, df2, ncp`
Gamma	`gamma`	`shape, rate (=1/scale)`
Geometrisch	`geom`	`prob`
Gleichförmig	`unif`	`min, max`
Hypergeometrisch	`hyper`	`m, n, k`
Log-normal	`lnorm`	`meanlog, sdlog`
Logistic	`logis`	`location, scale`
Negativ binomial	`nbinom`	`size, prob`
Normal	`norm`	`mean, sd`
Poisson	`pois`	`lambda`
Signed rank	`signrank`	`n`
Student's t	`t`	`df, ncp`
Weibull	`weibull`	`shape, scale`
Wilcoxon Rangsumme	`wilcox`	`m, n`
Wilcoxon Vorzeichen-Rang	`signrank`	`n`

Tabelle 1-1: Wahrscheinlichkeitsverteilungen in R

Vor die aufgelistete R-Bezeichnung (wie `norm`) kommt noch ein entsprechender Buchstabe:[1]

- `r` für „zufällige" Daten gemäß der Verteilung. *Beispiel*: `rnorm(10)`
- `d` für die Dichte. *Beispiel*: `dnorm(x)`
- `p` für die Verteilungsfunktion. *Beispiel*: `pnorm(x)`
- `q` für die Quantilfunktion. *Beispiel*: `qnorm(p)`

[1] Die Beipiele beziehen sich auf die Normalverteilung

2 Funktionsplot

In einer ggplot2-Grafik kann leicht mit `stat_function()` ein Funktionsplot in einer neuen Ebene hinzugefügt werden. Die zu zeichnende Funktion wird dabei durch das Argument `fun` bestimmt.

```
ggplot(data.frame(x = c(-4, 4)), aes(x)) +
  stat_function(fun = dnorm, colour = "red")
```

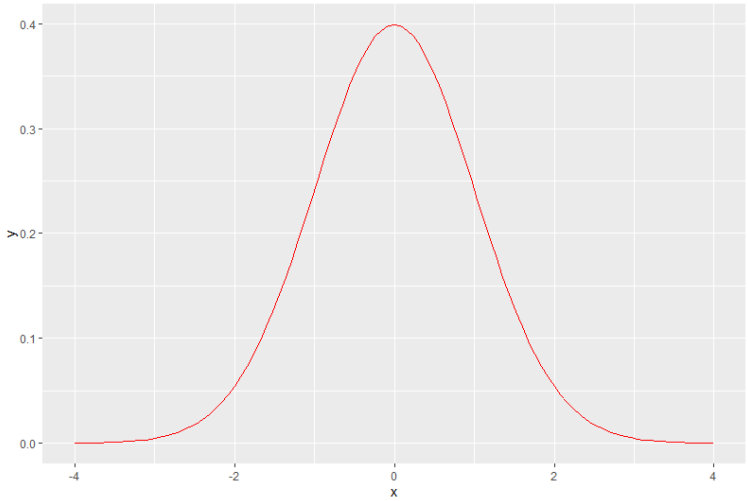


Abbildung 1: Dichte der Normalverteilung N(0;1)

Statt `dnorm` hätten wir auch eine andere Funktion wie z. B. `sin` oder den Bezeichner einer selbst geschriebenen numerischen Funktion angeben können. Ähnlich leicht können mehrere Ebenen in einer Grafik gleichzeitig angezeigt werden. Als Beispiel verwenden wir die Dichten der χ^2-Verteilung mit unterschiedlichen Freiheitsgraden:

```
?dchisq
sizeline <- 1
ggplot(data.frame(x = c(0, 10)), aes(x)) +
  stat_function(fun = dchisq, args = list(df=1), color = "cyan", size = sizeline) +
  stat_function(fun = dchisq, args = list(df=2), color = "red", size = sizeline) +
  stat_function(fun = dchisq, args = list(df=3), color = "blue", size = sizeline)+
  stat_function(fun = dchisq, args = list(df=4), color = "green3", size = sizeline)+
  labs(title = "Dichten der Chi-Quadrat-Verteilungen", subtitle ="Data Science, J. Willms")
```

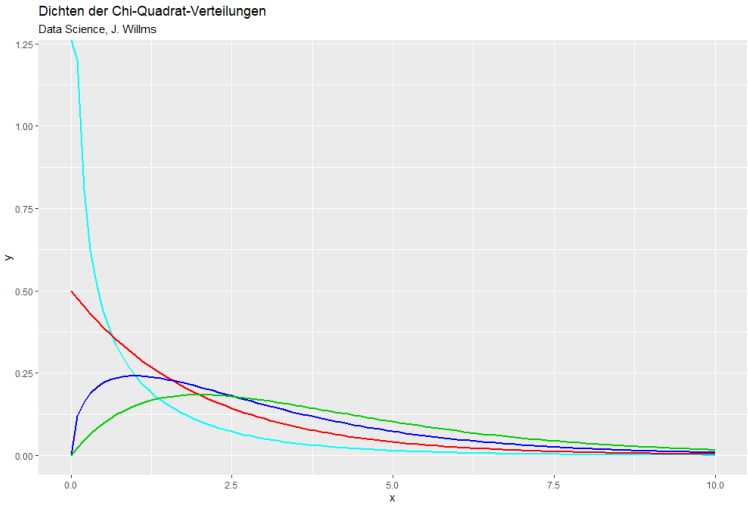


Abbildung 2: χ^2-Verteilung mit den Freiheitsgraden df = 1, 2, 3, 4

Wie wir sehen, können in `stat_function()` zusätzliche Argumente für den Funktionsaufruf mithilfe des Arguments `args` mithilfe einer Liste übergeben werden. Das nächste Beispiel zeigt die Dichten der *t*-Verteilung[2]:

```
sizeline <- 1
ggplot(data.frame(x = c(-4, 4)), aes(x)) +
  stat_function(fun = dt, args = list(df=1), color = "cyan", size = sizeline) +
  stat_function(fun = dt, args = list(df=2), color = "red", size = sizeline) +
  stat_function(fun = dt, args = list(df=3), color = "blue", size = sizeline)+
  stat_function(fun = dt, args= list(df=4), color = "green3", size = sizeline)+
  labs(title = "Dichten der t-Verteilungen", subtitle ="Data Science, J. Willms") +
  theme_light()
```

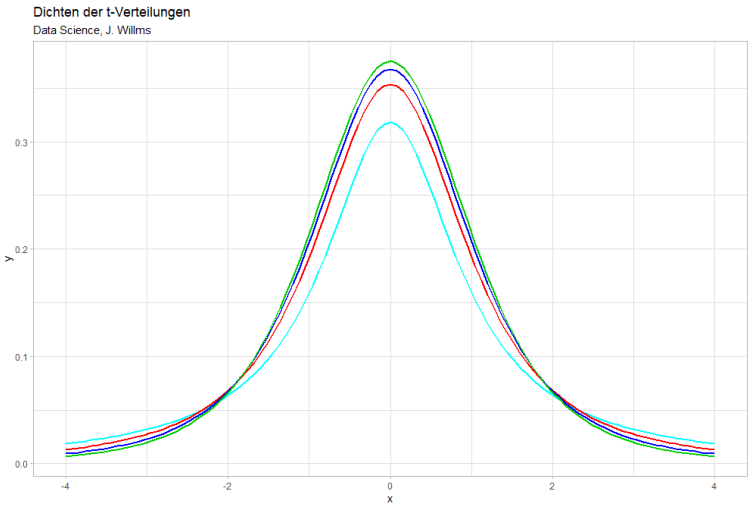


Abbildung 3: Dichte der *t*-Verteilung mit den Freiheitsgraden df = 1, 2, 3, 4

Im letzten Beispielplot haben wir ein anderes „Theme“ benutzt. `ggplot2()` unterstützt standardmäßig acht unterschiedliche „Themes“, mit denen die globalen Eigenschaften der Nicht-Daten-Elemente in einer Grafik gesetzt werden können, siehe `?theme` und `?theme_bw`.

Aufgabe

a) Die χ^2-Verteilung ist ein Spezialfall der Gamma-Verteilung. Zeigen Sie, dass Sie „Abbildung 2: χ^2-Verteilung mit den Freiheitsgraden df = 1, 2, 3, 4“ auch mit `dgamma()` statt mit `dchisq()` erzeugen können.

[2] manchmal auch mit Student-Verteilung bezeichnet

3 Histogramme

Wir beginnen mit einem einfachen Datensatz `x`. Wir haben bereits gesehen, dass mithilfe der Funktion `hist()`[3] ein Histogramm erstellt werden kann:

```
set.seed(1)
x = rnorm(50)
h ->hist(x, breaks = 10)
h
```

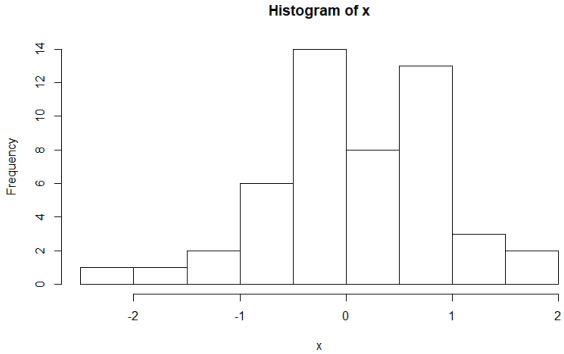


Abbildung 4: Histogramm mit `hist()` erzeugt

Als Nächstes wollen wir mithilfe von `ggplot()`ein ähnliches Histogramm erstellen und dann nacheinander weitere Ebenen in die Grafik bringen:

```
library(ggplot2)
ggplot(data.frame(x = x), aes(x)) +
  geom_histogram(bins=9)   # y: count
```

Überraschenderweise hat das so erzeugte Histogramm eine andere Form. Der Grund hierfür ist, dass die beiden Histogramme auf unterschiedlichen Gruppenaufteilungen basieren. Die Gestalt eines Histogramms hängt sehr wesentlich von der Anzahl der Gruppen („bins“), der einzelnen Gruppenbreiten sowie von der unteren Grenze der ersten Gruppe (bzw. von der oberen Grenze der letzten Gruppe) ab. Im Folgenden wollen wir dies anhand von Beispielen illustrieren.

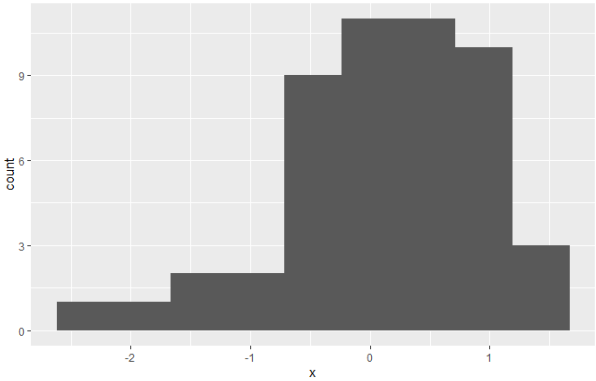


Abbildung 5: Histogramm mit ggplot2

Mithilfe von `str(h)` sehen wir, wie die Gruppen im ersten Histogramm gebildet worden sind.

```
> str(h)
List of 6
 $ breaks  : num [1:10] -2.5 -2 -1.5 -1 -0.5 0 0.5 1 1.5 2
 $ counts  : int [1:9] 1 1 2 6 14 8 13 3 2
 $ density : num [1:9] 0.04 0.04 0.08 0.24 0.56 0.32 0.52 0.12 0.08
 $ mids    : num [1:9] -2.25 -1.75 -1.25 -0.75 -0.25 0.25 0.75 1.25 1.75
 $ xname   : chr "x"
 $ equidist: logi TRUE
 - attr(*, "class")= chr "histogram"
```

Den gleichen Effekt erhalten wir, wenn wir in `geom_histogram()` als Grenze `boundary=-2.5` und als Gruppenbreite `binwidth = 0.5` angeben und zusätzlich `theme_minimal()` verwenden:

[3] Bestandteil von Base-R

```
ggplot(data.frame(x = x), aes(x)) +
  geom_histogram(boundary=-2.5, binwidth = 0.5, fill="white", color="black") +
  theme_minimal()
```

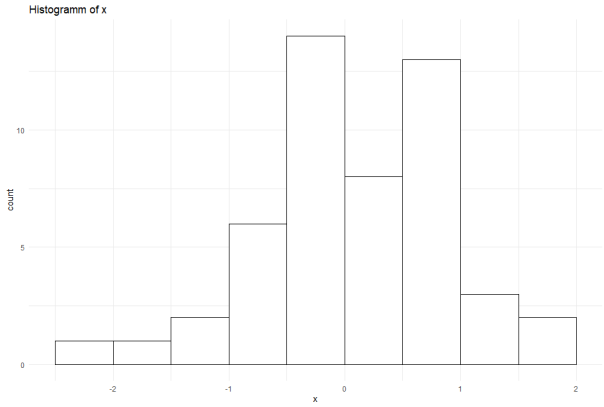


Abbildung 6: Angepasstes Histogramm

3.1 Unterschiedliche Anzahl der Gruppen

Es ist eigentlich nicht überraschend, dass die Gestalt eines Histogramms bei identischen Daten sehr wesentlich von der Anzahl der Gruppen („bins“) abhängt.

```
for(nBins in 7:11) {
  p <- ggplot(data.frame(x = x), aes(x)) +
    geom_histogram(boundary=-2.5, bins = nBins, fill="gray", color="black") +
    labs(title = "Histogramm of x") +
    theme_bw()
  print(p)
}
```

Die Ergebnisse sind in der nächsten Abbildung zusammengefasst:

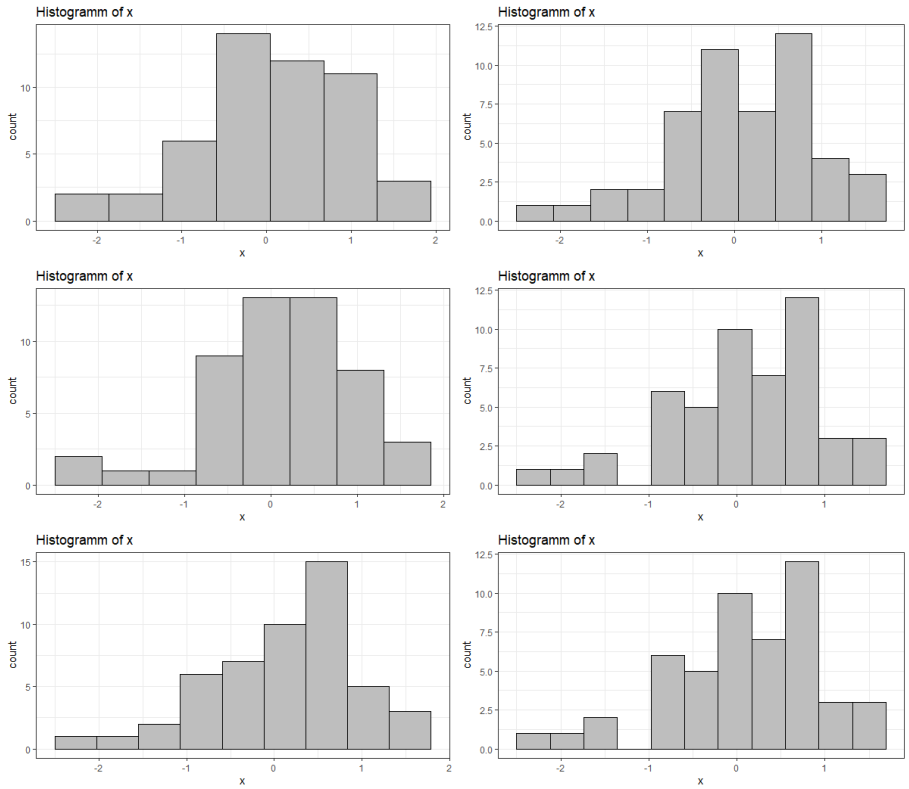


Abbildung 7: Identische Daten, aber eine unterschiedliche Anzahl von Gruppen

3.2 Unterschiedliche untere Grenzen

Überraschend ist vielleicht Folgendes: Selbst wenn die Anzahl der Gruppen und die Gruppenbreite konstant bleibt, kann die Gestalt eines Histogramms bei identischen Daten wesentlich von der unteren Grenze der ersten Gruppe (bzw. von der oberen Grenze der letzten Gruppe) abhängen!

Als Beispiel wollen wir einen bekannten Datensatz verwenden, der die Schneehöhen in Inches aus 63 hintereinander folgenden Wintern auflistet, gemessen in Buffalo im Bundesstaat NewYork, USA:[4]

```
sHoehe <- c(25.0 , 39.8 , 39.9 , 40.1 , 46.7 , 49.1 , 49.6 , 51.1 , 51.6,
            53.5 , 54.7 , 55.5 , 55.9 , 58.0 , 60.3 , 63.6 , 65.4 , 66.1,
            69.3 , 70.9 , 71.4 , 71.5 , 71.8 , 72.9 , 74.4 , 76.2 , 77.8,
            78.1 , 78.4 , 79.0 , 79.3 , 79.6 , 80.7 , 82.4 , 82.4 , 83.0,
            83.6 , 83.6 , 84.8 , 85.5 , 87.4 , 88.7 , 89.6 , 89.8 , 89.9,
            90.9 , 97.0 , 98.3 , 101.4 , 102.4 , 103.9 , 104.5 , 105.2 , 110.0,
            110.5 , 110.5 , 113.7 , 114.5 , 115.6 , 120.5 , 120.7 , 124.7 , 126.4)
df <- data.frame(x=sHoehe)
```

Wir wollen nun fünf Histogramme erstellen, die sich *lediglich* in der Position der unteren Grenze der ersten Gruppe unterscheiden (sowohl Anzahl der Gruppen als auch Gruppenbreite bleiben dabei konstant):

```
transparenz <- 0.2
for(b in seq(16, 24, by=2)) {
  p <- ggplot(df, aes(x)) +
    geom_histogram(binwidth=10, boundary=b, alpha=transparenz)
  print(p)
  transparenz <- transparenz+0.1
}
```

Die Ergebnisse sind in der nächsten Abbildung zusammengefasst:

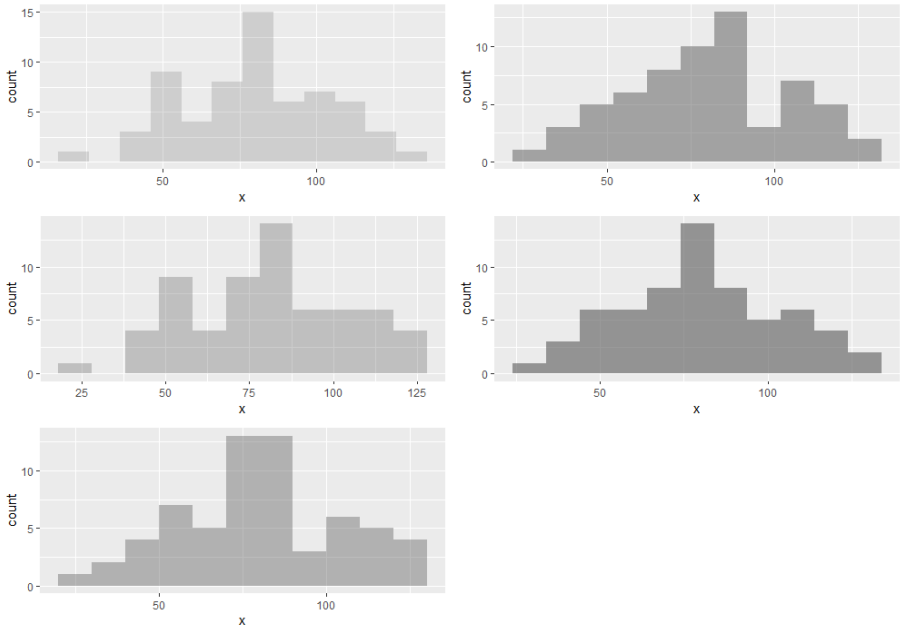


Abbildung 8: Unterschiedliche untere Grenzen *b* der jeweils ersten Gruppe: *b* = 16, 18, 20, 22, 24

[4] Eine detailliertere Behandlung dieses Beispiels finden Sie auch auf S. 11 im Abschnitt „*Histogramme lassen sich weiter verfeinern*“ von: Kockelkorn, Ulrich: *Statistik für Anwender*. Heidelberg: Spektrum Akademischer Verlag, 2012 oder oder Arens, Tilo ; Hettlich, Frank ; Karpfinger, Christian ; Kockelkorn, Ulrich ; Lichtenegger, Klaus ; Stachel, Hellmuth ; Hettlich, Frank ; Karpfinger, Christian ; Kockelkorn, Ulrich ; Lichtenegger, Klaus ; Stachel, Hellmuth: Ergänzungen und Ver-tiefungen zu Arens et al., Mathematik. 2. Aufl.. Berlin Heidelberg New York: Springer-Verlag, 2017, Kapitel 18, S 206 ff (auch als EBook in der FH-SWF verfügbar, Stand Oktober 2019)

Die Gestalt der fünf Histogramme ist sehr unterschiedlich, obwohl alle auf denselben Daten basieren; auch die Anzahl der Gruppen und die Gruppenbreiten sind identisch! Dies wird auch deutlich, wenn die fünf Histogramme in einer Grafik als Überlagerung dargestellt werden:

```
transparenz <- 0.2
p <- ggplot(df, aes(x))
for(b in seq(16, 24, by=2)) {
  p <- p + geom_histogram(binwidth=10, boundary=b, alpha=transparenz)
  transparenz <- transparenz+0.1
}
p + labs(title = "Überlagerung der fünf Histogramme")
```

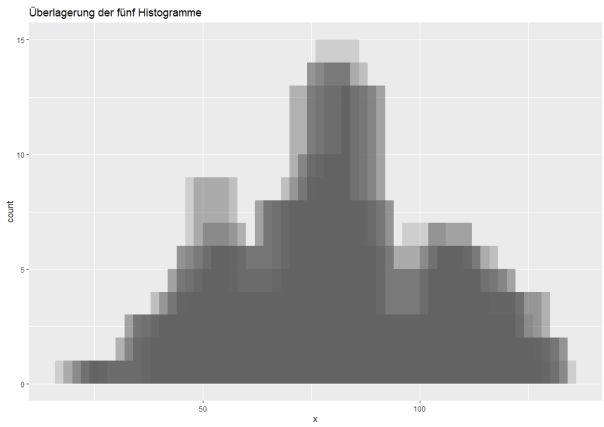


Abbildung 9: Überlagerung der fünf Histogramme

3.3 Kerndichteschätzer `geom_density()`

In der Überlagerung der fünf Histogramme scheint nun eine Struktur erkennbar, die in den einzelnen Histogrammen nicht so ersichtlich ist. Hier helfen oft Kerndichteschätzer wie `geom_density()`, die das Histogramm glätten und ein stetiges Bild des Histogramms vermitteln.

Damit wir `geom_density()` anwenden können, müssen wir `aes(x, stat(density))` statt `aes(x)` in `ggplot()` angeben. Statt der Anzahl (`count`) wird nun die Dichte (`density`) auf der y-Achse im Histogramm aufgetragen. Die Daten (Schneehöhen) übernehmen wir unverändert, ebenso die Gruppenbreite 10; die untere Grenze *b* der ersten Gruppe legen wir auf den Wert 16 fest:

```
ggplot(df, aes(x, stat(density))) +
  geom_histogram(binwidth=10, boundary=16, alpha=0.2) +
  geom_density(color="blue", size=1)
```

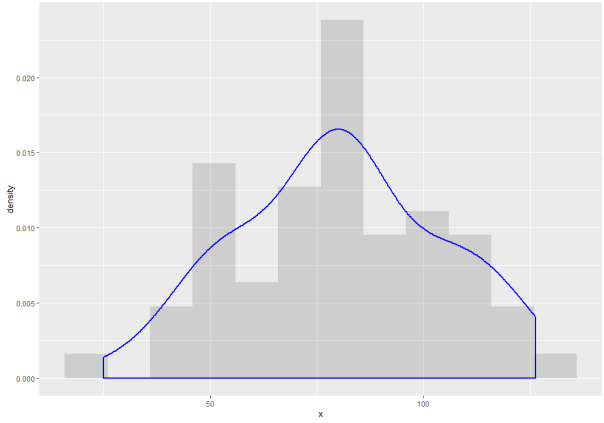


Abbildung 10: Histogramm mit `geom_density()`

Mit dem Argument `adjust` können wir die Dichteschätzung relativ zur Gruppenbreite anpassen; `adjust=0.5` nimmt die halbe Gruppenbreite als Grundlage der Schätzung:

```
ggplot(df, aes(x, stat(density))) +
  geom_histogram(binwidth=10, boundary=16, alpha=0.2) +
  geom_density(color="blue", size=1)+
  geom_density(adjust=0.5, color="red", size =1)
```

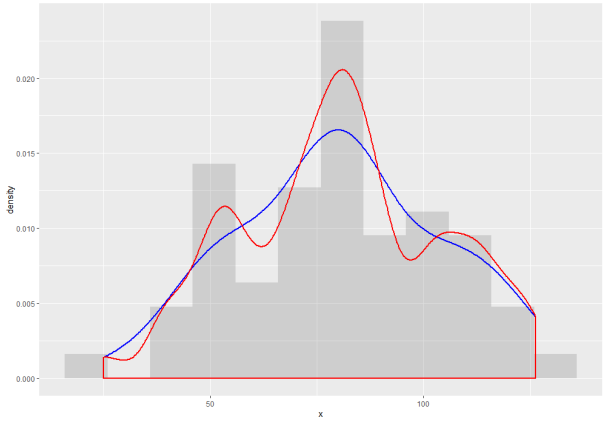


Abbildung 11: Histogramm mit `geom_density()` (rot: `adjust=0.5`)

Aus meiner Sicht wird die Funktionsweise des Dichteschätzers `geom_density()` noch deutlicher, wenn man die Überlagerung der Histogramme zugrunde legt:

```
transparenz <- 0.2
p <- ggplot(df, aes(x, stat(density)))
for(b in seq(16, 24, by=2)) {
  p <- p + geom_histogram(binwidth=10, boundary=b, alpha=transparenz)
  transparenz <- transparenz+0.1
}
p <- p + geom_density(color="blue", size=1)+
    geom_density(adjust=0.5, color="red", size =1)
p +
  theme_bw() +
  labs(title = "Überlagerung der Histogramme mit Dichteschätzer",
       subtitle ="Data Science, J. Willms")
```

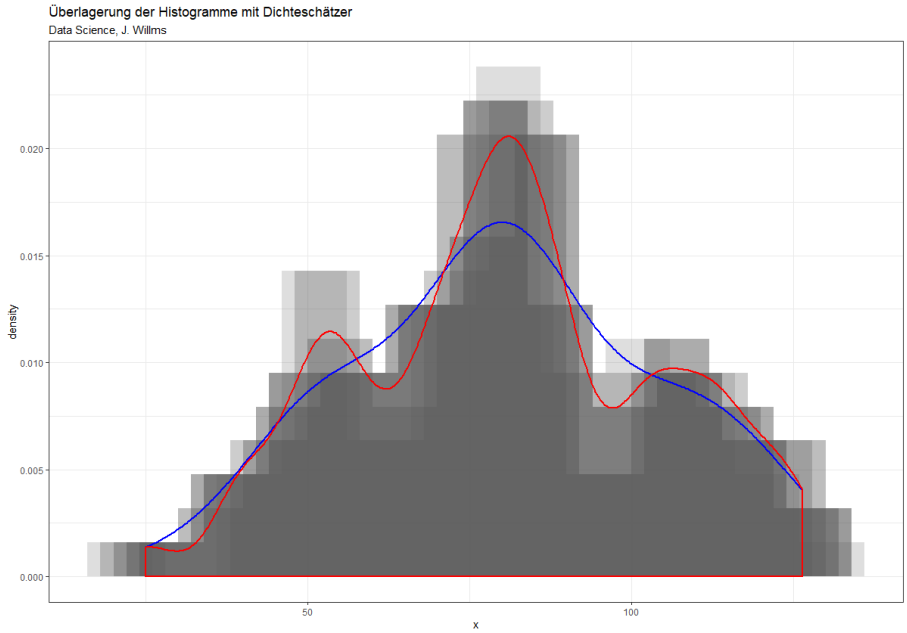


Abbildung 12: Überlagerung und Kerndichteschätzer

4 Beispiel: Zentraler Grenzwertsatz

Als nächstes Beispiel wollen wir die Konvergenzaussage des zentralen Grenzwertsatzes anhand eines elementaren Beispiels visualisieren. Abgesehen von `dnorm()`, der Dichtefunktion der Normalverteilung, wollen wir lediglich die Funktion `sample()` benutzen.

Als Erstes wollen wir einen zufälligen Wert `x` erzeugen, der mit Wahrscheinlichkeit 2/3 den Wert 0 und mit Wahrscheinlichkeit 1/3 den Wert 1 annimmt:

```
set.seed(1)
x1 <- sample(c(0,0,1),1)
```

Die (theoretische) Varianz der zugrundeliegenden zweiwertigen Zufallsvariablen X ist leicht berechnet: $\mathrm{Var}(X) = 2/3 \cdot 1/9 + 1/3 \cdot 4/9 = 2/9$.

Als Nächstes wollen nicht einen, sondern `n`=300 solcher Datensätze erzeugen:

```
n <- 300
x <- sample(c(0,0,1),n, replace = TRUE)
```

`x` ist also ein Vektor mit 300 Elementen und jedes Element hat mit Wahrscheinlichkeit 2/3 den Wert 0 und mit Wahrscheinlichkeit 1/3 den Wert 1:

```
> x
  [1] 1 0 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 1 0
 [44] 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1 0 1 1 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 0 0
 [87] 1 1 1 0 0 0 1 0 0 1 1 1 0 0 0 0 1 1 0 1 0 0 0 0 1 1 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 0
[130] 1 0 1 1 0 1 1 0 0 0 0 0 0 1 1 1 1 1 0 1 0 1 0 0 1 0 0 0 0 0 0 1 0 0 1 1 1 1 0 1 0 1 0
[173] 0 0 0 1 0 0 0 1 1 0 0 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
[216] 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 1 1 0 0 1 0 0 0 1 0 0 0
[259] 0 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 1 1 1 1 0 0 0 0 0 1 0 1 1 1 0 0 0 1 1 0 0 0 1 1 0
```

Als Nächstes wollen wir den Durchschnitt `sn` unserer zufälligen Daten `x` betrachten. Nach dem Gesetz der großen Zahlen sollte `sn` ungefähr den Wert 1/3 haben:

```
> sn <- sum(x)/n      # == mean(x)
> sn
[1] 0.3066667
```

`sn` kann als Realisierung einer Zufallsvariable S_n aufgefasst werden. Für große `n` macht der zentrale Grenzwertsatz eine Aussage über die Verteilung von S_n: sie sollte ungefähr normalverteilt sein. Dies wollen wir visuell mithilfe einer Simulation überprüfen. Dazu müssen wir mehrfach[5] und stochastisch unabhängig voneinander solche zufälligen Datensätze `x` erzeugen, `sn` berechnen und grafisch darstellen.

[5] in unserem Beispiel fünfzigmal

```
Anzahl <- 50    # gibt an, wie oft sn realisiert werden soll
s <- replicate(Anzahl, sum(sample(c(0,0,1),n, replace = TRUE))/n)
```

Aufgabe

b) Welche Verteilung hat die Zufallsvariable $n \cdot S_n$?

`replicate(n, expression)` ruft `sapply()` auf und führt `n`-mal den übergebenen Ausdruck `expression` aus und speichert das Ergebnis in einer möglichst einfachen Datenstruktur[6]. In unserem Beispiel enthält der Vektor `s` nun 50 Realisierungen der Zufallsvariable S_n .

```
> str(s)
 num [1:50] 0.313 0.303 0.347 0.367 0.317 ...
```

Das arithmetische Mittel von `s` sollte annähernd `1/3` betragen und `var(s)` müsste ungefähr gleich `2/9n` sein:

```
 > mean(s)
[1] 0.3365333
> var(s) - 2/(9*n)
[1] 3.475132e-05
```

Aufgabe

c) Erzeugen Sie den Vektor `s`, ohne die Funktion `replicate()` zu benutzen (Hinweis: `sapply()`).

Mithilfe eines Histogramms können wir die Mittelwerte `s` visualisieren:

```
df <- data.frame(x=s)
# Zeichne Histogramm
ggplot(df, aes(x, stat(density))) +
  geom_histogram(bins=8, alpha=0.4) +
  geom_density(color="blue", size =1.2)+
  labs(title = "Konvergenz gegen die Normalverteilung?",
       subtitle ="anzahl=50, n=300,  J. Willms") +
  theme_bw()
```

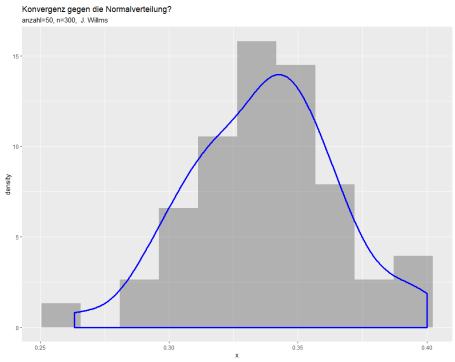


Abbildung 13: Histogramm von s

[6] In unserem Beispiel ist dies ein Vektor.

Die mit n multiplizierten so erzeugten s-Werte sind binomialverteilt. Nach dem zentralen Grenzwertsatz kann für große n diese Binomialverteilung durch eine Normalverteilung approximiert werden. Also fügen wir im Plot in einer zusätzlichen Ebene als Vergleich die Dichte der entsprechenden Normalverteilung hinzu. In unserem Fall ist dies die Normalverteilung N(n/3; 2/9n).[7]

Um flexibler zu sein, kapseln wir die Zeichenbefehle in einer Funktion:

```
zeichne <- function(df, nGruppen, n=300){
  supStr <- sprintf("anzahl=%d, n=300,  J. Willms", length(df$x))
  p <- ggplot(df, aes(x)) +
    geom_histogram(aes(x, stat(density)), bins=nGruppen, alpha=0.4)+
    geom_density(color="blue", size =1.2) +
    labs(title = "Konvergenz gegen die Normalverteilung?", subtitle = supStr) +
    stat_function(fun = dnorm, args= list(mean=1/3, sd=sqrt(2/(9*n))),
                  colour = "red", size = 1.1) +
    theme_bw()
  return(p)
}
```

Da der Untertitel angepasst werden muss, entspricht er dem char-vektor subStr, der mithilfe der Funktion sprintf() gemäß den übergebenen Argumenten gesetzt wird. sprintf() wird anlog wie die C-Funktion sprintf() (und somit auch wie printf()) aufgerufen.

Vergleichen wollen wir nun für die Anzahl 50 und die Anzahl 2000 die entsprechenden Histogramme (mit der Anzahl der Gruppen von 10 und 18), die durch folgende Anweisungen erzeugt werden:

```
set.seed(1)
n <- 300
x <- sample(c(0,0,1),n, replace = TRUE)
Anzahl <- 50    # gibt an, wie oft sn realisiert werden soll
s <- replicate(Anzahl, sum(sample(c(0,0,1),n, replace = TRUE))/n)
df <- data.frame(x=s)
zeichne(df,10,n)
```

```
set.seed(1)
n <- 300
x <- sample(c(0,0,1),n, replace = TRUE)
Anzahl <- 2000    # gibt an, wie oft sn realisiert werden soll
s <- replicate(Anzahl, sum(sample(c(0,0,1),n, replace = TRUE))/n)
df <- data.frame(x=s)
zeichne(df,18,n)
```

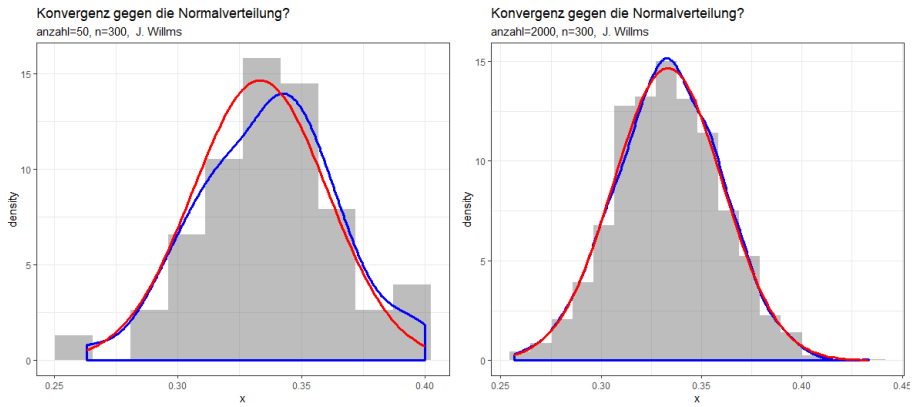


Abbildung 14: Annäherung an die Normalverteilung

[7] Wieso n/3 und wieso 2/9n?

Abbildung 14 zeigt in der Tat, dass die beiden Verteilungen recht gut durch die Normalverteilung N(n/3; 2/9n) approximiert werden können.

Aufgabe

d) Für n= 2,3,4,5 überprüfen Sie in ähnlicher Weise durch eine Simulation, wie sich die Summe n identisch verteilter und unabhängiger Zufallsvariablen verhält, die N(0; 1) verteilt sind.

5 Quantilplot

Ein *Quantilplot*[8] bietet die Möglichkeit, zwei Verteilungen auf Gleichheit visuell zu überprüfen. In einem Quantilplot werden die jeweiligen α-Quantile zweier Verteilungen in einem Koordinatensystem gegeneinander aufgetragen. Liegen die gezeichneten Punkte annähernd auf einer Geraden, dann liegt die Vermutung nahe, dass es sich um ähnliche Verteilungen handelt. Wir wollen sogenannte *NP-Plot* benutzen, die eine Verteilung mit der Normalverteilung vergleicht.

Als einführendes Beispiel wollen wir einen Datensatz verwenden, von dem wir wissen, dass er N(4; 4)-verteilt ist.

```
set.seed(0)
s <- rnorm(400 ,mean=4, sd=2)  # N(4;4)-verteilt
ggplot(data.frame(s), aes(sample = s)) +
  stat_qq() + stat_qq_line()  +
  labs(title = "NP-PLot", subtitle = "N(4;4)-verteilt")
```

Wir erhalten folgenden NP-Plot:

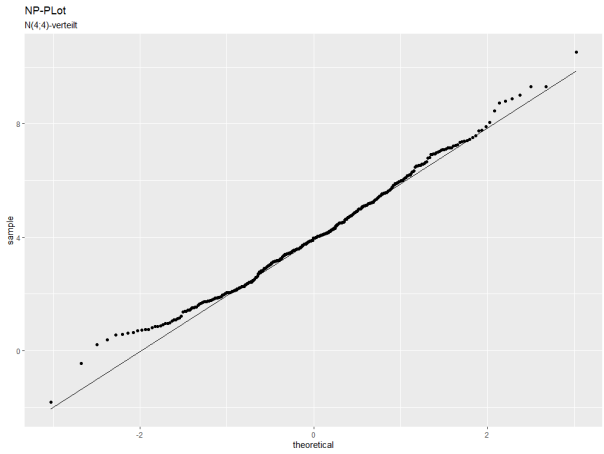


Abbildung 15: NP-Plot von N(4; 4) verteilten Daten

Im Idealfall müssten alle Punkte auf einer Geraden liegen. Unser Beispiel ist „zufällig“ recht schlecht: Werden in unserem konkreten Beispiel andere „Seed“-Werte als 0 gewählt, ergeben sich sehr oft bessere Übereinstimmungen. Abweichungen finden sich im Allgemeinen häufig bei sehr kleinen oder sehr großen Quantil-Werten. Eine bessere Übereinstimmung erhält man in der Regel auch, wenn die Anzahl der Daten erhöht wird.

Aus dem y-Achsenabschnitt der eingezeichneten Geraden kann der Erwartungswert und aus der Steigung der Geraden kann die Streuung der theoretisch ermittelten Normalverteilung, die am „besten“ zu dem Datensatz passt, abgelesen werden. Unabhängig vom „Seed“ sollten in unserem Beispiel stets ungefähr 1/3 als Erwartungswert und 2 als Steigung abzulesen sein.

Aufgabe

e) Erstellen Sie für unterschiedliche Seeds und unterschiedliche Datensatzgrößen entsprechende NP-Plots für Daten, die N(2; 9)-verteilt sind.

[8] Siehe z. B.: S. Kockelkorn, Ulrich: *Statistik für Anwender*. Heidelberg: Spektrum Akademischer Verlag, 2012, S. 173

Als Nächstes wollen wir die Daten aus dem letzten Kapitel durch einen NP-Plot visualisieren.

```
set.seed(1)
n <- 300
x <- sample(c(0,0,1),n, replace = TRUE)
s50 <- replicate(50, sum(sample(c(0,0,1),n, replace = TRUE))/n)
ggplot(data.frame(s50), aes(sample = s50))+
  stat_qq() + stat_qq_line() +
  labs(title = "NP-PLot", subtitle = "Anzahl=50")
```

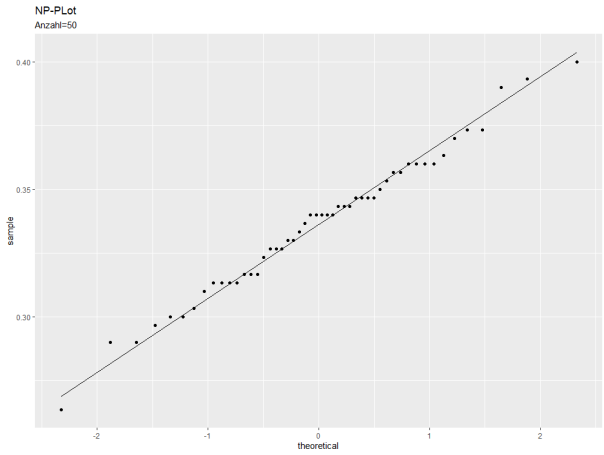


Abbildung 16: NP-Plot mit 50 Werten

Unabhängig vom „Seed“ sollten stets ungefähr 1/3 als Erwartungswert und $\sqrt{2/9n}$ (ca. 0.027 für den benutzten Wert n=300) als Steigung abzulesen sein. Ein ähnliches Bild ergibt sich auch für den zweiten Datensatz aus dem vorherigen Kapitel:

```
set.seed(1)
x <- sample(c(0,0,1),n, replace = TRUE)
s2000 <- replicate(2000, sum(sample(c(0,0,1),n, replace = TRUE))/n)
ggplot(data.frame(s2000), aes(sample = s2000))+
  stat_qq() + stat_qq_line() +
  labs(title = "NP-PLot", subtitle = "Anzahl=2000")
```

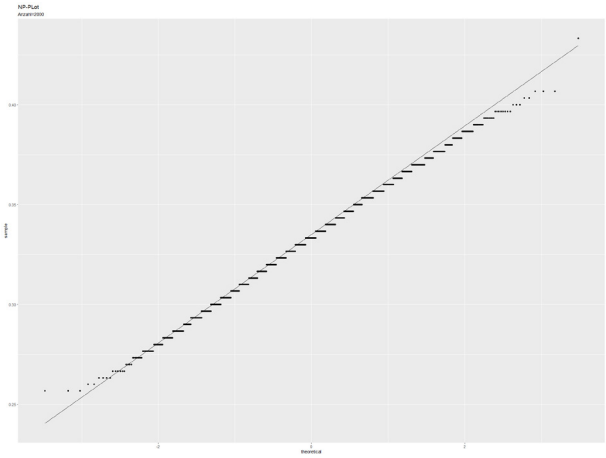


Abbildung 17: NP-Plot mit 2000 Werten

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

Spezielle Themen und Fallstudien 4

Datenimport

1 AUFGABE 2

2 ZU KONTROLLZWECKEN: ERZEUGUNG DER DATEN 3

3 IMPORTIEREN 4

1 Aufgabe

In dieser Fallstudie wollen wir uns mit dem Import von Daten beschäftigen. Weiterführende Ausführungen zu diesem Thema finden Sie im Kapitel „11 Data import“ in [R4DS][1].

Genauer wollen wir uns mit folgender Aufgabenstellung beschäftigen:

Gegeben ist die CSV-Datei „*diamonds100.csv*“, deren erste Zeilen wie folgt lauten:

```
1,21;Very Good;I;VS2;63,4;57; 5387;6,79;6,72;4,28
0,31;Ideal;E;VS1;61,2;55;  722;4,40;4,43;2,70
1,54;Premium;J;VS1;60,9;61; 6194;7,49;7,45;4,55
2,52;Good;G;SI2;63,7;59;17608;8,57;8,52;5,44
2,12;Ideal;H;SI2;62,4;57;16466;8,19;8,16;5,10
0,37;Ideal;E;VVS2;61,8;54;  896;4,61;4,64;2,86
0,57;Ideal;D;VS2;62,1;56; 1864;5,29;5,34;3,30
0,34;Ideal;G;VS2;62,1;55;  765;4,49;4,46;2,78
```

Die Datei „*diamonds100.csv*“ ist auf unserer e-learning-Plattform FH-SWF hinterlegt. Aufgabe ist es nun, diese Daten zu importieren und ein Objekt `diam` zu erzeugen, das *genau* folgende Eigenschaften hat:

```
> dd
# A tibble: 100 x 11
   carat cut       color clarity depth table   EUR     x     y     z Verfügbar
   <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl> <lgl>
 1  0.52 Good      H     VS2      63.5    58  1165  5.11  5.12  3.25 TRUE
 2  0.71 Ideal     E     VS1      61.5    55  3389  5.78  5.75  3.55 TRUE
 3  0.5  Very Good F     VS1      62.3    55  1546  5.06  5.09  3.16 TRUE
 4  0.54 Ideal     E     VS1      61      56  1845  5.35  5.24  3.23 FALSE
 5  0.9  Premium   D     SI1      61.2    60  3892  6.19  6.14  3.77 TRUE
 6  1.21 Good      J     SI2      63.7    55  3860  6.81  6.75  4.32 TRUE
 7  1.71 Ideal     G     VVS2     62.1    55 16819  7.63  7.66  4.75 TRUE
 8  0.71 Premium   F     SI2      59.6    59  2178  5.79  5.75  3.44 FALSE
 9  0.72 Ideal     I     VS2      61.7    56  2396  5.75  5.8   3.56 TRUE
10  0.31 Ideal     F     VS2      61.6    55   565  4.36  4.37  2.69 TRUE
# ... with 90 more rows
```

Aufgabe

a) Was bedeuten `<dbl>`, `<ord>`, `<int>`, `<lgl>` in der obigen Ausgabe?

```
> str(dd)
Classes ‘tbl_df’, ‘tbl’ and 'data.frame':                                    100 obs. o
f  11 variables:
 $ carat    : num  0.52 0.71 0.5 0.54 0.9 1.21 1.71 0.71 0.72 0.31 ...
 $ cut      : Ord.factor w/ 5 levels "Fair"<"Good"<..: 2 5 3 5 4 2 5 4 5 5 ...
 $ color    : Ord.factor w/ 7 levels "D"<"E"<"F"<"G"<..: 5 2 3 2 1 7 4 3 6 3 ...
 $ clarity  : Ord.factor w/ 8 levels "I1"<"SI2"<"SI1"<..: 4 5 5 5 3 2 6 2 4 4 ...
 $ depth    : num  63.5 61.5 62.3 61 61.2 63.7 62.1 59.6 61.7 61.6 ...
 $ table    : num  58 55 55 56 60 55 55 59 56 55 ...
 $ EUR      : int  1165 3389 1546 1845 3892 3860 16819 2178 2396 565 ...
 $ x        : num  5.11 5.78 5.06 5.35 6.19 6.81 7.63 5.79 5.75 4.36 ...
 $ y        : num  5.12 5.75 5.09 5.24 6.14 6.75 7.66 5.75 5.8 4.37 ...
 $ z        : num  3.25 3.55 3.16 3.23 3.77 4.32 4.75 3.44 3.56 2.69 ...
 $ Verfügbar: logi  TRUE TRUE TRUE FALSE TRUE TRUE ...
```

[1] https://r4ds.had.co.nz/transform.html

Wie im Datensatz `diamonds` sind die geordneten Level der einzelnen Faktoren:

```
cut: Fair < Good < Very Good < Premium < Ideal
color: D < E < F < G < H < I < J
clarity: I1 < SI2 < SI1 < VS2 < VS1 < VVS2 < VVS1 < IF
```

Damit Sie leicht Ihre importierten Daten mit den ursprünglichen Daten abgleichen können, zeigen wir im nächsten Kapitel zu Kontrollzwecken, wie die Daten ursprünglich erzeugt wurden. In Kapitel 3 zeigen wir mögliche Lösungswege auf.

2 Zu Kontrollzwecken: Erzeugung der Daten

Grundlage für unsere Daten ist der Datensatz `diamonds` (siehe auch Kapitel 8 des Skripts), aus dem wir 100 zufällige Daten wählen. Dazu nutzen wir die Funktion `sample_n` aus dem `dplyr`-Paket:

```
set.seed(071219)
dd <- sample_n(diamonds, 100)     #dplyr
```

Alternativ hätten wir auch in Base-R schreiben können:

```
set.seed(071219)
ddTmp <- diamonds[sample(nrow(diamonds), 100), ]
```

Wie wir sehen, erhalten wir identische Ergebnisse:

```
> identical(dd, ddTmp)
[1] TRUE
```

Zusätzlich wird die `price`-Spalte von Dollar in Euro umgerechnet und eine zusätzliche Spalte „`Verfügbar`“ mit ziemlich willkürlichen Werten eingeführt:

```
dd <- dd %>% mutate(price=as.integer(round(0.9043*price,2))) %>%  # in EURO
  rename(EUR=price) %>%
  mutate(Verfügbar = (round(EUR)%%3>0) )
```

Schließlich wird das Objekt `dd` als CSV-Datei in das aktuelle Verzeichnis geschrieben:

```
write_csv2(dd, "diamonds100.csv")
```

3 Importieren

Wie wir in Kapitel 5 im Skript gesehen haben, können wir CSV-Dateien mit `read.csv()` bzw. mit `read.csv2()` einlesen:

```
diam1 <- read.csv2("diamonds100.csv")
```

oder besser:

```
diam2 <- read.csv2("diamonds100.csv", fileEncoding = "UTF-8")
```

Aufgabe

b) Was ist der Unterschied zwischen `diam1` und `diam2`?

Wir erhalten:

```
> all.equal(dd, diam2)
[1] "Factor levels not equal for column `cut`"
[2] "Factor levels not equal for column `clarity`"
[3] "Incompatible type for column `table`: x numeric, y integer"
> identical(dd, diam2)
[1] FALSE
```

Es müssen also noch einige Änderungen an `diam2` vorgenommen werden, bevor die Objekte `diam` und `dd` exakt gleich sind. Wir wollen aber diesen Weg nicht weiterverfolgen. Stattdessen werden die CSV-Daten mit der Funktion `read_csv2()` aus dem `readr`-Paket einlesen. Das `readr`-Paket ist genau wie das `dplyr`-Paket Bestandteil von `tidyverse`. Die Funktion `read_csv2()` bzw. `read_csv()` kann auch mit komprimierten Dateien wie zip-Dateien umgehen. Ferner werden Dateinamen, die mit „`http://`“, „`https://`“, „`ftp://`“ oder „`ftps://`“ beginnen, automatisch aus dem Netz heruntergeladen. Im Vergleich zu `read.csv2()` scheint die Funktion `read_csv2()` insbesondere bei großen Eingabedateien viel schneller die Daten einzulesen.

```
> diam <- read_csv2("diamonds100.csv")
Using ',' as decimal and '.' as grouping mark. Use read_delim() for more control.
Parsed with column specification:
cols(
  carat = col_double(),
  cut = col_character(),
  color = col_character(),
  clarity = col_character(),
  depth = col_double(),
  table = col_double(),
  EUR = col_double(),
  x = col_double(),
  y = col_double(),
  z = col_double(),
  Verfügbar = col_logical()
)
```

Mithilfe von `str()` sehen wir, dass `diam` im Gegensatz zu `dd` keinerlei Faktoren als Spalten hat. Dies wird auch durch Folgendes deutlich:

```
> names(dd)[!sapply((1:11), function(i) {identical(dd[i], diam[i])})]
[1] "cut"     "color"   "clarity" "EUR"
```

Aufgabe

c) Erklären Sie die letzte Anweisung. Was ist der Grund für die Unterschiede?

Eine Möglichkeit, dies zu korrigieren, ist:

```
diam$EUR <- as.integer(diam$EUR)
diam$cut <- factor(diam$cut, levels=c("Fair","Good","Very Good","Premium","Ideal"),
                        ordered=TRUE)
diam$color <- factor(diam$color, ordered=TRUE)
diam$clarity <- factor(diam$clarity,levels=c("I1","SI2","SI1","VS2","VS1","VVS2","VVS1","IF"),
                          ordered=TRUE)
```

Wir wollen stattdessen einen anderen Weg verfolgen. Wir kopieren die Informationen `column specification`, die wir beim Aufruf `read_csv2()` erhalten haben, und passen Sie unseren Erfordernissen entsprechend an:

```
diam <- read_csv2("diamonds100.csv",col_types = cols(
  carat = col_double(),
  cut = col_factor(levels=c("Fair","Good","Very Good","Premium","Ideal"),
                        ordered =TRUE),
  color = col_factor(levels=LETTERS[4:10],
                          ordered =TRUE),
  clarity = col_factor(levels=c("I1","SI2","SI1","VS2","VS1","VVS2","VVS1","IF"),
                            ordered =TRUE),
  depth = col_double(),
  table = col_double(),
  EUR = col_integer(),
  x = col_double(),
  y = col_double(),
  z = col_double(),
  Verfügbar = col_logical())
)
```

Unser Ziel haben wir nun fast erreicht:

```
> all.equal(dd, diam)
[1] TRUE
> identical(dd, diam)
[1] FALSE
```

Aufgabe

d) Wieso sind `diam` und `dd` nicht identisch?[2]

Wie im Abschnitt „tibble subclass“ in https://www.tidyverse.org/blog/2018/12/readr-1-3-1/ erklärt, liefert aber erst Indizieren (subsetting) den richtigen Datentyp. Dies können wir leicht erreichen durch:

```
> diam <- diam[]
> identical(dd, diam)
```

Alternativ hätten wir dies auch erhalten durch:

```
diam <- diam %>% select(everything())
```

[2] Siehe: https://www.tidyverse.org/blog/2018/12/readr-1-3-1/ Abschnitt: tibble subclass

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

Spezielle Themen und Fallstudien 5

Vergleich der Teststärke des Shapiro-Wilk-Tests mit dem Kolmogorov-Smirnov-Test

1 SHAPIRO-WILK-TEST 2

1.1 FUNKTIONEN ZUR MEHRFACHEN WIEDERHOLUNG DES SHAPIRO-WILK-TESTS 3
1.2 TESTGÜTE DES SHAPIRO-WILK-TESTS BEI GLEICHVERTEILUNG 5

2 VERGLEICH MIT DEM KOLMOGOROV-SMIRNOV-TEST 6

1 Shapiro-Wilk-Test

In dieser Fallstudie wollen wir zuerst exemplarisch auf den Shapiro-Wilk-Test eingehen. Der Shapiro-Wilk-Test ist ein statistischer Signifikanztest, der die Hypothese überprüft, ob eine Normalverteilung vorliegt.

Die *Nullhypothese* H_0 lautet: *Die Daten der Grundgesamtheit sind normalverteilt*. Die Alternativhypothese H_1 lautet somit: *Die Daten der Grundgesamtheit sind nicht normalverteilt*.

Als *Signifikanzniveau* wählen wir erst einmal $\alpha = 0.05$[1].

Es ist in R sehr leicht, den Shapiro-Wilk-Test durchzuführen. Viel schwieriger ist es, die richtigen Schlüsse aus den berechneten Testergebnissen zu ziehen.

Zuerst wollen wir den Pseudo-Zufallsgenerator initialisieren:

```
set.seed(17122019)
```

Als Nächstes wollen wir eine Stichprobe bestehend aus 40 Elementen aus einer normalverteilen Grundgesamtheit testen:

```
> stichprobe <- rnorm(40, mean = 5, sd = 3)
> shapiro.test(stichprobe)

                 Shapiro-Wilk normality test

data:  stichprobe
W = 0.97987, p-value = 0.6846
```

Wie wir sehen, ist der p-Wert recht groß; wir haben also keinen Grund, die Nullhypothese H_0 abzulehnen. Selbst ein großer p-Wert belegt nicht, dass H_0 wahr ist. Dies Aussagekraft des Ergebnisses ist also in diesem Fall recht schwach.

Bei einer Stichprobe, die aus einer auf [0,1] gleichverteilten Grundgesamtheit stammt, sollten die Testergebnisse anders aussehen:

```
> stichprobe <- runif(40)
> shapiro.test(stichprobe)

                 Shapiro-Wilk normality test

data:  stichprobe
W = 0.88576, p-value = 0.0007558
```

Da der p-Wert unter dem gewählten Signifikanzniveau α liegt, ist die Nullhypothese H_0 in diesem Fall abzulehnen. Wir können aber nicht ausschließen, dass H_0 trotzdem richtig ist! Wird H_0 aufgrund des Tests (fälschlicherweise) abgelehnt, obwohl H_0 wahr ist (*Fehler 1. Art*), dann repräsentiert die Stichprobe ein sehr seltenes Ereignis, das höchstens mit Wahrscheinlichkeit α eintritt. Daraus lässt sich aber **nicht** ableiten, dass die Wahrscheinlichkeit, dass H_0 wahr ist, kleiner als das gewählte Signifikanzniveau α ist.[2]

[1] Dies ist ein üblicher, aber trotzdem sehr willkürlicher Wert
[2] Leider wird dieser Schluss fälschlicherweise sehr oft gezogen

Im Folgenden sehen wir, dass die Funktion `shapiro.test()` eine Liste mit vier Elementen zurückgibt und dass `p.value` dem p-Wert entspricht:

```
> sTest <- shapiro.test(stichprobe)
> str(sTest)
List of 4
 $ statistic: Named num 0.886
  ..- attr(*, "names")= chr "W"
 $ p.value  : num 0.000756
 $ method   : chr "Shapiro-Wilk normality test"
 $ data.name: chr "stichprobe"
 - attr(*, "class")= chr "htest"
```

1.1 Funktionen zur mehrfachen Wiederholung des Shapiro-Wilk-Tests

Wir wollen als Nächstes für unterschiedliche Stichprobengrößen ähnliche Tests mehrfach wiederholen und jeweils untersuchen, wie oft die entsprechende Nullhypothese verworfen wird. Dafür verwenden wir vier Funktionen. Die erste Funktion werden wir benutzen, um unsere erhaltenen Ergebnisdaten zu visualisieren:

```
myPlot <- function(titel, nWerte, Anteil){
  g <- ggplot(data.frame(nWerte, Anteil), aes(x = nWerte, y = Anteil)) +
    ggtitle(titel, subtitle =
              expression('H'[0]*': Normalverteilung liegt vor')) +
    ylab(expression('H'[0]*' verworfen')) +
    xlab("Stichproben-Größe: n") +
    scale_y_continuous(labels = scales::percent)
  return(g)
}
```

Wir haben hier den Funktionsaufruf `scale_y_continuous(labels = scales::percent)` benutzt, um die y-Achse mit den entsprechenden Prozentzahlen zu skalieren. Neu ist auch `expression('H'[0]*': Normalverteilung liegt vor')) .`
Damit wird der folgende Text im Plot erzeugt: „H_0 : `Normalverteilung liegt vor`“. Der mit `[]` umschlossene Buchstabe wird also tiefergestellt.[3] Auf die genaue Syntax wollen wir an dieser Stelle nicht eingehen, sondern lediglich ein weiteres Beispiel anführen:

```
ggplot() + ggtitle(expression('Beispiel: A'['1,2']* ' und B' ^'1,2'))
```

Die obige Anweisung ergibt den folgenden (leeren) Plot:

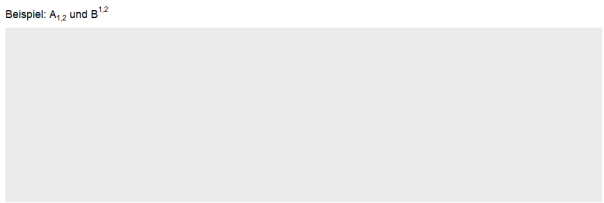


Abbildung 1: Leer Plot mit tiefer und höher gestellten Buchstaben im Titel

[3] Um einen Buchstaben hoch zu stellen, wird ^ statt `[]` benutzt.

Die zweite Funktion `pShapiro(n)` erzeugt in [-1,1] gleichverteilte Stichproben der Größe `n`, führt den Shapiro-Wilk-Test durch und gibt den entsprechenden p-Wert zurück. Wollen wir eine andere Wahrscheinlichkeitsverteilung zugrunde legen, dann muss diese Funktion angepasst werden.

```
# Shapiro-Wilk-Test
# Stichproben gleichverteilt
# n: Stichprobengröße
pShapiroTest <- function(n){
  stichprobe <- runif(n, min=-1, max=1)
  h <- shapiro.test(stichprobe)
  return(h$p.value)
}
```

Die dritte Funktion `replicateTest()` ruft `nTest`-mal die übergebene Funktion `FUN(n)` auf und gibt den Anteil zurück, wie oft der entsprechende p-Wert unter dem Signifikanzniveau α gelegen hat.

```
# n: Stichprobengröße
# FUN: Testfunktion (Default: pShapiroTest)
# nTest: Anzahl der Tests (Wiederholungen)
# alpha: entspricht dem Signifikanzniveau
replicateTest <- function(n, FUN=pShapiroTest, nTest, alpha){
  res <- replicate(nTest, FUN(n))
  return(sum(res <= alpha)/nTest)
}
```

In der vierten Funktion `testShapiro()` schließlich wird festgelegt:

- die Stichprobengröße `n`
- welcher Test benutzt werden soll (der Defaultwert ist die Funktion `pShapiroTest()`)
- wie oft (bei gegebener Stichprobengröße) ein Test ausgeführt werden soll (`nTest`)
- das Signifikanzniveau `alpha`

und es wird die Funktion `replicateTest()` aufgerufen:

```
testShapiro <- function(n){
  replicateTest(n, FUN=pShapiroTest, nTest=1000, alpha=0.05)
}
```

1.2 Testgüte des Shapiro-Wilk-Tests bei Gleichverteilung

Nun können wir sehr leicht die Testgüte des Shapiro-Wilk-Tests untersuchen und die Ergebnisse visuell darstellen. Nach wie vor lautet die Nullhypothese H_0: *Die Daten der Grundgesamtheit sind normalverteilt*. Die Alternativhypothese H_1 lautet somit: *Die Daten der Grundgesamtheit sind nicht normalverteilt*. Wir wissen, dass die Stichproben aus einer gleichverteilten Grundgesamtheit stammen, also ist die Nullhypothese H_0 falsch. Wir wollen nun untersuchen, *wie oft* die Nullhypothese H_0 durch den Test tatsächlich verworfen wird. Welche qualitativen Ergebnisse erwarten wir? Für relativ große Stichprobengrößen sollte ein Test in der Lage sein, in fast 100% der Fälle die Nullhypothese H_0 zu verwerfen. Je kleiner die Stichprobengröße hingegen ist, desto öfter erwarten wir, dass aufgrund des p-Wertes die Nullhypothese H_0 nicht abgelehnt wird. Dies können wir nun leicht nachprüfen. Als Stichprobengröße n wählen wir
n = 3, 4, 5,…,100:

```
set.seed(17122019)
nWerte <- 3:100
AnteilShapiro <- sapply(nWerte, testShapiro)

myPlot("Shapiro-Wilk-Test: gleichverteilte Stichproben (nTest=1000)",
                                                  nWerte, AnteilShapiro)  +
  geom_point() + geom_smooth()
```

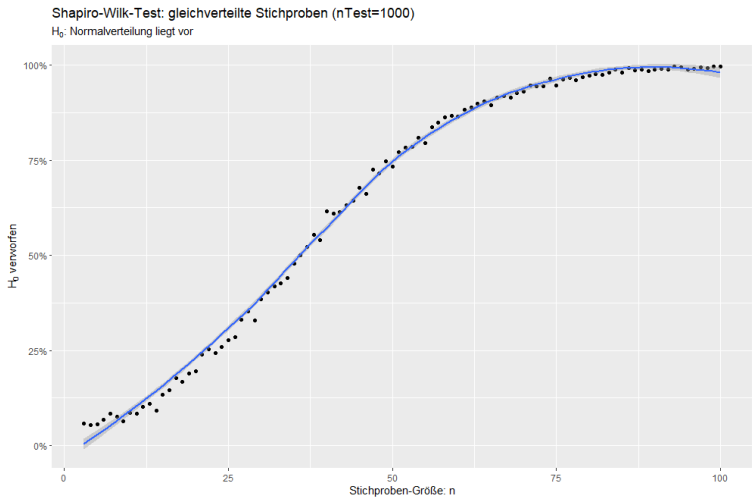


Abbildung 2: Testgüte des Shapiro-Wilk-Tests

Ändert sich der Plot wesentlich, wenn die Untersuchung mit anderen zufälligen Werten wiederholt wird? Auch dies können wir sofort nachprüfen.

```
AnteilShapiro2 <- sapply(nWerte, testShapiro)

myPlot("Shapiro-Wilk-Test: gleichverteilte Stichproben (nTest=1000)", nWerte, AnteilS
hapiro)  +
  geom_point() + geom_smooth() +
  geom_point(aes(x = nWerte, y = AnteilShapiro2), color = "green")
```

Die neu hinzu gekommenen Daten wurden in den letzten Plot als grüne Punkte eingefügt. Wie man sieht, gibt es nur geringfügige Abweichungen.

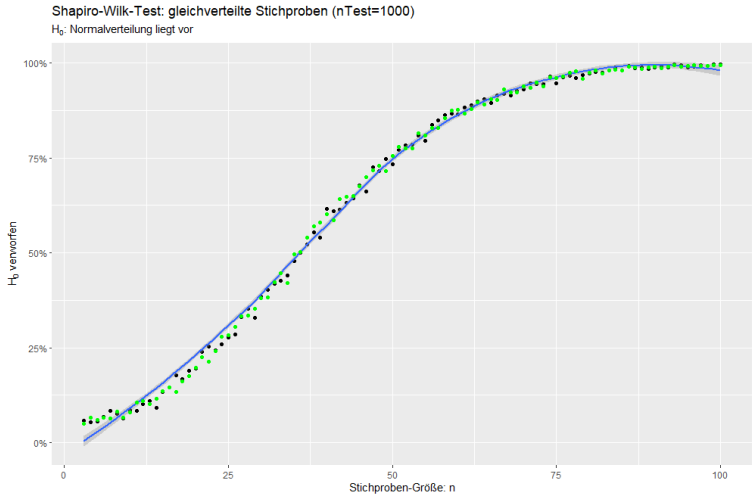


Abbildung 3: Wiederholung mit anderen zufälligen Werten

Aufgabe

a) Wiederholen Sie die Untersuchung mit einer χ^2-Verteilung (Chi-Quadrat-Verteilung) mit zwei Freiheitsgraden anstelle der Gleichverteilung. Wie unterscheiden sich die Ergebnisse? Plotten Sie zuerst die χ^2-Verteilung mit zwei Freiheitsgraden.

b) Wiederholen Sie die Untersuchung mit einer Normalverteilung anstelle der Gleichverteilung. Was ändert sich nun grundsätzlich? Welchen Einfluss hat nun die Stichprobengröße `n` und das Signifikanzniveau `alpha`. Testen Sie dies!

2 Vergleich mit dem Kolmogorov-Smirnov-Test

Mithilfe unserer oben definierten Funktionen ist es nun sehr leicht, den Shapiro-Wilk-Test `shapiro.test()` durch den Kolmogorov-Smirnov-Test `ks.test()` zu ersetzen und beide Tests direkt zu vergleichen.

```
pKSTest <- function(n){
  stichprobe <- runif(n, min=-1, max=1)
  h <- ks.test(stichprobe, "pnorm")
  return(h$p.value)
}

testKS <- function(n){
  replicateTest(n, FUN=pKSTest, nTest=1000, alpha=0.1)
}
```

```
set.seed(17122019)
nWerte <- 3:100
AnteilKS <- sapply(nWerte, testKS)
```

Ähnlich wie im vorherigen Abschnitt können wir die berechneten Anteile (`AnteilKS`), wie oft die Nullhypothese H_0 verworfen wurden, visualisieren:

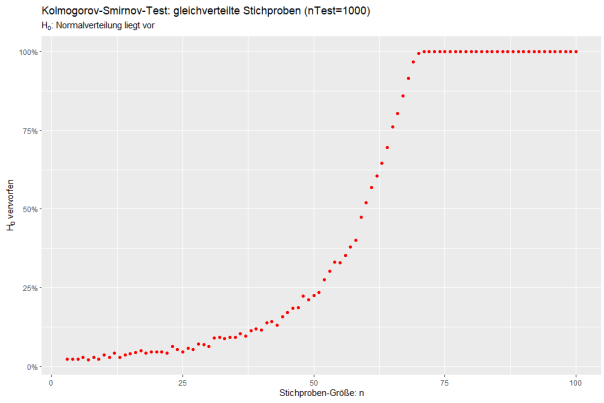


Abbildung 4: Testgüte des Kolmogorov-Smirnov-Tests

Um die Ergebnisse besser vergleichen zu können, erweitern wir Abbildung 2 um die neu berechneten roten Punkte und erhalten Abbildung 5:

```
myPlot("Gleichverteilte Stichproben (nTest=1000)",
       nWerte, AnteilShapiro)  +
  geom_point() + geom_smooth() +
  geom_point(aes(x = nWerte, y = AnteilKS), color = "red")
```

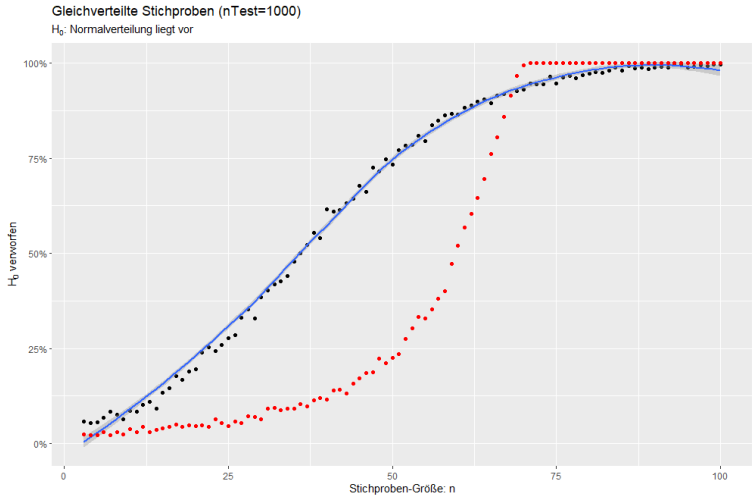


Abbildung 5: Vergleich der Testgüte: Shapiro-Wilk-Test (schwarz) und Kolmogorov-Smirnov-Tests (rot)

Wir können die Ergebnisse auch in Abbildung 3 einfügen, sollten dann aber auch eine entsprechende Legende hinzufügen:

```
myPlot("Gleichverteilte Stichproben (nTest=1000)", nWerte, AnteilShapiro)  +
  geom_point(aes(color = "Shapiro 1")) + geom_smooth() +
  geom_point(aes(x = nWerte, y = AnteilShapiro2, color = "Shapiro 2")) +
  geom_point(aes(x = nWerte, y = AnteilKS, color = "KS")) +
  labs(color = "Legende")
```

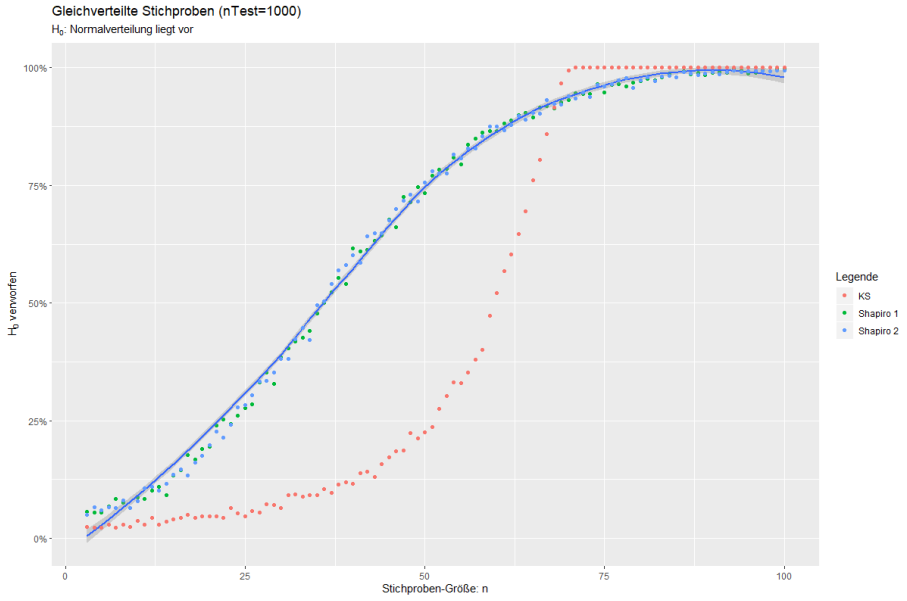


Abbildung 6: Alle Ergebnisse

In der Tat sehen wir, dass in unserem speziellen Testmodell der Shapiro-Wilk-Test für Stichprobengrößen $n \leq 65$ besser[4] als der Kolmogorov-Smirnov-Test abschneidet.

Aufgabe

c) Wie sieht der Vergleich des Shapiro-Wilk-Tests mit dem Kolmogorov-Smirnov-Test aus für Stichprobengrößen n zwischen 4 und 100, wenn statt der Gleichverteilung eine χ^2-Verteilung (Chi-Quadrat-Verteilung) mit zwei Freiheitsgraden zugrunde gelegt wird?

[4] Für $n \leq 60$ sogar deutlich besser

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse

Spezielle Themen und Fallstudien 6

Datenanalyse am Beispiel des Datensatzes diamonds

1 DATENANALYSE DER X-, Y- UND Z-WERTE 2

1.1 NA-WERTE 3
1.2 BEREINIGUNG DER X-, Y- UND Z-WERTE 4
1.3 WEITERE AUSREIßER 7
1.4 LINEARE REGRESSION 8
1.5 VERGLEICH DER X- UND Z-VARIABLEN 11

2 ALLGEMEINER ÜBERBLICK 12

```
require(tidyverse)
```

1 Datenanalyse der x-, y- und z-Werte

Dieser Fallstudie liegt der Datensatz `diamonds` zugrunde. In einer Übung haben wir bereits Auffälligkeiten in der Verteilung der Preis-Variablen aufgedeckt. In dieser Fallstudie wollen wir nach Auffälligkeiten in den `x`-, `y`- und `z`-Variablen suchen und die gefundenen Werte im Datensatz kennzeichnen.

```
dd <- diamonds
```

Aufgabe

a) Fügen Sie ein weiteres Attribut `nr` dem Datensatz `dd` hinzu. Dieses Attribut soll das erste Attribut des Datensatzes `dd` sein und der Zeilennummer entsprechen (Hinweis: `row_number()`):

```
> dd
# A tibble: 53,940 x 11
      nr carat cut       color clarity depth table price     x     y     z
   <int> <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
 1     1 0.23  Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
 2     2 0.21  Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
 3     3 0.23  Good      E     VS1      56.9    65   327  4.05  4.07  2.31
 4     4 0.290 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
 5     5 0.31  Good      J     SI2      63.3    58   335  4.34  4.35  2.75
 6     6 0.24  Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
 7     7 0.24  Very Good I     VVS1     62.3    57   336  3.95  3.98  2.47
 8     8 0.26  Very Good H     SI1      61.9    55   337  4.07  4.11  2.53
 9     9 0.22  Fair      E     VS2      65.1    61   337  3.87  3.78  2.49
10    10 0.23  Very Good H     VS1      59.4    61   338  4     4.05  2.39
# … with 53,930 more rows
```

Aufgabe

b) Für wie viele Datensätze gilt $x \geq y$ und für wie viele $x < y$? Ändern Sie die einzelnen Datensätze, so dass stets $x \geq y$ gilt.[1]

c) Für welche Datensätze gilt: $z > y$?

In dieser Fallstudie werden wir erst einmal lediglich die Werte der Variablen `x`, `y` und `z` in unsere Datenanalyse einfließen lassen. Dabei sollen zunächst zwei Aspekte beleuchtet werden:

- Gibt es bezüglich der drei Variablen `x`, `y` und `z` Werte, die nicht definiert oder offensichtlich fehlerhaft sind? Der Datensatz soll entsprechend bereinigt und die ausgeschlossenen Datensätze dokumentiert werden.
- Gibt es in den so bereinigten Daten noch *extrem* auffällige `x`, `y` oder `z` Werte?

[1] In einer zusätzlichen Übungsaufgabe wird behandelt, wie in einem Tibble zeilenweise bezüglich mehrerer Spalten sortiert werden kann.

1.1 NA-Werte

Zu Testzwecken wollen wir zuerst den folgenden Datensatz betrachten:

```
d <- tibble(x=1:100, y=2*x, z=x%%7)
d$x[99] <- NA
d$y[22:26] <- NA
d$z[c(23,33)] <- NA
```

Um alle Datenzeilen herauszufinden, die einen `NA`-Wert enthalten, können wir schreiben:

```
> d %>% filter(is.na(x)|is.na(y)|is.na(z))
# A tibble: 7 x 3
      x     y     z
  <int> <dbl> <dbl>
1    22    NA     1
2    23    NA    NA
3    24    NA     3
4    25    NA     4
5    26    NA     5
6    33    66    NA
7    NA   198     1
```

Alternativ können wir auch die Funktion `complete.cases()` benutzen. Sie gibt einen booleschen Vektor zurück, der angibt, ob eine entsprechende Datenzeile einen `NA`-Wert enthält:

```
> complete.cases(d)
  [1]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
 [15]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE  TRUE
 [29]  TRUE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
 [43]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
 [57]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
 [71]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
 [85]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
 [99] FALSE  TRUE
```

Somit erhalten wir:

```
d[!complete.cases(d),]
# A tibble: 7 x 3
      x     y     z
  <int> <dbl> <dbl>
1    22    NA     1
2    23    NA    NA
3    24    NA     3
4    25    NA     4
5    26    NA     5
6    33    66    NA
7    NA   198     1
```

```
> sum(!complete.cases(d))
[1] 7
```

Des Weiteren gibt es die `tidyr`-Funktion `drop_na()`, die aus einem Datensatz alle Zeilen entfernt, die mindestens einen `NA`-Wert enthalten:

```
> d0 <- d %>% drop_na()
> sum(!complete.cases(d0))
[1] 0
```

Also können wir nun leicht feststellen, dass unsere Diamantendaten `dd` keine `NA`-Werte enthalten:

```
sum(!complete.cases(dd))
[1] 0
```

1.2 Bereinigung der x-, y- und z-Werte

Wie verschaffen wir uns am besten einen ersten Überblick über die Verteilung der `x`-Werte? Es folgen vier Vorschläge:

```
summary(dd$x)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  0.000   4.740   5.730   5.757   6.560  58.900 
```

```
ggplot(data = dd, mapping = aes(x="", y = x)) +
  geom_boxplot()
```

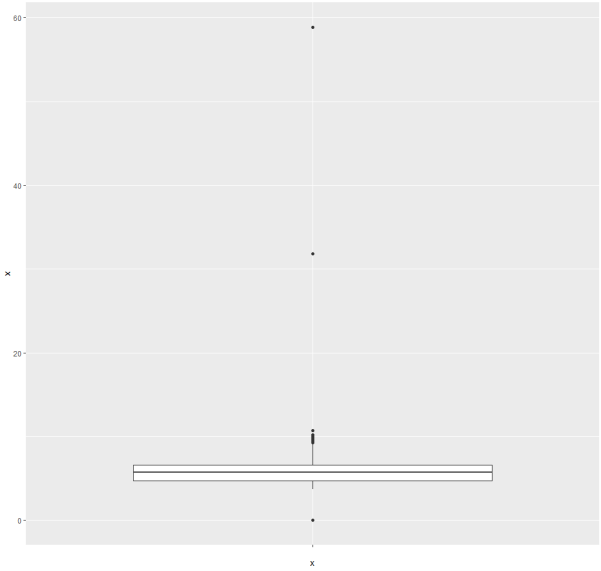


Abbildung 1: Ein erster Boxplot der x-Werte

```
ggplot(dd, aes(x = x)) + geom_histogram(binwidth=0.5, boundary=0)
```

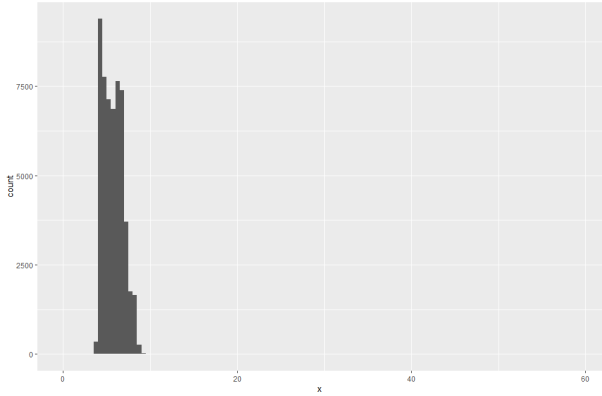


Abbildung 2: Ein Histogramm der x-Werte

```
dd %>% count(cut_width(x, 0.5, boundary = 0))
# A tibble: 18 x 2
   `cut_width(x, 0.5, boundary = 0)`     n
   <fct>                             <int>
 1 [0,0.5]                               7
 2 (3.5,4]                             337
 3 (4,4.5]                            9385
 4 (4.5,5]                            7767
 5 (5,5.5]                            7139
 6 (5.5,6]                            6863
 7 (6,6.5]                            7640
 8 (6.5,7]                            7393
 9 (7,7.5]                            3702
10 (7.5,8]                            1752
11 (8,8.5]                            1655
12 (8.5,9]                             256
13 (9,9.5]                              30
14 (9.5,10]                              7
15 (10,10.5]                             4
16 (10.5,11]                             1
17 (31.5,32]                             1
18 (58.5,59]                             1
```

Ähnlich können wir auch mit den y- und z-Werten verfahren. Es ist leicht, sich die fraglichen Datensätze genauer anzuschauen:

```
dd %>% filter(x==0 | y==0 | z==0 )
# A tibble: 20 x 11
      nr carat cut       color clarity depth table price     z     x     y
   <int> <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
 1  2208  1    Premium   G     SI2      59.1    59  3142     0  6.55  6.48
 2  2315  1.01 Premium   H     I1       58.1    59  3167     0  6.66  6.6
 3  4792  1.1  Premium   G     SI2      63      59  3696     0  6.5   6.47
 4  5472  1.01 Premium   F     SI2      59.2    58  3837     0  6.5   6.47
 5 10168  1.5  Good      G     I1       64      61  4731     0  7.15  7.04
 6 11183  1.07 Ideal     F     SI2      61.6    56  4954     0  6.62  0
 7 11964  1    Very Good H     VS2      63.3    53  5139     0  0     0
 8 13602  1.15 Ideal     G     VS2      59.2    56  5564     0  6.88  6.83
 9 15952  1.14 Fair      G     VS1      57.5    67  6381     0  0     0
10 24395  2.18 Premium   H     SI2      59.4    61 12631     0  8.49  8.45
11 24521  1.56 Ideal     G     VS2      62.2    54 12800     0  0     0
12 26124  2.25 Premium   I     SI1      61.3    58 15397     0  8.52  8.42
13 26244  1.2  Premium   D     VVS1     62.1    59 15686     0  0     0
14 27113  2.2  Premium   H     SI1      61.2    59 17265     0  8.42  8.37
15 27430  2.25 Premium   H     SI2      62.8    59 18034     0  0     0
16 27504  2.02 Premium   H     VS2      62.7    53 18207     0  8.02  7.95
17 27740  2.8  Good      G     SI2      63.8    58 18788     0  8.9   8.85
18 49557  0.71 Good      F     SI2      64.1    60  2130     0  0     0
19 49558  0.71 Good      F     SI2      64.1    60  2130     0  0     0
20 51507  1.12 Premium   G     I1       60.4    59  2383     0  6.71  6.67
```

```
dd %>% filter(x>30 | y>30 | z>30 )
# A tibble: 3 x 11
     nr carat cut       color clarity depth table price     z     x     y
  <int> <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1 24068  2    Premium   H     SI2      58.9  57   12210  8.06 58.9   8.09
2 48411  0.51 Very Good E     VS1      61.8  54.7  1970 31.8   5.15  5.12
3 49190  0.51 Ideal     E     VS1      61.8  55    2075  5.12 31.8   5.15
```

Als Nächstes wollen wir die fraglichen Werte auf `NA` setzen:

```
dd <- dd %>%
  mutate (x=ifelse(x==0 | x>30, NA, x) ) %>%
  mutate (y=ifelse(y==0 | y>30, NA, y) ) %>%
  mutate (z=ifelse(z==0 | z>30, NA, z) )
```

und danach die Daten überprüfen:

```
> summary(dd$x)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
  3.730   4.740   5.730   5.756   6.560  10.740       9
> summary(dd$y)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   3.68    4.70    5.68    5.71    6.51   10.54       8
> summary(dd$z)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   1.07    2.91    3.53    3.54    4.04    8.06      21
```

Beim Aufruf von `geom_boxplot()` erhalten wir nun die Warnung: „`Removed 9 rows containing non-finite values (stat_boxplot).`“:

```
dd %>% ggplot(aes(x="", y = x)) + geom_boxplot()
```

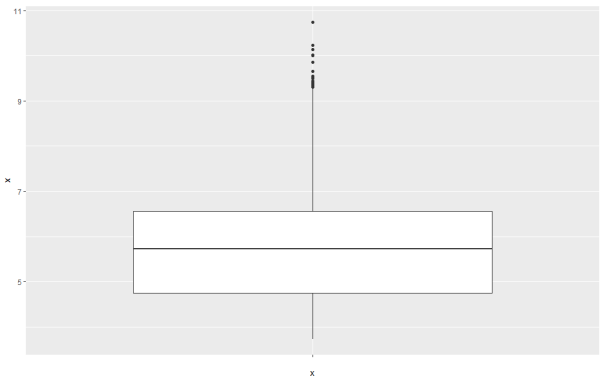


Abbildung 3: Ein erster Boxplot der bereinigten x-Werte

Zu Dokumentationszwecken speichern wir die fraglichen Datensätze ab und schreiben sie in eine CSV-Datei:

```
> dd.na <- dd[!complete.cases(dd),]
> dd.na
# A tibble: 23 x 11
      nr carat cut       color clarity depth table price     z     x     y
   <int> <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
 1  2208  1    Premium   G     SI2      59.1    59  3142 NA     6.55  6.48
 2  2315  1.01 Premium   H     I1       58.1    59  3167 NA     6.66  6.6
 3  4792  1.1  Premium   G     SI2      63      59  3696 NA     6.5   6.47
 4  5472  1.01 Premium   F     SI2      59.2    58  3837 NA     6.5   6.47
 5 10168  1.5  Good      G     I1       64      61  4731 NA     7.15  7.04
 6 11183  1.07 Ideal     F     SI2      61.6    56  4954 NA     6.62 NA
 7 11964  1    Very Good H     VS2      63.3    53  5139 NA    NA    NA
 8 13602  1.15 Ideal     G     VS2      59.2    56  5564 NA     6.88  6.83
 9 15952  1.14 Fair      G     VS1      57.5    67  6381 NA    NA    NA
10 24068  2    Premium   H     SI2      58.9    57 12210  8.06 NA     8.09
# … with 13 more rows
```

```
write_csv2(dd.na, "diamonds_fraglich1.csv")
```

Danach bereinigen wir unsere Daten, in dem wir alle Datensätze löschen, die einen `NA`-Wert enthalten. Da dies nur einen äußerst geringen Anteil der gesamten Daten betrifft, verlieren wir hierdurch nicht viel Information.

```
dd <- dd %>% drop_na()
```

1.3 Weitere Ausreißer

Wie man z. B. in Abbildung 3 sieht, gibt es in den `x`-Werten nach oben einige deutliche Ausreißer. Ob dies fehlerhafte oder fragliche Werte sind, können wir ohne spezielles Anwenderwissen nicht beurteilen. Für die weitere Analyse scheint es sinnvoll zu sein, die Zusammenhänge zwischen den `x`- und `y`-Werten genauer zu betrachten. Einen ersten Hinweis erhalten wir über die Korrelation:

```
cor(dd$x, dd$y)
[1] 0.9995239
```

Es gibt also einen starken linearen Zusammenhang zwischen den x- und y-Werten. Dies sieht man auch im folgenden Plot:

```
ggplot(dd, aes(x = x, y = y)) +
  geom_point(alpha =0.5) +
  geom_smooth(method='lm')
```

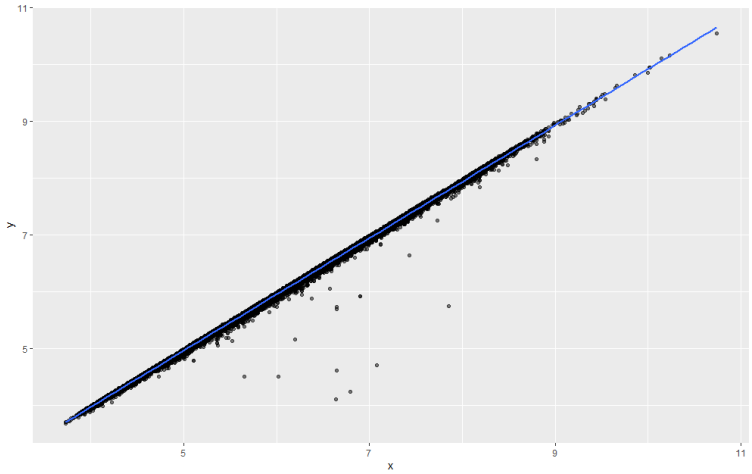


Abbildung 4: y- und x-Werte mit einer Ausgleichsgeraden (geom_smooth())

Bei diesem Plot haben wir mithilfe von `geom_smooth(method='lm')` eine Ausgleichgerade erzeugt. `'lm'` steht hier für „linear regression“. Im nächsten Abschnitt wollen wir mithilfe lineare Regression untersuchen, wie wir die Ausreißer unterhalb der Geraden herausfiltern.

1.4 Lineare Regression

Mithilfe linearer Regression können wir die Ausgleichsgerade in Abbildung 4 auch direkt berechnen. Dazu nutzen wir die mächtige Funktion `lm()`:

```
> mod_xy <- lm(y ~ x , data = dd)
```

Der Ausdruck `y ~ x` beschreibt eine Formel (`formula`) in R. Eine Formel ist ein eigener Datentyp; sie ist typischerweise von der Form `y ~ model`, wobei `model` aus einer Reihe von Variablen (Einflussgrößen) besteht, die die Größe `y` beschreiben sollen und für die entsprechenden Modell-Parameter[2] bestimmt werden sollen (wie zum Beispiel durch Kleinste-Quadrat-Schätzer der Regressionskoeffizienten für lineare Modelle). Tabelle 1 listet einfache lineare Modelle auf.

Formel	Modell
`y ~ x`	$y = \beta x + \alpha$
`y ~ x-1`	$y = \beta x$
`y ~ x,z`	$y = \beta x + \gamma z + \alpha$

Tabelle 1: Einfache lineare Modelle

```
> mod_xy

Call:
lm(formula = y ~ x, data = dd)

Coefficients:
(Intercept)            x
    0.00308      0.99142
```

[2] In der folgenden Tabelle sind die zu bestimmenden Modell-Parameter: α, β und γ

Die Funktion `lm()` gibt eine Liste zurück, genauer ein Objekt der Klasse `lm`. In unserem einfachen Fall erhalten wir in der Ausgabe die Steigung und den y-Achsenabschnitt der gefundenen Ausgleichsgeraden. Im Listenelement `coefficients` sind die berechneten Modell-Parameter hinterlegt. In unserem Fall sind dies die Steigung und der y-Achsenabschnitt der berechneten Ausgleichsgeraden.

```
alpha <- mod_xy$coefficients[1]
beta <- mod_xy$coefficients[2]

ggplot(dd, aes(x = x, y = y)) +
  geom_point(alpha = 0.5) +
  geom_abline(intercept = beta, slope = alpha, color = "magenta", size = 1) +
  ggtitle("Berechnete Ausgleichsgerade")
```

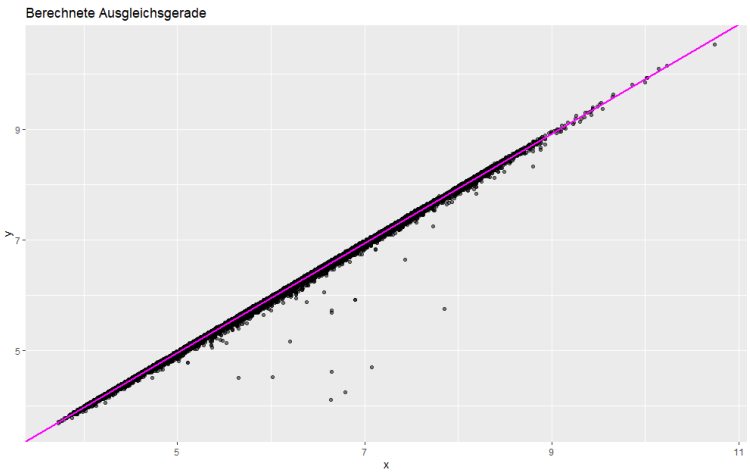


Abbildung 5: Berechnete Ausgleichsgerade

Die *Residuen*, also die Abstände zwischen den Datenpunkten und den jeweils korrespondierenden Punkten[3] auf der Ausgleichsgeraden erhalten wir durch: `dd$y-(beta*dd$x+alpha)`. Alternativ können wir diese Werte auch über das Objekt `mod_xy` erhalten:

```
all.equal(as.double(dd$y-(beta*dd$x+alpha)), as.double(mod_xy$residuals))
[1] TRUE
```

Die Funktion `lm()` berechnet noch viel mehr. Dies wird deutlich beim Aufruf von `summary(mod_xy)`. Hierdurch erhält man zusätzliche Informationen über die Güte des Schätzers, die Residuen und die daraus abgeleiteten statistischen Kenngrößen (wie Standard-Fehler, t-Test- und F-Test-Werte), auf die wir aber hier nicht weiter eingehen werden:

[3] Zu einem Datenpunkt mit den Koordinaten (x,y) ist dies der Punkt auf der Ausgleichsgeraden mit derselben x-Koordinate

```
> summary(mod_xy)

Call:
lm(formula = y ~ x, data = dd)

Residuals:
     Min       1Q   Median       3Q      Max 
-2.49483 -0.00830  0.00431  0.01458  0.06636 

Coefficients:
             Estimate Std. Error  t value Pr(>|t|)    
(Intercept) 0.0030796  0.0007728    3.985 6.77e-05 ***
x           0.9914212  0.0001318 7521.858  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.03427 on 53915 degrees of freedom
Multiple R-squared:  0.999, Adjusted R-squared:  0.999 
F-statistic: 5.658e+07 on 1 and 53915 DF,  p-value: < 2.2e-16
```

In unserem Fall sehen wir, dass die Bestimmtheitsmaße `Multiple R-squared` und `Adjusted R-squared` nahe bei 1 liegen; dies bedeutet, dass das gewählte Modell[4] sehr gut zu den Daten passt. Dies überrascht nicht, da Abbildung 4 und Abbildung 5 denselben Schluss nahelegen.

Als Nächstes speichern wir die Residuen in einer neuen Spalte:

```
dd <- dd %>% mutate(res_xy = y-(beta*x+alpha))
```

Im Folgenden wollen wir diejenigen Diamanten kennzeichnen, die bezüglich der Variablen `res_xy` auffällige Werte haben. Hierzu betrachten wir ähnlich wie bei einem Boxplot die Ausreißer:

```
> schranke <-  median(dd$res_xy) - 10 * IQR(dd$res_xy)
> dd %>% filter( abs(res_xy) > abs(schranke)) %>% count()
# A tibble: 1 x 1
      n
  <int>
1    29
```

`IQR(dd$res_xy)` gibt den mittleren Quartilsabstand zurück, entspricht also der Differenz `quantile(dd$res_xy, 0.75) - quantile(dd$res_xy, 0.25)`. Der Faktor 10 wurde hier mehr oder weniger willkürlich gewählt. Wie die obige Ausgabe zeigt, gibt es 29 Ausreißer (beim Faktor 10). In `dd` fügen wir nun eine neue Spalte ein, die die Ausreißer kennzeichnet:

```
dd <- dd %>% mutate(Atypisch = "NONE") %>%
  mutate(Atypisch = ifelse( abs(res_xy) > abs(schranke), "Shape_xy", Atypisch)) %>%
  select(nr, Atypisch, everything())

dd$Atypisch <- as.factor(dd$Atypisch)
```

[4] In unserem Fall ein einfaches lineares Modell

Die auffälligen Diamanten geordnet nach den Residuen sind dann somit:

```
> dd %>% filter(Atypisch != "NONE" ) %>%
+   arrange(res_xy)
# A tibble: 29 x 13
      nr Atypisch carat cut     color clarity depth table price     z     x     y res_xy
   <int> <fct>    <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>  <dbl>
 1  2275 Shape_xy  1.22 Premium J     SI2      62.6    59  3156  3.76  6.79  4.24 -2.49
 2 15236 Shape_xy  1.12 Premium H     VS1      61.6    58  6115  3.7   6.64  4.11 -2.48
 3 17827 Shape_xy  1.51 Fair    H     VS2      65.6    56  7188  4     7.08  4.7  -2.32
 4 19503 Shape_xy  1.57 Fair    H     VS1      67.3    66  8133  3.87  7.85  5.75 -2.04
 5 33103 Shape_xy  0.34 Good    E     VS2      57      61   816  2.64  6.65  4.61 -1.99
 6 34283 Shape_xy  0.39 Premium H     SI2      61.2    58   468  4.44  6.02  4.51 -1.46
 7 49916 Shape_xy  0.7  Premium F     SI1      62.6    55  2184  3     5.65  4.5  -1.10
 8 48833 Shape_xy  0.53 Ideal   F     VVS2     62.7    56  2030  3.25  6.2   5.16 -0.990
 9 14030 Shape_xy  1.22 Ideal   H     SI2      61.1    57  5703  4.22  6.9   5.92 -0.924
10 14345 Shape_xy  1.22 Ideal   H     SI2      61.1    57  5807  4.22  6.9   5.92 -0.924
# … with 19 more rows
```

Die Ergebnisse können leicht visualisiert werden:

```
ggplot(dd, aes(x = x, y = y, color=Atypisch)) +
  geom_point(alpha = 0.5) +
  scale_colour_brewer(palette="Dark2") +
  geom_abline(intercept = alpha, slope = beta,size = 1) +
  ggtitle("Auffällige x-, und y-Werte")
```

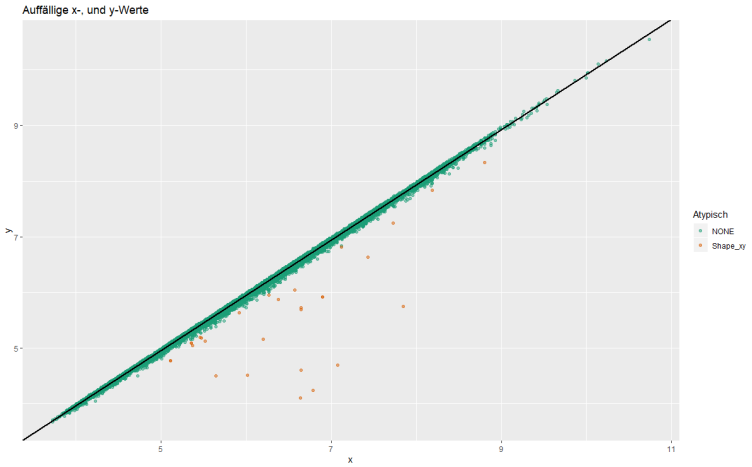


Abbildung 6: Ausreißer bzgl. x- und y-Werte

1.5 Vergleich der x- und z-Variablen

Ähnlich kann nun auch beim Vergleich der x- und z-Variablen vorgegangen werden. Wie der nächste Plot zeigt, scheint es auch hier Ausreißer zu geben:

```
ddd <- dd %>% filter(Atypisch == "NONE" )

ggplot(ddd, aes(x = x, y = z)) +
  geom_point(alpha =0.5) +
  geom_smooth(method='lm')
```

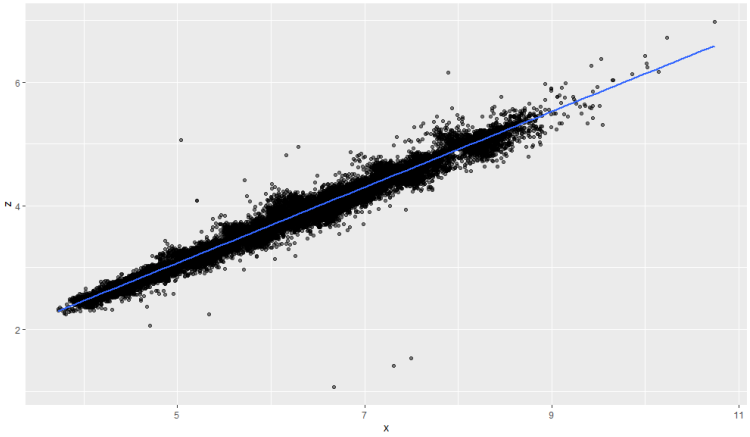


Abbildung 7: Ausreißer bzgl. x- und z-Werte

Die restliche Analyse der x- und z-Variablen und der y- und z-Variablen wird als zusätzliche Übungsaufgabe gestellt.

2 Allgemeiner Überblick

In diesem Kapitel wollen wir kurz verschiedene Möglichkeiten vorstellen, wie man sich relativ schnell einen ersten groben Überblick über einen Datensatz verschaffen kann. Als Beispiel benutzten wir den Datensatz `diamonds`. Da `diamonds` ein Tibble ist, ist es leicht erste Informationen zu erhalten:[5]

```
> diamonds
# A tibble: 53,940 x 10
   carat cut       color clarity depth table price     x     y     z
   <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
 1 0.23  Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
 2 0.21  Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
 3 0.23  Good      E     VS1      56.9    65   327  4.05  4.07  2.31
 4 0.290 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
 5 0.31  Good      J     SI2      63.3    58   335  4.34  4.35  2.75
 6 0.24  Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
 7 0.24  Very Good I     VVS1     62.3    57   336  3.95  3.98  2.47
 8 0.26  Very Good H     SI1      61.9    55   337  4.07  4.11  2.53
 9 0.22  Fair      E     VS2      65.1    61   337  3.87  3.78  2.49
10 0.23  Very Good H     VS1      59.4    61   338  4     4.05  2.39
# … with 53,930 more rows
```

Informativ ist auch:

```
> summary(diamonds)
     carat               cut        color       clarity          depth      
 Min.   :0.2000   Fair     : 1610   D: 6775   SI1    :13065   Min.   :43.00  
 1st Qu.:0.4000   Good     : 4906   E: 9797   VS2    :12258   1st Qu.:61.00  
 Median :0.7000   Very Good:12082   F: 9542   SI2    : 9194   Median :61.80  
 Mean   :0.7979   Premium  :13791   G:11292   VS1    : 8171   Mean   :61.75  
 3rd Qu.:1.0400   Ideal    :21551   H: 8304   VVS2   : 5066   3rd Qu.:62.50  
 Max.   :5.0100                     I: 5422   VVS1   : 3655   Max.   :79.00  
                                    J: 2808   (Other): 2531                  
     table           price             x                y                z       
 Min.   :43.00   Min.   :  326   Min.   : 0.000   Min.   : 0.000   Min.   : 0.000  
 1st Qu.:56.00   1st Qu.:  950   1st Qu.: 4.710   1st Qu.: 4.720   1st Qu.: 2.910  
 Median :57.00   Median : 2401   Median : 5.700   Median : 5.710   Median : 3.530  
 Mean   :57.46   Mean   : 3933   Mean   : 5.731   Mean   : 5.735   Mean   : 3.539  
 3rd Qu.:59.00   3rd Qu.: 5324   3rd Qu.: 6.540   3rd Qu.: 6.540   3rd Qu.: 4.040  
 Max.   :95.00   Max.   :18823   Max.   :10.740   Max.   :58.900   Max.   :31.800  
```

[5] Für einen normalen Data Frame wäre `str()` die vielleicht bessere Lösung.

Mithilfe von Korrelationen kann die lineare Abhängigkeit der einzelnen Variablen untersucht werden:

```
cor(diamonds)
Fehler in cor(diamonds) : 'x' muss numerisch sein
```

Also besser:

```
> d <- diamonds  %>%  select(carat, depth, price,x,y,z)
> cor(d)
            carat       depth      price           x           y          z
carat 1.00000000  0.02822431  0.9215913  0.97509423  0.95172220 0.95338738
depth 0.02822431  1.00000000 -0.0106474 -0.02528925 -0.02934067 0.09492388
price 0.92159130 -0.01064740  1.0000000  0.88443516  0.86542090 0.86124944
x     0.97509423 -0.02528925  0.8844352  1.00000000  0.97470148 0.97077180
y     0.95172220 -0.02934067  0.8654209  0.97470148  1.00000000 0.95200572
z     0.95338738  0.09492388  0.8612494  0.97077180  0.95200572 1.00000000
```

Für die nächste Grafik in Abbildung 8 wollen wir das R-Paket `GGalley` benutzen, das `ggplot2` erweitert. Es ist nicht Bestandteil von `tidyverse` und muss daher getrennt installiert werden. Aus dem `GGalley`-Paket wollen wir lediglich die Funktion `ggpairs()` nutzen, die in einer Matrixform je zwei Spalten vergleicht und grafisch darstellt. Insgesamt werden dabei 10×10 Teilgrafiken automatisch erzeugt. Wird der komplette Datensatz benutzt, dann braucht die Erzeugung dieser Grafik viel Zeit und manche Teilgrafiken sind auch durch die vielen Punkte nicht mehr aussagekräftig. Daher werden für die Erzeugung der Grafik lediglich 5000 zufällig ausgewählte Datensätze[6] zugrunde gelegt.

```
diamondsSample <- sample_n(diamonds, 5000)
diamondsSample %>%
  ggpairs(title = "Diamonds", mapping =ggplot2::aes(alpha = 0.2))
```

Aufgabe

d) Erklären Sie die Grafiken in Abbildung 8.

[6] Ein willkürlich gewählter Wert.

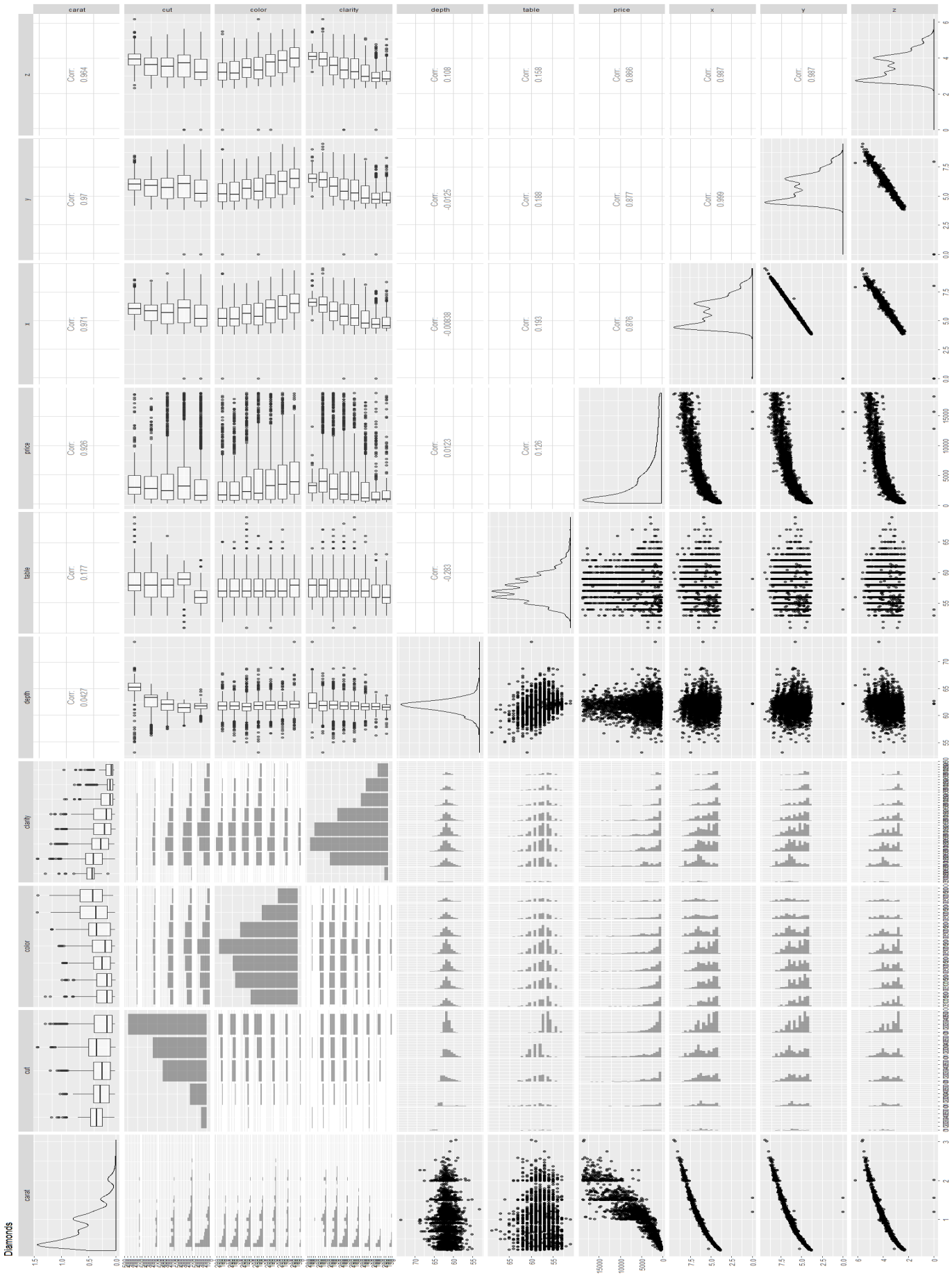


Abbildung 8: Mit `ggpairs()` erstellte Auswertung

Programmierung für statistische Datenanalyse

Übungen zu Kapitel 2

Alle Aufgaben beziehen sich auf die Programmiersprache R! In den Übungen werden an manchen Stellen R-Funktionen benutzt, die im Skript nicht detailliert erläutert werden. Benutzen Sie deswegen ausgiebig die entsprechenden Manual-Einträge und Hilfe-Systeme.

Übung 1

a) Im Downloadbereich finden Sie Skripte, die alle R-Beispiele dieses Skriptes enthalten. Öffnen Sie das Skript „02_jw_psd_x.R“ und führen Sie es schrittweise aus.

b) Erstellen Sie ein Skript „01_ersteBeispiele2.R“, indem Sie das Skript aus Aufgabenteil a) leicht abwandeln. Untersuchen Sie, wie im Editor Fehler unmittelbar angezeigt werden und testen Sie die *Autovervollständigung* in der Konsole und im Editor.

Übung 2

Erstellen Sie im RStudio ein Projekt „psd_1“ in einem neu zu erstellenden Verzeichnis „psd_1“. Alle relevanten Quelldateien für den ersten Teil des Skripts und der entsprechenden Übungen sollen in Ihrem Verzeichnis „psd_1“ gespeichert sein.

Übung 3

a) Welchen Wert haben folgende Anweisungen?

`1/0`	
`0/0`	
`1/0 - 1/0`	
Was ist der Unterschied zwischen `Inf` und `NaN` ? (siehe `?NaN`):	
`47%%10`	
`47%/%10`	
Hinweis: siehe `?'%/%'`	
`-9%%5`	
`-11%%5`	
`3.1%%3`	
`pi%%3`	
`factorial(5)`	
`factorial(60)`	
`abs(-3)`	
`choose(4,2)`	
`choose(6,1)`	
`choose(6,2)`	
`choose(6,3)`	

`choose(6,4)`	
`choose(49,6)`	
Was hat der letzte Ausdruck mit Lotto zu tun?	
`x <- 1:6`	
`tan(atan(x))`	
`log(exp(x))`	

b) Wenn **`x <- c(2,3,5,2)`**, was wird ausgegeben und wieso? Erstellen Sie zusätzlich eine kurze Beschreibung der Funktionen.

`sum(x)`	
`prod(x)`	
`cumsum(x)`	
`cumprod(x)`	
`diff(x)`	
`min(x)`	
`max(x)`	
`range(x)`	
`sort(x)`	
`mean(x)`	
`var(x)`	
`sd(x)`	
`x %% 5`	
`x %% 5 - 3`	
`x + c(1,2)`	
`x + 1:8`	
`8:4`	
`3.8 : 7.2`	
`7.2 : 3.8`	
`rep(x,3)`	

`x <- c(2,3,5,2); x[6] <- 10`	
`x`	
`sum(x)`	
`sum(x, na.rm = TRUE)`	
`mean(x)`	
`mean(p, na.rm = TRUE)`	

c) Gegeben ist:

```
x <- c(5,3,5,7)
y <- c(2,3,c(1,2),7)
z <- c(x,y,17)
z <- c(z,z)
```

Was wird ausgegeben und wieso?

`x`	
`y`	
`length(z)`	

d) Vervollständigen Sie die folgenden beiden Tabellen, die eine kurze Zusammenfassung kennengelernter R-Funktionen geben.

Elementare Funktionen

`abs(x)`	
`choose(n,k)`	
`factorial(n)`	
`log(x)`	natürlicher Logarithmus von x
`log10(x)`	Logarithmus von x zur Basis 10
`exp(x)`	Exponentialfunktion angewandt auf x
`sin(x)`	Sinus von x
`cos(x)`	Kosinus von x
`tan(x)`	Tangens von x
`asin(x)`	Arkussinus von x
`acos(x)`	Arkuskosinus von x
`atan(x)`	Arkustangens von x

Funktionen für Vektoren x

`sort(x)`	
`length(x)`	Anzahl der Elemente von x
`any(x,cond)`	TRUE, falls mindestens ein Element von x die übergebene Bedingung `cond` erfüllt
`all(x,cond)`	TRUE, falls alle Elemente von x die übergebene Bedingung cond erfüllt
`min(x)`	Minimum der Elemente von x
`max(x)`	Maximum der Elemente von x
`range(x)`	
`sum(x)`	
`prod(x)`	
`cumsum(x)`	
`cumprod(x)`	

Übung 4

Gegeben ist ein numerischer Vektor wie z. B. `p <- c(2,3,5,-7,11,13,17,19,-23,-29)` und `dp <- diff(p)`. Wie kann man mithilfe von `dp` und `p[1]` in einer Anweisung die Werte von `p` wieder zurückgewinnen?

Übung 5

`sample(x,k)` `sample(x,k,TRUE)`	Zufällige Auswahl von `k` Elementen aus `x` (ohne und mit Zurücklegen)

a) Gegeben ist `set.seed(1); x <- 1:5`. Was gibt die Funktion `sample()` zurück?

`sample(5,3)`	
`sample(5,3)`	
`sample(5,5)`	
`sample(5,5)`	
`sample(5,3,TRUE)`	
`sample(5,3,TRUE)`	
`sample(5,5,TRUE)`	
`sample(5,5,TRUE)`	

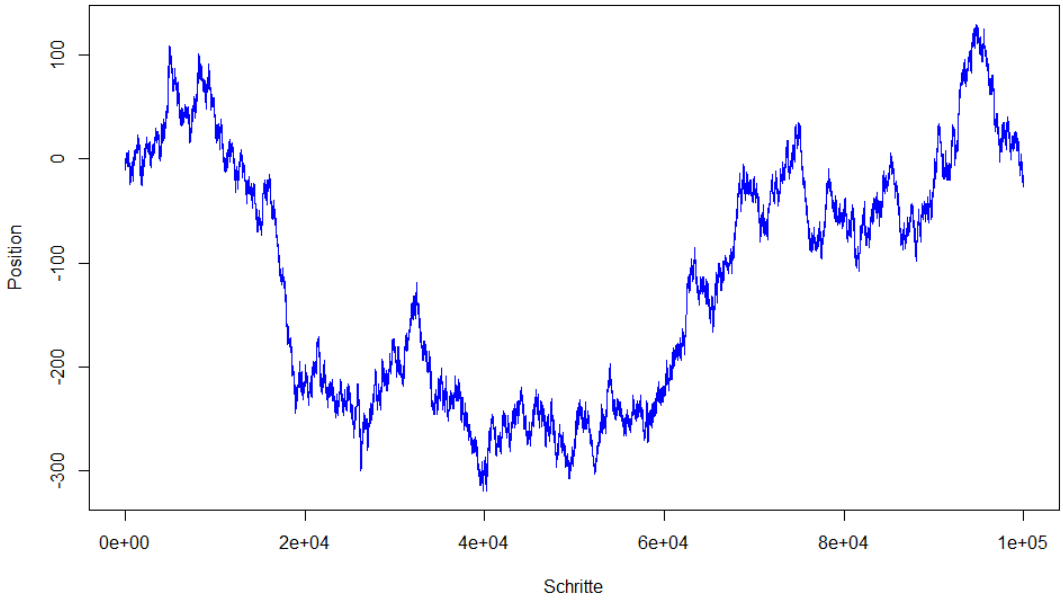
b) Erzeugen Sie eine zufällige Folge von Nullen und Einsen der Länge 1000 (HINWEIS: `sample()`)

c) Erzeugen Sie fünf Permutationen der Ziffern von 0 bis 9

Übung 6 (Random Walk, siehe auch: https://de.wikipedia.org/wiki/Random_Walk)

Ein Betrunkener läuft mit Wahrscheinlichkeit p=0.5 einen Schritt nach rechts und mit gleicher Wahrscheinlichkeit einen Schritt nach links. Die Schritte sind alle gleich lang und die Entscheidung, in welche Richtung er geht, hängt nicht von der Vergangenheit ab. Visualisieren Sie die Realisierung eines möglichen Wegs, der 100000 Schritte umfasst.

Eine Realisierung eines möglichen Wegs könnte wie folgt aussehen:



WICHTIG

- `%/%`
- `log(), log10(), exp()`
- `sin(), cos(), tan()`
- `asin(), acos(), atan()`
- `abs()`
- `choose(n,k)`
- `factorial(n)`
- `sort()`
- `min(), max(), range()`
- `sum(), prod()`
- `cumsum(), cumprod()`
- `sample()`

Programmierung für statistische Datenanalyse
Übungen zu Kapitel 3

Aufgabe 1

Was sind die Werte der folgenden Ausdrücke:

a) `'+' (7,'*' (2, 4))`

b) `1e3`

c) `10e3`

d) `log(0)`

e) `0 * log(0)`

Aufgabe 2

Welchen Wert haben die folgenden Vektoren und was ist ihr Typ?

```
i <- 1:10
gerade <-  function(x){
  x %% 2 == 0
}

g <- gerade(i)
x <- 2.1*i
c <- 4*x +1i*x
z <- as.character(i)     # Nicht im Skript
```

	Wert	Type
g		
x		
c		
z		

Was bewirkt die Funktion `as.character()`? Gibt es auch `as.double()` und `as.integer()` ?

Aufgabe 3

Gegeben ist: `x <- 1; for(i in 1:20) x <- c(x,x)`

a) Welchen Wert hat `x`?
b) Welchen Datentyp hat `x`?
c) Schätzen Sie die Größe von `x`: zuerst recht grob im Kopf und danach genauer
d) Wie hätte man `x` viel einfacher und schneller erzeugen können?
e) Wie kann `x` wieder gelöscht werden?
f) Was ändert sich, wenn statt `x <- 1` nun `x <- 1L` geschrieben wird?

Aufgabe 4

a) Erzeugen Sie 1000 (pseudo-)zufällige Werte gemäß der Normalverteilung mit Erwartungswert 3 und Varianz 4.

b) Testen Sie die erzeugten Daten.

Aufgabe 5

Vereinfachen Sie folgende Schleife:

```
repeat {
  if(n >= 10 && count >= 6 ) break
  print(n^2)
  n <- n+2
}
```

Hinweis: Der Operator && bezieht sich bei Vektoren nur auf das erste Element; bei Vektoren sollte man statt && den Operator & benutzen! Ähnliches gilt für die Operatoren || und | (siehe z. B. `?'&'`).

Aufgabe 6

a) Implementieren Sie eine Funktion `skalierenW()`, die einen nichtnegativen numerischen Vektor `x` als Häufigkeiten interpretiert und den entsprechenden Wahrscheinlichkeitsvektor `p` zurückgibt, d. h. folgende Aussagen sind dann wahr:

```
all(p >= 0)
sum(p) == 1
all(x/sum(x) == p)
```

b) Implementieren Sie eine Funktion `skalierenAffin()`, die einen numerischen Vektor `x` folgendermaßen linear skaliert (d.h. mithilfe einer affinen Abbildung):

- Bezeichne mit x_{min} und x_{max} den kleinsten bzw. größten Wert von `x`
- x_{min} soll auf 0 und x_{max} auf 1 abgebildet werden
- Da abgesehen von einer Verschiebung die Skalierung linear ist, wird also ein Element x_i von `x` auf $(x_i - x_{min}) / (x_{max} - x_{min})$ abgebildet.

Beispiele

```
> skalierenAffin(1:5)
[1] 0.00 0.25 0.50 0.75 1.00
> skalierenAffin(6:10)
[1] 0.00 0.25 0.50 0.75 1.00
> skalierenAffin(c(0,1,20))
[1] 0.00 0.05 1.00
> skalierenAffin(c(0,1,20)+10)
[1] 0.00 0.05 1.00
> skalierenAffin(c(-0.5, -1, 1, 0.5))
[1] 0.25 0.00 1.00 0.75
```

Testen Sie Ihre Lösung auch mit folgenden problematischen Eingabewerten. Wie sieht Ihre Lösung aus?

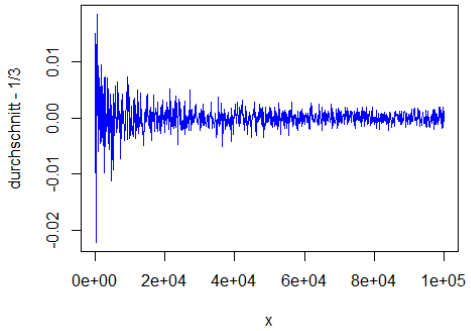
```
> skalierenAffin(c((1:5), NA))
[1] 0.00 0.25 0.50 0.75 1.00   NA
> skalierenAffin(1/(-2:2))
[1] 0.25 0.00  Inf 1.00 0.75
```

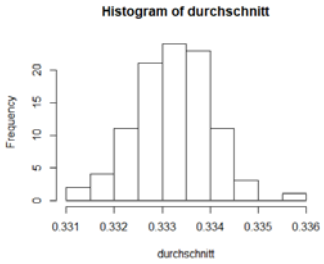
Aufgabe 7

Ein Sensor einer Maschine, der auf einer geradlinigen Leiste angebracht ist, fährt nacheinander unterschiedliche Punkte $P_1, P_2, \ldots, P_n$ ab. Die eindimensionalen Koordinaten $x_1, x_2, \ldots, x_n$ der n Punkte sind dabei gleichförmig im Intervall [0,1] und unabhängig verteilt. Wie groß ist der durchschnittliche Weg (in Abhängigkeit von n?), die der Sensor zurücklegt?

Die Lösung kann relativ leicht analytisch berechnet werden. Sie sollen jedoch die Lösung nicht analytisch bestimmen, sondern dies für unterschiedliche n -Werte in R simulieren, die Ergebnisse visualisieren und aufgrund der ermittelten Daten eine Vermutung aufstellen. Implementieren Sie hierfür eine Funktion `rstrecken(n, min = 0, max = 1)`, die ähnlich wie `runif()` oder `rnorm()` „zufällige“ Werte zurückgibt und die (im Gegensatz zu `runif()` oder `rnorm()`) als zurückgelegte Strecken des Sensors interpretiert werden können.

Eine Visualisierung könnte wie folgt aussehen:





Aufgabe 8

a) Implementieren Sie eine Funktion `llaenge1()`, die die *Lauflängen-Codierung* einer binären Folge (bestehend aus Nullen und Einsen) zurückgibt. In der folgenden Tabelle sehen Sie einige Beispiele:

x	llaenge1(x)	llaenge(x)
0,0,0,1,1,0,0,0,0,1,0,1,1	3,2,4,1,1,2	3,2,4,1,1,2
1,1,1,1,0,0	4,2	4,2
0,0,0,0,1,1	4,2	4,2
"a","a","a","a","a","a","b","b","b","b","a"	FEHLER	6,4,1
"rot", "rot", "blau"	FEHLER	2,1

b) Implementieren Sie eine Funktion `llaenge()`, die die *Lauflängen-Codierung* einer binären Folge (bestehend aus beliebigen zwei Symbolen) zurückgibt.

c) Implementieren Sie eine Funktion `llaengeInverse(y, first=0L, second=1L)`, die aus einer Lauflängen-Codierung eine binäre Folge erzeugt, die aus den durch `first` und `second` spezifizierten Symbolen besteht. In der folgenden Tabelle sehen Sie einige Beispiele:

r	llaengeInverse(r)	llaengeInverse(r, first=-1L)	llaengeInverse(r, first="+", second="-")
3,2,1	0,0,0,1,1,0	-1,-1,-1,1,1,-1	"+","+","+","-","-","+"
1,4	0,1,1,1,1	-1,1,1,1,1	"+","-","-","-","-"

d) Testen Sie Ihre implementierten Funktionen mithilfe der folgenden Funktion:

```
testLaufLaenge <- function(anzahl = 16L, maxLaenge = 12) {
  for(i in 1:anzahl){
    n <- sample(1:maxLaenge,1)
    x <- sample(c(0L,1L),n,TRUE)
    r <- llaenge(x)
    print(r)
    y <- llaengeInverse(r, 1L, 0L)
    if(y[1] != x[1])
      y <- 1-y
    if(any(x != y))
      stop("Test nicht bestanden: ", x)
  }
}
```

Hinweis:

`stop()`	Stoppt die Ausführung der aktuellen Anweisung und gibt eine entsprechende Fehlermeldung aus

WICHTIG

- `as.character()`
- `as.double()`
- `as.integer()`
- `var()`
- `stop()`

Programmierung für statistische Datenanalyse
Übungen zu Kapitel 4

Aufgabe 1

Was wird zurückgegeben?

```
ifelse(1:8%%2 == 0, letters, LETTERS)
```

Aufgabe 2

a) Was wird ausgegeben? Wieso?

```
set.seed(1)
x <- sample(1:9, 10,TRUE)
x
order(x)
x[order(x)]
sort(x)
```

b) Beschreiben Sie kurz die Funktion `order()`:

`order(x)`	

c) Ist für einen Vektor `x` der Aufruf `x[order(x)]` identisch zu `sort(x)`?

Aufgabe 3

Was bewirkt die folgende Zuweisung?

```
d <- d[!is.na(d)]
```

Aufgabe 4

In den Übungen zu Kapitel 3 haben Sie eine Funktion `llaenge()` implementiert, die die *Lauflängen-Codierung* einer binären Folge (bestehend aus beliebigen zwei Symbolen) zurückgibt. Implementieren Sie eine neue Version dieser Funktion, deren Funktionsrumpf nur drei kurze Anweisungen umfasst!

Hinweis

- Betrachten Sie für einen Eingabevektor `x`
  `v <- x[-1L] != x[-n]`
- Wenden Sie zuerst `which` und dann `diff` auf `v` an

Aufgabe 5

a) Die Entropie H eines Wahrscheinlichkeitsvektors $p=(p_1, p_2, \dots, p_n)$ (siehe Übung 6 zu Kapitel 3) ist definiert durch:

$$\mathrm{H(p)} = -\sum_{k=0}^{n} p_i \cdot log_2\,(p_i)$$

Falls $p_i = 0$ gilt, wie ist dann sinnvollerweise $p_i \cdot log_2(p_i)$ definiert? Wieso? (**Hinweis**: Regel von de l'Hospital).

Implementieren Sie zwei Versionen einer Funktion `entropie()`, die für einen Wahrscheinlichkeitsvektor $p=(p_1, p_2, \dots, p_n)$ die Entropie zurückgibt. Zum Herausfiltern des Falls $p_i = 0$ soll die erste Version der Funktion `ifelse` benutzen, während die zweite Version stattdessen von der Vektor-Indizierung Gebrauch macht.

b) Bestimmen Sie die Entropie von

$p = (0.25, 0.25, 0.25, 0.25)$

$p = (0.25, 0, 0.25, 0.25, 0.25)$

$p = (0.2, 0.2, 0.2, 0.2, 0.2)$

$p = (0.04, 0.02, 0.04, 0.9)$

c) Implementieren Sie eine Funktion, die Ihre beiden Versionen der Entropie-Funktion auf Gleichheit testet. Überprüfen Sie zusätzlich, dass stets $0 \leq \mathrm{H(p)} \leq log_2(n)$ gilt. Für welche Wahrscheinlichkeitsvektoren nimmt die Entropie den maximalen Wert $log_2(n)$ an?

Aufgabe 6

a) Was sind die Werte der folgenden Vektoren? Wieso?

```
2^1:16
2^(1:16)
(0+1i)^(1:16)
exp(2*pi*1i/16)^(1:16)
```

b) Gibt es einen Unterschied zwischen den folgenden beiden Funktionsaufrufen?

```
complex(argument = 2*pi/16)
exp(2*pi*1i/16)
```

c) Gegeben ist `z <- exp(2*pi*1i/16)^(1:16)`. Beschreiben Sie `z`. Welchen Wert hat `z^16` ?

Aufgabe 7

a) Welchen Wert haben die Ausdrücke `4+T`, `4+F` und `T+T`?
b) Gegeben ist ein Vektor `x`. Was geben folgende Funktionsaufrufe zurück?

```
sum(x==2)
length(x[x==2])
length(which(x == 2))
```

c) Erklären Sie:

```
> a <- 2^(1/2)
> x <- a^(1:6)
> x
[1] 1.414214 2.000000 2.828427 4.000000 5.656854 8.000000
> sum(x==2)
[1] 0
> length(x[x==2])
[1] 0
> length(which(x == 2))
[1] 0
> all(x == 2)
[1] FALSE
> any(x == 2)
[1] FALSE
> all.equal(2,x[2])
[1] TRUE
```

Aufgabe 8

a) Wie kann man `x <- 1:20 *-1` kürzer schreiben?

b) Welche Werte haben dann `x[-(1:5)]` und `x[6:10]`?

Aufgabe 9

Gegeben ist ein numerischer Vektor `x` der Länge 1000 wie z. B. `x <- runif(1000)`.

a) Bestimmen Sie jeweils den Mittelwert des dritten Viertels von x und den Mittelwert der zweiten Hälfte von x.

b) Wie können Sie die Anzahl der Elemente im dritten Viertel von `x` bestimmen, die größer als 0.5 sind?

c) Geben Sie alle Elemente von `x[i]` aus, die die beiden folgenden Bedingungen erfüllen:

- $51 \leq i \leq 100$ oder $301 \leq i \leq 400$
- `x[i] ≥ 0.8`

d) Wieso ist `x[x[c(51:100, 301:400)] >= 0.9]` keine Lösung von c)

Aufgabe 10

Was bewirkt `x[ x < 0] <- -x[ x < 0]` bei einem numerischen Vektor `x`? Wie kann dies kürzer und auch klarer ausgedrückt werden?

Aufgabe 11

Gegeben ist ein numerischer Vektor `x` wie z. B. `x <- (1:1000*131)%%17`.

a) Was gibt `table(x)` aus?

b) Ändern Sie `x`, so dass alle Werte von `x`, die größer gleich 12 sind, den Wert 20 erhalten. Testen Sie Ihre Lösung mithilfe der Funktion `table()`.

Aufgabe 12

Gegeben ist folgender Quelltext:

```
x <- 1:10
aendern <- function(i){
  x[i] <- 99L
}
aendern(1)
```

a) Hat sich der Vektor `x` durch den Funktionsaufruf `aendern(1)` geändert?

b) Wie muss man den Quelltext entsprechend umschreiben, so dass der Vektor `x` durch `aendern(1)` den Wert 99 erhält?

Aufgabe 13

Gegeben ist:

```
a <- LETTERS[c(T,F,F)]
b <- LETTERS[c(F,T,F)]
worte <- paste(a,b, sep = "")
auto <- 1:9
names(auto) <- worte
```

a) Welche Werte haben dann `a`, `b`, `worte` und `auto`?

b) Welchen Wert hat dann `auto["Vw"]` und welchen Wert `auto["VW"]`? Wieso?

c) Was wird durch folgende Kommandos ausgegeben? Wieso?

```
paste(a,b)
paste(a,b, sep = "")
paste(a,b, sep = "::")
paste(a,b, sep="::", collapse = ", ")
paste(a,b, sep="", collapse = ", ")
paste(a,b, sep="", collapse = "")
```

Aufgabe 14

Gegeben ist folgender Quelltext:

```
nchar("Data Sciences")
strn <- paste(LETTERS[1:8], collapse = "")
strn
nchar(strn)
substring(strn,1,1)
substring(strn,1,2)
substring(strn,1,4)
substring(strn,2,4)
```

Was wird ausgegeben? Siehe `?nchar` und `?substring`.

Aufgabe 15

Erzeugen Sie einen Vektor `w`, so dass Folgendes gilt:

```
> w
a1 b1 c1 d1 e1 f1 g1 h1 a2 b2 c2 d2 e2 f2 g2 h2 a3 b3 c3 d3 e3 f3 g3 h3 a4 b4 c4 d4 e4 f4
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
g4 h4 a5 b5 c5 d5 e5 f5 g5 h5 a6 b6 c6 d6 e6 f6 g6 h6 a7 b7 c7 d7 e7 f7 g7 h7 a8 b8 c8 d8
31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
e8 f8 g8 h8
61 62 63 64
> str(w)
 Named int [1:64] 1 2 3 4 5 6 7 8 9 10 ...
 - attr(*, "names")= chr [1:64] "a1" "b1" "c1" "d1" ...
```

Aufgabe 16

Erzeugen Sie eine Matrix `A`, so dass Folgendes gilt:

```
> A
  1 2 3 4 5 6 7 8
a 1 1 1 1 1 1 1 1
b 1 1 1 1 1 1 1 1
c 1 1 1 1 1 1 1 1
d 1 1 1 1 1 1 1 1
e 1 1 1 1 1 1 1 1
f 1 1 1 1 1 1 1 1
g 1 1 1 1 1 1 1 1
h 1 1 1 1 1 1 1 1
> str(A)
 num [1:8, 1:8] 1 1 1 1 1 1 1 1 1 1 ...
 - attr(*, "dimnames")=List of 2
  ..$ : chr [1:8] "a" "b" "c" "d" ...
  ..$ : chr [1:8] "1" "2" "3" "4" ...
```

Aufgabe 17

Gegeben ist wie im Skript:

```
A <- matrix(1:6, nrow=2)
colnames(A) <- c("a", "b", "c")
rownames(A) <- c("X", "Y")
```

Was wird dann durch folgende Aufrufe ausgegeben und wieso?

```
A
A["X", "b"]
A["Y", "c"]
A["Yc"]
A["Y", ]
A[, "a"]
```

Aufgabe 18

Was bewirkt folgende Funktion, wenn `x` und `y` Vektoren gleicher Länge sind?
Beispiel: `zusammenfuehren(letters,LETTERS)`

```
zusammenfuehren <- function(x,y){
  A <- matrix(c(x,y), nrow=2, byrow=TRUE)
  #print(A)
  as.vector(A)
}
```

Aufgabe 19

Gegeben ist die Matrix `A` durch: `A <- matrix(1:20, nrow=4)`

a) Löschen Sie die dritte Spalte und die zweite Zeile

b) Fügen Sie eine neuen Zeile am Ende hinzu mit den Werten 1,2,3,4

c) Bestimmen Sie die Determinante von A

d) Bestimmen Sie die Transponierte A^t von A

Aufgabe 20

Gegeben ist die Matrix `A` durch: `A <- matrix(1:20, nrow=4).` Berechnen Sie $A^t \cdot A$, $A \cdot A^t$ und `A*A`.

Aufgabe 21

Erzeugen Sie 1000 unterschiedliche quadratische 20×20-Matrizen mit gleichverteilten zufälligen Werten zwischen -2 und 2. Speichern Sie die Determinanten der so erzeugten Matrizen in einem Vektor d. Verschaffen Sie sich einen Überblick, wie die Werte von d verteilt sind.

a) Matrizen, derer Wert der Determinante nahe bei 0 liegt, sind bei manchen Berechnungen kritisch. Wie viele solcher kritischen Matrizen gibt es in dem erzeugten Datensatz?

b) Haben Sie zwei Matrizen erzeugt, deren Determinanten identisch sind?

Aufgabe 22

Gegeben ist:

```
A <- matrix(1:32, nrow=4)

aus <- function(x){
  print(paste(x, collapse = "->"))
  return(NULL)
}
```

a) Was wird dann durch die Funktionsaufrufe `apply(A, 1, aus)` und `apply(A, 2, aus)` ausgegeben?

b) Was wird dann durch die Funktionsaufrufe `apply(A, 1, summary)` und `apply(A, 2, summary)` ausgegeben?

c) Setze `b <- c(letters, LETTERS)`. Was wird dann durch die Funktionsaufrufe `apply(A, 1, function(x) {b[x]})` und `apply(A, 2, function(x) {b[x]})` ausgegeben?

WICHTIG

- `order()`
- `table()`
- `nchar()`
- `substring()`
- `par()`
- `paste()`

Programmierung für statistische Datenanalyse
Übungen zu Kapitel 5

Teil 1

Aufgabe 1

Betrachten Sie folgenden Quelltext:

```
x <- rnorm(1e4)
h <- hist(x)
```

a) Welchen Datentyp hat `h`?

b) Ist `h` eine Liste?

c) Was gibt `is.list(h)` zurück?

d) Was gibt `is.vector(h)` zurück?

e) Welche Elemente hat `h`?

f) Welchen Datentyp hat `unlist(h)`?

Aufgabe 2

Gegeben ist ein Objekt `x` in R. Ist dann `is.vector(as.vector(x))` stets wahr?

Aufgabe 3

Gegeben ist:

```
v <- 1:4
w <- LETTERS[1:8]
l1 <- list(v,T, w[-1])
```

Mit welcher Anweisung können Sie innerhalb der Liste `l1` den Buchstaben *C* (der durch `w` einfügt wurde) durch den Buchstaben *U* ersetzen?

Aufgabe 4 [1]

Gegeben ist eine Liste `l1` der Form:

```
l1 <- list("1"=4, "3"=8, "5"=12, "7"=16)
```

Wie können Sie mithilfe von `sum()`

a) die Summe der Listenelemente (also 4+8+12+16) bestimmen?

b) die Summe der Namen der Listenelemente (also 1+3+5+7) bestimmen?

Aufgabe 5

Gegeben ist eine Liste `l1` mit mehr als einem Element. Was gibt dann `length(l1[2])` zurück?

Aufgabe 6

Gegeben ist eine Liste `l1`:

```
l1 <- list(10:1, "Data Science",  c("Data Science", "FH SWF"), c(T,T,F))
```

a) Was gibt `length(l1)` zurück?

b) Bestimmen Sie die Länge des zweiten und des dritten Listenelements.

c) Wie können Sie mit einem Aufruf die Länge aller Listenelemente als Vektor erhalten?

Aufgabe 7

Ändern Sie die Liste `l1` in der vorherigen Aufgabe so, dass `str(l1)` Folgendes zurückgibt:

```
> str(l1)
List of 4
 $ A: int [1:10] 10 9 8 7 6 5 4 3 2 1
 $ B: chr "Data Science"
 $ C: chr [1:2] "Data Science" "FH SWF"
 $ D: logi [1:3] TRUE TRUE FALSE
```

[1] Hinweis: `unlist()`

Aufgabe 8

Gegeben ist:

```
> set.seed(42)
> data <- sample(LETTERS[1:5], 100, replace=TRUE)
> f1 <- factor(data, levels= c( "A", "B","C"), ordered = TRUE)
> f1
  [1] A    <NA> A    A    B    <NA> B    B    A    <NA> A    <NA> <NA> B    B    C    A
 [18] A    C    <NA> <NA> <NA> <NA> <NA> B    <NA> C    B    A    B    C    B    <NA> <NA>
 [35] B    <NA> <NA> <NA> <NA> B    B    C    A    <NA> B    B    B    <NA> C    <NA> B
 [52] B    B    <NA> A    A    <NA> <NA> B    A    <NA> <NA> <NA> A    C    C    <NA> <NA>
 [69] <NA> <NA> <NA> B    B    A    B    <NA> <NA> <NA> A    <NA> B    B    C    <NA> <NA>
 [86] C    <NA> <NA> A    <NA> A    A    <NA> C    A    B    <NA> A    <NA> <NA>
Levels: A < B < C
```

a) Ersetzen Sie in `f1` den Wert `B` durch `X`, so dass sich folgende Ausgabe ergibt:

```
> f1
  [1] A    <NA> A    A    X    <NA> X    X    A    <NA> A    <NA> <NA> X    X    C    A
 [18] A    C    <NA> <NA> <NA> <NA> <NA> X    <NA> C    X    A    X    C    X    <NA> <NA>
 [35] X    <NA> <NA> <NA> <NA> X    X    C    A    <NA> X    X    X    <NA> C    <NA> X
 [52] X    X    <NA> A    A    <NA> <NA> X    A    <NA> <NA> <NA> A    C    C    <NA> <NA>
 [69] <NA> <NA> <NA> X    X    A    X    <NA> <NA> <NA> A    <NA> X    X    C    <NA> <NA>
 [86] C    <NA> <NA> A    <NA> A    A    <NA> C    A    X    <NA> A    <NA> <NA>
Levels: A < X < C
```

b) Ersetzen Sie in `f1` die Werte `A` und `C` durch `Y`.

c) Löschen Sie in `f1` alle `NA`-Werte:

```
> f1
 [1] Y Y Y X X X Y Y X X Y Y Y Y X Y X Y X Y X X X X Y Y X X X Y X X X Y Y X Y Y
Y Y X X Y X Y X X Y Y Y
[51] Y Y Y Y X Y
Levels: Y < X
```

Aufgabe 9

Machen Sie sich mit den Funktionen `rev()` und `unique()` vertraut, die im einfachsten Fall auf einen Vektor angewandt werden.

a) Ist `identical(rev(1:100),100:1)` stets wahr?

b) Gegeben ist: `d <- sample(1:10, 50, replace = TRUE)`.
Ist `identical(sort(unique(d)), 1:10)` stets wahr?

Bestimmen Sie per Simulation die ungefähre Wahrscheinlichkeit, dass `identical(sort(unique(d)), 1:10)` falsch ist.

Wie ändert sich diese so ermittelte Wahrscheinlichkeit, wenn man statt den Wert 50, die Werte 10, 20, 30, …, 100 zugrunde legt?

c) Gegeben ist `f1` wie am Beginn der vorherigen Aufgabe. Bestimmen Sie `rev(f1)` und `unique(f1)`.

Aufgabe 10

Wir haben im Abschnitt „Data Frame erzeugen“ gesehen, dass die Namen der Argumente automatisch übernommen werden, wenn kein Name explizit angegeben wird. Beispiel:

```
bezeichnung <- paste("M", 1:6, sep="")
nr <- 1:6
df <- data.frame(nr, bezeichnung)
```

Gilt das auch für Listen?

Teil 2

Für die Aufgaben im Teil 2 betrachten wir ein Data Frame `student`. Erzeugt werden kann `student` durch:

```
set.seed(1)
anzahl <-  100
Punkte <-  rnorm(anzahl, mean = 65, sd = 20)
Punkte <- as.integer(Punkte)
MatNr <- sample(1E6:1E7-1, anzahl)
names(MatNr) <- sample(c("ET","MB"), anzahl, replace=TRUE)
Bestanden <- F
student <- data.frame(MatNr, Punkte, Bestanden)
student$Stdgang <- names(MatNr)
rm(Punkte, MatNr,Bestanden,anzahl)
```

Die Ausgabe von `head(student)` lautet:

```
> head(student)
    MatNr Punkte Bestanden Stdgang
1 9075256     52     FALSE      MB
2 5998360     68     FALSE      ET
3 7390976     48     FALSE      MB
4 2188111     96     FALSE      ET
5 2972641     71     FALSE      MB
6 7484652     48     FALSE      ET
```

Aufgabe 11

a) Aus wie vielen Variablen und aus wie vielen Observationen besteht `student`?

b) Die Funktionen `head(x)` und `tail(x)` liefern den ersten und letzten Teil von `x` zurück. Was wird ausgegeben durch:

```
head(student, n=10)
head(student, n=-95)
head(student, n=-93)
tail(student)
```

c) Was wird ausgegeben durch:

```
student[1]
student[2]
```

d) Geben Sie die erste Zeile von Student aus.

e) Um die Klausur zu bestehen, reichen 50 Punkte. Ändern Sie `student` entsprechend!

```
> head(student)
    MatNr Punkte Bestanden Stdgang
1 9075256     52      TRUE      MB
2 5998360     68      TRUE      ET
3 7390976     48     FALSE      MB
4 2188111     96      TRUE      ET
5 2972641     71      TRUE      MB
6 7484652     48     FALSE      ET
```

Aufgabe 12

In den Übungen zu Kapitel 4 haben wir bereits die Funktion `order()` kennengelernt.

a) Was geben die Funktionen zurück:

- `order(student$Punkte)`
- `order(student$MatNr)`
- `order(student$Stdgang)`

b) Was wird zurückgegeben:

- `student[order(student$Punkte), ]`
- `student[order(student$MatNr), ]`
- `student[order(student$Stdgang), ]`

c) Geben Sie geordnet nach Punkten die Ergebnisse im Studiengang MB aus. Die Ausgabe der ersten zehn Zeilen in unserem Beispiel lautet dann:

```
     MatNr Punkte Bestanden Stdgang
84 7589118     34     FALSE      MB
35 9742530     37     FALSE      MB
75 3243959     39     FALSE      MB
97 4658829     39     FALSE      MB
54 5707891     42     FALSE      MB
3  7390976     48     FALSE      MB
46 6188906     50      TRUE      MB
72 9286466     50      TRUE      MB
1  9075256     52      TRUE      MB
13 1926848     52      TRUE      MB
```

Aufgabe 13

Als Nächstes sollen Noten in `student` eingefügt werden. Zugrunde liegt das folgende Notenschema:

	Punkte		*Punkte*		*Punkte*		*Punkte*
Note 4:	50–58	**Note 3:**	59-69	**Note 2:**	70-80	**Note 1:**	>=81

a) Benutzen Sie die Funktion `cut()`, um aus `student$Punkte` einen Faktor zu erzeugen, der als Note in `student` eingefügt werden kann. Die Ausgabe der ersten zehn Zeilen in unserem Bespiel lautet dann:

```
      MatNr Punkte Bestanden Stdgang Note
1   9075256     52      TRUE      MB    4
2   5998360     68      TRUE      ET    3
3   7390976     48     FALSE      MB    5
4   2188111     96      TRUE      ET    1
5   2972641     71      TRUE      MB    2
6   7484652     48     FALSE      ET    5
7   5912142     74      TRUE      MB    2
8   3578183     79      TRUE      MB    2
9   4222106     76      TRUE      ET    2
10  8963286     58      TRUE      MB    4
```

b) Geben Sie nach Punkten sortiert alle Studierenden aus, die mindestens 48 und höchstens 52 Punkte erhalten haben:

```
      MatNr Punkte Bestanden Stdgang Note
3   7390976     48     FALSE      MB    5
6   7484652     48     FALSE      ET    5
46  6188906     50      TRUE      MB    4
65  5817860     50      TRUE      ET    4
72  9286466     50      TRUE      MB    4
45  1784475     51      TRUE      ET    4
1   9075256     52      TRUE      MB    4
13  1926848     52      TRUE      MB    4
52  1922892     52      TRUE      ET    4
```

c) Geben Sie nach Punkten sortiert alle Studierenden aus, die mindestens 79 und höchstens 82 Punkte erhalten haben:

```
      MatNr Punkte Bestanden Stdgang Note
8   3578183     79      TRUE      MB    2
94  8871878     79      TRUE      ET    2
22  9438067     80      TRUE      MB    2
40  3007020     80      TRUE      ET    2
48  3108410     80      TRUE      MB    2
19  8663430     81      TRUE      MB    1
50  8606356     82      TRUE      ET    1
```

d) Erstellen Sie für den Notenspiegel ein entsprechendes Balkendiagramm:[2]

e) Ist `summary(student)` hilfreich?

WICHTIG

- `rev()`
- `unique()`

[2] Benutzt wurden die Argumente: `main = "Notenspiegel", xlab="Noten", ylab="Anzahl"`

Programmierung für statistische Datenanalyse
Übungen zu Kapitel 6

Aufgabe 1

Erstellen Sie die nebenstehende Grafik:

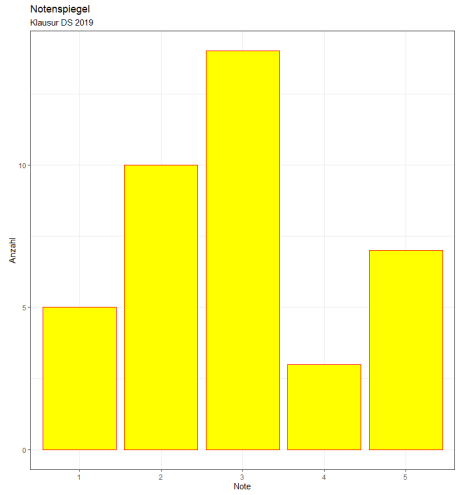


Abbildung 1: Farbiges Histogramm

Hinweis

```
anzahl <- c(5,10,14,3,7)
```

Aufgabe 2

Gegeben sind folgende Daten:

```
n <- 32
x <- y <- c(1:n, 1:n, 1:(n/2), 1:(n/4))
```

Was ist der Unterschied zwischen den beiden Plots:

```
ggplot(data = data.frame(x,y)) +
  geom_point(mapping = aes(x = x, y = y))
```

```
ggplot(data = data.frame(x,y)) +
  geom_point(mapping = aes(x = x, y = y), position = "jitter")
```

Was würden Sie bevorzugen?

Aufgabe 3

Gegeben sind ähnlich wie in den Übungen zu Kapitel 5 folgende Daten:

```
set.seed(1)
anzahl <-  100
MatNr <- sample(1E6:1E7-1, anzahl)
names(MatNr) <- sample(c("ET","MB", "WIng", "WInfo"), anzahl, replace=TRUE)
student <- data.frame(MatNr)
student$Stdgang <- names(MatNr)
student$Punkte <- as.integer(rnorm(anzahl, mean = 68, sd = 12))
```

Es soll wieder das folgende Notenschema zugrunde liegen:

	Punkte		*Punkte*		*Punkte*		*Punkte*
Note 4:	50–58	**Note 3:**	59-69	**Note 2:**	70-80	**Note 1:**	>=81

Erzeugen Sie die folgenden zwei Grafiken:

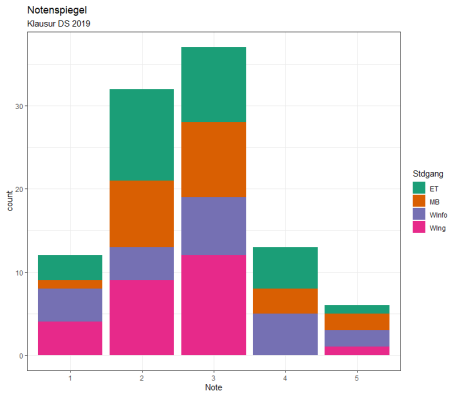


Abbildung 2: Notenspiegel (Variante 1)

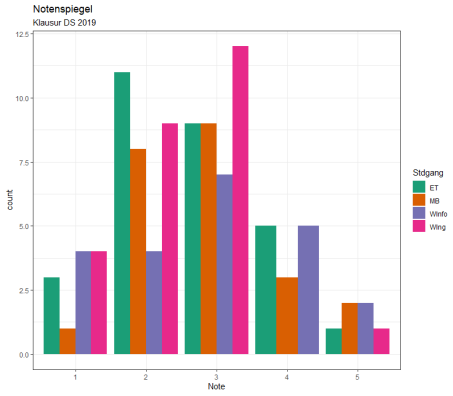


Abbildung 3: Notenspiegel (Variante 2)

Hinweis

…… `geom_bar(……) + scale_fill_brewer(palette="Dark2")`[1] `+ theme_bw()`

[1] Siehe auch http://www.cookbook-r.com/Graphs/Colors_(ggplot2)/

Aufgabe 4

Erzeugen Sie mit den Daten der letzten Übungsaufgabe die folgenden vier Grafiken:

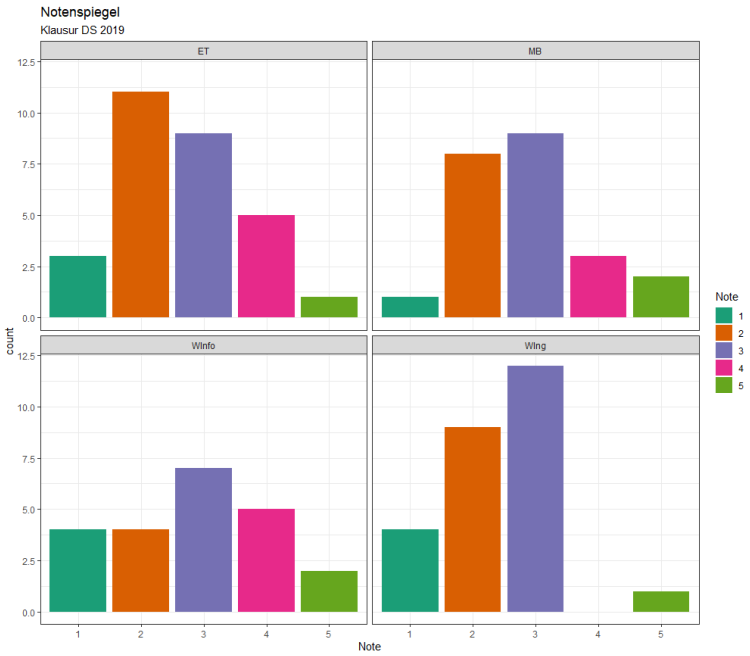


Abbildung 4: Notenspiegel mit Facetten (Variante 1)

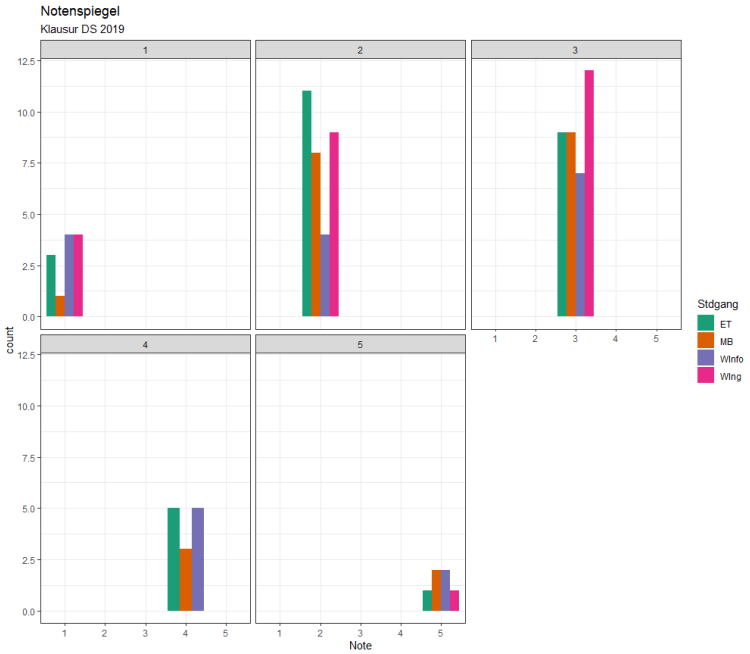


Abbildung 5: Notenspiegel mit Facetten (Variante 2)

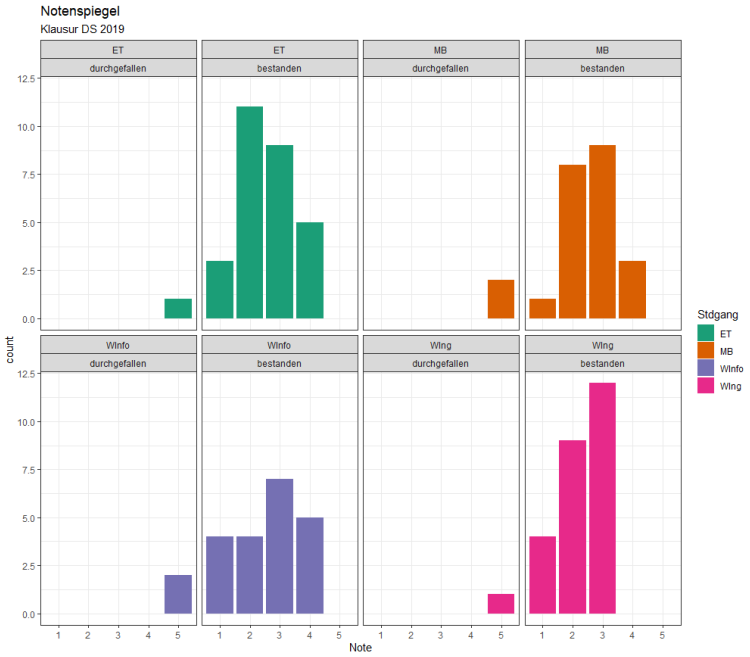


Abbildung 6: Noten aufgeteilt in „bestanden“ und „durchgefallen“ (Variante 1)

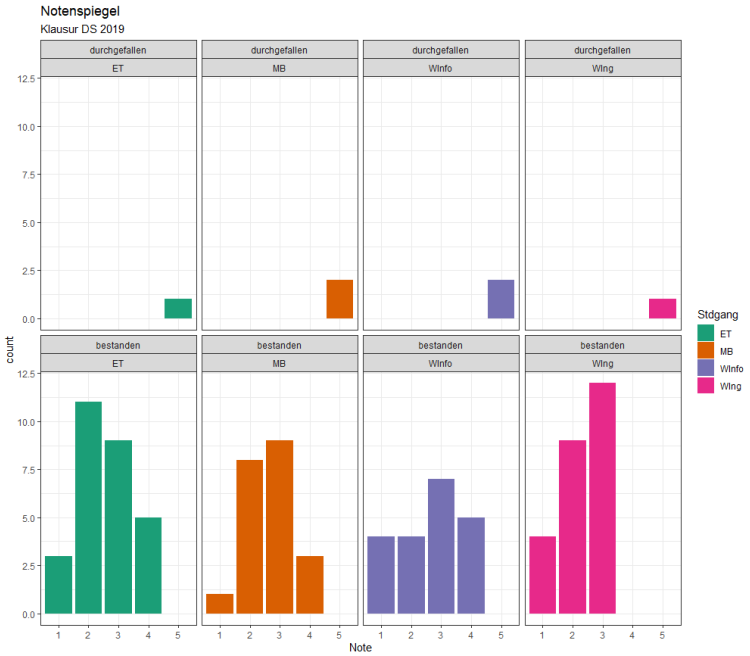


Abbildung 7: Noten aufgeteilt in „bestanden“ und „durchgefallen“ (Variante 2)

Programmierung für statistische Datenanalyse
Übungen zu Kapitel 7

```
require(tidyverse)
require(nycflights13)
```

Die `folgenden` Aufgaben beziehen sich auf den Datensatz `flights` aus dem Paket `nycflights13.`

Aufgabe 1

Welche der drei Funktionsaufrufe sind identisch? Was ist der Unterschied?

```
filter(flights, month == 1, day == 1)
filter(flights, month == 1 & day == 1)
filter(flights, month == 1 && day == 1
```

Aufgabe 2

a) Gesucht sind alle Flüge, die mehr als 150 Minuten verspätet den Zielflughafen erreichten.

b) Gesucht sind alle Flüge, die mehr als 150 Minuten verspätet den Zielflughafen erreichten, obwohl sie pünktlich gestartet waren.

c) Alle Flüge in den Monaten September, Oktober, November und Dezember, die mit mehr als 150 Minuten Verspätung den Zielflughafen erreichten.

Aufgabe 3

Sind Verspätungen von mehr als 150 Minuten in den Monaten April bis September deutlich seltener als in den restlichen sechs Monaten des Jahres? Was sind die genauen Prozentzahlen?

Aufgabe 4

a) Gesucht sind alle Flüge, die mit mehr als 60 Minuten Verspätung gestartet sind, aber mehr als 60 Minuten im Flug aufgeholt haben.

b) Gesucht sind alle Flüge, die mit mehr als 60 Minuten Verspätung gestartet sind, aber mehr als 60 Minuten im Flug aufgeholt haben und nicht verspätet gelandet sind.

Aufgabe 5

a) Bei wie vielen Flügen ist die Abflugzeit nicht aufgelistet?

b) Bei wie vielen Flügen ist die Verspätung der Abflugzeit nicht aufgelistet?

c) Bei wie vielen Flügen ist die Verspätung der Abflugzeit aufgelistet, aber nicht die Abflugzeit?

Aufgabe 6

Was ist der Unterschied zwischen:

```
arrange(flights, year, month, day)
arrange(flights, day, month, year)
```

Aufgabe 7

a) Wie lang ist die längste Flugroute? Von wo nach wo?

b) Listen Sie sortiert nach Länge alle Flugrouten auf, die länger als 4000 km sind. Ausgegeben werden sollen pro Zeile: Start- und Zielflughafen und die Länge in Meilen und km (Umrechnung: 1.000 Meilen entsprechen 1.609,344 km). Benutzen Sie den Pipe-Operator.

Aufgabe 8

Was wird jeweils ausgegeben?

a)

```
select(flights, year, month, year, day, year)
```

b)

```
spalten <- c("year", "month", "day", "dep_delay", "arr_delay")
```

c)

```
select(flights, contains("DEP"))
```

d)

```
flights %>%
  select(flight, contains("DEP")) %>%
  head(4)
```

Aufgabe 9

Die Zeitformate `dep_time` und `sched_dep_time` in `flights` benutzen das Format HHMM oder HMM; 1607 entspricht also 16:07 Uhr. Erstellen Sie eine Tabelle mit allen Flüge mit den Spalten

`year`, `month`, `day`, `flight`, `dep_time`, `sched_dep_time`

Dabei sollen `dep_time` und `sched_dep_time` eine Darstellung haben, die der Anzahl der Minuten nach Mitternacht entspricht. Schreiben Sie hierfür eine Funktion `zeitInMinuten()`. Die mögliche Zeitangabe 2400 für Mitternacht soll dabei auf 0 Minuten abgebildet werden (d.h. `zeitInMinuten(2400)` soll den Wert 0 zurückgeben).

Aufgabe 10

Erklären Sie folgende Ausgabe:

```
flights %>%
  select(flight, distance) %>%
  unique() %>%
  group_by(flight)  %>%
  mutate(dMin = min(distance), dMax = max(distance)) %>%
  mutate(r = min_rank(distance)) %>%
  filter(dMax - dMin > 2500) %>%
  arrange(dMax - dMin, distance) %>%
  print(n = Inf)
```

Programmierung für statistische Datenanalyse
Übungen zu Kapitel 8 (EDA)

```
require(tidyverse)
```

Aufgabe 1

Gegeben ist eine diskrete Variable `dx` und eine kontinuierliche Variable `kx`. Mithilfe von `ggplot2` soll die Verteilung der Werte von `dx` visualisiert werden; ebenso soll in einem separaten Plot die Verteilung der Werte von `dx` visualisiert werden.

a) Wann benutzen Sie `geom_bar()` und wann `geom_histogram()`?

b) Bei der Auswertung des Datensatzes `diamonds` wurden folgende Plots vorgeschlagen. Was ist falsch oder zumindest fraglich und warum?

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut))

ggplot(data = diamonds) +
  geom_histogram(mapping = aes(x = carat))

ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = carat))

ggplot(data = diamonds) +
  geom_histogram(mapping = aes(x = cut))
```

Aufgabe 2

a) Sind folgende Funktionsaufrufe identisch? Was bewirken Sie?

```
diamonds %>% tally()
diamonds %>% count()
diamonds %>% summarize(n=n())
tibble(n=nrow(diamonds))
```

b) Sind folgende Funktionsaufrufe identisch? Was bewirken Sie?

```
diamonds %>% group_by(carat) %>% summarize(n=n())
diamonds %>% group_by(carat) %>% tally()
diamonds %>% group_by(carat) %>% count()
```

c) Wie kann man mithilfe von `count()` oder `tally()` Folgendes kürzer schreiben?

```
diamonds %>% group_by(carat) %>% summarize(n=n())
```

d) Wie kann man mithilfe von `count()` oder `tally()` Folgendes kürzer schreiben?

```
diamonds %>% summarize(n=sum(carat))
```

a) Wie kann man mithilfe von `count()` oder `tally()` Folgendes kürzer schreiben??

```
tibble(n=diamonds %>% select(carat) %>% sum())
```

Aufgabe 3

Erklären Sie den folgenden Quelltext:

```
dd <- diamonds[1:8,]
dd$carat
cut_width(dd$carat, 0.5)
dd %>% count(cut_width(carat, 0.5))
count(dd, cut_width(carat, 0.5))
```

Aufgabe 4

a) Was gibt die Funktion ganzOderWeniger() zurück?

```
ganzOderWeniger <- function(x, delta) {
  return(floor(x+delta) == ceiling(x))
}
```

b) Erklären Sie den folgenden Quelltext und werten Sie die entsprechende Ausgabe aus. Was ist daran auffällig?

```
diamonds %>%
  filter(ganzOderWeniger(carat,0.05)) %>%
  count(carat)

# äqivalent hierzu:
diamonds %>%
  filter(ganzOderWeniger(carat,0.05)) %>%
  group_by(carat) %>%
  summarize(n=n())
```

c) Vergleichen Sie folgende Variante mit Aufgabenteil b)? Gibt es die Auffälligkeiten immer noch?

```
diamonds %>%
  filter(ganzOderWeniger(2*carat,0.1)) %>%
  count(carat) %>%
  print(n=Inf)
```

Programmierung für statistische Datenanalyse

Übung 9

Tidy data, Relational data, Iteration

```
require(tidyverse)
require(nycflights13)    # Für die Datensätze flights und airports[1].
```

Aufgabe 1

Gegeben sind jeweils vier gemessene Laufzeiten eines Algorithmus in Abhängigkeit von der Anzahl der Eingabedaten n:

```
zeiten1 <- tibble(
  n = (4:9)*1e3L,
  mess1 = c(3.656, 4.672, 5.719, 6.720, 7.641, 8.750),
  mess2 = c(3.781, 4.735, 5.672, 6.782, 7.673, 8.798),
  mess3 = c(3.719, 4.719, 5.672, 6.688, 7.704, 9.048),
  mess4 = c(3.766, 4.688, 5.625, 6.610, 7.750, 8.735)
)

zeiten1
# A tibble: 6 x 5
      n mess1 mess2 mess3 mess4
  <int> <dbl> <dbl> <dbl> <dbl>
1  4000  3.66  3.78  3.72  3.77
2  5000  4.67  4.74  4.72  4.69
3  6000  5.72  5.67  5.67  5.62
4  7000  6.72  6.78  6.69  6.61
5  8000  7.64  7.67  7.70  7.75
6  9000  8.75  8.80  9.05  8.74
```

```
> print(zeiten2, n=Inf)
# A tibble: 24 x 3
       n Nr     zeit
   <int> <chr> <dbl>
 1  4000 mess1  3.66
 2  5000 mess1  4.67
 3  6000 mess1  5.72
 4  7000 mess1  6.72
 5  8000 mess1  7.64
 6  9000 mess1  8.75
 7  4000 mess2  3.78
 8  5000 mess2  4.74
 9  6000 mess2  5.67
10  7000 mess2  6.78
11  8000 mess2  7.67
12  9000 mess2  8.80
13  4000 mess3  3.72
14  5000 mess3  4.72
15  6000 mess3  5.67
16  7000 mess3  6.69
17  8000 mess3  7.70
18  9000 mess3  9.05
19  4000 mess4  3.77
20  5000 mess4  4.69
21  6000 mess4  5.62
22  7000 mess4  6.61
23  8000 mess4  7.75
24  9000 mess4  8.74
```

a) Wandeln Sie `zeiten1` in `zeiten2` um (siehe Auflistung rechts).

b) Handelt es sich bei `zeiten1` bzw. bei `zeiten2` um „Tidy Data“ im Sinne von Wickham?

c) Mit welcher Anweisung kann aus `zeiten2` der ursprüngliche Datensatz `zeiten1` erzeugt werden?

d) Berechnen Sie in Abhängigkeit von n die Mittelwerte der Laufzeiten. Gehen Sie zuerst davon aus, dass lediglich `zeiten1` gegeben ist, danach gehen Sie davon aus, dass nur `zeiten2` gegeben ist.

[1] siehe `library(nycflights13)`

Aufgabe 2

Ähnlich wie in der vorherigen Aufgabe sind unterschiedliche Laufzeiten eines Algorithmus in Abhängigkeit von `n` gegeben:

```
n <- (4:9)*1e3L
mess1 <- c(3.656, NA, 5.719, NA, 7.641, 8.750)
mess2 <- c(3.781, 4.735, NA, 6.782,NA , 8.798)
mess3 <- c(NA, 4.719, 5.672, 6.688, 7.704, 9.048)
mess4 <- c(NA, 4.688, 5.625, 6.610, 7.750, NA)
mess5 <- c(3.766, 4.672,NA, 6.720, 7.673, 8.735  )
mess6 <- c(3.719, NA, NA,NA, NA,NA)
zeiten1 <- tibble(n, mess1, mess2, mess3, mess4, mess5, mess6)
```

```
> zeiten1
# A tibble: 6 x 7
      n mess1 mess2 mess3 mess4 mess5 mess6
  <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  4000  3.66  3.78 NA    NA     3.77  3.72
2  5000 NA     4.74  4.72  4.69  4.67 NA
3  6000  5.72 NA     5.67  5.62 NA    NA
4  7000 NA     6.78  6.69  6.61  6.72 NA
5  8000  7.64 NA     7.70  7.75  7.67 NA
6  9000  8.75  8.80  9.05 NA     8.74 NA
```

a) Wandeln Sie ähnlich wie in der vorherigen Aufgabe `zeiten1` in `zeiten2` um (siehe Auflistung rechts).

b) Handelt es sich bei `zeiten1` bzw. bei `zeiten2` um „Tidy Data“ im Sinne von Wickham?

c) Mit welcher Anweisung kann aus `zeiten2` der ursprüngliche Datensatz `zeiten1` erzeugt werden?

d) Berechnen Sie in Abhängigkeit von `n` die Mittelwerte der Laufzeiten ausgehend jeweils von `zeiten1` und `zeiten2`.

```
> print(zeiten2, n=Inf)
# A tibble: 23 x 3
       n Nr     zeit
   <int> <fct> <dbl>
 1  4000 mess1  3.66
 2  6000 mess1  5.72
 3  8000 mess1  7.64
 4  9000 mess1  8.75
 5  4000 mess2  3.78
 6  5000 mess2  4.74
 7  7000 mess2  6.78
 8  9000 mess2  8.80
 9  5000 mess3  4.72
10  6000 mess3  5.67
11  7000 mess3  6.69
12  8000 mess3  7.70
13  9000 mess3  9.05
14  5000 mess4  4.69
15  6000 mess4  5.62
16  7000 mess4  6.61
17  8000 mess4  7.75
18  4000 mess5  3.77
19  5000 mess5  4.67
20  7000 mess5  6.72
21  8000 mess5  7.67
22  9000 mess5  8.74
23  4000 mess6  3.72
```

Aufgabe 3

Welche Zielflughäfen sind in `flights` (als `dest`) aufgelistet, erscheinen aber nicht als FAA-Kennung in `airports`?

Aufgabe 4

Geben Sie diejenigen Datensätze aus `airports` aus, die als Zielflughafen in `flights` (als `dest`) vorkommen.

Aufgabe 5

Geben Sie diejenigen Datensätze aus `airports` aus, die als Abflughafen in `flights` (als `origin`) vorkommen.

Aufgabe 6

Geben Sie Breitengrad und Längengrad der drei Abflughäfen im Datensatz `flights` aus. Unterscheiden sich die Koordinatenangaben deutlich und was sind die entsprechenden Mittelwerte?

Aufgabe 7

a) Nennen Sie den Datensatz `flights` in den Datensatz `flights_withCoord` um und fügen Sie aus dem Datensatz `airports` den Breitengrad (Latitude) und den Längengrad (Longitude) des jeweiligen Zielflughafens in den Datensatz `flights` ein.

b) Wie groß sind die Korrelationen zwischen
- Breitengrad und Fluglänge
- Längengrad und Fluglänge? (Hinweis: `use="complete.obs"`)

Ändern sich diese Korrelationen, wenn man die Fluglänge in Kilometer statt in Meilen bei der Berechnung der Korrelation zugrundelegt?

c) Betrachten Sie nun für jeden Flug die Differenz der Längen- und Breitengrade von Abflug- und Zielflughafen. Bestimmen Sie die Wurzel aus der Summe der Quadrate dieser beiden Differenzen und ermitteln Sie die Korrelation mit der Fluglänge. Was für einen Wert erwarten Sie?

Aufgabe 8

Was geben die folgenden Anweisungen aus und wieso?

```
map(1:10, rnorm)
map(1:10, ~rnorm(.))
map(1:10, ~rnorm(4, .))
```

Aufgabe 9

Berechnen Sie die Mittelwerte aller Spalten in `mtcars`.

Aufgabe 10

Implementieren Sie mithilfe der Funktionen `is.numeric()` und `map_lgl()` eine Funktion `numerisch()`, die als Argument einen Tibble `df` erwartet und den Tibble zurückgibt, der genau aus allen numerischen Spalten von `df` besteht. Insbesondere soll dann möglich sein:

```
> cor(numerisch(diamonds), use="complete.obs")
           carat       depth      table      price           x           y          z
carat 1.00000000  0.02822431  0.1816175  0.9215913  0.97509423  0.95172220 0.95338738
depth 0.02822431  1.00000000 -0.2957785 -0.0106474 -0.02528925 -0.02934067 0.09492388
table 0.18161755 -0.29577852  1.0000000  0.1271339  0.19534428  0.18376015 0.15092869
price 0.92159130 -0.01064740  0.1271339  1.0000000  0.88443516  0.86542090 0.86124944
x     0.97509423 -0.02528925  0.1953443  0.8844352  1.00000000  0.97470148 0.97077180
y     0.95172220 -0.02934067  0.1837601  0.8654209  0.97470148  1.00000000 0.95200572
z     0.95338738  0.09492388  0.1509287  0.8612494  0.97077180  0.95200572 1.00000000
```

```
> cor(numerisch(mpg), use="complete.obs")
           displ         year        cyl         cty          hwy
displ  1.0000000  0.147842816  0.9302271 -0.79852397 -0.766020021
year   0.1478428  1.000000000  0.1222453 -0.03723229  0.002157643
cyl    0.9302271  0.122245347  1.0000000 -0.80577141 -0.761912354
cty   -0.7985240 -0.037232291 -0.8057714  1.00000000  0.955915914
hwy   -0.7660200  0.002157643 -0.7619124  0.95591591  1.000000000
```

Aufgabe 11

Erklären Sie folgenden Quelltext:

```
> d <- diamonds %>% drop_na() %>% mutate(vol=x*y*z)

> lmd <- lm(price ~ vol, data = d)
> sum_lmd <- summary(lmd)
> sum_lmd$r.squared
[1] 0.8142979

> models <- d %>%
+   split(.$cut) %>%
+   map(~lm(price ~ vol, data = .))

> models %>%
+   map(summary) %>%
+   map_dbl(~.$r.squared)
     Fair      Good Very Good   Premium     Ideal
0.7391252 0.8437127 0.8523177 0.7516382 0.8616549
```

Programmierung für statistische Datenanalyse

Übung 10

Objektorientierte Programmierung in R, Laufzeitverhalten, three dots ellipsis

Aufgabe 1

Woran erkennt man, dass ein R-Objekt kein Objekt einer S3 Klasse ist?

Aufgabe 2

Implementieren Sie eine S3-Klasse `binseq`, die binäre (nicht-leere) Folgen modelliert. Eine binäre Folge ist eine endliche Folge, die nur aus den Elementen -1 und +1 besteht. Die Klasse soll sich wie folgt verhalten:

```
> s <- binseq(c(1,1,1,-1,1,-1,-1))
> str(s)
 'binseq' int [1:7] 1 1 1 -1 1 -1 -1
 - attr(*, "n")= int 7
> print(s)
[1] "Binäre Folge der Länge 7"
[1]  1  1  1 -1  1 -1 -1
> attributes(s)
$n
[1] 7

$class
[1] "binseq"
```

```
> binseq(c(1,1,-1.0,1))
[1] "Binäre Folge der Länge 4"
[1]  1  1 -1  1
> binseq(c(1,3,-1.0,1))
 Fehler: Alle Elemente dürfen nur den Wert 1 oder -1 haben
> binseq(double())
 Fehler: n==0
> binseq(c("a", "b"))
Fehler: Alle Elemente dürfen nur den Wert 1 oder -1 haben
Zusätzlich: Warnmeldung:
In binseq(c("a", "b")) :
 Fehler: Alle Elemente dürfen nur den Wert 1 oder -1 haben
> binseq(-1)
[1] "Binäre Folge der Länge 1"
[1] -1
> binseq(c(1,1,1,0))
 Fehler: Alle Elemente dürfen nur den Wert 1 oder -1 haben
> binseq(c(T,T,T,F))
[1] "Binäre Folge der Länge 4"
[1]  1  1  1 -1
> identical(binseq(c(T,T,T,F)), binseq(c(1,1,1,-1)))
[1] TRUE
```

Gehen Sie dabei genauso vor wie in Kapitel 13 in „*Advanced R*“ von Hadley Wickham https://adv-r.hadley.nz/ [1] beschrieben:

- Implementieren Sie die Funktionen `new_binseq()`, `validate_binseq()` und `binseq()`
- Erzeugen Sie die Methode `print.binseq()` für die generische Funktion `print()`

Aufgabe 3

a) Was sind generische Funktionen?
b) Was ist der Unterschied zwischen einer generischen Funktion und einer Methode?
c) Gibt es in Base-R mehr als 100 generische Funktionen?
d) Was gibt die Funktion `methods()` zurück? (Zum Beispiel: `methods(mean)` oder `methods(print)`)

Aufgabe 4

Eine ***binäre Folge der Länge n*** ist eine endliche Folge $s = (s_1, s_2, \cdots, s_n)$ mit $s_j \in \{-1, +1\}$ für alle $1 \le j \le n$. Für $0 < k < n$ ist die ***aperiodische Autokorrelation*** *(der Ordnung k)* definiert durch:

$$C_k(s) = \sum_{j=1}^{n-k} s_j \cdot s_{j+k}$$

In R kann s leicht durch einen Integer-Vektor dargestellt und die aperiodische Autokorrelation beispielsweise wie folgt implementiert werden:

```
autokorrelation1 <- function(s) {
  n <- length(s)
  res <- vector(mode = "integer", length = n - 1)
  for (k in 1:(n-1)) {
    res[k] <- as.integer(t(s[1:(n - k)]) %*% s[(k + 1):n])
  }
  return(res)
}
```

a) Modifizieren Sie die Funktion `autokorrelation1()`, indem Sie die Schleife durch `map_int()` (aus `purrr`) ersetzen. Nennen Sie die modifizierte Version `autokorrelation2()`.

b) Vergleichen Sie die Laufzeiten der beiden Funktionen für große `n` mithilfe von `system.time()`, beispielsweise durch:

```
randBinSeq <- function(n) {
  return(sample(c(-1L, 1L), n, replace = TRUE))
}
s <- randBinSeq(2E4)
autokorrelation1(s)
autokorrelation2(s)
```

c) Erstellen Sie einen weiteren Laufzeittest; erklären und interpretieren Sie die Ergebnisse:

```
require(compiler)
f <- cmpfun(autokorrelation1)
system.time(f(s))
```

HINWEIS: Byte Code Compilation und JIT-Compiler

[1] Dies ist die Web-Seite der zweiten Auflage des Buchs *Advanced R (Chapman & Hall's R Series)* von Hadley Wickham

Aufgabe 5

Erzeugen Sie eine neue generische Funktion `autokor()` und implementieren Sie für die Klasse `binseq` (siehe oben) eine entsprechende Methode, die die aperiodische Autokorrelation zurückgibt. Wann ist dieses Vorgehen sinnvoll?

Aufgabe 6

Erklären Sie den folgenden Quelltext:

```
foo1 <- function(x,...) {
  print(x)
  args <- list(...)
  for(a in args){
    print(a)
  }
}

foo1(1)
foo1(1,2,3,4, mean=6)
```

```
foo2 <- function(n, ...) {
  return(rnorm(n, ...))
}

foo2(4)
foo2(4,100)
foo2(4, mean=100)
foo2(4, mean=100, sd=500)
foo2(4, meen=100, sd=500)
```

```
foo3 <- function(x, ...) {
  return(c(mean(x,...), median(x,...)))
}

x <- c(1,2,100,4)
foo3(x)
x <- c(1,2,NA, 100,4)
foo3(x)
foo3(x, na.rm=TRUE)
foo3(x, rm.na=TRUE)
```

HINWEIS (nicht direkt klausurrelevant)

Das in R wichtige Konzept `environment` haben wir bisher nur indirekt behandelt. Wer mehr wissen will, den verweise ich auf https://adv-r.hadley.nz/environments.html.

Jürgen Willms

Fachhochschule Südwestfalen
University of Applied Sciences

Programmierung für statistische Datenanalyse
Probeklausur WS 19/20

16.1.2020, **Bearbeitungszeit: 90 min**

.....	.....	.....	.....	.....
Name	Vorname	Matrikel-Nr.	Studiengang	Unterschrift

1 *(10)*	**2** *(6)*	**3** *(6)*	**4** *(6)*	**5** *(8)*	**6** *(4)*	**7** *(9)*	**8** *(6)*	**9** *(10)*	**10** *(8)*	**11** *(6)*	**12** *(6)*	**13** *(5)*		**Prüfer**	**Note**

Diese Klausur besteht aus **13 Aufgaben mit insgesamt 90 Punkten**. Wenn Sie aus Platzgründen eine Teilaufgabe auf einem Zusatzblatt lösen, ist dieses ausdrücklich auf dem Aufgabenblatt zu vermerken. Sollte aus Ihrer Sicht eine Aufgabenstellung unklar (oder gar fehlerhaft) sein, so treffen Sie sinnvolle Annahmen und vermerken Sie dies zusammen mit einer Begründung am Anfang Ihrer Lösung.

Abgesehen von den vier Cheat-Sheets **sind keine weiteren Hilfsmittel zugelassen!** Der genaue Zeitpunkt der Abgabe wird bekanntgegeben. Nach Beendigung der Klausur obliegt es Ihrer Verantwortung, die vollständige Klausur unverzüglich abzugeben. Eine Täuschung oder ein Täuschungsversuch führt zum Nichtbestehen der Klausur.

Unter „zufällig“ wollen wir im Folgenden immer „pseudo-zufällig“ verstehen (Ähnliches gilt, wenn wir von „Zufallszahlen“ reden). Alle Aufgaben beziehen sich auf die Programmiersprache R! Bei allen Aufgaben und Lösungen können Sie davon ausgehen, dass das `tidyverse`-Package geladen ist. Im Folgenden gilt: mehrere Anweisungen, die durch Pipe-Operatoren verbunden werden, zählen nur als eine Anweisung

Viel Erfolg!

Aufgabe 1 (10 Punkte)

Welchen Wert geben folgende Anweisungen zurück?

`3/0`	
`0/0`	
`10%/%4`	
`10e2`	
`10e-2`	
`0*log(0)`	
`'%%' (1:6, '-'(6,3))`	
`choose(8,3)`	
`cumsum(c(1,2,3,4,5))`	
`1:4 + 1:2`	

Aufgabe 2 (6 Punkte)

a) Gegeben ist:

```
x <- 1:5
setze1 <- function(i, wert){
  x[i] <- wert
}
setze1(1,99)
```

Was gibt dann x aus?

b) Gegeben ist:

```
x <- 1:5
setze2 <- function(x, i, wert){
  x[i] <- wert
}
setze1(1,99)
```

Was gibt dann x aus?

Aufgabe 3 (6 Punkte)

Gegeben ist die Matrix A durch: `A <- matrix(1:12, nrow=4)`

a) Löschen Sie die dritte Zeile und die zweite Spalte.

b) Fügen Sie eine neue Spalte am Ende mit den Werten 0 hinzu.

Aufgabe 4 (6 Punkte)

Gegeben ist eine diskrete Variable `dx` und eine kontinuierliche Variable `kx`. Mithilfe von `ggplot2` soll die Verteilung der Werte von `dx` visualisiert werden; ebenso soll in einem separaten Plot die Verteilung der Werte von `dx` visualisiert werden.

a) Wann benutzen Sie `geom_bar()` und wann `geom_histogram()`?

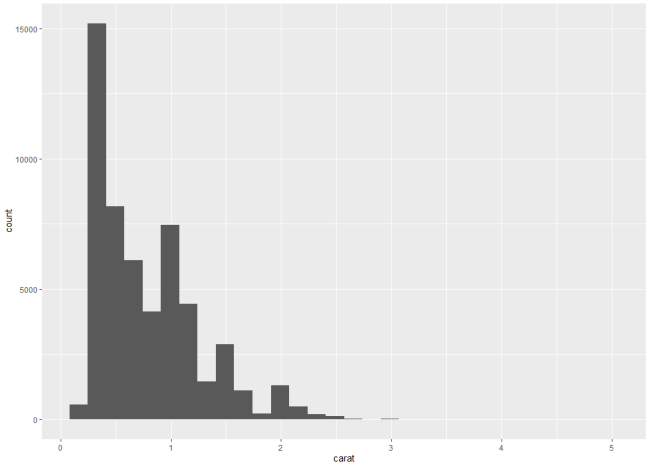
b) Bei der Auswertung des Datensatzes `diamonds` wurden folgende Plots vorgeschlagen. Was ist falsch oder zumindest fraglich und warum?

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut))

ggplot(data = diamonds) +
  geom_histogram(mapping = aes(x = carat))

ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = carat))

ggplot(data = diamonds) +
  geom_histogram(mapping = aes(x = cut))
```



Aufgabe 5 (8 Punkte)

a) Gegeben ist die obige Grafik. Sie wurde durch eine Anweisung erzeugt, die ganz ähnlich einer der folgenden Anweisungen ist, in der sich leider ein Fehler eingeschlichen hat. Mit welcher Anweisung wurde die obige Grafik erzeugt?

```
ggplot(data = diamonds) +
  geom_histogram(mapping = aes(z = carat))

ggplot(data = diamonds) +
  geom_histogram(mapping = aes(z = carat, y = count))

ggplot(data = diamonds) +
  geom_bar(mapping = aes(z = carat))

ggplot(data = diamonds) +
  geom_bar(mapping = aes(z = carat, y = count))

ggplot(data = diamonds) +
  geom_point(mapping = aes(z = carat))

ggplot(data = diamonds) +
  geom_point(mapping = aes(z = carat, y = count))
```

b) In der obigen Grafik soll durch zusätzliche Farbinformationen verdeutlicht werden, wie die dargestellten Häufigkeiten bzgl. der Variablen „cut“ aufgeteilt sind. Wie kann dies erreicht werden?

Aufgabe 6 (4 Punkte)

Gegeben ist folgender Tibble:

```
df <- tibble(nr= 1:100, x = nr%%5 , y = nr%%3, z = nr%%7, w = z%%3)
```

Was bewirkt die folgende Schleife?

```
for (i in seq.int(nrow(df))) {
  sdaten <- df[i,c("x", "y", "z", "w")]
  df[i,c("x", "y", "z","w")] <- sdaten[order(unlist(sdaten))]
}
```

Aufgabe 7 (9 Punkte)

Gegeben ist folgender Tibble `stud`

```
A tibble: 40 x 5
   MatNr Studiengang Punkte1 Punkte2 Punkte3
   <dbl> <fct>         <int>   <int>   <int>
 1   467 WInfo           61      81      59
 2   549 ET              62      46      10
 3   184 MB              79     111     130
 4   581 WInfo           63      55      84
 5   425 WInfo           64      28      81
 6   720 WInfo           51      57      67
 7   453 WInfo           58      65      80
 8   917 MB              66     114      63
 9   910 WInfo           53      63      78
10   147 ET              75      71      57
# … with 30 more rows
```

a) Bestimmen Sie mit einer Anweisung die Durchschnittspunktzahl der Spalte `Punkte1` innerhalb der jeweiligen Studiengänge. Die Aussage sollte so aussehen:

```
# A tibble: 4 x 2
  Studiengang Durchschnitt
  <fct>              <dbl>
1 ET                  72.6
2 MB                  73.2
3 WInfo               58.7
4 WIng                64.9
```

b) Fügen Sie in einer Anweisung eine neue Variable `Gesamt` in `stud` ein, die der Summe der drei Punktwerte entspricht.

c) Geben Sie mithilfe einer Anweisung den Tibble `stud` in aufsteigender Reihenfolge gemäß der eingefügten Spalte `Gesamt` aus. Die Ausgabe sollte so aussehen:

```
A tibble: 40 x 6
   MatNr Studiengang Punkte1 Punkte2 Punkte3 Gesamt
   <dbl> <fct>         <int>   <int>   <int>  <int>
 1   317 WIng             39      16      50    105
 2   369 WIng             27      51      58    136
 3   187 ET               44      79      20    143
 4   479 MB               66      82      14    162
 5   696 ET               73      41      53    167
 6   206 ET               57      53      58    168
 7   628 MB               69      52      47    168
 8   865 ET               72      24      74    170
 9   217 WIng             44      36      95    175
10   642 ET               63      52      63    178
# … with 30 more rows
```

Aufgabe 8 (6 Punkte)

Implementieren Sie mithilfe der Funktionen `is.numeric()` und `map_lgl()` eine Funktion `numerisch()`, die als Argument einen Tibble `df` erwartet und den Tibble zurückgibt, der genau aus allen numerischen Spalten von `df` besteht.

Aufgabe 9 (10 Punkte)

Gegeben sind jeweils vier gemessene Laufzeiten eines Algorithmus in Abhängigkeit von der Anzahl der Eingabedaten `n` im Tibble `zeiten1`:

```
zeiten1
# A tibble: 6 x 5
      n mess1 mess2 mess3 mess4
  <int> <dbl> <dbl> <dbl> <dbl>
1  4000  3.66  3.78  3.72  3.77
2  5000  4.67  4.74  4.72  4.69
3  6000  5.72  5.67  5.67  5.62
4  7000  6.72  6.78  6.69  6.61
5  8000  7.64  7.67  7.70  7.75
6  9000  8.75  8.80  9.05  8.74
```

a) Wandeln Sie `zeiten1` in `zeiten2` um (siehe Auflistung rechts).

```
> print(zeiten2, n=Inf)
# A tibble: 24 x 3
       n Nr     zeit
   <int> <chr> <dbl>
 1  4000 mess1  3.66
 2  5000 mess1  4.67
 3  6000 mess1  5.72
 4  7000 mess1  6.72
 5  8000 mess1  7.64
 6  9000 mess1  8.75
 7  4000 mess2  3.78
 8  5000 mess2  4.74
 9  6000 mess2  5.67
10  7000 mess2  6.78
11  8000 mess2  7.67
12  9000 mess2  8.80
13  4000 mess3  3.72
14  5000 mess3  4.72
15  6000 mess3  5.67
16  7000 mess3  6.69
17  8000 mess3  7.70
18  9000 mess3  9.05
19  4000 mess4  3.77
20  5000 mess4  4.69
21  6000 mess4  5.62
22  7000 mess4  6.61
23  8000 mess4  7.75
24  9000 mess4  8.74
```

b) Berechnen Sie in Abhängigkeit von `n` die Mittelwerte der Laufzeiten. Gehen Sie zuerst davon aus, dass lediglich `zeiten1` gegeben ist, danach gehen Sie davon aus, dass nur `zeiten2` gegeben ist.

Aufgabe 10 (8 Punkte)

Gegeben sind (ähnlich wie `flights` und `airport` aus dem Paket `nycflights13`):

```
> fluege
# A tibble: 336,776 x 8
   flight tailnum origin dest  dep_time dep_delay arr_time arr_delay
    <int> <chr>   <chr>  <chr>    <int>     <dbl>    <int>     <dbl>
 1   1545 N14228  EWR    IAH        517         2      830        11
 2   1714 N24211  LGA    IAH        533         4      850        20
 3   1141 N619AA  JFK    MIA        542         2      923        33
 4    725 N804JB  JFK    BQN        544        -1     1004       -18
 5    461 N668DN  LGA    ATL        554        -6      812       -25
 6   1696 N39463  EWR    ORD        554        -4      740        12
 7    507 N516JB  EWR    FLL        555        -5      913        19
 8   5708 N829AS  LGA    IAD        557        -3      709       -14
 9     79 N593JB  JFK    MCO        557        -3      838        -8
10    301 N3ALAA  LGA    ORD        558        -2      753         8
# … with 336,766 more rows
```

```
flughafen
# A tibble: 1,458 x 5
   faa   name                            lat    lon   alt
   <chr> <chr>                         <dbl>  <dbl> <dbl>
 1 04G   Lansdowne Airport              41.1  -80.6  1044
 2 06A   Moton Field Municipal Airport  32.5  -85.7   264
 3 06C   Schaumburg Regional            42.0  -88.1   801
 4 06N   Randall Airport                41.4  -74.4   523
 5 09J   Jekyll Island Airport          31.1  -81.4    11
 6 0A9   Elizabethton Municipal Airport 36.4  -82.2  1593
 7 0G6   Williams County Airport        41.5  -84.5   730
 8 0G7   Finger Lakes Regional Airport  42.9  -76.8   492
 9 0P2   Shoestring Aviation Airfield   39.8  -76.6  1000
10 0S9   Jefferson County Intl          48.1 -123.    108
# … with 1,448 more rows
```

a) Bestimmen Sie mit einer Anweisung die Zielflughäfen in `fluege` (siehe Variable `dest`), deren FAA-Kennung (siehe Variable `faa`) nicht in `flughafen` aufgelistet sind?

b) Bestimmen Sie mit einer Anweisung diejenigen Datensätze aus `flughafen`, die als Zielflughafen in `fluege` (siehe Variable `dest`) vorkommen.

Aufgabe 11 (6 Punkte)

a) Was sind generische Funktionen?

b) Was ist der Unterschied zwischen einer generischen Funktion und einer Methode?

c) Was gibt die Funktion `methods()` zurück? (Zum Beispiel: `methods(mean)` oder `methods(print)`)

Aufgabe 12 (6 Punkte)

Was gibt folgender Quelltext aus?

```
foo3 <- function(x, ...) {
  return(c(mean(x,...), median(x,...)))
}

x <- c(1,3,100,4)
foo3(x)
x <- c(1,3,NA, 100,4)
foo3(x)
foo3(x, na.rm=TRUE)
foo3(x, rm.na=TRUE)
```

Aufgabe 13 (5 Punkte)

Gegeben ist:

```
dataScience <- function(x) {
  structure(
    x,
    class = "dataScience"
  )
}

print.dataScience <- function(x, ...) {
  print(paste("DataScience",length(x) ))
  print(unclass(x))
}

a <- dataScience(1:4);
```

Was gibt folgender Quelltext aus?

```
print(a)
```